



UNIVERSITAT POLITÈCNICA DE CATALUNYA  
BARCELONATECH

Facultat d'Informàtica de Barcelona



# DIAGNÓSTICO AUTOMATIZADO DE LINFOMAS MALIGNOS EN BIOPSIAS H&E MEDIANTE APRENDIZAJE AUTOMÁTICO

**AFTAB AHMED CHOUDHRY**

**Director/a**

JORDI DELGADO PIN (Departamento de Ciencias de la Computación)

**Titulación**

Grado en Ingeniería Informática (Computación)

**Memoria del trabajo de fin de grado**

**Facultat d'Informàtica de Barcelona (FIB)**

**Universitat Politècnica de Catalunya (UPC) - BarcelonaTech**

20 de junio de 2025

## Abstract

The purpose of this Final Degree Project is to explore, experiment with, and develop an automated system that uses deep learning techniques to classify histological lymphoma images into three subtypes: CLL, FL, and MCL. This classification has high clinical value, as it can enable faster and more accurate diagnoses, thus supporting medical specialists in decision-making.

For this purpose, the “Malignant Lymphoma Classification” dataset available on Kaggle was used, consisting of a total of 374 images distributed across the three classes. Given the small size of the dataset, various data augmentation techniques were applied to enrich the data and improve the model's generalization capability. The images, originally in .tif format, were converted to .png, resized to 224x224 pixels, and normalized. They were then split in a stratified manner into training, validation, and test subsets (80%-10%-10%).

Multiple pre-trained convolutional neural network architectures were tested, specifically InceptionV3, MobileNetV2, DenseNet201, EfficientNetV2B0, ResNet50V2, and VGG16, using transfer learning in two phases: feature extraction and fine-tuning. During training, optimization techniques such as the Adam algorithm and data augmentation were applied. The performance of each model was evaluated using metrics such as accuracy, classification report, confusion matrix, and multiclass performance indicators.

The results show a high level of accuracy on both validation and test data, indicating that the model performs well even with previously unseen images. Class-specific analysis demonstrates that the system reliably distinguishes between the three lymphoma subtypes, achieving a good balance between correctly identifying cases and minimizing errors.

In conclusion, this project demonstrates the feasibility of applying deep learning techniques to the medical field with moderate resources and a sustainable approach. The proposed solution can be a valuable support tool in clinical diagnosis, reducing the workload of healthcare professionals and improving the accuracy of lymphoma detection. The project also reflects deep learning in both the technical and ethical sense, with clear awareness of its economic, social, and environmental implications.

All source code developed for this project is publicly available in my GitHub repository [56].

## Resumen

El propósito de este Trabajo de Fin de Grado es explorar, experimentar y desarrollar un sistema automatizado que utilice técnicas de aprendizaje profundo para clasificar imágenes histológicas de linfoma en tres subtipos: CLL, FL y MCL. Esta clasificación tiene un alto valor clínico, ya que puede facilitar un diagnóstico más rápido y preciso, apoyando así a los especialistas médicos en la toma de decisiones.

Para ello se utilizó el conjunto de datos "Malignant Lymphoma Classification" disponible en Kaggle, compuesto por un total de 374 imágenes distribuidas entre las tres clases. Dado el tamaño reducido del dataset, se aplicaron diversas técnicas de data augmentation para enriquecer los datos y mejorar la capacidad de generalización del modelo. Las imágenes, originalmente en formato .tif, fueron convertidas a .png, redimensionadas a 224x224 píxeles y normalizadas. Posteriormente, se dividieron de forma estratificada en subconjuntos de entrenamiento, validación y prueba (80%-10%-10%).

Se experimentó con múltiples arquitecturas de redes neuronales convolucionales preentrenadas, concretamente InceptionV3, MobileNetV2, DenseNet201, EfficientNetV2B0, ResNet50V2 y VGG16, utilizando la técnica de transfer learning en dos fases: feature extraction y fine tuning. Durante el entrenamiento, se aplicaron técnicas de optimización como el algoritmo Adam y data augmentation. El rendimiento de cada modelo fue evaluado utilizando métricas como la precisión, el informe de clasificación, la matriz de confusión y las curvas ROC multiclase.

Los resultados obtenidos muestran un alto nivel de precisión tanto en los datos de validación como en los de prueba, lo que indica que el modelo funciona bien incluso con imágenes que no ha visto antes. El análisis del rendimiento por clase demuestra que el sistema identifica de forma equilibrada los tres tipos de linfoma, logrando un buen compromiso entre detectar correctamente los casos y evitar errores.

En conclusión, este TFG demuestra que es factible aplicar técnicas de deep learning al ámbito médico con recursos moderados y un enfoque sostenible. La solución propuesta puede constituir un valioso apoyo al diagnóstico clínico, reduciendo la carga de trabajo de los profesionales y mejorando la precisión en la detección de linfomas. El proyecto también refleja un aprendizaje profundo en términos técnicos y éticos, con una clara conciencia de sus implicaciones económicas, sociales y medioambientales.

Todo el código fuente desarrollado para este proyecto se encuentra disponible públicamente en mi repositorio de GitHub [56].

## Resum

L'objectiu d'aquest Treball de Fi de Grau és explorar, experimentar i desenvolupar un sistema automatitzat que utilitzi tècniques d'aprenentatge profund per classificar imatges histològiques de limfoma en tres subtipus: CLL, FL i MCL. Aquesta classificació té un alt valor clínic, ja que pot facilitar un diagnòstic més ràpid i precís, donant suport als especialistes mèdics en la presa de decisions.

Per a això s'ha utilitzat el conjunt de dades "Malignant Lymphoma Classification" disponible a Kaggle, compost per un total de 374 imatges distribuïdes entre les tres classes. Donada la mida reduïda del conjunt de dades, s'han aplicat diverses tècniques d'augment de dades per enriquir la informació disponible i millorar la capacitat de generalització del model. Les imatges, originalment en format .tif, es van convertir a .png, redimensionar a 224x224 píxels i normalitzar. Posteriorment, es van dividir de manera estratificada en subconjunts d'entrenament, validació i prova (80%-10%-10%).

S'ha experimentat amb múltiples arquitectures de xarxes neuronals convolucionals preentrenades, concretament InceptionV3, MobileNetV2, DenseNet201, EfficientNetV2B0, ResNet50V2 i VGG16, utilitzant la tècnica d'aprenentatge per transferència en dues fases: extracció de característiques i ajust fi (fine tuning). Durant l'entrenament, s'han aplicat tècniques d'optimització com l'algorisme Adam i augment de dades. El rendiment de cada model s'ha avaluat amb mètriques com la precisió, l'informe de classificació, la matriu de confusió i indicadors de rendiment per classe.

Els resultats obtinguts mostren un alt nivell de precisió tant en les dades de validació com en les de prova, cosa que indica que el model funciona bé fins i tot amb imatges que no ha vist abans. L'anàlisi per classes demostra que el sistema identifica de forma equilibrada els tres tipus de limfoma, assolint un bon compromís entre la detecció correcta dels casos i la minimització d'errors.

En conclusió, aquest TFG demostra que és factible aplicar tècniques d'aprenentatge profund en l'àmbit mèdic amb recursos moderats i un enfocament sostenible. La solució proposada pot constituir un suport valuós per al diagnòstic clínic, reduint la càrrega de treball dels professionals sanitaris i millorant la precisió en la detecció de limfomes. El projecte també reflecteix un aprenentatge profund tant en l'àmbit tècnic com ètic, amb una clara consciència de les seves implicacions econòmiques, socials i ambientals.

Tot el codi font desenvolupat per a aquest projecte està disponible públicament al meu repositori de GitHub [56].



# Índice

|         |                                                                  |    |
|---------|------------------------------------------------------------------|----|
| 1       | Introducción y Contextualización.....                            | 10 |
| 1.1     | Introducción.....                                                | 10 |
| 1.2     | Contexto.....                                                    | 10 |
| 1.3     | Conceptos.....                                                   | 11 |
| 1.3.1   | Linfoma.....                                                     | 11 |
| 1.3.1.1 | Leucemia Linfocítica Crónica (CLL).....                          | 12 |
| 1.3.1.2 | Linfoma Folicular (FL).....                                      | 12 |
| 1.3.1.3 | Linfoma de Células del Manto (MCL).....                          | 12 |
| 1.3.2   | Aprendizaje Automático.....                                      | 13 |
| 1.3.2.1 | Aprendizaje Profundo.....                                        | 13 |
| 1.3.2.2 | Redes neuronales.....                                            | 13 |
| 1.4     | Conceptos claves de Deep Learning utilizados en el proyecto..... | 14 |
| 1.4.1   | Redes neuronales convolucionales.....                            | 15 |
| 1.4.2   | Aprendizaje por transferencia.....                               | 16 |
| 1.4.2.1 | Inception V3.....                                                | 17 |
| 1.4.2.2 | MobileNet V2.....                                                | 18 |
| 1.4.2.3 | DenseNet 201.....                                                | 19 |
| 1.4.2.4 | EfficientNet V2B0.....                                           | 20 |
| 1.4.2.5 | ResNet 50V2.....                                                 | 21 |
| 1.4.2.6 | VGG16.....                                                       | 22 |
| 1.5     | Identificación del problema.....                                 | 23 |
| 1.6     | Partes interesadas.....                                          | 23 |
| 2       | Justificación.....                                               | 24 |
| 2.1     | Estudio de soluciones existentes.....                            | 24 |
| 2.2     | Otras alternativas.....                                          | 24 |
| 3       | Alcance.....                                                     | 25 |
| 3.1     | Objetivos.....                                                   | 25 |
| 3.2     | KPIs y Métricas para Evaluar el Éxito del Proyecto.....          | 25 |
| 3.2.1   | Nivel técnico.....                                               | 25 |
| 3.2.2   | Nivel de gestión del proyecto.....                               | 26 |
| 3.3     | Requerimientos.....                                              | 26 |
| 3.4     | Obstáculos y riesgos.....                                        | 26 |
| 4       | Metodología y rigor.....                                         | 27 |
| 5       | Planificación temporal.....                                      | 28 |
| 5.1     | Descripción de las tareas.....                                   | 28 |
| 5.1.1   | Gestión y seguimiento del proyecto.....                          | 28 |
| 5.1.1.1 | Contextualización y alcance.....                                 | 28 |
| 5.1.1.2 | Planificación temporal.....                                      | 28 |
| 5.1.1.3 | Presupuesto.....                                                 | 28 |

|         |                                                           |    |
|---------|-----------------------------------------------------------|----|
| 5.1.1.4 | Informe de sostenibilidad.....                            | 28 |
| 5.1.1.5 | Integración del documento final.....                      | 29 |
| 5.1.1.6 | Documentación.....                                        | 29 |
| 5.1.1.7 | Presentación.....                                         | 29 |
| 5.1.1.8 | Reuniones.....                                            | 29 |
| 5.1.2   | Trabajo previo.....                                       | 29 |
| 5.1.2.1 | Familiarización con Keras y TensorFlow.....               | 29 |
| 5.1.2.2 | Estudio de las CNN's.....                                 | 29 |
| 5.1.3   | Desarrollo de la CNN.....                                 | 30 |
| 5.1.3.1 | Diseñar la arquitectura de la CNN.....                    | 30 |
| 5.1.3.2 | Entrenar la CNN.....                                      | 30 |
| 5.1.3.3 | Evaluar el modelo.....                                    | 30 |
| 5.1.3.4 | Optimizar el modelo.....                                  | 30 |
| 5.1.3.5 | Analizar los patrones.....                                | 30 |
| 5.2     | Recursos.....                                             | 31 |
| 5.2.1   | Recursos humanos.....                                     | 31 |
| 5.2.2   | Recursos de hardware.....                                 | 31 |
| 5.2.3   | Recursos de software.....                                 | 31 |
| 5.2.4   | Recursos materiales.....                                  | 31 |
| 5.3     | Tabla de resumen.....                                     | 31 |
| 5.4     | Estimaciones y Gantt.....                                 | 33 |
| 5.5     | Gestión de riesgos: Planes alternativos y obstáculos..... | 33 |
| 6       | Presupuesto.....                                          | 34 |
| 6.1     | Identificación de los costes.....                         | 34 |
| 6.2     | Estimación de los costes.....                             | 36 |
| 6.3     | Control de gestión.....                                   | 37 |
| 7       | Sostenibilidad.....                                       | 38 |
| 7.1     | Autoevaluación.....                                       | 38 |
| 7.2     | Hito Inicial.....                                         | 39 |
| 7.2.1   | Dimensión Económica.....                                  | 39 |
| 7.2.2   | Dimensión Ambiental.....                                  | 39 |
| 7.2.3   | Dimensión Social.....                                     | 40 |
| 7.3     | Hito Final.....                                           | 40 |
| 7.3.1   | Dimensión Económica.....                                  | 40 |
| 7.3.1.1 | Proyecto Puesto en Producción.....                        | 41 |
| 7.3.1.2 | Vida Útil.....                                            | 41 |
| 7.3.1.3 | Riesgos.....                                              | 41 |
| 7.3.2   | Dimensión Ambiental.....                                  | 41 |
| 7.3.2.1 | Proyecto Puesto en Producción.....                        | 41 |
| 7.3.2.2 | Vida Útil.....                                            | 42 |
| 7.3.2.3 | Riesgos.....                                              | 42 |
| 7.3.3   | Dimensión Social.....                                     | 42 |

|         |                                                                                                               |    |
|---------|---------------------------------------------------------------------------------------------------------------|----|
| 7.3.3.1 | Proyecto Puesto en Producción.....                                                                            | 42 |
| 7.3.3.2 | Vida Útil.....                                                                                                | 42 |
| 7.3.3.3 | Riesgos.....                                                                                                  | 43 |
| 7.4     | Tabla de resumen.....                                                                                         | 43 |
| 8       | Trabajo Previo.....                                                                                           | 44 |
| 8.1     | Familiarización con Keras y TensorFlow.....                                                                   | 44 |
| 8.2     | Estudio de las Redes Neuronales Convolucionales.....                                                          | 44 |
| 9       | Experimento Preliminar antes del Desarrollo Principal.....                                                    | 45 |
| 9.1     | Selección de los datos.....                                                                                   | 45 |
| 9.2     | Preprocesamiento de los datos.....                                                                            | 45 |
| 9.3     | Diseño de la CNN.....                                                                                         | 46 |
| 9.4     | Entrenamiento del modelo.....                                                                                 | 46 |
| 9.5     | Evaluación del modelo.....                                                                                    | 46 |
| 10      | Desarrollo de la CNN para el Proyecto.....                                                                    | 47 |
| 10.1    | Trabajo previo con los mismos datos.....                                                                      | 47 |
| 10.2    | Selección de los datos.....                                                                                   | 48 |
| 10.3    | Configuración previa del entorno.....                                                                         | 48 |
| 10.4    | Preprocesamiento de los datos.....                                                                            | 49 |
| 10.4.1  | Aumento de datos explícito.....                                                                               | 49 |
| 10.4.2  | Aumento de datos implícito.....                                                                               | 50 |
| 10.5    | Protocolo común de diseño y entrenamiento de modelos.....                                                     | 50 |
| 10.5.1  | Inception V3.....                                                                                             | 51 |
| 10.5.2  | MobileNet V2.....                                                                                             | 51 |
| 10.5.3  | DenseNet 201.....                                                                                             | 52 |
| 10.5.4  | EfficientNet V2B0.....                                                                                        | 52 |
| 10.5.5  | ResNet 50V2.....                                                                                              | 52 |
| 10.5.6  | VGG16.....                                                                                                    | 53 |
| 10.6    | Comparación entre Modelos CNN Implementados.....                                                              | 53 |
| 11      | Dificultades Encontradas durante el Desarrollo del Proyecto.....                                              | 55 |
| 11.1    | Adquisición de los fundamentos teóricos y prácticos de las CNNs y su implementación con TensorFlow/Keras..... | 55 |
| 11.2    | Problemas de compatibilidad entre versiones de paquetes en el entorno virtual.....                            | 55 |
| 11.3    | Conjunto de datos muy reducido.....                                                                           | 55 |
| 11.4    | Aplicación incorrecta del Fine Tuning en las primeras pruebas.....                                            | 56 |
| 11.5    | Limitaciones de espacio en la GPU.....                                                                        | 56 |
| 11.6    | Imposibilidad de implementar técnicas de interpretación visual por falta de tiempo y conocimiento.....        | 56 |
| 12      | Aplicación e integración de conocimientos previos.....                                                        | 57 |
| 13      | Consideraciones Éticas y Licencias de Uso de los Datos.....                                                   | 57 |
| 14      | Conclusiones.....                                                                                             | 58 |
| 15      | Trabajos Futuros.....                                                                                         | 59 |
| 16      | Agradecimientos.....                                                                                          | 59 |



|    |                   |    |
|----|-------------------|----|
| 17 | Glosario.....     | 60 |
| 18 | Bibliografia..... | 61 |
| 19 | Anexos.....       | 66 |

## Índice de Figuras

|                                                                          |    |
|--------------------------------------------------------------------------|----|
| Figura 1: Linfoma no Hodgkin (NHL). [13].....                            | 11 |
| Figura 2: Comparación entre linfocitos normales y CLL. [14].....         | 12 |
| Figura 3: Comparación entre ganglio linfático normal y con FL. [15]..... | 12 |
| Figura 4: Evolución del MCL. [16].....                                   | 12 |
| Figura 5: Jerarquía de IA, ML, NN y DL. [17].....                        | 13 |
| Figura 6: Funcionamiento de las CNN's. [18].....                         | 15 |
| Figura 7: Transfer Learning. [45].....                                   | 16 |
| Figura 8: Feature Extraction and Fine Tuning. [57].....                  | 16 |
| Figura 9: Arquitectura de Inception V3. [44].....                        | 17 |
| Figura 10: Arquitectura de MobileNet V2. [58].....                       | 18 |
| Figura 11: Arquitectura de DenseNet 201. [42].....                       | 19 |
| Figura 12: Arquitectura de EfficientNet V2B0. [41].....                  | 20 |
| Figura 13: Arquitectura de ResNet 50V2. [43].....                        | 21 |
| Figura 14: Arquitectura de VGG16. [46].....                              | 22 |
| Figura 15: Organización en Trello (Elaboración propia).....              | 27 |
| Figura 16: Diagrama de Gantt (Elaboración propia).....                   | 33 |

## Índice de Tablas

|                                                                  |    |
|------------------------------------------------------------------|----|
| Tabla 1: Resumen de las tareas (Elaboración propia).....         | 33 |
| Tabla 2: Estructura de Presupuesto (Elaboración propia).....     | 36 |
| Tabla 3: Salario de cada rol (Elaboración propia).....           | 37 |
| Tabla 4: Resumen de la sostenibilidad (Elaboración propia).....  | 44 |
| Tabla 5: Comparación entre los modelos (Elaboración propia)..... | 54 |

# 1 Introducción y Contextualización

El cáncer es la principal causa de muerte en todo el mundo y representa uno de los desafíos más complejos para la salud pública. La detección temprana y la clasificación precisa de sus diferentes tipos son fundamentales para mejorar la tasa de supervivencia, pero los métodos tradicionales pueden verse limitados por la heterogeneidad de las muestras y la variabilidad en los procesos de tinción.

En este sentido, el creciente avance en técnicas de inteligencia artificial y, en particular, en el uso de modelos de aprendizaje automático, abre nuevas posibilidades para optimizar el diagnóstico. La aplicación de estos métodos en el análisis de imágenes histológicas promete reducir la dependencia de la interpretación subjetiva y mejorar la consistencia de los diagnósticos, permitiendo una intervención médica más precisa y eficaz en el tratamiento del paciente.

## 1.1 Introducción

Este Trabajo de Fin de Grado (TFG) en Ingeniería Informática, especialidad en Computación, se ha realizado en la Facultat d'Informàtica de Barcelona (FIB) de la Universitat Politècnica de Catalunya, bajo la dirección de Jordi Delgado Pin del departamento de Ciencias de la Computación.

El objetivo principal del proyecto es explorar y desarrollar un modelo de aprendizaje automático que clasifique tres tipos de linfomas malignos (CLL, FL y MCL) a partir de imágenes de biopsias teñidas con Hematoxilina/Eosina (H&E). De esta forma, se busca mejorar la precisión diagnóstica, facilitando un diagnóstico más consistente y menos dependiente de la experiencia de patólogos altamente especializados.

## 1.2 Contexto

Este trabajo de investigación se centra en el desarrollo de técnicas de machine learning para la clasificación de tres subtipos de linfoma no Hodgkin. El objetivo es explorar y comparar diferentes enfoques de aprendizaje automático para identificar cuál de ellos resulta más efectivo a la hora de diferenciar entre estos subtipos.

Se desarrollará una red neuronal especializada en analizar imágenes de biopsias teñidas, permitiendo distinguir de forma automática entre los tres subtipos de linfoma no Hodgkin. Esta red se entrenará con datos reales, lo que ayudará a mejorar la precisión en el diagnóstico y a reducir la dependencia de la interpretación manual.

En resumen, este proyecto se enmarca en un contexto en el que la inteligencia artificial está transformando el campo de la medicina, aportando herramientas innovadoras que permiten un diagnóstico más rápido, preciso y menos dependiente de la experiencia individual del especialista.

## 1.3 Conceptos

Para facilitar la comprensión del proyecto, a continuación se definen los conceptos clave que se abordarán a lo largo del trabajo.

### 1.3.1 Linfoma

El linfoma es un tipo de cáncer del sistema inmunitario que se manifiesta cuando los glóbulos blancos o linfocitos, que son células de la sangre que produce la médula ósea y que ayudan a defender el organismo contra las infecciones, pierden el control, se dividen de manera anormal o no mueren cuando deberían [1]. Es la enfermedad oncológica de la sangre más común entre los adultos y se estima que cada año se diagnostica linfoma a más de 735.000 personas en todo el mundo. Se divide en dos tipos principales:

- El linfoma de Hodgkin (HL)
- El linfoma no Hodgkin (NHL)

Según el cirujano oncólogo Leandro Nikisch, miembro de la Asociación Argentina de Cirugía, el mayor porcentaje de casos está en los linfomas no Hodgkin [2]. Hay más de 60 tipos diferentes de NHL, que también pueden ser clasificados como indolente (de crecimiento lento) o agresivo (de crecimiento rápido), dependiendo de la rapidez con la que se desarrolla [3].

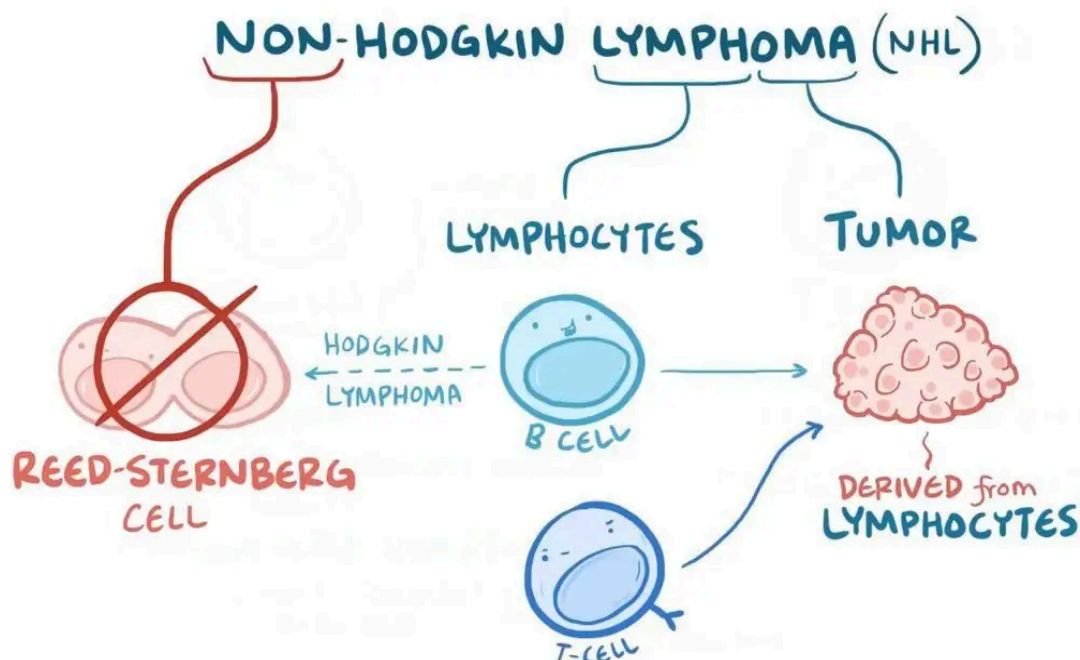


Figura 1: Linfoma no Hodgkin (NHL). [13]



### 1.3.1.1 Leucemia Linfocítica Crónica (CLL)

La leucemia linfocítica crónica (CLL), un subtipo de NHL, es la forma más común de leucemia en adultos en países occidentales. Se caracteriza por la acumulación progresiva de linfocitos B maduros en la sangre, médula ósea y otros tejidos linfoides. Aunque en muchos casos su evolución es indolente y puede permanecer asintomática durante años, en ciertos pacientes la enfermedad progresa, requiriendo tratamientos específicos para controlar su avance [4].

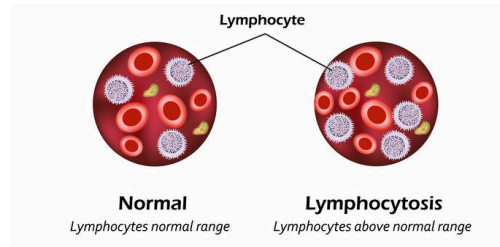


Figura 2: Comparación entre linfocitos normales y CLL. [14]

### 1.3.1.2 Linfoma Folicular (FL)

El linfoma folicular (FL) es el subtipo más común de NHL de crecimiento lento, que se origina en los linfocitos B. Afecta principalmente a los ganglios linfáticos y, en ocasiones, puede extenderse a la médula ósea o al bazo. La mayoría de los pacientes con FL son mayores de 50 años en el momento del diagnóstico. En algunos casos, el FL puede seguir un curso indolente y no requerir tratamiento inmediato, aunque existe el riesgo de que evolucione hacia una forma más agresiva [5].

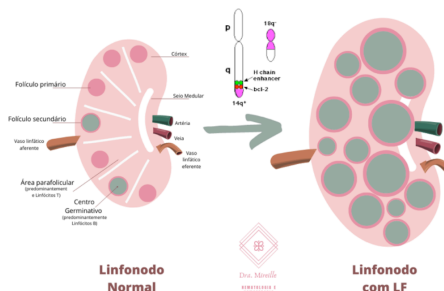


Figura 3: Comparación entre ganglio linfático normal y con FL. [15]

### 1.3.1.3 Linfoma de Células del Manto (MCL)

El linfoma de células del manto (MCL) es un subtipo poco frecuente de NHL que se origina en las células del manto, localizadas en la periferia de los ganglios linfáticos. Se caracteriza por presentar un comportamiento clínico generalmente agresivo, lo que exige un abordaje terapéutico intensivo. Además, este subtipo suele diagnosticarse en etapas avanzadas, lo que complica la aplicación de tratamientos convencionales y resalta la necesidad de estrategias terapéuticas innovadoras [6].

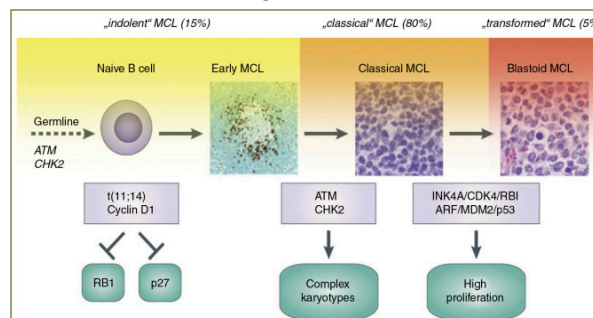


Figura 4: Evolución del MCL. [16]

### 1.3.2 Aprendizaje Automático

El aprendizaje automático (ML) es una rama de la inteligencia artificial (IA) que permite a las máquinas aprender a realizar tareas específicas sin necesidad de ser programadas explícitamente para cada una de ellas. Para ello, se basan en técnicas estadísticas que identifican patrones y realizan predicciones, siendo especialmente eficaces cuando se dispone de grandes volúmenes de datos. En el ámbito del diagnóstico, el ML ha demostrado ser una herramienta útil para la detección y clasificación de diversos tipos de cáncer, alcanzando en algunos casos una precisión comparable o superior a la obtenida mediante métodos tradicionales. Además, la aplicación de la IA y el ML tiene el potencial de mejorar no solo el diagnóstico y el pronóstico, sino también la calidad de vida de los pacientes con distintas enfermedades [7].

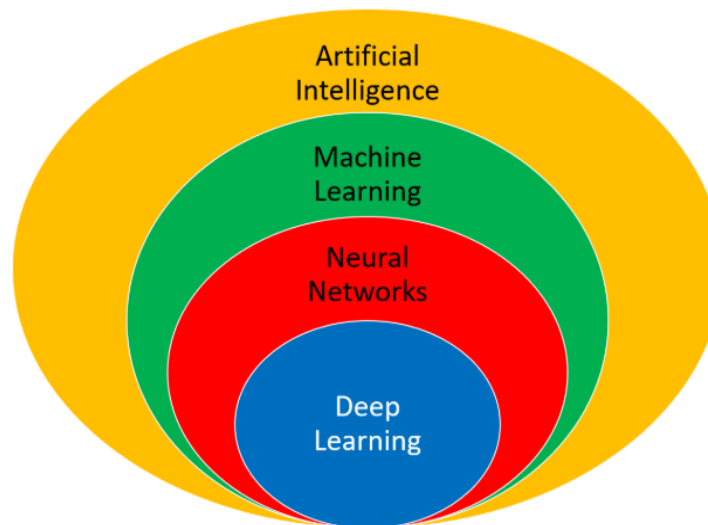


Figura 5: Jerarquía de IA, ML, NN y DL. [17]

#### 1.3.2.1 Aprendizaje Profundo

El aprendizaje profundo (en inglés, deep learning DL) es una rama del aprendizaje automático que se basa en el uso de algoritmos para modelar y comprender datos complejos. Estos algoritmos se inspiran en la estructura y función del cerebro humano, utilizando múltiples capas de procesamiento para extraer y transformar información de manera iterativa. El objetivo es crear modelos que puedan aprender representaciones abstractas de los datos, lo que les permite realizar tareas como el reconocimiento de patrones, la clasificación y la predicción con alta precisión [19].

#### 1.3.2.2 Redes neuronales

Las redes neuronales artificiales (ANN o NN) son modelos de ML inspirados en la estructura y el funcionamiento de las redes neuronales biológicas del cerebro. Estos modelos se componen de unidades interconectadas, llamadas neuronas artificiales, que simulan, de manera aproximada, el comportamiento de las neuronas reales. Cada neurona recibe señales numéricas de otras, las procesa mediante una función de activación no lineal y transmite el resultado a otras neuronas, siendo la fuerza de cada conexión determinada por un peso que se ajusta durante el proceso de aprendizaje.

Por lo general, las neuronas se organizan en capas. La capa de entrada recibe los datos, las capas ocultas transforman la información y la capa de salida genera el resultado final. Cuando una red cuenta con dos o más capas ocultas, se clasifica como una red neuronal profunda [8].

## 1.4 Conceptos claves de Deep Learning utilizados en el proyecto

En el desarrollo del presente trabajo se emplean diversas técnicas y terminología propias del campo del aprendizaje profundo (deep learning). A continuación, se describen brevemente algunos de los conceptos más relevantes que serán utilizados en fases posteriores del proyecto:

- **Regularización:** Conjunto de técnicas aplicadas durante el entrenamiento de modelos con el objetivo de evitar que estos se ajusten en exceso a los datos de entrenamiento (overfitting). Entre los métodos más comunes se encuentra la penalización de pesos (como L2) o el uso de capas dropout, que desactivan aleatoriamente ciertas neuronas durante el entrenamiento para fomentar una representación más generalizada.
- **Gradiente:** En el contexto del aprendizaje automático, el gradiente representa la dirección y magnitud del cambio necesario en los pesos del modelo para reducir el error (función de pérdida). Es una herramienta fundamental para el proceso de entrenamiento, ya que permite optimizar el modelo mediante técnicas como el descenso del gradiente (gradient descent). Cuanto más profundo es el modelo, más difícil puede ser propagar correctamente el gradiente, lo que a menudo requiere mecanismos adicionales como normalización o clasificadores auxiliares.
- **Data augmentation (aumento de datos):** Técnica que consiste en aplicar transformaciones aleatorias a las imágenes del conjunto de datos (rotaciones, recortes, escalados, volteos, etc.) para generar nuevas muestras sintéticas. Esto permite aumentar la diversidad del conjunto de entrenamiento sin necesidad de recopilar más datos, mejorando la capacidad del modelo para generalizar.
- **Overfitting y underfitting:** El overfitting ocurre cuando un modelo aprende demasiado bien los datos de entrenamiento, incluyendo el ruido, y no generaliza bien en datos nuevos. En cambio, el underfitting indica que el modelo no ha logrado aprender adecuadamente los patrones subyacentes, ni siquiera en el conjunto de entrenamiento.
- **Batch y epoch:** Una epoch equivale a un recorrido completo por todos los ejemplos del conjunto de entrenamiento. En cambio, un batch es un subconjunto de datos utilizado para actualizar los pesos del modelo. Entrenar en batches permite gestionar eficientemente grandes volúmenes de datos y estabilizar el aprendizaje.
- **Learning rate (tasa de aprendizaje):** Parámetro que determina la magnitud de los ajustes realizados en los pesos del modelo en cada iteración del entrenamiento. Si es demasiado alto, el modelo puede no converger; si es demasiado bajo, el proceso de aprendizaje será excesivamente lento.
- **Loss function (función de pérdida):** Es la función que mide la diferencia entre la predicción del modelo y el valor real. Su objetivo es cuantificar el error cometido, y su valor se utiliza como guía para actualizar los pesos del modelo durante el entrenamiento.
- **Optimizer (optimizador):** Algoritmo encargado de actualizar los pesos del modelo a partir del valor de la función de pérdida. Existen diversos optimizadores, como SGD, Adam o RMSprop, cada uno con características que influyen en la velocidad y estabilidad del aprendizaje.

Estos elementos son esenciales para comprender el funcionamiento interno del modelo propuesto y las decisiones adoptadas durante su diseño, entrenamiento y evaluación.

### 1.4.1 Redes neuronales convolucionales

Las redes neuronales convolucionales (conocidas como CNN, por sus siglas en inglés) son un tipo especial de red neuronal diseñada específicamente para procesar imágenes. Son ampliamente utilizadas en tareas como el reconocimiento facial, la detección de objetos, la clasificación de imágenes médicas, y muchas otras aplicaciones donde es importante que un sistema "vea" y entienda imágenes de forma automática.

A diferencia de las redes neuronales tradicionales, que tratan cada píxel de una imagen como una entrada independiente, las CNN tienen una estructura inspirada en cómo funciona la visión en los seres vivos. En lugar de analizar la imagen entera de una vez, van detectando primero pequeños patrones simples (como bordes o esquinas) y luego, a través de múltiples capas, combinan esa información para reconocer formas más complejas (como una prenda de ropa, una célula o una letra escrita a mano). Estas redes están compuestas principalmente por tres tipos de capas:

- **Capas de convolución:** aplican filtros (pequeñas ventanas que recorren la imagen) para detectar patrones locales. Por ejemplo, un filtro puede destacar los bordes horizontales, otro los verticales, etc.
- **Capas de activación (ReLU):** introducen no linealidad, permitiendo a la red aprender relaciones complejas.
- **Capas de pooling:** reducen el tamaño de los datos, manteniendo la información más importante, lo que ayuda a que el modelo sea más eficiente y menos propenso al sobreajuste.

Al final de estas capas, la red suele tener una o más capas densas (totalmente conectadas) que toman las características extraídas y generan la predicción final, como asignar una etiqueta a una imagen.

Una de las grandes ventajas de las CNN es que aprenden automáticamente qué características son importantes para distinguir entre clases, sin necesidad de que una persona defina manualmente qué buscar en la imagen. Sin embargo, este tipo de modelos requiere gran capacidad de cálculo para entrenarse, especialmente cuando se trabaja con conjuntos de datos grandes o redes muy profundas. Por ello, suelen entrenarse usando unidades de procesamiento gráfico (GPU), que aceleran notablemente los cálculos.

Gracias a su eficacia y flexibilidad, las CNN se han convertido en una herramienta fundamental en el campo de la inteligencia artificial aplicada a la visión por computadora [9].

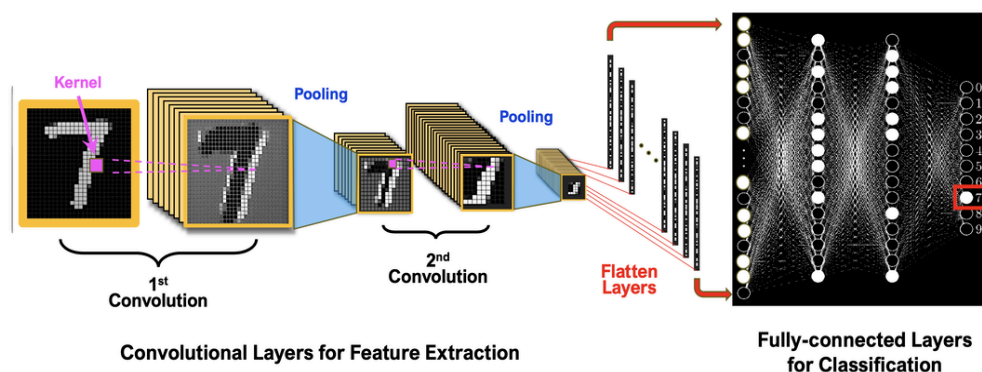


Figura 6: Funcionamiento de las CNN's. [18]

## 1.4.2 Aprendizaje por transferencia

El aprendizaje por transferencia (en inglés, transfer learning TL) se basa en el concepto de reutilización de elementos de los modelos de ML preentrenados en nuevos modelos que se utilizarán para fines similares. Esto permite optimizar los recursos y los datos etiquetados necesarios para el entrenamiento.

El TL se usa generalmente cuando el entrenamiento de un modelo de ML para resolver una nueva tarea requiere muchos recursos. El proceso reutiliza partes relevantes de un modelo de ML existente para solventar un problema nuevo similar [32].

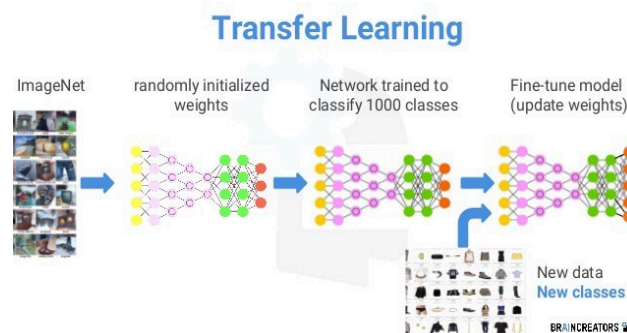


Figura 7: Transfer Learning. [45]

Dentro del TL, existen dos enfoques principales:

- **Feature Extraction:** Consiste en utilizar las capas convolucionales de un modelo preentrenado como un extractor fijo de representaciones. En este enfoque, se congelan los pesos del modelo base y solo se entrena una nueva capa de clasificación adaptada a la tarea específica. Es especialmente útil cuando se dispone de pocos datos o cuando la tarea es similar a la original, ya que permite aprovechar directamente las representaciones aprendidas sin modificar el resto del modelo.
- **Ajuste fino (Fine Tuning):** Implica continuar el entrenamiento de parte del modelo preentrenado para adaptarlo mejor a la nueva tarea. A diferencia de la extracción de características, se descongelan algunas de las capas superiores del modelo base y se actualizan sus pesos junto con los de la nueva capa de salida. Este enfoque permite al modelo refinar sus representaciones para la nueva tarea, siendo más eficaz cuando se cuenta con una cantidad moderada de datos y cuando la tarea difiere parcialmente de la original.

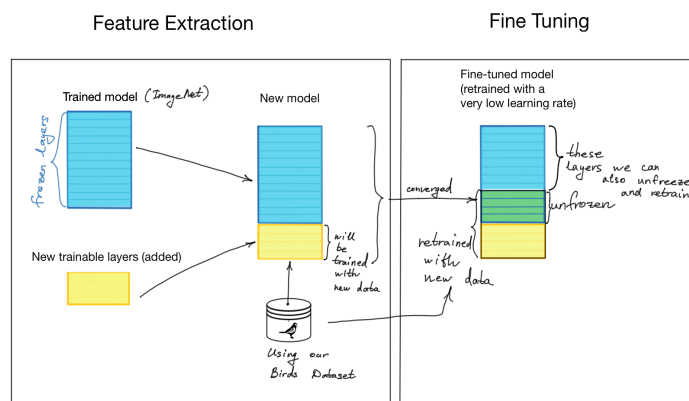


Figura 8: Feature Extraction and Fine Tuning. [57]

### 1.4.2.1 Inception V3

Inception V3 es una red neuronal convolucional avanzada creada para tareas de análisis y clasificación de imágenes, así como para la detección de objetos. Forma parte de la familia de redes Inception, desarrolladas inicialmente por Google como evolución del modelo GoogLeNet, que destacó en competiciones de reconocimiento de imágenes como ImageNet.

Esta tercera versión de Inception mejora la eficiencia y profundidad de la red sin aumentar demasiado la cantidad de parámetros, es decir, sin hacerla demasiado pesada o lenta. Por ejemplo, Inception V3 tiene menos de 25 millones de parámetros, mucho menos que otras redes antiguas como AlexNet, que tiene más de 60 millones [33].

Las innovaciones principales de Inception V3 son:

- **Factorización de convoluciones:** Para hacer el procesamiento más eficiente, la red divide algunas operaciones grandes en varias más pequeñas. Por ejemplo, en lugar de usar un filtro grande de  $7 \times 7$ , lo descompone en filtros más pequeños, como  $1 \times 7$  y  $7 \times 1$ , que requieren menos cálculo sin perder precisión.
- **Suavizado de etiquetas (label smoothing):** Esta es una técnica que ayuda a que el modelo no se confíe demasiado en las respuestas exactas durante el entrenamiento. En lugar de dar una respuesta “perfecta” para una clase, el modelo aprende a ser un poco más flexible, lo que mejora su capacidad para generalizar y evitar errores con datos nuevos.
- **Clasificadores auxiliares:** Son pequeñas redes dentro de la red principal que ayudan durante el entrenamiento a que la señal que indica cómo mejorar el modelo (llamada “gradiente”) llegue mejor a las capas más profundas. Esto facilita que la red aprenda de manera más estable y rápida.
- **Normalización por lotes (batch normalization):** Esta técnica se aplica en diferentes partes de la red para mantener los datos equilibrados mientras la red aprende, mejorando la velocidad y la estabilidad del entrenamiento. También se usa en los clasificadores auxiliares.

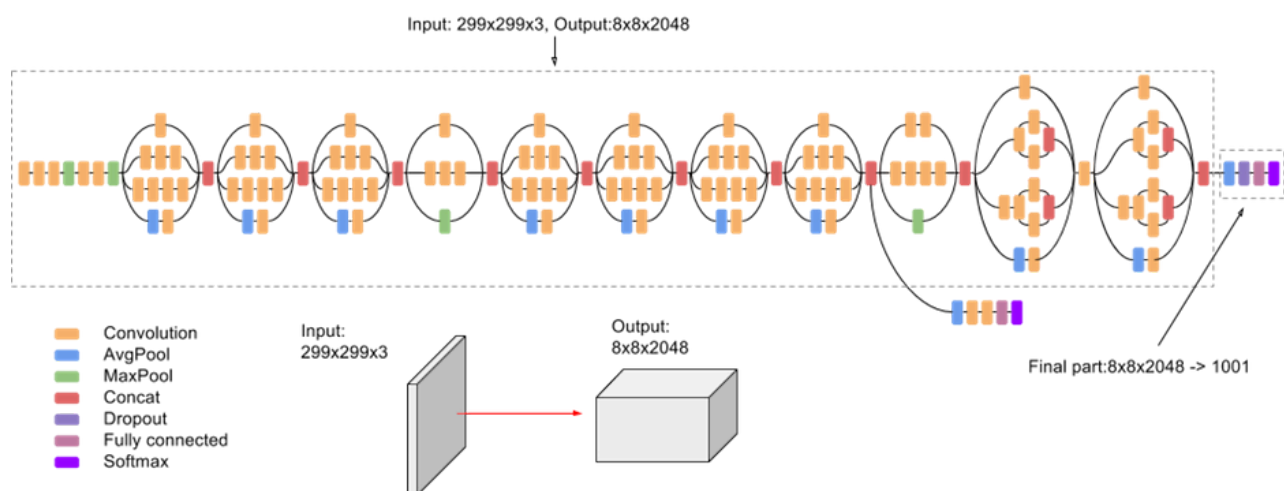


Figura 9: Arquitectura de Inception V3. [44]



### 1.4.2.2 MobileNet V2

MobileNet V2 es una arquitectura de red neuronal convolucional (CNN) diseñada para ser especialmente eficiente y ligera. Está pensada para funcionar correctamente incluso en dispositivos con recursos limitados, como teléfonos móviles, sistemas embebidos o aplicaciones de inteligencia artificial que se ejecutan en el borde (edge computing). Gracias a su bajo consumo de energía, su rapidez y su tamaño reducido, MobileNet V2 es ideal para tareas como la clasificación de imágenes o la detección de objetos en tiempo real. Además, es compatible con plataformas como TensorFlow Lite, pensadas para su uso en dispositivos móviles [58].

Publicada en 2019, esta versión introdujo dos mejoras importantes respecto a su predecesora:

- **Capas residuales invertidas:** En redes profundas, a veces es difícil que la información fluya correctamente entre las distintas capas. Para solucionarlo, se utilizan conexiones que permiten a la red "recordar" información anterior. MobileNet V2 reorganiza estas conexiones de una forma especial: en lugar de reducir primero la cantidad de información y luego expandirla, hace lo contrario. Primero amplía la información para trabajar con más detalles, luego aplica filtros que identifican patrones y, finalmente, reduce la información otra vez para simplificar el resultado. Esta estrategia permite mantener la calidad de la información mientras se reduce el coste computacional.
- **Cuellos de botella lineales:** En la mayoría de redes, al final de cada bloque se aplica una función llamada "ReLU", que introduce cierta simplificación al resultado para hacerlo más útil. Sin embargo, cuando se trabaja con poca información (como en las capas finales de MobileNet V2), aplicar esta simplificación puede hacer que se pierdan detalles importantes. Por eso, MobileNet V2 decide no aplicar esa función al final de estos bloques, manteniendo la información sin distorsiones innecesarias. Esto ayuda a que la red sea más precisa sin complicarla más.

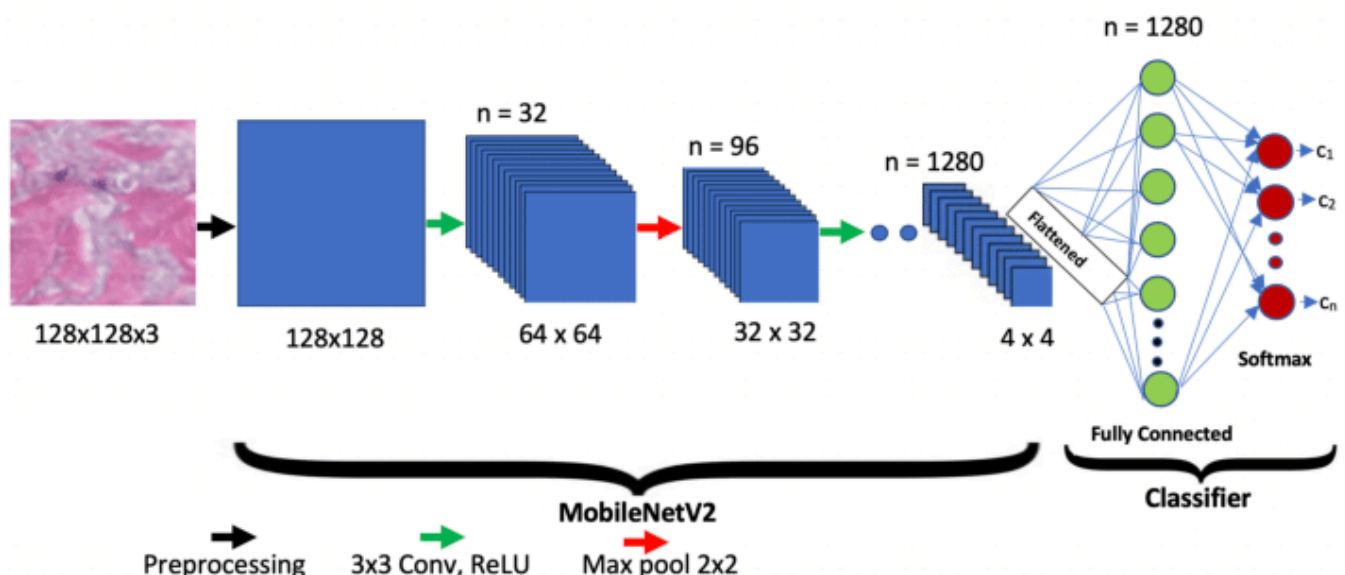


Figura 10: Arquitectura de MobileNet V2. [58]

### 1.4.2.3 DenseNet 201

DenseNet-201 es una red neuronal convolucional profunda que pertenece a la familia DenseNet (de Densely Connected Convolutional Networks). Fue diseñada para mejorar la eficiencia y el rendimiento en tareas de clasificación de imágenes, manteniendo una arquitectura más ligera y aprovechando mejor la información aprendida durante el entrenamiento.

Lo que hace especial a DenseNet-201 es su estructura de conexiones densas. En lugar de que cada capa solo reciba información de la capa anterior (como sucede en la mayoría de redes tradicionales), en DenseNet cada capa recibe también las salidas de todas las capas anteriores dentro de un mismo bloque. Esto crea una especie de “red de caminos” donde la información fluye de manera más directa y completa a lo largo del modelo [32].

Sus características principales son:

- **Conexiones densas entre capas:** Este diseño permite reutilizar la información aprendida en etapas previas, lo cual mejora el rendimiento, acelera el entrenamiento y evita que se pierda información útil a medida que la imagen atraviesa la red.
- **Mejor propagación del gradiente:** Al tener múltiples caminos para que la señal fluya hacia atrás durante el aprendizaje, DenseNet ayuda a resolver problemas comunes en redes profundas, como el desvanecimiento del gradiente, que puede dificultar que el modelo aprenda correctamente.
- **Eficiencia en el uso de parámetros:** Aunque tiene una gran profundidad (201 capas), DenseNet-201 no necesita tantos parámetros como otras redes similares. Esto se debe a que, gracias a su diseño, no necesita “aprender desde cero” en cada capa, sino que reutiliza muchos de los patrones ya aprendidos.
- **Entrenamiento previo en ImageNet:** DenseNet-201 puede utilizar pesos ya entrenados en el conjunto de datos ImageNet, lo que le proporciona una sólida base visual y permite reutilizar el modelo para tareas nuevas mediante técnicas como el fine-tuning.

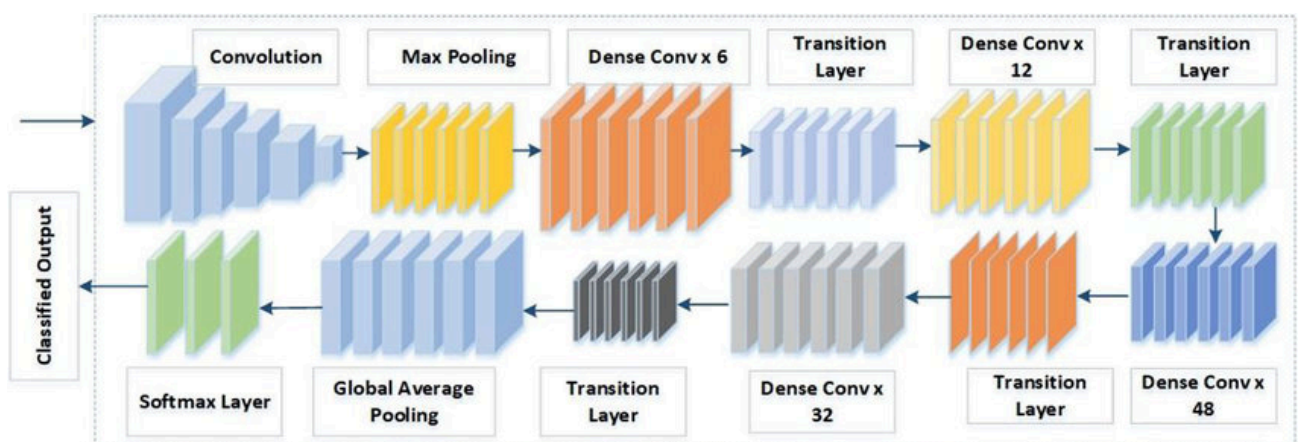


Figura 11: Arquitectura de DenseNet 201. [42]



#### 1.4.2.4 EfficientNet V2B0

EfficientNetV2B0 es un modelo de red neuronal convolucional diseñado para resolver tareas de visión por computadora, como clasificar imágenes, de forma precisa y rápida. Lo más destacado de esta arquitectura es que logra muy buenos resultados utilizando menos recursos computacionales, lo que la hace ideal para usar en dispositivos con limitaciones de memoria o potencia, como móviles o pequeños ordenadores.

Forma parte de la segunda generación de modelos EfficientNet, una familia de redes conocida por su eficiencia. Esta nueva versión ha sido optimizada para que el proceso de entrenamiento (es decir, el aprendizaje del modelo) sea más rápido y al mismo tiempo mantenga una alta precisión.

Entre sus principales características, se destacan:

- **Diseño eficiente de bloques internos:** Utiliza una combinación de bloques llamados MBConv (ya presentes en MobileNetV2) y otros más simples conocidos como Fused-MBConv. Aunque los nombres son técnicos, la idea principal es que estos bloques permiten procesar las imágenes de forma más rápida y con menos cálculos, aprovechando mejor los recursos del sistema.
- **Entrenamiento más rápido:** Gracias a mejoras en la estructura del modelo, EfficientNetV2B0 puede aprender más rápido sin perder calidad. Esto es especialmente útil cuando se trabaja con grandes cantidades de datos o se tiene poco tiempo para entrenar el modelo.
- **Uso de técnicas avanzadas para evitar errores y mejorar el rendimiento:** Durante el entrenamiento, se aplican métodos como la regularización (que ayuda a evitar que el modelo memorice demasiado y se vuelva poco flexible) y la mejora de datos (data augmentation), que genera nuevas imágenes artificiales a partir de las originales para que el modelo aprenda mejor.

Además, esta red utiliza una estrategia llamada escalado compuesto, que consiste en ajustar de forma equilibrada tres factores clave: la profundidad del modelo (cuántas capas tiene), el ancho (cuántas unidades por capa) y el tamaño de entrada de las imágenes.

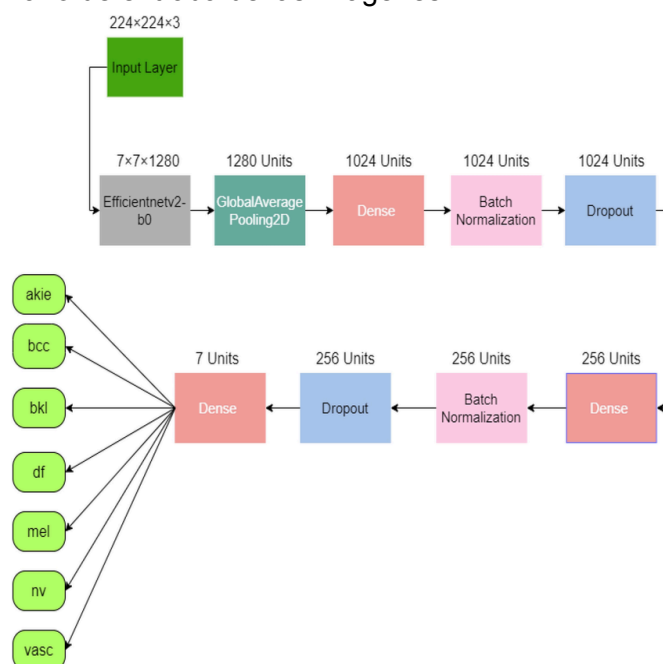


Figura 12: Arquitectura de EfficientNet V2B0. [41]

### 1.4.2.5 ResNet 50V2

ResNet-50V2 es una red neuronal convolucional de 50 capas que pertenece a la familia ResNet (abreviatura de Residual Network). Estas redes están diseñadas para poder ser muy profundas sin que su rendimiento se vea afectado, gracias a un mecanismo llamado conexión residual o skip connection, que ayuda a que la información fluya más fácilmente a través de las capas.

En redes muy profundas, uno de los problemas habituales es que, al avanzar por muchas capas, el modelo puede “olvidar” o distorsionar la información inicial, lo que dificulta el aprendizaje. Para evitar esto, ResNet introduce caminos alternativos que permiten que los datos “salten” algunas capas, facilitando así el entrenamiento. En la versión mejorada (V2), se reorganiza la estructura interna de los bloques para hacerlo aún más estable y eficaz.

Las principales características de ResNet-50V2 son:

- **Conexiones residuales mejoradas:** Estas conexiones permiten que parte de la información original pase directamente de una capa a otra sin ser modificada, lo que ayuda a mantenerla intacta y mejora el proceso de aprendizaje, sobre todo en redes profundas.
- **Estructura de bloques reorganizada (pre-activación):** En esta versión, el orden de operaciones dentro de cada bloque ha sido modificado. Primero se normalizan los datos (para que tengan una escala adecuada), luego se aplica la función de activación (ReLU), y por último se realiza la operación de convolución. Este cambio mejora la estabilidad durante el entrenamiento y facilita que la red aprenda mejor.
- **Uso de pesos preentrenados:** ResNet-50V2 se puede cargar ya entrenada con millones de imágenes del conjunto de datos ImageNet. Esto significa que ya ha aprendido a reconocer una gran variedad de patrones visuales generales, lo que permite reutilizar ese conocimiento para nuevas tareas con menos esfuerzo y datos.

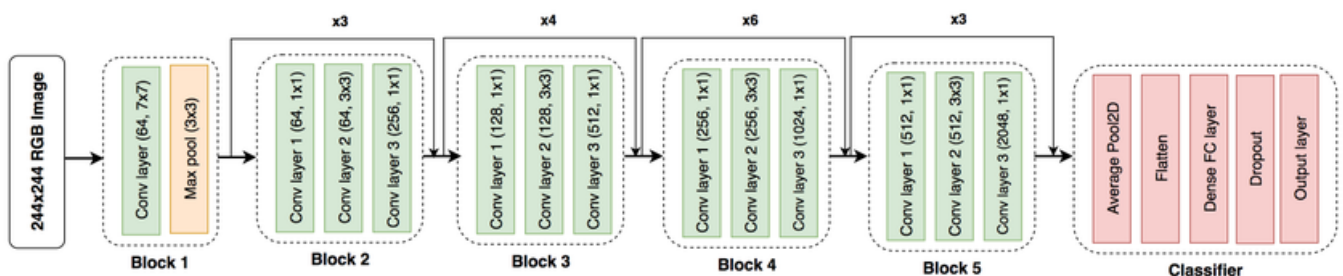


Figura 13: Arquitectura de ResNet 50V2. [43]

#### 1.4.2.6 VGG16

VGG16 es una red neuronal convolucional profunda desarrollada por el grupo de investigación Visual Geometry Group (VGG) de la Universidad de Oxford. Su nombre hace referencia a su arquitectura, compuesta por 16 capas que contienen parámetros entrenables (pesos), y fue una de las redes más influyentes en el desarrollo de modelos de visión por computadora modernos.

Una de las principales ventajas de VGG16 es su estructura simple y uniforme. En lugar de utilizar filtros grandes y complejos, la red se construye apilando muchas capas pequeñas de convoluciones de 3×3 píxeles, lo que permite detectar patrones visuales con mayor detalle y precisión. Estas capas se intercalan con operaciones llamadas pooling, que reducen progresivamente el tamaño de la imagen y ayudan a concentrar la información más importante.

Sus principales características son:

- **Arquitectura profunda y regular:** VGG16 está formada por una secuencia repetitiva de capas sencillas, lo que la hace fácil de entender y modificar. Esta estructura permite que la red aprenda representaciones visuales cada vez más complejas a medida que avanza por sus capas.
- **Uso de ReLU como función de activación:** Cada capa aplica una función matemática llamada ReLU (Rectified Linear Unit), que ayuda a que el modelo aprenda de forma más rápida y eficaz durante el entrenamiento.
- **Pesos preentrenados en ImageNet:** VGG16 puede utilizar conocimientos previamente aprendidos en una enorme base de datos de imágenes llamada ImageNet. Esto significa que ya ha aprendido a detectar una amplia variedad de patrones visuales, por lo que se puede reutilizar fácilmente para nuevas tareas, ajustando solo una parte del modelo (proceso conocido como transfer learning).

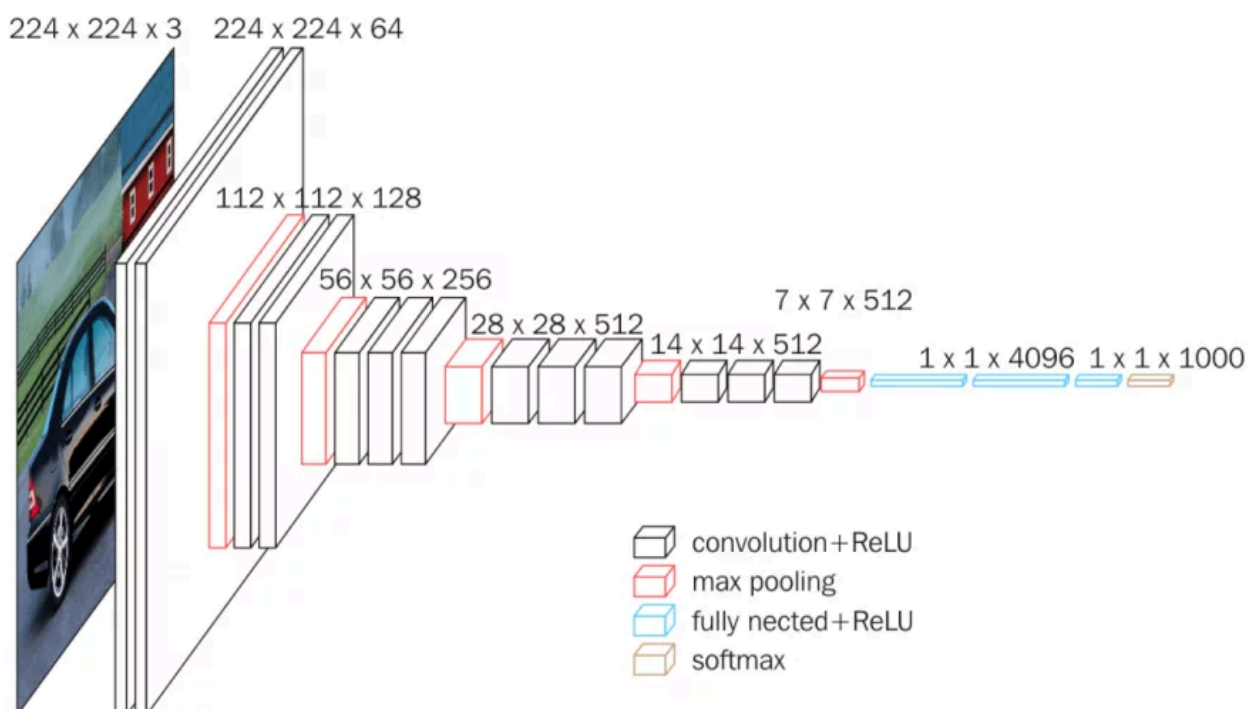


Figura 14: Arquitectura de VGG16. [46]

## 1.5 Identificación del problema

El diagnóstico preciso de los linfomas malignos es crucial para definir el tratamiento adecuado y mejorar el pronóstico del paciente. No obstante, el proceso diagnóstico actual se enfrenta a desafíos importantes:

- **Variabilidad en la tinción:** Las biopsias teñidas con Hematoxilina/Eosina (H&E) pueden presentar diferencias significativas en función de las técnicas y condiciones de laboratorio, lo que afecta la uniformidad de las muestras.
- **Subjetividad en la interpretación:** La evaluación de las imágenes histológicas depende en gran medida de la experiencia del patólogo, lo que puede generar discrepancias en el diagnóstico, especialmente en casos complejos o en etapas tempranas de la enfermedad.
- **Heterogeneidad de las muestras:** Las diferencias naturales en la forma de las células y en la forma en que se presentan los linfomas complican la clasificación exacta de sus subtipos (CLL, FL y MCL).

Estos factores pueden retrasar el diagnóstico y, en consecuencia, la aplicación de tratamientos adecuados, lo que repercute negativamente en la evolución y calidad de vida de los pacientes. Por ello, se identifica la necesidad de desarrollar herramientas automatizadas que integren técnicas de ML, capaces de analizar las imágenes H&E de forma consistente y precisa, minimizando la dependencia de la interpretación subjetiva y mejorando la eficiencia en la detección y clasificación de estos linfomas.

## 1.6 Partes interesadas

Las partes interesadas en este proyecto son diversas y abarcan tanto el ámbito clínico como el tecnológico y académico. Entre ellas se destacan:

- **Profesionales de la salud:** Patólogos, oncólogos, hematólogos y demás especialistas que se beneficiarán de un diagnóstico más preciso, rápido y consistente al contar con herramientas automatizadas que complementen su labor.
- **Pacientes y sus familias:** Un diagnóstico temprano y exacto puede conducir a tratamientos más adecuados, lo que repercute en una mejora del pronóstico y de la calidad de vida.
- **Instituciones sanitarias y centros de investigación:** La incorporación de tecnologías innovadoras en el proceso diagnóstico puede optimizar los recursos y mejorar la eficiencia en el manejo de casos clínicos.
- **Industria tecnológica y de software médico:** El desarrollo y la aplicación de modelos de ML en el ámbito de la salud abren nuevas oportunidades para el diseño de soluciones avanzadas que pueden integrarse en productos comerciales o servicios tecnológicos.

## 2 Justificación

El presente proyecto se justifica por la necesidad de mejorar la precisión diagnóstica en el análisis de linfomas malignos. Actualmente, la variabilidad en las técnicas de tinción y la interpretación subjetiva de las imágenes histológicas pueden ocasionar retrasos en el inicio de tratamientos adecuados, comprometiendo el pronóstico del paciente. La aplicación de modelos de aprendizaje automático (Machine Learning) se plantea como una alternativa prometedora para automatizar y estandarizar el diagnóstico, logrando un proceso más rápido, preciso y confiable.

Además de su relevancia clínica y tecnológica, este trabajo surge de un interés personal por el área de la oncología y, en particular, por el diagnóstico de linfomas. Esta motivación ha impulsado la elección del tema y ha guiado el desarrollo del proyecto con especial compromiso e implicación. El objetivo principal es explorar el potencial de las redes neuronales convolucionales para asistir en la clasificación de imágenes histológicas, contribuyendo, en la medida de lo posible, a la mejora de los procesos diagnósticos en este campo.

### 2.1 Estudio de soluciones existentes

En el campo del diagnóstico asistido por computadora, se han desarrollado diversas soluciones basadas en técnicas de deep learning para el análisis de imágenes histológicas, con el objetivo de mejorar la clasificación y detección de subtipos de linfoma.

Por ejemplo, el estudio publicado en Wiley [10] presenta un método que combina transferencia de aprendizaje (TL) y análisis de componentes principales (PCA) para clasificar subtipos de NHL. En este enfoque, las imágenes digitales se preprocesan mediante segmentación y división, se extraen características utilizando modelos preentrenados, y posteriormente se mapean con PCA para ser clasificadas mediante una red neuronal. Los resultados muestran precisiones muy elevadas (alrededor del 98.93% de exactitud global), lo que evidencia el potencial de esta técnica para automatizar y mejorar el diagnóstico en entornos clínicos.

De forma complementaria, otro estudio disponible en IEEE Access [11] se centra en el desarrollo de un método basado en una red neuronal convolucional residual integrada en un autoencoder (FusionNet). Este enfoque se aplica a la clasificación de subtipos de linfoma. Los autores destacan que entrenar la red de manera end-to-end, evitando depender de características patológicas predefinidas, permite extraer información de alta resolución y mejorar la precisión en la clasificación.

### 2.2 Otras alternativas

Inicialmente, se planteó realizar un TFG cuyo objetivo principal era entrenar un modelo de machine learning capaz de diferenciar, a partir de imágenes celulares, entre células benignas y malignas, para posteriormente clasificar las células malignas en HL y NHL. Para ello, se preveía emplear técnicas de procesamiento de imágenes y algoritmos de clasificación supervisada. Sin embargo, durante la fase de exploración se observó que no existían conjuntos de datos públicos para implementar este enfoque de manera sólida.

En una búsqueda en Kaggle relacionada con linfoma, el conjunto de datos que resultó más interesante fue aquel destinado a la clasificación de tres subtipos de NHL. Esta limitación en el acceso a datos determinó la necesidad de ajustar el enfoque del proyecto, orientándose hacia la clasificación de estos tres subtipos, lo que también permite aportar una contribución innovadora en el análisis de imágenes histológicas.

## 3 Alcance

En este apartado se define el alcance del proyecto, delimitando sus objetivos generales y específicos, así como los requerimientos y posibles obstáculos y riesgos.

### 3.1 Objetivos

Tal y como se ha mencionado en el apartado 1.1, el objetivo principal del proyecto es explorar y desarrollar un sistema basado en modelos de ML que permita clasificar tres subtipos de NHL (CLL, FL y MCL) a partir de imágenes de biopsias teñidas con Hematoxilina/Eosina (H&E). Para alcanzar este objetivo se seguirán los siguientes pasos:

- Entrenar un modelo de red neuronal convolucional (CNN) utilizando un conjunto de datos de imágenes, de manera que, a partir de una nueva imagen, se pueda predecir el subtipo de linfoma.
- Evaluar los resultados y la eficiencia del modelo mediante métricas de rendimiento específicas.
- Analizar las características extraídas de las imágenes para identificar aquellas que tengan mayor impacto en la clasificación.

### 3.2 KPIs y Métricas para Evaluar el Éxito del Proyecto

Para medir el éxito del proyecto se establecerán indicadores clave de rendimiento (KPIs) y métricas tanto a nivel técnico como en la gestión del proyecto.

#### 3.2.1 Nivel técnico

A nivel técnico se considerarán:

- **Exactitud (Accuracy):** Se espera que el modelo alcance una exactitud global superior al 90% en la clasificación de las imágenes, garantizando la fiabilidad de las predicciones para los subtipos de NHL.
- **Recall para MCL:** Dado que el linfoma de células del manto (MCL) es la clase prioritaria, se establecerá como objetivo que el recall para MCL supere el 95%, minimizando los falsos negativos en esta categoría.
- **Precisión y F1-Score:** Estas métricas se evaluarán para cada subtipo, buscando un equilibrio óptimo entre la detección correcta y la minimización de falsos positivos.
- **Tiempo de procesamiento:** Se medirá el tiempo medio que tarda el modelo en clasificar una imagen, con el fin de mantenerlo en un rango aceptable para su posible aplicación en entornos clínicos.

### 3.2.2 Nivel de gestión del proyecto

A nivel de gestión del proyecto se utilizarán KPIs adicionales, tales como:

- **Cumplimiento de plazos:** Se controlará el porcentaje de tareas completadas según el cronograma establecido en el diagrama de Gantt.
- **Calidad de la documentación:** Medida a través de revisiones periódicas y feedback del tutor y director del TFG, para asegurar que toda la información relevante se documente correctamente.

## 3.3 Requerimientos

Para garantizar el éxito del proyecto, es fundamental establecer una serie de requerimientos que aseguren tanto el rendimiento óptimo del modelo como su viabilidad práctica en entornos reales:

- **Rendimiento de clasificación:** Los resultados del modelo deberán demostrar una alta exactitud (accuracy) global y precisión en la clasificación de los subtipos de linfoma. En particular, se priorizará la maximización del recall para el MCL, dada la naturaleza agresiva de esta clase en comparación con CLL y FL. Se reconoce que alcanzar altos valores en todas las métricas simultáneamente es complejo, por lo que se hará especial énfasis en mejorar el recall en MCL.
- **Eficiencia:** El modelo debe ser eficiente tanto en términos de tiempo de ejecución como en el uso de recursos computacionales. Esto implica que el sistema sea capaz de procesar y clasificar las imágenes de forma rápida, lo cual es esencial para su integración en entornos clínicos sin requerir hardware excesivamente costoso.
- **Generalización y robustez:** Es crucial que el modelo obtenga buenos resultados sin sobreajustarse a los datos de entrenamiento. De esta manera, al aplicarse a nuevas instancias o conjuntos de datos, el sistema mantendrá un rendimiento consistente y fiable, demostrando una capacidad de generalización robusta.

## 3.4 Obstáculos y riesgos

Para el desarrollo exitoso del proyecto se han identificado diversos obstáculos y riesgos que deberán gestionarse:

- **Fecha límite de entrega:** El proyecto se encuentra condicionado por un plazo de entrega fijo, lo que limita el tiempo disponible para su desarrollo. Es fundamental definir de manera precisa la distribución y planificación de las tareas para evitar retrasos.
- **Falta de experiencia en ML aplicado a imágenes:** Aunque se cuenta con experiencia en el manejo de datos numéricos, este proyecto implica ML aplicado al análisis de imágenes. La curva de aprendizaje en esta área podría ralentizar el progreso y requiere dedicar tiempo adicional para adquirir los conocimientos necesarios.
- **Disponibilidad de tiempo reducida por prácticas:** El autor del proyecto se encuentra realizando prácticas profesionales, lo que reduce el tiempo disponible para dedicar al desarrollo de la tesis. Esta situación demanda una gestión eficiente del tiempo para asegurar el cumplimiento de las distintas etapas del proyecto.



## 4 Metodología y rigor

El desarrollo del proyecto se ha estructurado siguiendo una metodología que integra los enfoques teóricos adquiridos durante la carrera y que se adapta a la naturaleza dinámica de la investigación en ML. Se ha optado por la metodología Agile, ya que su enfoque iterativo y flexible permite responder de manera rápida a los cambios en los requerimientos o descubrimientos durante el proceso de desarrollo. La metodología adoptada se compone de las siguientes fases:

- **Contextualización:** Se inicia con una revisión de la literatura y el análisis de soluciones existentes, lo que permite contextualizar el proyecto y fundamentar la elección de técnicas y modelos de ML.
- **Diseño y planificación del proyecto:** A partir de los conocimientos adquiridos en la formación académica, se diseña una planificación detallada que divide el proyecto en etapas: recopilación y preprocesamiento de datos, desarrollo y entrenamiento de modelos, validación y análisis de resultados.
- **Implementación y experimentación:** Durante esta fase se desarrollan y entrenan los modelos de redes neuronales, en especial las redes convolucionales (CNN), para la clasificación de los subtipos de linfoma. Se aplican técnicas de validación cruzada y se utilizan métricas específicas (accuracy, precisión, recall, etc.) para evaluar el rendimiento de los modelos.
- **Seguimiento:** Para asegurar una gestión eficaz del proyecto, se empleará la herramienta Trello, que permitirá organizar y monitorizar las tareas, asignar responsabilidades y establecer hitos temporales. Esta herramienta facilitará la visualización del progreso y la identificación temprana de posibles desviaciones en el plan, garantizando así el cumplimiento de los plazos establecidos.
- **Análisis de resultados y documentación:** Finalmente, se realiza un análisis exhaustivo de los resultados obtenidos, evaluando tanto la eficacia del modelo como su capacidad de generalización. Esta fase incluye la elaboración de informes técnicos y la documentación completa del proceso.

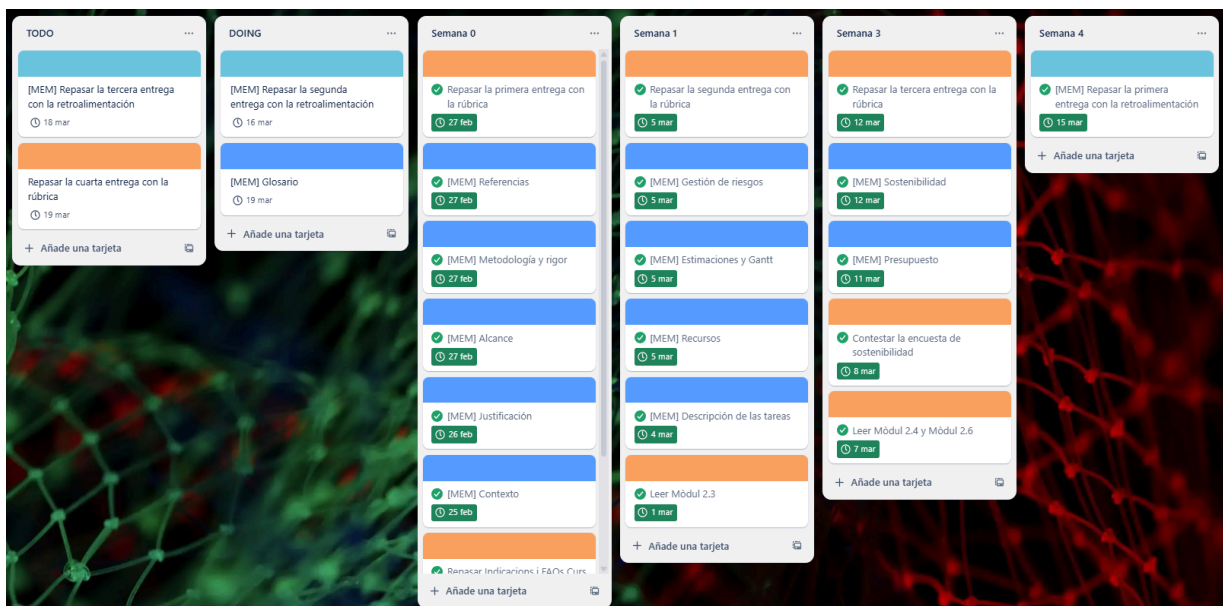


Figura 15: Organización en Trello (Elaboración propia).



## 5 Planificación temporal

En este apartado se detalla la planificación temporal del proyecto, incluyendo la descripción de las tareas a realizar durante su desarrollo, los recursos humanos y materiales necesarios, las estimaciones de tiempo representadas en un diagrama de Gantt, y la gestión de riesgos mediante la definición de planes alternativos y el análisis de obstáculos.

El proyecto empieza el 19 de febrero de 2025 y se tiene prevista la defensa del TFG para el 26 de junio de 2025. La fecha de finalización del proyecto se ha fijado para el 16 de junio de 2025, reservándose un margen de 10 días para afrontar posibles imprevistos. Esto implica disponer de un total de 117 días para la ejecución del proyecto, en los cuales se invertirán aproximadamente 470 horas. Se espera dedicar en promedio 4 horas diarias durante este periodo. Todas las actividades se realizarán utilizando el ordenador del autor como única herramienta física para el desarrollo del proyecto.

### 5.1 Descripción de las tareas

En este apartado se describen en detalle todas las tareas que se llevarán a cabo, organizadas en bloques. Se estima la duración de cada actividad y se identifican las dependencias entre ellas, en caso de que existan.

#### 5.1.1 Gestión y seguimiento del proyecto

La gestión y seguimiento del proyecto son fundamentales para asegurar su correcto desarrollo. Una planificación detallada resulta crucial para alcanzar los objetivos previstos. Se estima que estas actividades representarán aproximadamente 210 horas del tiempo total asignado.

##### 5.1.1.1 Contextualización y alcance

El primer paso para garantizar el éxito del proyecto consiste en definir de forma precisa los objetivos que se desean alcanzar. Además, se ofrece una descripción del contexto en el que se sitúa el tema abordado. Esta tarea requiere aproximadamente 35 horas.

##### 5.1.1.2 Planificación temporal

Una planificación temporal detallada es esencial para el correcto desarrollo del proyecto. En esta fase se definen los plazos, se asigna el tiempo necesario a cada tarea y se establecen las dependencias entre ellas, garantizando así una ejecución coordinada y eficiente. Esta tarea requiere aproximadamente 25 horas.

##### 5.1.1.3 Presupuesto

Se elabora un presupuesto en el que se cuantifican todos los costes asociados al proyecto, tanto en términos económicos como temporales. Esta tarea requiere aproximadamente 20 horas.

##### 5.1.1.4 Informe de sostenibilidad

Se elabora un breve informe de sostenibilidad que evalúa el impacto ambiental, económico y social del proyecto. Esta tarea requiere aproximadamente 15 horas.

#### **5.1.1.5 Integración del documento final**

Se integra de manera coherente los tres documentos (Contextualización y alcance, Planificación temporal, y Presupuesto y sostenibilidad) incorporando el feedback del tutor de GEP. Esta tarea requiere aproximadamente 5 horas.

#### **5.1.1.6 Documentación**

La documentación del proyecto se llevará a cabo de forma continua y paralela al desarrollo, de modo que cada idea, decisión y resultado quede registrado a medida que se avanza. Esta estrategia evita dejar aspectos relevantes sin documentar. Esta tarea requiere aproximadamente 70 horas.

#### **5.1.1.7 Presentación**

Finalmente, se desarrollará la presentación del proyecto destinada a la defensa ante el tribunal. Esta fase incluye la elaboración de diapositivas, la estructuración de la exposición y la realización de ensayos para perfeccionar la defensa. Esta tarea requiere aproximadamente 40 horas.

#### **5.1.1.8 Reuniones**

Se mantendrán reuniones periódicas con el director del proyecto para garantizar un seguimiento adecuado, identificar áreas de mejora y optimizar el desarrollo del mismo. Estas reuniones se programarán con una frecuencia de una vez a la semana o cada dos semanas. Por lo tanto, el tiempo destinado a estas sesiones no se ha incluido en la estimación total de horas.

### **5.1.2 Trabajo previo**

Para el desarrollo de la red neuronal convolucional (CNN), es fundamental llevar a cabo un trabajo previo que permita adquirir los conocimientos y habilidades necesarios. Se estima que estas actividades representarán aproximadamente 110 horas del tiempo total asignado. A continuación, se detallan dos áreas principales de estudio:

#### **5.1.2.1 Familiarización con Keras y TensorFlow**

Dado que el autor cuenta con experiencia limitada en el uso de Keras y TensorFlow, se dedicará un tiempo considerable a familiarizarse con estas herramientas esenciales para el desarrollo de modelos de deep learning. Esta fase incluirá el estudio de documentación, la realización de tutoriales y la práctica con ejemplos básicos, con el objetivo de adquirir una comprensión sólida de su funcionamiento. Esta tarea requiere aproximadamente 40 horas.

#### **5.1.2.2 Estudio de las CNN's**

Se investigará en profundidad el funcionamiento teórico y práctico de las CNN. Para ello, se consultarán diversas fuentes, incluyendo tres libros recomendados por el director del proyecto, así como material adicional obtenido de investigaciones y recursos en línea. El objetivo es comprender los fundamentos y técnicas que rigen las CNN, lo cual es crucial para su implementación y optimización en el proyecto. Esta tarea requiere aproximadamente 70 horas.

### 5.1.3 Desarrollo de la CNN

Una vez adquiridos los conocimientos en Keras, TensorFlow y redes neuronales convolucionales (CNN), se procederá al desarrollo de la CNN destinada a clasificar los tres subtipos de NHL. Se estima que esta fase requerirá aproximadamente 150 horas. En este apartado se describen las subtareas esenciales para garantizar el desarrollo exitoso de la red, organizadas de la siguiente manera:

#### 5.1.3.1 Diseñar la arquitectura de la CNN

El primer paso para desarrollar la CNN consiste en diseñar su arquitectura, teniendo en cuenta las características de los datos de entrada, el tamaño de las imágenes y la complejidad del problema a resolver. Esta tarea requiere aproximadamente 40 horas.

#### 5.1.3.2 Entrenar la CNN

Una vez definida la arquitectura de la red, se procederá a su entrenamiento. En esta fase se ajustarán los parámetros del modelo utilizando el conjunto de entrenamiento. Además, se realizarán iteraciones de validación para asegurar que la red sea robusta y apta para su aplicación práctica. Esta tarea requiere aproximadamente 15 horas.

#### 5.1.3.3 Evaluar el modelo

Una vez finalizado el entrenamiento, se evaluará el rendimiento de la red neuronal. En esta fase se aplicarán métricas de evaluación (como exactitud o accuracy, precisión, recall y F1-score) sobre el conjunto de test para determinar la capacidad del modelo de generalizar a nuevos datos. Esta tarea requiere aproximadamente 15 horas.

#### 5.1.3.4 Optimizar el modelo

Tras evaluar el modelo, se procederá a identificar e implementar mejoras que aumenten tanto su calidad como su eficiencia. Esta fase consistirá en ajustar la arquitectura, los parámetros y las técnicas de entrenamiento, de modo que cada modificación permita alcanzar un mejor rendimiento. Cada vez que se realice un cambio, se volverá a entrenar y evaluar el modelo, repitiendo este ciclo iterativo hasta obtener la versión óptima. Esta tarea requiere aproximadamente 50 horas.

#### 5.1.3.5 Analizar los patrones

Por último, con el modelo final implementado, se incorporarán técnicas de visualización que permitan identificar y destacar los elementos clave en las imágenes que influyen en la clasificación realizada por la red neuronal. De esta forma, se podrá observar de manera clara qué características o regiones de la imagen son determinantes para asignarla a una categoría específica. Esta tarea requiere aproximadamente 30 horas.

## 5.2 Recursos

Todo proyecto requiere disponer de los recursos necesarios para su adecuada organización y ejecución. En este sentido, se han identificado y clasificado cuatro grupos esenciales de recursos: recursos humanos, hardware, software y materiales.

### 5.2.1 Recursos humanos

El éxito del proyecto depende en gran medida de la participación de diversos actores. En este sentido, se identifican tres recursos humanos esenciales:

- **Autor del proyecto:** Será el responsable de desarrollar el proyecto, lo que incluye la planificación, investigación, experimentación y el análisis de los resultados obtenidos.
- **Director del proyecto:** Actuará como mentor y supervisor, proporcionando orientación y asesoramiento para asegurar el correcto desarrollo del proyecto.
- **Tutor de GEP:** Brindará apoyo durante el primer mes del proyecto, colaborando en la gestión y organización inicial para establecer una base sólida en la ejecución de las tareas.

### 5.2.2 Recursos de hardware

El desarrollo del proyecto se llevará a cabo utilizando como única herramienta física un portátil Victus HP 16. Este equipo cuenta con 16GB de RAM y un procesador AMD Ryzen 7 5800H con gráficos Radeon, operando a una velocidad de 3.20 GHz.

### 5.2.3 Recursos de software

El desarrollo del proyecto se apoyará en diversas herramientas de software que facilitarán tanto la planificación como la implementación y seguimiento del mismo. Entre estas herramientas se encuentran:

- **Trello:** Para organizar y dar seguimiento a las tareas y actividades del proyecto.
- **Ganttter:** Para la gestión de la planificación temporal, mediante diagrama de Gantt.
- **Calculadora:** Para realizar algunos cálculos simples.
- **Google Colab:** Para experimentar con ejemplos sencillos utilizando Keras y TensorFlow.
- **Jupyter Notebook:** Para escribir y ejecutar el código relacionado con el desarrollo del proyecto.
- **GitHub:** Para compartir y versionar el código, facilitando la colaboración y manteniendo informado al director del proyecto sobre los avances.
- **Autocorrector:** Para corregir faltas ortográficas y garantizar la calidad lingüística de la memoria.

### 5.2.4 Recursos materiales

Por último, se consultarán estudios previos y diversas fuentes de información, incluyendo documentos académicos, artículos científicos y libros especializados en aprendizaje automático.

## 5.3 Tabla de resumen

En la tabla a continuación se resumen las tareas descritas anteriormente, detallando el código asignado, el nombre descriptivo, el tiempo estimado, las dependencias temporales y los recursos requeridos.

| Código      | Nombre descriptivo                     | Tiempo estimado (h) | Dependencias temporales | Recursos                                         |
|-------------|----------------------------------------|---------------------|-------------------------|--------------------------------------------------|
| <b>T1</b>   | Gestión y seguimiento del proyecto     | 210                 |                         | PC y Trello                                      |
| <b>T1.1</b> | Contextualización y alcance            | 35                  |                         | PC y artículos                                   |
| <b>T1.2</b> | Planificación temporal                 | 25                  | T1.1                    | PC, Ganttter y calculadora                       |
| <b>T1.3</b> | Presupuesto                            | 20                  | T1.2                    | PC y calculadora                                 |
| <b>T1.4</b> | Informe de sostenibilidad              | 15                  | T1.3                    | PC                                               |
| <b>T1.5</b> | Integración del documento final        | 5                   | T1.4                    | PC                                               |
| <b>T1.6</b> | Documentación                          | 70                  |                         | PC, Trello y los resultados obtenidos            |
| <b>T1.7</b> | Presentación                           | 40                  | T1.6, T2 y T3           | PC y los resultados obtenidos                    |
| <b>T1.8</b> | Reuniones                              |                     |                         | Director del proyecto                            |
| <b>T2</b>   | Trabajo previo                         | 110                 |                         | PC, artículos, libros y Google Collab            |
| <b>T2.1</b> | Familiarización con Keras y TensorFlow | 40                  |                         | PC, artículos, libros y Google Collab            |
| <b>T2.2</b> | Estudio de las CNN's                   | 70                  |                         | PC, artículos, libros y Google Collab            |
| <b>T3</b>   | Desarrollo de la CNN                   | 150                 | T2                      | PC, artículos, libros, Jupyter Notebook y Github |
| <b>T3.1</b> | Diseñar la arquitectura de la CNN      | 40                  | T2                      | PC, artículos, libros, Jupyter Notebook y Github |
| <b>T3.2</b> | Entrenar la CNN                        | 15                  | T3.1                    | PC, artículos, libros, Jupyter Notebook y Github |
| <b>T3.3</b> | Evaluar el modelo                      | 15                  | T3.2                    | PC, artículos, libros, Jupyter Notebook y Github |
| <b>T3.4</b> | Optimizar el modelo                    | 50                  | T3.3                    | PC, artículos, libros, Jupyter Notebook y Github |
| <b>T3.5</b> | Analizar los patrones                  | 30                  | T3.4                    | PC, artículos, libros, Jupyter Notebook y Github |

Tabla 1: Resumen de las tareas (Elaboración propia).

## 5.4 Estimaciones y Gantt

El siguiente diagrama de Gantt ofrece una visión global de la planificación temporal del proyecto, mostrando la duración estimada de cada tarea y sus dependencias.

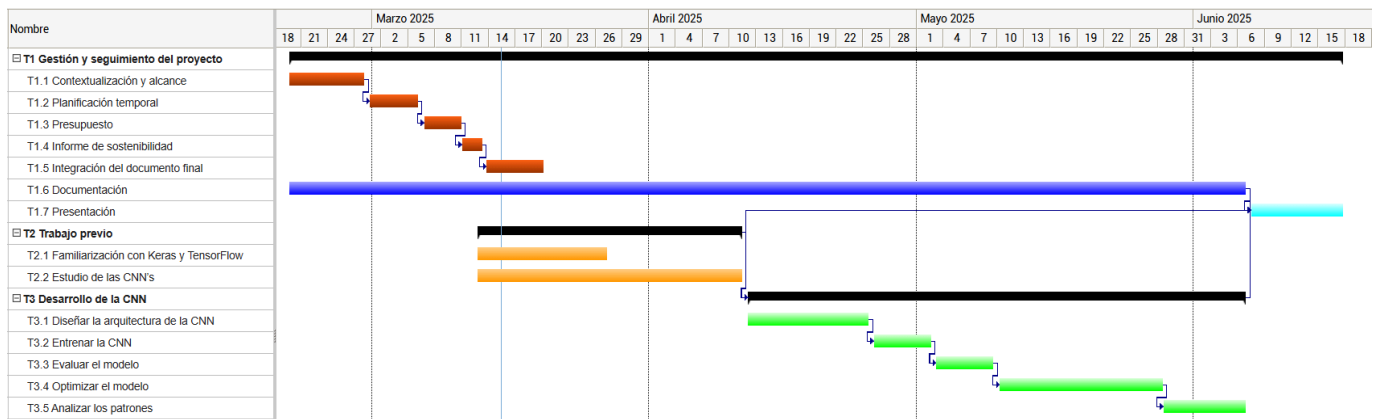


Figura 16: Diagrama de Gantt (Elaboración propia).

## 5.5 Gestión de riesgos: Planes alternativos y obstáculos

A lo largo del desarrollo del proyecto pueden surgir obstáculos que dificulten su avance. Aunque los riesgos potenciales ya se han identificado en el apartado 3.4, en esta sección se detalla el plan de acción para afrontarlos, mediante la incorporación de nuevas tareas y la reorganización de la planificación. Asimismo, se asignará a cada riesgo un nivel de prioridad para facilitar la toma de decisiones.

- **Fecha límite de entrega [Alto riesgo]:** Este riesgo puede originarse en una subestimación de las tareas y de su duración, ya que la planificación se realiza antes de comenzar con el desarrollo del proyecto. Para mitigar este riesgo, se propone realizar una revisión y actualización de la planificación en un punto intermedio del proyecto. Esta replanificación se llevará a cabo utilizando el PC, el software GanttProject y una calculadora, con el fin de ajustar las estimaciones de tiempo. En caso de que, tras la revisión, el tiempo siga siendo insuficiente, se considerará aumentar el número de horas de trabajo diarias para cumplir con la fecha límite de entrega.
- **Falta de experiencia en ML aplicado a imágenes [Riesgo medio]:** La falta de experiencia en aplicar técnicas de ML a imágenes podría dificultar el desarrollo del proyecto, debido a la complejidad en la implementación y ajuste de modelos. Para mitigar este riesgo, se complementará la formación mediante tutoriales y la consulta de artículos y libros especializados, contando además con el asesoramiento continuo del director del proyecto.
- **Disponibilidad de tiempo reducida por prácticas [Bajo riesgo]:** El autor de este proyecto realiza prácticas laborales a tiempo parcial, lo que reduce el tiempo disponible para desarrollar el proyecto. Para mitigar este riesgo, se asignará un número mínimo de horas semanales exclusivamente al proyecto y se definirán con antelación las distintas etapas. De este modo, se podrán identificar y solucionar posibles imprevistos con suficiente antelación, garantizando el cumplimiento de los plazos de entrega.

## 6 Presupuesto

Tras la planificación temporal, en este apartado se estimará el presupuesto total del proyecto. Para ello, se identificarán y clasificarán los diferentes costes, incluyendo los costes de personal asignados a cada tarea, los costes genéricos y otros gastos asociados. Además, se establecerán mecanismos de control de gestión que permitan monitorizar y corregir posibles desviaciones en el presupuesto a lo largo del desarrollo del proyecto.

### 6.1 Identificación de los costes

Para identificar los costes del proyecto, se ha considerado el uso de distintos recursos. En primer lugar, se han definido roles específicos en el ámbito de los recursos humanos para asignar y ejecutar cada tarea, lo que permitirá estimar de manera más precisa el coste asociado a cada actividad. A continuación, se detallan los roles y sus responsabilidades:

- **Redactor Técnico:** Es el encargado de elaborar toda la documentación técnica del proyecto redactando informes que reflejen el desarrollo del proyecto y los resultados obtenidos.
- **Investigador:** Responsable de la búsqueda y análisis de información teórica. Este rol se centrará en la consulta de artículos y libros especializados aportando la base conceptual necesaria para el desarrollo del proyecto.
- **Ingeniero de Deep Learning:** Encargado del desarrollo, implementación y optimización de la red neuronal. Diseñará la arquitectura, entrenará el modelo, evaluará su rendimiento y aplicará ajustes para maximizar su eficiencia, asegurando el correcto funcionamiento del sistema.
- **Director del proyecto:** Se encargará de presentar el proyecto ante el tribunal, garantizando el cumplimiento de los objetivos establecidos.

En segundo lugar, se consideran los recursos materiales, que incluyen hardware y software. Todo el software utilizado será gratuito, mientras que el hardware representa el componente de mayor coste en esta categoría. Asimismo, se tendrán en cuenta otros recursos como el espacio de trabajo, el consumo de electricidad, el mobiliario y el acceso a Internet. Por último, se incluye el transporte necesario para acudir a reuniones con el director del proyecto.

En la Tabla 2 se identifican y clasifican todos los costes del proyecto, divididos en Costes por Actividad (CPA), Costes Generales (CG) y Costes de Contingencias e Incidentes.

| Actividad                            | Importe (€) | Observaciones                   |
|--------------------------------------|-------------|---------------------------------|
| T1.1 Contextualización y alcance     | 894,66      | Redactor Técnico, 35 horas      |
| T1.2 Planificación temporal          | 639,04      | Redactor Técnico, 25 horas      |
| T1.3 Presupuesto                     | 511,24      | Redactor Técnico, 20 horas      |
| T1.4 Informe de sostenibilidad       | 383,43      | Redactor Técnico, 15 horas      |
| T1.5 Integración del documento final | 127,81      | Redactor Técnico, 5 horas       |
| T1.6 Documentación                   | 1.789,33    | Redactor Técnico, 70 horas      |
| T1.7 Presentación                    | 1.584,83    | Director del proyecto, 40 horas |



|                                                     |                  |                                                                    |
|-----------------------------------------------------|------------------|--------------------------------------------------------------------|
| T2.1 Familiarización con Keras y TensorFlow         | 803,37           | Investigador, 40 horas                                             |
| T2.2 Estudio de las CNN's                           | 1.405,90         | Investigador, 70 horas                                             |
| T3.1 Diseñar la arquitectura de la CNN              | 1.460,67         | Ingeniero de Deep Learning, 40 horas                               |
| T3.2 Entrenar la CNN                                | 547,75           | Ingeniero de Deep Learning, 15 horas                               |
| T3.3 Evaluar el modelo                              | 547,75           | Ingeniero de Deep Learning, 15 horas                               |
| T3.4 Optimizar el modelo                            | 1.825,84         | Ingeniero de Deep Learning, 50 horas                               |
| T3.5 Analizar los patrones                          | 1.095,51         | Ingeniero de Deep Learning, 30 horas                               |
| <b>Total CPA (Costes de personal por actividad)</b> | <b>13.617,13</b> |                                                                    |
| <b>Hardware</b>                                     |                  |                                                                    |
| Portátil                                            | 73,08            | Victus HP 16, 470 horas. Precio de compra: 1.107,14€               |
| Ratón y teclado                                     | 2,11             | BAKTH, 470 horas. Precio de compra: 31,99€                         |
| <b>Software</b>                                     |                  |                                                                    |
| Trello                                              | 0                | Uso gratuito                                                       |
| Ganttter                                            | 0                | Uso gratuito                                                       |
| Calculadora                                         | 0                | Uso gratuito                                                       |
| Google Colab                                        | 0                | Uso gratuito                                                       |
| Jupyter Notebook                                    | 0                | Uso gratuito                                                       |
| Github                                              | 0                | Uso gratuito                                                       |
| Autocorrector                                       | 0                | Uso gratuito                                                       |
| <b>Espacio</b>                                      |                  |                                                                    |
| Electricidad                                        | 320              | 80€ al mes. La duración del proyecto es de unos 4 meses            |
| Muebles                                             | 239,98           | Mesa + Silla                                                       |
| Acceso a Internet                                   | 152              | 38€ al mes. La duración del proyecto es de unos 4 meses            |
| <b>Transporte</b>                                   | 88               | 2 T-Jove (tarjeta de transporte) para hacer reuniones con el tutor |
| <b>Total CG (Costes generales)</b>                  | <b>875,17</b>    |                                                                    |
| <b>Total Costes (Total CPA + Total CG)</b>          | <b>14.492,3</b>  |                                                                    |
| <b>Contingencia</b>                                 | <b>2.173,85</b>  | Margen de contingencia = 15%                                       |
| <b>Total CD+CI +Contingencia</b>                    | <b>16.666,15</b> |                                                                    |
| Fecha límite de entrega                             | 594,31           | Coste: Director del proyecto, 30 horas. Riesgo: 50%                |
| Falta de experiencia en ML aplicado a imágenes      | 120,51           | Coste: Investigador, 20 horas. Riesgo: 30%                         |
| Disponibilidad de tiempo reducida por prácticas     | 118,86           | Coste: Director del proyecto, 15 horas. Riesgo: 20%                |
| <b>Total Imprevistos:</b>                           | <b>833,68</b>    |                                                                    |
| <b>TOTAL</b>                                        | <b>17.499,83</b> |                                                                    |

Tabla 2: Estructura de Presupuesto (Elaboración propia).

## 6.2 Estimación de los costes

Tal como se indicó anteriormente, el coste total del proyecto se estima considerando tres componentes principales: los Costes por Actividad (CPA), los Costes Generales (CG) y los Costes de Contingencias e Incidentes. En la Tabla 2 se presenta de forma detallada el coste total del proyecto.

Para calcular los CPA, se asigna a cada tarea el coste correspondiente al rol encargado, multiplicando el coste por hora de dicho rol por el número total de horas estimadas para la realización de la tarea. En la Tabla 3 se recogen los salarios de cada rol, incluyendo el coste adicional de la seguridad social (calculado aplicando un factor de 1,3 sobre el salario anual, considerando un contrato de 1780 horas al año). Estos datos se han obtenido a partir de fuentes como GlassDoor [12], que proporciona valores promedio en el sector. El coste total de los recursos humanos para este proyecto es de 13.617,13€.

En cuanto a los CG, se consideran los recursos materiales, que comprenden hardware, espacio y transporte. Dado que el software utilizado es gratuito, el análisis se centra principalmente en el hardware, cuya amortización se calcula mediante la fórmula:  $\frac{\text{coste total del producto} * \text{horas totales del proyecto}}{\text{horas de uso estimadas en 4 años}}$ . Por ejemplo, para el portátil con un coste de 1.107,14€, si se estima que se utilizará durante 470 horas en el proyecto y suponiendo un uso de 1780 horas anuales, su amortización se calcula:  $\frac{1.107,14 * 470}{4 * 1780}$  que aproximadamente es 73,08€. Se aplica el mismo método para el resto de los equipos. Además, se incluyen los costes asociados al espacio (electricidad, mobiliario, internet) y al transporte, situando el total de los CG en 875,17€.

Sumando los CPA y los CG se obtiene un coste base de 14.492,30€. A este importe se añade un margen de contingencia del 15% para cubrir posibles imprevistos, lo que eleva el coste total a 16.666,15€. Finalmente, se incorporan los costes incidentales, resultando en un coste total final del proyecto de 17.499,83€.

| Rol                        | Salario anual | Total incluyendo SS | Precio por hora | Total de horas | Coste total del rol |
|----------------------------|---------------|---------------------|-----------------|----------------|---------------------|
| Redactor Técnico           | 35.000,00     | 45.500,00           | 25,56           | 170            | 4.345,50            |
| Investigador               | 27.500,00     | 35.750,00           | 20,08           | 110            | 2.209,27            |
| Ingeniero de Deep Learning | 50.000,00     | 65.000,00           | 36,52           | 150            | 5.477,53            |
| Director del proyecto      | 54.250,00     | 70.525,00           | 39,62           | 40             | 1.584,83            |

Tabla 3: Salario de cada rol (Elaboración propia).

## 6.3 Control de gestión

En esta sección se describen los procedimientos para controlar el presupuesto del proyecto, así como los indicadores que permitirán supervisar las desviaciones de costes durante su ejecución. Un mecanismo de control consiste en comparar los recursos reales consumidos con los recursos estimados, de modo que se pueda identificar cualquier desviación.

Por ejemplo, para los costes de recursos humanos (CPA), se calcula:

**Desviación de recursos humanos** = (Coste estimado por hora – Coste real por hora) × Total de horas consumidas.

De forma similar, para los costes generales (CG) y, en particular, para el hardware, es necesario evaluar la desviación en la amortización:

**Desviación de amortización** = (Horas de uso estimadas – Horas de uso reales) × Precio por hora.

En cuanto a los costes fijos de espacio y transporte, no se prevén desviaciones, salvo en el caso del suministro eléctrico, para el que se aplica:

**Desviación de costes de electricidad** = (Horas de uso estimadas – Horas de uso reales) × Precio por hora.

Para los costes incidentales, que pueden presentar mayores variaciones si el proyecto se retrasa, se define:

**Desviación de costes incidentales** = (Horas incidentales estimadas – Horas incidentales reales) × Total de horas incidentales.

Finalmente, la desviación total de costes se obtiene sumando todas las desviaciones anteriores:

**Desviación total de costes** = Desviación de recursos humanos + Desviación de amortización + Desviación de costes de electricidad + Desviación de costes incidentales.

Estos indicadores permitirán identificar dónde se producen desviaciones en el presupuesto y, al comparar el coste total real del proyecto con el coste total estimado, se podrá evaluar si el margen de contingencia ha sido calculado correctamente.

## 7 Sostenibilidad

La sostenibilidad del proyecto se analiza desde una perspectiva integral, considerando las dimensiones económica, ambiental y social. Este enfoque garantiza que tanto el desarrollo como la posible aplicación futura del proyecto sean viables y responsables a largo plazo. A continuación, se presenta una autoevaluación general y un análisis detallado en dos fases: el hito inicial, correspondiente a la planificación y diseño del proyecto, y el hito final, correspondiente a la finalización del desarrollo y su posible implementación en un contexto real.

### 7.1 Autoevaluación

Al responder el cuestionario sobre sostenibilidad, descubrí que mi comprensión inicial era bastante superficial, limitada a la idea de que la sostenibilidad sólo implicaba minimizar el impacto ambiental. Sin embargo, esta experiencia me obligó a replantear mi visión y a considerar que la sostenibilidad abarca también dimensiones económicas y sociales, que son igual de esenciales para el éxito a largo plazo de cualquier proyecto.

Durante el cuestionario se hizo evidente que existían numerosos indicadores que deben evaluarse para medir el impacto en cada una de estas áreas. Por ejemplo, en el ámbito económico es fundamental analizar la viabilidad del presupuesto y los costes a lo largo del ciclo de vida del proyecto, mientras que en el ambiental se deben considerar aspectos como el consumo de recursos y la generación de residuos. En cuanto a la dimensión social, es crucial valorar cómo el proyecto contribuye al bienestar y la equidad en la comunidad generando beneficios.

La realización de esta autoevaluación me permitió identificar que, antes de este ejercicio, mi conocimiento sobre sostenibilidad era limitado y estaba centrado únicamente en la dimensión ecológica. Ahora comprendo que una evaluación integral debe tener en cuenta las tres dimensiones: económica, ambiental y social. Este cuestionario me ha motivado a profundizar en cada una de ellas y a incorporar prácticas y medidas que aseguren un desarrollo más responsable y equilibrado.

En resumen, esto me ha permitido reconocer áreas de mejora en mi enfoque hacia la sostenibilidad, enfatizando la importancia de medir y gestionar de forma conjunta los impactos económicos, ambientales y sociales. Esto no solo me ayudará a aplicar estos conocimientos en el presente proyecto, sino que también servirá como base para futuros desarrollos.

## 7.2 Hito Inicial

En esta fase se analiza la sostenibilidad del proyecto tal como fue concebido al inicio, durante la etapa de planificación. Se evalúan los recursos necesarios, los posibles impactos ambientales derivados de la ejecución inicial, y las implicaciones sociales previstas. Este análisis permite establecer una base para contrastar con los resultados y decisiones tomadas durante el desarrollo del proyecto.

### 7.2.1 Dimensión Económica

En el apartado 6 se han detallado tanto la reflexión sobre el coste estimado del proyecto como los mecanismos de control de gestión implementados. Desde mi punto de vista, el presupuesto es razonable y viable para un proyecto real, al integrar de forma equilibrada los Costes por Actividad (CPA), los Costes Generales (CG) y un margen de contingencia.

En relación a la vida útil, la solución actual aborda los costes mediante la optimización de recursos, como el uso de software gratuito y la aplicando un adecuado proceso de amortización del hardware. Estas estrategias permiten mantener bajos los costes operativos a lo largo del tiempo y asegurar que el impacto económico del proyecto sea sostenible durante su ciclo de vida.

Además, mi solución presenta mejoras económicas significativas respecto a las metodologías existentes. Al emplear técnicas de aprendizaje automático para automatizar el proceso de diagnóstico, se reduce la dependencia de la intervención humana y se optimiza el uso de recursos tecnológicos.

### 7.2.2 Dimensión Ambiental

Debido a la naturaleza del proyecto, resulta difícil estimar su impacto ambiental real, ya que se desarrollará utilizando únicamente una computadora portátil, lo que implica un consumo energético y de recursos muy reducido, lo cual representa una gran ventaja. No obstante, se han adoptado medidas adicionales para minimizar aún más este impacto; por ejemplo, se utiliza software gratuito, evitando la adquisición de licencias o hardware adicional que pudiera incrementar la huella ecológica.

En lo que respecta a la puesta en producción, el proyecto no genera desperdicio material, aunque sí puede implicar un elevado consumo eléctrico durante la fase experimental. Se realizarán múltiples iteraciones para optimizar el modelo, lo que llevará a un mayor consumo de electricidad en ese periodo.

Por otro lado, en relación a la vida útil, los métodos actuales que abordan problemas similares suelen requerir infraestructuras más amplias, lo que se traduce en un mayor consumo energético y generación de residuos electrónicos. En contraste, mi solución se diferencia al operar con una única máquina ofreciendo así una opción más sostenible.

### 7.2.3 Dimensión Social

Desde el punto de vista social, la realización de este proyecto aporta beneficios tanto a nivel personal como colectivo. En primer lugar, a nivel personal, el desarrollo del proyecto me permitirá adquirir conocimientos avanzados en el ámbito del aprendizaje automático aplicado a imágenes médicas. Esta experiencia enriquecerá mis competencias técnicas, impulsará mi formación profesional y me abrirá nuevas oportunidades en sectores de alta demanda tecnológica y sanitaria.

Respecto al proyecto puesto en producción, la solución propuesta aborda un problema que actualmente se resuelve mediante métodos tradicionales, como la interpretación manual de imágenes histológicas, proceso que suele ser subjetivo y propenso a errores. La automatización del diagnóstico mediante redes neuronales no solo aumentará la precisión y la rapidez en la clasificación de los subtipos de linfoma, sino que también contribuirá a una atención médica más oportuna. Esto, a su vez, tiene un impacto social directo, ya que se traduce en tratamientos más adecuados y, por ende, en una mejora en la calidad de vida de los pacientes y sus familias.

Por último, existe una necesidad real de este proyecto. El diagnóstico temprano y preciso de los linfomas es crucial para definir tratamientos eficaces y mejorar el pronóstico de los pacientes. La implementación de esta solución tecnológica responde a una demanda creciente en el ámbito de la salud. En resumen, este proyecto no solo representa un avance personal y profesional, sino que también tiene el potencial de generar un impacto positivo en la sociedad, elevando la calidad de vida a través de diagnósticos más precisos y eficientes.

## 7.3 Hito Final

Este apartado examina la sostenibilidad del proyecto una vez finalizado su desarrollo. Se evalúan aspectos como la viabilidad económica si el sistema se llevase a producción, su impacto ambiental en un uso prolongado y su influencia social. Además, se consideran posibles riesgos y la vida útil del sistema, permitiendo así valorar la madurez y sostenibilidad real del trabajo realizado.

### 7.3.1 Dimensión Económica

En la fase final del proyecto, la dimensión económica se ha abordado con una visión orientada a la producción. Se han estimado y justificado los costes derivados del desarrollo, destacando decisiones tomadas para reducir gastos, como el uso de herramientas gratuitas y la optimización del tiempo dedicado. Además, se ha reflexionado sobre el coste total a lo largo de la vida útil del proyecto, incluyendo posibles ajustes y mantenimientos necesarios para garantizar su continuidad. Por último, se han identificado riesgos económicos potenciales que podrían afectar la viabilidad, como requerimientos de infraestructura más exigente o problemas de compatibilidad futura, proponiendo estrategias de mitigación para garantizar una sostenibilidad económica razonable en un contexto real.

### 7.3.1.1 Proyecto Puesto en Producción

Durante el desarrollo del proyecto se estimaron los costes humanos y materiales, siendo el principal el tiempo invertido en el aprendizaje y experimentación con nuevas tecnologías. Para reducir costes, se utilizaron recursos propios y herramientas de software libre, evitando gastos en licencias o infraestructuras externas. Aunque no se cuantificó el ahorro exacto, se considera significativo. El coste final se ajustó al previsto, aunque se identificaron como lección aprendida ciertos costes en tiempo derivados del uso de herramientas gratuitas, que requieren una mayor dedicación técnica.

### 7.3.1.2 Vida Útil

Durante su vida útil, el proyecto tendrá un coste bajo gracias al uso de software libre y recursos ya disponibles. Los principales gastos estarían relacionados con mantenimiento, actualizaciones y posibles mejoras del modelo. Aunque no se ha hecho una estimación económica precisa, se han considerado estas tareas y se plantea minimizar su impacto mediante buenas prácticas de desarrollo y uso eficiente de recursos.

### 7.3.1.3 Riesgos

Existen ciertos escenarios que podrían afectar la viabilidad económica del proyecto en producción, como la necesidad de infraestructura más potente para escalar el modelo, costes asociados a mantenimiento prolongado, o incompatibilidades con nuevas versiones de librerías. Sin embargo, la planificación contempla medidas para mitigar estos riesgos, como el uso de entornos virtuales y la escalabilidad gradual según recursos disponibles.

## 7.3.2 Dimensión Ambiental

La dimensión ambiental en la fase de puesta en producción y vida útil del proyecto se caracteriza por un bajo impacto, gracias al uso de una única computadora portátil y software gratuito que minimizan el consumo de recursos y residuos. No obstante, es importante considerar que procesos intensivos como el reentrenamiento del modelo podrían aumentar el consumo energético. Se han identificado posibles riesgos que podrían elevar la huella ecológica, pero con una adecuada planificación y optimización de recursos, es factible mantener el impacto ambiental en niveles mínimos, contribuyendo así a una solución tecnológica sostenible a largo plazo.

### 7.3.2.1 Proyecto Puesto en Producción

Aunque no se ha cuantificado con precisión el impacto ambiental del proyecto, se ha considerado desde el inicio. Para reducirlo, se han tomado decisiones como trabajar con software gratuito, evitar compras de hardware adicional y ejecutar los experimentos en un equipo personal. Estas medidas han minimizado el consumo energético y de recursos. Si se repitiera el proyecto, podrían optimizarse aún más los entrenamientos reduciendo el número de experimentos o utilizando técnicas más eficientes, lo que disminuiría aún más el uso energético.



### 7.3.2.2 Vida Útil

Durante su vida útil, el proyecto requerirá principalmente recursos computacionales para su mantenimiento y posibles reentrenamientos, lo que implica un consumo energético moderado. Sin embargo, este impacto ambiental es limitado en comparación con soluciones tradicionales que implican mayor intervención humana y uso de infraestructuras físicas más complejas. Además, al automatizar procesos de diagnóstico, el proyecto puede contribuir a reducir desplazamientos, consumo de papel y uso innecesario de recursos clínicos, mejorando así la huella ecológica global.

### 7.3.2.3 Riesgos

Existen escenarios que podrían incrementar la huella ecológica del proyecto, como la necesidad de realizar múltiples reentrenamientos frecuentes o el uso prolongado de hardware sin optimización energética. Además, la dependencia de recursos computacionales cada vez más potentes podría aumentar el consumo eléctrico. Sin embargo, con una gestión adecuada y optimización del código y recursos, estos riesgos pueden minimizarse para mantener un impacto ambiental bajo.

## 7.3.3 Dimensión Social

La dimensión social del proyecto se fundamenta en su capacidad para aportar beneficios significativos tanto a nivel individual como colectivo. Durante su desarrollo, se han producido reflexiones importantes en torno a los aspectos éticos y profesionales asociados a la automatización del diagnóstico médico, resaltando la necesidad de mantener siempre la supervisión clínica. A lo largo de su vida útil, el proyecto beneficiará principalmente a pacientes y profesionales sanitarios, al facilitar diagnósticos más rápidos y precisos que mejoran la calidad de vida, aunque se debe considerar el posible impacto en colectivos vulnerables y evitar una dependencia excesiva de la tecnología que pueda debilitar la autonomía de los usuarios. Por tanto, el éxito social del proyecto dependerá de su correcta integración en los procesos clínicos, garantizando un uso responsable que minimice riesgos y maximice sus beneficios.

### 7.3.3.1 Proyecto Puesto en Producción

La realización del proyecto ha generado reflexiones importantes a nivel personal y profesional, especialmente en cuanto al impacto ético de la automatización en el diagnóstico médico. Se ha valorado la responsabilidad de garantizar precisión y fiabilidad en los modelos para evitar posibles errores que afecten a los pacientes. Además, el proceso ha reforzado la conciencia sobre la importancia de la colaboración entre tecnología y profesionales sanitarios para mejorar la calidad asistencial, asegurando que la innovación tecnológica se utilice de manera ética y beneficiosa para la sociedad.

### 7.3.3.2 Vida Útil

Durante su vida útil, el proyecto beneficiará principalmente a profesionales sanitarios y pacientes, al facilitar diagnósticos más rápidos y precisos de linfomas, mejorando la toma de decisiones clínicas y el pronóstico de los pacientes. Aunque no se identifican colectivos claramente perjudicados, es importante considerar posibles resistencias o desconfianza inicial en la adopción de sistemas automatizados por parte de algunos profesionales. En cuanto al problema planteado, el proyecto ofrece una solución tecnológica efectiva que contribuye a reducir la subjetividad en el diagnóstico histológico, agilizando el proceso y aumentando la precisión, lo que responde directamente a la necesidad de mejorar el análisis de linfomas malignos.

### 7.3.3.3 Riesgos

Existen escenarios en los que el proyecto podría ser perjudicial para ciertos grupos, especialmente si la automatización del diagnóstico no se complementa adecuadamente con la supervisión médica, lo que podría conducir a errores o malinterpretaciones que afecten a pacientes vulnerables. Asimismo, el uso del sistema podría generar cierta dependencia tecnológica en los profesionales, disminuyendo la confianza en el juicio clínico propio o la capacitación tradicional, lo que podría colocar a los usuarios en una posición de debilidad ante fallos del sistema o falta de recursos alternativos. Por ello, es crucial mantener un equilibrio entre la tecnología y la supervisión humana para mitigar estos riesgos.

## 7.4 Tabla de resumen

A continuación se presenta una tabla resumen que sintetiza los aspectos clave de la sostenibilidad del proyecto en las tres dimensiones analizadas: Ambiental, Económica y Social, en relación con las tres fases principales consideradas: Proyecto Puesto en Producción (PPP), Vida Útil y Riesgos.

|                  | PPP                                                                                                                         | Vida Útil                                                                                                     | Riesgos                                                                                                                           |
|------------------|-----------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| <b>Ambiental</b> | Consumo energético y generación mínima de residuos durante el desarrollo, uso eficiente de recursos tecnológicos            | Huella ecológica reducida, limitada al consumo eléctrico del hardware durante su uso prolongado               | Posible aumento del consumo energético si el modelo se escala o si se requieren más recursos para mantenimiento o actualizaciones |
| <b>Económico</b> | Coste de recursos humanos y materiales ajustado, planificación y control de costes eficiente                                | Plan de viabilidad simplificado que incluye mantenimiento, actualizaciones y amortización del hardware        | Riesgos de desviación en tiempos y costes que pueden afectar la rentabilidad o viabilidad económica                               |
| <b>Social</b>    | Impacto formativo y profesional significativo en los desarrolladores, con reflexiones éticas sobre el uso de la IA en salud | Mejora en la calidad de vida de pacientes y optimización del trabajo clínico; posible dependencia tecnológica | Riesgo de impactos negativos si se genera dependencia excesiva o errores en el diagnóstico automatizado                           |

Tabla 4: Resumen de la sostenibilidad (Elaboración propia).

## 8 Trabajo Previo

Antes de iniciar el desarrollo del proyecto, fue necesario dedicar una cantidad considerable de tiempo a una fase de preparación técnica y teórica, con el objetivo de adquirir los conocimientos esenciales sobre deep learning y redes neuronales convolucionales (CNN). Esta etapa fue especialmente relevante dado que, al comenzar el TFG, no contaba con experiencia previa en este ámbito, ni en el uso de frameworks como TensorFlow o Keras. Por ello, gran parte del trabajo inicial estuvo centrado en el aprendizaje autodidacta de estos conceptos, lo cual resultó crucial para poder avanzar con garantías en el desarrollo del proyecto.

### 8.1 Familiarización con Keras y TensorFlow

Uno de los primeros pasos fue aprender a utilizar las bibliotecas Keras y TensorFlow, dado que son las herramientas principales para la construcción y entrenamiento de modelos de deep learning. Para ello, se llevó a cabo una investigación práctica y teórica que incluyó el estudio de la documentación oficial, la realización de tutoriales interactivos y la implementación de modelos básicos. Esta fase permitió comprender aspectos como la estructura de una CNN en Keras, la definición de capas, la configuración de parámetros de entrenamiento, y el uso de funciones de pérdida y métricas de evaluación. Gracias a este trabajo previo, fue posible afrontar con mayor solvencia la implementación de los modelos más avanzados utilizados en el proyecto.

### 8.2 Estudio de las Redes Neuronales Convolucionales

En paralelo al aprendizaje técnico, se llevó a cabo un estudio teórico sobre las CNN, abarcando desde los conceptos básicos de convolución y pooling hasta arquitecturas modernas. Esta parte incluyó la lectura de capítulos clave en manuales especializados, recursos recomendados por el director del TFG y artículos académicos disponibles en línea. Una vez adquiridos estos fundamentos, se intentaron diseñar varias arquitecturas de CNN propias para resolver el problema del experimento preliminar. Estas pruebas incluyeron variaciones en la cantidad de capas, tipos de activación, tamaños de filtros y funciones de pérdida.

Si bien estos primeros modelos ofrecieron resultados prometedores en ciertas métricas, también presentaron limitaciones en términos de generalización y estabilidad durante el entrenamiento. Esta experiencia fue determinante para entender los retos del diseño de arquitecturas desde cero y motivó la posterior decisión de trabajar con modelos preentrenados (como MobileNet, VGG16 o ResNet), adaptándolos mediante técnicas de transferencia de aprendizaje para aprovechar su capacidad de representación sin necesidad de entrenar redes profundas desde cero.

## 9 Experimento Preliminar antes del Desarrollo Principal

Antes de iniciar el desarrollo principal del proyecto, se consideró conveniente llevar a cabo una experimentación preliminar, con el objetivo de adquirir experiencia práctica en el uso de herramientas clave como Keras y TensorFlow, fundamentales para la implementación de modelos de aprendizaje profundo en este trabajo. Dado que el autor contaba con una experiencia limitada en el desarrollo de redes neuronales convolucionales (CNN) aplicadas al reconocimiento de imágenes, se optó por un primer ejercicio controlado y de menor complejidad. Esta decisión permitió familiarizarse de forma gradual con el flujo de trabajo necesario para construir, entrenar y evaluar modelos de clasificación de imágenes.

Como entorno de experimentación se eligió el conjunto de datos Fashion MNIST [36], ampliamente utilizado en contextos educativos e investigadores, al estar integrado en la librería Keras. Este experimento tenía un doble propósito:

- Desarrollar fluidez en la implementación práctica de una CNN básica.
- Evaluar el comportamiento del modelo y analizar los errores de predicción.

### 9.1 Selección de los datos

El conjunto de datos utilizado en este experimento preliminar es Fashion MNIST, que consiste en imágenes en escala de grises de 28×28 píxeles correspondientes a distintos artículos de ropa, calzado y accesorios, organizados en 10 clases. Para este trabajo se han seleccionado tres subclases específicas:

- Camiseta/top (etiqueta 0)
- Jersey (etiqueta 2)
- Camisa (etiqueta 6)

Estas categorías presentan una alta similitud visual entre sí, lo que permite recrear, a pequeña escala, un escenario de clasificación complejo y exigente, similar al que se abordará más adelante en el contexto clínico con imágenes de linfomas.

### 9.2 Preprocesamiento de los datos

Para preparar los datos del experimento, se filtraron únicamente las tres clases de interés del conjunto original Fashion MNIST: camiseta/top (etiqueta 0), jersey (etiqueta 2) y camisa (etiqueta 6). Esto permitió construir un subconjunto específico compuesto por 18.000 imágenes para entrenamiento y 3.000 para prueba, distribuidas equitativamente entre las tres categorías seleccionadas. Posteriormente, se normalizaron todas las imágenes dividiendo los valores de los píxeles por 255, lo que redujo el rango de intensidades de [0, 255] a [0, 1]. Esta normalización facilita la estabilidad del entrenamiento y mejora la eficiencia del modelo al trabajar con valores en una escala uniforme.

## 9.3 Diseño de la CNN

En esta fase del experimento preliminar, se diseñaron dos modelos de red neuronal con arquitecturas diferentes con el fin de comparar su rendimiento en la tarea de clasificación de imágenes. Ambas redes comienzan con una capa de aplanamiento (Flatten) que convierte las imágenes de entrada en vectores unidimensionales, permitiendo su procesamiento por capas densas. El primer modelo se compone de dos capas ocultas: una capa Dense con 100 neuronas y función de activación ReLU, seguida de otra con 50 neuronas y activación sigmoide. Finalmente, una capa de salida con 3 neuronas y activación softmax permite obtener la probabilidad de pertenencia a cada una de las clases. El segundo modelo incrementa la profundidad de la red, incorporando cinco capas ocultas: una primera con 200 neuronas (ReLU), seguida de capas con 150 (sigmoide), 100 (ReLU), 50 (ReLU) y 25 (sigmoide) neuronas, antes de la capa de salida softmax. Este diseño más complejo busca evaluar si un mayor número de capas y neuronas puede mejorar la capacidad de la red para distinguir entre clases visualmente similares.

## 9.4 Entrenamiento del modelo

Para el entrenamiento de ambos modelos diseñados, se utilizó el optimizador Adam, ampliamente empleado por su eficiencia y capacidad de adaptación durante el proceso de aprendizaje. La función de pérdida seleccionada fue sparse categorical crossentropy, adecuada para problemas de clasificación multiclase con etiquetas codificadas como enteros. Como métrica de evaluación durante el entrenamiento se empleó la precisión (accuracy), con el objetivo de medir el porcentaje de predicciones correctas. Ambos modelos fueron entrenados durante 30 épocas, lo que permitió observar la evolución del aprendizaje y evaluar si la red estaba alcanzando una convergencia adecuada sin incurrir en sobreajuste.

## 9.5 Evaluación del modelo

La evaluación del rendimiento de ambos modelos se realizó utilizando la matriz de confusión, una herramienta fundamental en problemas de clasificación que permite visualizar de manera detallada el número de aciertos y errores en la predicción para cada clase específica. Esta métrica facilita la identificación de patrones de confusión entre categorías y ofrece una comprensión más profunda del comportamiento del modelo más allá de métricas globales como la precisión. A continuación, se presentan las matrices de confusión obtenidas para cada modelo evaluado:

|   |     |     |     |
|---|-----|-----|-----|
| 0 | 835 | 21  | 144 |
| 1 | 17  | 860 | 123 |
| 2 | 127 | 73  | 800 |
|   | 0   | 1   | 2   |

|   |     |     |     |
|---|-----|-----|-----|
| 0 | 756 | 16  | 228 |
| 1 | 12  | 906 | 82  |
| 2 | 67  | 108 | 825 |
|   | 0   | 1   | 2   |

# 10 Desarrollo de la CNN para el Proyecto

En este apartado se detalla el proceso completo seguido para desarrollar, entrenar y evaluar distintos modelos de redes neuronales convolucionales (CNN) aplicados a la clasificación automática de imágenes histológicas de linfomas malignos. El objetivo principal ha sido diseñar un sistema capaz de distinguir entre tres tipos de linfomas a partir de biopsias teñidas con Hematoxilina y Eosina (H+E), utilizando arquitecturas de deep learning con pesos preentrenados y adaptadas al conjunto de datos empleado.

A lo largo de este capítulo se abordan las etapas fundamentales del flujo de trabajo: desde la exploración inicial de los datos hasta la configuración del entorno de desarrollo, el preprocesamiento de imágenes, y la implementación de distintas arquitecturas CNN. Cada modelo ha sido entrenado y validado bajo condiciones similares para permitir una comparación objetiva del rendimiento. Asimismo, se describen las estrategias empleadas para ajustar los modelos a las características específicas del conjunto de datos, y se presentan los resultados obtenidos mediante métricas y visualizaciones que facilitan su interpretación.

## 10.1 Trabajo previo con los mismos datos

El conjunto de datos utilizado en este proyecto proviene de una colección pública disponible en la plataforma Kaggle, que agrupa imágenes de biopsias teñidas con Hematoxilina y Eosina (H+E) de tres tipos principales de linfomas malignos: leucemia linfocítica crónica (CLL), linfoma folicular (FL) y linfoma de células del manto (MCL).

Este dataset fue elaborado por diferentes patólogos en varios centros, lo que implica una notable variabilidad en la tinción de las muestras, reflejando las condiciones reales que se encuentran en la práctica clínica. La diferenciación precisa entre estos tipos de linfomas mediante análisis visual tradicional es un desafío incluso para patólogos especializados, ya que generalmente se requiere el uso de sondas específicas para lograr una clasificación confiable.

En la literatura asociada a este conjunto de datos, como el artículo de Orlov et al. (2010), se han explorado métodos automáticos de clasificación de imágenes basados en características globales extraídas mediante transformadas matemáticas. Estos estudios han demostrado el potencial del procesamiento computacional para mejorar la consistencia y rapidez en el diagnóstico de linfomas malignos.

Aunque la licencia del conjunto de datos no está especificada claramente en la fuente, el uso académico y de investigación es generalmente aceptado, siempre citando la referencia original.

## 10.2 Selección de los datos

Una vez descargado el conjunto de datos desde la plataforma Kaggle, se procedió a organizar las imágenes de manera estructurada para facilitar su uso durante el desarrollo del proyecto. Para ello, se creó una carpeta denominada dataset en el mismo directorio donde se encuentra el cuaderno principal del proyecto (TFG.ipynb).

Dentro de la carpeta dataset, se organizaron las imágenes en tres subcarpetas correspondientes a las clases presentes en el conjunto de datos:

- **CLL**: contiene las imágenes de muestras correspondientes a leucemia linfocítica crónica.
- **FL**: contiene las imágenes de linfoma folicular.
- **MCL**: contiene las imágenes de linfoma de células del manto.

Esta estructura de carpetas, con una subcarpeta por clase, es compatible con las funciones de carga y preprocesamiento de imágenes que ofrece la biblioteca Keras, permitiendo etiquetar automáticamente las imágenes en función del nombre de la subcarpeta donde se encuentran. Este formato también simplifica el proceso de división en conjuntos de entrenamiento, validación y prueba.

## 10.3 Configuración previa del entorno

A diferencia del experimento preliminar realizado en Google Colab —una plataforma en la nube que permite ejecutar código sin necesidad de configuración local— el desarrollo completo del Trabajo de Fin de Grado se llevó a cabo en un entorno local con Jupyter Notebook, utilizando el gestor de entornos Anaconda. Esta decisión se tomó con el objetivo de aprovechar el rendimiento de la GPU del equipo disponible y garantizar una mayor autonomía y control sobre las dependencias del proyecto.

Para evitar posibles conflictos entre paquetes, se creó un entorno virtual específico con Python 3.10, en el que se instalaron cuidadosamente todas las bibliotecas necesarias. La instalación se realizó a través del terminal de Anaconda (Anaconda Prompt), siguiendo los siguientes pasos:

- `conda create -n py310 python=3.10`
- `conda activate py310`
- `conda install -c conda-forge cudatoolkit=11.2 cudnn=8.1.0`
- `python -m pip install "tensorflow==2.10"`
- `conda install scikit-learn`
- `conda install numpy=1.26`
- `conda install matplotlib`
- `conda install pillow`
- `conda install pandas`
- `conda install pydot`
- `conda install -c conda-forge graphviz`
- `conda install seaborn`



Este entorno de trabajo permitió ejecutar el código de entrenamiento y evaluación de modelos de deep learning con soporte para aceleración por GPU, lo que mejoró significativamente los tiempos de procesamiento respecto a una ejecución en CPU. Además, el uso de bibliotecas como TensorFlow, NumPy, Pandas o Matplotlib fue fundamental tanto para el desarrollo como para la visualización y análisis de los resultados.

## 10.4 Preprocesamiento de los datos

El preprocesamiento de los datos comenzó adaptando el formato de las imágenes descargadas desde la plataforma Kaggle. Estas imágenes estaban originalmente en formato .tif, que no es compatible con ciertas funciones de carga de datos de TensorFlow, como `image_dataset_from_directory()`. Por ello, se implementó una rutina inicial que convierte todas las imágenes al formato .png, manteniendo la estructura original de carpetas organizada por clase (CLL, FL y MCL).

Una vez convertidas las imágenes, se verificó que TensorFlow reconociera correctamente la GPU del sistema, con el objetivo de asegurar un entrenamiento más eficiente y aprovechar al máximo el rendimiento del hardware disponible.

Posteriormente, se procedió a cargar las imágenes ya convertidas y organizarlas en tres subconjuntos: un 80% para el entrenamiento del modelo, un 10% para la validación durante el entrenamiento y un 10% para la evaluación final. Esta división se realizó de forma estratificada (`stratify`) para asegurar una distribución equilibrada de clases en cada subconjunto, lo que garantiza una evaluación más robusta y representativa del rendimiento del modelo.

Antes del entrenamiento, todas las imágenes se redimensionaron desde su tamaño original de 1388×1040 píxeles a 224×224 píxeles, de modo que fueran compatibles con la entrada esperada por los modelos preentrenados utilizados. Además, se normalizaron dividiendo los valores de los píxeles entre 255 para escalar los datos al rango [0, 1], facilitando así una convergencia más estable durante el proceso de entrenamiento.

A continuación, se realizó una visualización aleatoria de algunas imágenes del conjunto de entrenamiento para asegurar que la carga y la asignación de etiquetas se habían realizado correctamente. Finalmente, se prepararon dos enfoques distintos para aplicar técnicas de aumento de datos (*data augmentation*), con el objetivo de mejorar la robustez del modelo y reducir el sobreajuste:

### 10.4.1 Aumento de datos explícito

Este enfoque consistió en generar de forma anticipada imágenes nuevas a partir del conjunto de entrenamiento, aplicando transformaciones aleatorias como rotaciones, desplazamientos horizontales y verticales, ampliaciones, recortes, inversiones horizontales y verticales, entre otras. Estas transformaciones se aplicaron a cada imagen original para obtener múltiples versiones aumentadas, que se añadieron al conjunto de entrenamiento original. De esta forma, se multiplicó el tamaño del conjunto de entrenamiento, manteniendo intactos los conjuntos de validación y prueba. Esta técnica permite simular una mayor diversidad de muestras sin necesidad de recolectar nuevas imágenes.

## 10.4.2 Aumento de datos implícito

Como alternativa al enfoque anterior, se incorporó también un pipeline de aumento de datos directamente dentro de la arquitectura del modelo. En este caso, las transformaciones aleatorias se aplican dinámicamente durante cada época de entrenamiento, sin modificar los datos originales en memoria. Las transformaciones incluyen inversiones horizontales, rotaciones aleatorias, pequeños desplazamientos y cambios de escala. Esta técnica permite introducir variabilidad continua en cada iteración de entrenamiento, mejorando la capacidad del modelo para generalizar.

Ambas estrategias de aumento de datos se implementaron con el propósito de ser comparadas posteriormente durante la fase de entrenamiento.

## 10.5 Protocolo común de diseño y entrenamiento de modelos

Con el objetivo de asegurar una comparación justa y coherente entre los distintos modelos CNN utilizados en este trabajo, se estableció un protocolo común de diseño, entrenamiento y evaluación. Este protocolo se aplicó de forma sistemática a las seis arquitecturas preentrenadas seleccionadas: Inception V3, MobileNet V2, DenseNet 201, EfficientNet V2B0, ResNet 50V2 y VGG16.

En primer lugar, se empleó la técnica de feature extraction. En esta etapa inicial, se cargó cada modelo sin su capa de clasificación original (`include_top=False`), utilizando los pesos preentrenados en ImageNet, y se congelaron todas sus capas para preservar el conocimiento adquirido previamente. A continuación, se añadieron nuevas capas superiores adaptadas al problema de clasificación de linfomas (con tres clases), y se entrenó este modelo modificado durante un número determinado de épocas.

En segundo lugar, se aplicó fine tuning. En esta fase, se descongelaron parcialmente las últimas capas del modelo base (normalmente las últimas 50 capas) para permitir que se ajustaran a las características específicas del conjunto de datos de biopsias. Esta fase se llevó a cabo con una tasa de aprendizaje más baja y se continuó el entrenamiento desde donde finalizó la etapa anterior.

Además, con cada modelo se llevaron a cabo diez experimentos distintos para evaluar el impacto de distintas configuraciones de entrenamiento. Los primeros experimentos se realizaron sin data augmentation con 15, 20 y 30 épocas respectivamente. A partir de ahí, se exploró el uso de data augmentation tanto de forma explícita como implícita:

- **Data augmentation explícito:** se generaron nuevas imágenes artificialmente antes del entrenamiento, utilizando técnicas como rotaciones, desplazamientos, zoom y volteos. Estas imágenes aumentaron el conjunto de entrenamiento, duplicando, triplicando o cuadruplicando su tamaño original.
- **Data augmentation implícito:** se aplicaron transformaciones aleatorias a las imágenes durante el entrenamiento, en tiempo real y sin modificar el conjunto de datos original.

Finalmente, también se evaluaron combinaciones de ambas técnicas, aplicando data augmentation implícito junto con un conjunto de entrenamiento ampliado mediante técnicas explícitas.

La evaluación de los modelos se llevó a cabo utilizando el conjunto de prueba, analizando las métricas de precisión, pérdida, y curvas ROC por clase. Los resultados detallados de cada experimento están disponibles en el Anexo para cada arquitectura.

Este enfoque sistemático permitió analizar de manera controlada el impacto de las diferentes configuraciones de entrenamiento y comparar objetivamente el rendimiento de las arquitecturas seleccionadas.

### 10.5.1 Inception V3

Para la arquitectura Inception V3 se siguió el protocolo común descrito previamente, aplicando feature extraction y posteriormente fine tuning, así como diversas configuraciones de data augmentation. Los resultados completos de los diez experimentos realizados se encuentran en el apartado del Anexo, identificados como Experimento 1 al Experimento 10.

Los primeros entrenamientos sin data augmentation ofrecieron resultados moderados, pero el rendimiento del modelo mejoró significativamente al aplicar técnicas de data augmentation, tanto de forma explícita como implícita. El mejor resultado se obtuvo en el Experimento 10, en el que se combinaron ambas estrategias de aumento de datos, cuadruplicando el número de imágenes en el conjunto de entrenamiento. En este experimento, el modelo alcanzó una precisión del 79% sobre el conjunto de test, consolidándose como la configuración más eficaz para esta arquitectura.

### 10.5.2 MobileNet V2

En el caso de MobileNet V2, se llevaron a cabo un total de diez experimentos, cuyos resultados detallados pueden consultarse en el Anexo (Experimentos 11 al 20). A diferencia de otras arquitecturas, esta red presentó un comportamiento más inestable frente a las técnicas de data augmentation. De hecho, se observó que su rendimiento tendía a empeorar al aplicar aumentos de datos, especialmente en las fases de fine tuning.

Durante los primeros ocho experimentos (del 11 al 18), se aplicó el mismo protocolo utilizado con Inception V3, incluyendo combinaciones de data augmentation implícito y explícito, y la descongelación de las últimas 50 capas de la red para el ajuste fino. Sin embargo, los resultados obtenidos fueron poco satisfactorios, ya que el rendimiento del modelo disminuye tras el fine tuning, a pesar de haber mostrado una precisión razonable durante la fase de feature extraction.

Con el objetivo de mitigar esta caída de rendimiento, en los dos últimos experimentos (19 y 20) se optó por reducir la cantidad de capas descongeladas durante el fine tuning: 20 capas en el Experimento 19 y solo 5 en el Experimento 20. No obstante, esta estrategia tampoco logró mejorar significativamente los resultados, confirmando que el fine tuning no era beneficioso para esta arquitectura en el contexto del proyecto.

En resumen, MobileNet V2 fue la arquitectura con peor rendimiento general entre las evaluadas. El mejor resultado se obtuvo en el Experimento 15, donde se aplicó únicamente data augmentation explícito duplicando el conjunto de entrenamiento alcanzando una precisión del 71%.

### 10.5.3 DenseNet 201

La arquitectura DenseNet 201 ha demostrado un rendimiento global notablemente superior en comparación con MobileNet V2, manteniéndose estable y ofreciendo buenos resultados en todos los experimentos realizados. En total, se llevaron a cabo 12 experimentos con esta red, cuyos resultados pueden consultarse en el Anexo (Experimentos 21 al 32). El experimento con peor desempeño alcanzó una precisión del 74%, mientras que el mejor logró un 82%, marcando el mayor rendimiento conseguido hasta ese momento en el proyecto.

Los diez primeros experimentos (del 21 al 30) siguieron el mismo planteamiento que con Inception V3, aplicando distintas combinaciones de data augmentation (implícito y/o explícito), así como una descongelación de las últimas 50 capas durante la fase de fine tuning. Dado que DenseNet 201 respondía mejor al entrenamiento y era más robusta frente a modificaciones, se decidió realizar dos experimentos adicionales (31 y 32), en los que se amplió la cantidad de capas descongeladas a 100 y 200 respectivamente, manteniendo las mismas condiciones del Experimento 30 (ambas técnicas de data augmentation y cuadruplicación del conjunto de entrenamiento).

A pesar de los buenos resultados de estos últimos experimentos, el mejor desempeño global se alcanzó en el Experimento 27, en el cual se aplicó únicamente data augmentation explícito, cuadruplicando el número de imágenes del conjunto de entrenamiento. Este experimento logró una precisión del 82%, consolidando a DenseNet 201 como una de las arquitecturas más efectivas empleadas en este proyecto.

### 10.5.4 EfficientNet V2B0

En esta arquitectura se realizaron los 10 experimentos descritos en el protocolo común de entrenamiento, cuyos resultados pueden consultarse en el Anexo, desde el Experimento 33 hasta el Experimento 42. EfficientNet V2B0 mostró un comportamiento más variable que DenseNet 201, obteniendo tanto algunos de los resultados más bajos como uno de los más altos del proyecto.

El peor rendimiento registrado fue de 61% de accuracy, mientras que el mejor resultado se alcanzó en el Experimento 38, donde se aplicaron ambas técnicas de data augmentation (implícito y explícito) duplicando el número de imágenes del conjunto de entrenamiento, logrando una precisión del 84%. Esto posiciona a EfficientNet V2B0 como una arquitectura con alto potencial, aunque algo más sensible a la configuración del entrenamiento y las técnicas de regularización aplicadas.

### 10.5.5 ResNet 50V2

En el caso de la arquitectura ResNet 50V2 se llevaron a cabo 8 experimentos en total, cuyos resultados pueden consultarse en el Anexo, desde el Experimento 43 hasta el Experimento 50. A diferencia de otras arquitecturas, no fue posible completar los experimentos que requerían cuadruplicar las imágenes del conjunto de entrenamiento, ya que durante la fase de fine tuning se produjo un error de memoria relacionado con la capacidad de la GPU, incluso cuando la fase de feature extraction funcionaba correctamente.

El peor rendimiento observado fue de 61% de accuracy, mientras que el mejor resultado se obtuvo en el Experimento 47, donde se aplicó únicamente data augmentation explícito duplicando las imágenes del

conjunto de entrenamiento, alcanzando una precisión del 74%. Estos resultados reflejan que, aunque ResNet 50V2 puede alcanzar un rendimiento razonable, su escalabilidad en este entorno específico estuvo limitada por restricciones técnicas.

### 10.5.6 VGG16

Durante el diseño experimental con la arquitectura VGG16, surgieron limitaciones relacionadas con la memoria de la GPU, lo que impidió inicialmente ejecutar los entrenamientos. Para solventarlo, se redujo el tamaño del batch a 8, lo que permitió llevar a cabo la mayoría de los experimentos planificados.

En total, se realizaron 9 experimentos, cuyos resultados pueden consultarse en el Anexo, desde el Experimento 51 hasta el Experimento 59. No fue posible completar el décimo experimento correspondiente al protocolo común (con ambas técnicas de data augmentation y cuadruplicando las imágenes del conjunto de entrenamiento), ya que el problema de memoria persistía incluso con el batch size reducido.

Los resultados obtenidos con esta arquitectura fueron en general muy satisfactorios, con una precisión mínima del 61% y un máximo de 87%, lo que representa el mejor resultado global obtenido en todo el trabajo. Este rendimiento máximo se alcanzó en el Experimento 55, aplicando únicamente data augmentation explícito y duplicando la cantidad de imágenes en el conjunto de entrenamiento.

## 10.6 Comparación entre Modelos CNN Implementados

Tras el diseño, entrenamiento y evaluación de los seis modelos CNN seleccionados, se ha podido realizar una comparación global de su rendimiento, su comportamiento ante técnicas de data augmentation y su sensibilidad al proceso de fine tuning.

En primer lugar, cabe destacar que todos los modelos se entrenaron siguiendo un protocolo experimental común, con ligeras adaptaciones según las necesidades específicas de cada arquitectura. Este protocolo incluyó aproximadamente 10 experimentos por modelo, variando el número de épocas, el uso de data augmentation explícito (modificando directamente el conjunto de entrenamiento) e implícito (aplicado dinámicamente en cada epoch), así como el número de capas descongeladas durante el proceso de fine tuning. Sin embargo, debido a limitaciones de hardware (memoria GPU), algunos modelos no pudieron completar todos los experimentos previstos.

De los modelos evaluados, VGG16 ha sido el que ha obtenido el mejor rendimiento general, alcanzando un 87% de precisión en su mejor experimento (Experimento 55), aplicando data augmentation explícito y duplicando las imágenes del conjunto de entrenamiento. A pesar de requerir un batch size reducido por limitaciones de memoria, su comportamiento ha sido notablemente superior al del resto de arquitecturas.

EfficientNet V2B0 también ha mostrado un rendimiento sólido, logrando un 84% de precisión en su mejor experimento (Experimento 38), al aplicar ambas técnicas de data augmentation y duplicar las imágenes del conjunto de entrenamiento. Esta arquitectura se ha comportado de forma robusta y consistente durante los experimentos.

DenseNet201 ha ofrecido resultados estables y elevados a lo largo de todos los experimentos, con un mínimo de 74% y un máximo de 82% de precisión (Experimento 27). Dada su buena respuesta, se

realizaron dos experimentos adicionales variando el número de capas descongeladas durante el fine tuning, confirmando su fiabilidad incluso con ajustes más agresivos.

Por su parte, Inception V3 mostró una mejora progresiva con el uso de data augmentation, alcanzando su mejor rendimiento en el Experimento 10, con un 79% de precisión aplicando ambas técnicas de data augmentation y cuadruplicando el conjunto de entrenamiento. Su comportamiento confirma que la combinación de técnicas de aumento de datos y fine tuning puede optimizar significativamente los resultados.

En contraste, ResNet50V2 presentó limitaciones importantes. A pesar de obtener resultados razonables (máximo del 74% de precisión en el Experimento 47), no fue posible completar los experimentos con cuadruplicación de datos debido a errores por falta de memoria en la GPU. Además, el rendimiento se vio afectado negativamente al aplicar fine tuning en los conjuntos más grandes, evidenciando una sensibilidad a la carga computacional.

Finalmente, MobileNetV2 fue la arquitectura que peores resultados ofreció, con un máximo de 71% de precisión (Experimento 15) y una clara tendencia a empeorar su rendimiento al aplicar data augmentation o fine tuning. Esto sugiere que, aunque es un modelo ligero y eficiente, puede no ser el más adecuado para tareas complejas con conjuntos de datos limitados como el presente.

En resumen, la tabla siguiente sintetiza el mejor resultado obtenido por cada modelo, junto con las condiciones específicas del experimento correspondiente:

| Modelo                   | Mejor Accuracy | Mejor Experimento | Observaciones                                                                       |
|--------------------------|----------------|-------------------|-------------------------------------------------------------------------------------|
| <b>VGG16</b>             | 87%            | Experimento 55    | Mejor rendimiento global. Limitaciones de memoria superadas con batch size reducido |
| <b>EfficientNet V2B0</b> | 84%            | Experimento 38    | Excelente rendimiento y alta estabilidad                                            |
| <b>DenseNet 201</b>      | 82%            | Experimento 27    | Resultados consistentes y buen desempeño con fine tuning                            |
| <b>Inception V3</b>      | 79%            | Experimento 10    | Mejora clara con ambas técnicas de data augmentation                                |
| <b>ResNet 50V2</b>       | 74%            | Experimento 47    | Limitado por memoria GPU y sensible al fine tuning                                  |
| <b>MobileNet V2</b>      | 71%            | Experimento 15    | Peor rendimiento y el fine tuning degradó el desempeño                              |

Tabla 5: Comparación entre los modelos (Elaboración propia).

Esta comparación permite concluir que, si bien todos los modelos presentan capacidades razonables para la clasificación de imágenes en este problema concreto, modelos como VGG16, EfficientNet V2B0 y DenseNet201 destacan claramente por su fiabilidad y precisión.

# 11 Dificultades Encontradas durante el Desarrollo del Proyecto

Durante el desarrollo de este Trabajo Fin de Grado se presentaron diversas dificultades técnicas y conceptuales que influyeron en el ritmo de avance del proyecto. A continuación, se detallan los principales obstáculos encontrados.

## 11.1 Adquisición de los fundamentos teóricos y prácticos de las CNNs y su implementación con TensorFlow/Keras

Al inicio del proyecto, no contaba con experiencia previa en el uso de redes neuronales convolucionales ni en los entornos de desarrollo como TensorFlow o Keras. Esto hizo que la curva de aprendizaje fuese más pronunciada de lo esperado. Dediqué una parte considerable de tiempo a estudiar los conceptos teóricos fundamentales de las CNNs, la estructura de los modelos preentrenados, el funcionamiento del fine tuning, la función de pérdida, los optimizadores, entre otros aspectos necesarios para una correcta implementación. Este proceso formativo fue esencial pero ralentizó el avance en las etapas iniciales del proyecto.

## 11.2 Problemas de compatibilidad entre versiones de paquetes en el entorno virtual

Otra dificultad importante surgió al configurar el entorno de desarrollo. Inicialmente, se produjeron múltiples conflictos entre diferentes versiones de paquetes, especialmente entre TensorFlow y otros módulos necesarios para el procesamiento y visualización de datos. El código no se ejecutaba correctamente y, aunque intenté resolverlo probando diferentes versiones, siempre surgían nuevos conflictos. Este problema se prolongó durante varios días, hasta que tras consultar distintas fuentes y tutoriales online, logré identificar una combinación de versiones compatibles. Este contratiempo no estaba previsto y consumió un tiempo significativo que retrasó el inicio del entrenamiento de los modelos.

## 11.3 Conjunto de datos muy reducido

El conjunto de datos disponible estaba compuesto únicamente por 374 imágenes, lo cual representa una cantidad extremadamente limitada para entrenar redes neuronales profundas desde cero. Para abordar esta limitación, comencé a investigar técnicas de data augmentation con el objetivo de ampliar artificialmente el conjunto de datos. Apliqué tanto data augmentation implícito (realizado durante el entrenamiento) como explícito (duplicando, triplicando o cuadruplicando las imágenes), lo cual resultó ser una estrategia efectiva para mejorar el rendimiento de varios modelos.



## 11.4 Aplicación incorrecta del Fine Tuning en las primeras pruebas

Durante las primeras fases de entrenamiento, cometí el error de realizar el fine tuning creando un nuevo modelo desde cero, lo que generaba peores resultados y dificultaba la interpretación de los mismos. Tras investigar más a fondo la metodología adecuada, comprendí que el fine tuning debe aplicarse sobre el modelo previamente entrenado con feature extraction, reutilizando sus pesos como punto de partida. Esta corrección permitió mejorar notablemente el rendimiento de los modelos en las siguientes fases.

## 11.5 Limitaciones de espacio en la GPU

Al trabajar con arquitecturas complejas y conjuntos de datos aumentados, varios experimentos requerían una gran cantidad de memoria en la GPU, lo que generó errores de ejecución. Este problema se acentuaba especialmente durante la fase de fine tuning. Para solventarlo, me vi obligado a reducir el batch size en varios experimentos, lo que permitió completar el entrenamiento sin comprometer excesivamente el rendimiento.

## 11.6 Imposibilidad de implementar técnicas de interpretación visual por falta de tiempo y conocimiento

En la planificación inicial del proyecto estaba previsto incorporar técnicas de visualización que permitiesen destacar qué regiones de las imágenes influían más en la decisión del modelo. Sin embargo, debido a la falta de tiempo y de conocimientos avanzados sobre estas técnicas, no fue posible llevar a cabo esta parte del proyecto. No obstante, se reconoce su relevancia y se considera una posible línea de trabajo futuro para complementar este estudio.

## 12 Aplicación e integración de conocimientos previos

El desarrollo de este Trabajo de Fin de Grado ha supuesto una oportunidad para aplicar, consolidar y ampliar los conocimientos adquiridos a lo largo del grado, especialmente en la asignatura de Aprendizaje Automático (APA). En dicha asignatura se trabajaron diversos tipos de problemas de predicción, tanto de clasificación como de regresión, utilizando conjuntos de datos principalmente tabulares. Las actividades prácticas se centraron en problemas con variables numéricas, donde el objetivo era predecir una etiqueta o valor a partir de las características de entrada.

Aunque en APA no se abordó el análisis de imágenes como tipo de dato, los fundamentos aprendidos —como la preparación del conjunto de datos, la división en subconjuntos de entrenamiento y prueba, la evaluación del rendimiento del modelo, o el ajuste de hiperparámetros— resultaron fundamentales para enfrentar el reto del proyecto. Gracias a esta base, ya estaba familiarizado con la lógica general de los modelos predictivos, lo que facilitó la comprensión de conceptos más avanzados en el ámbito del deep learning.

Este trabajo ha representado, por tanto, una extensión natural de lo aprendido, aplicando esos mismos principios en un contexto más complejo y especializado: la clasificación de imágenes médicas mediante redes neuronales convolucionales. De esta forma, se ha producido una integración efectiva entre los conocimientos teóricos previos y las nuevas competencias técnicas adquiridas específicamente para este proyecto.

## 13 Consideraciones Éticas y Licencias de Uso de los Datos

Los datos utilizados en el presente proyecto han sido obtenidos de la plataforma pública Kaggle, la cual proporciona conjuntos de datos con condiciones de uso específicas claramente indicadas por los autores o las instituciones que los publican.

Antes de su utilización, se ha verificado que el conjunto de datos empleado cuenta con una licencia que permite su uso con fines de investigación y desarrollo académico. Por tanto, el uso de dichos datos se encuentra en conformidad con las normativas establecidas por la plataforma, y con los principios éticos de reutilización de datos abiertos para fines científicos.

Cabe destacar que el dataset no contiene información sensible ni datos personales, por lo que no está sujeto a restricciones derivadas de legislaciones específicas en materia de protección de datos, como el Reglamento General de Protección de Datos (GDPR). En consecuencia, se puede afirmar que el proyecto cumple con los principios éticos y legales aplicables al uso de datos en investigaciones académicas.

# 14 Conclusiones

El presente Trabajo Fin de Grado ha tenido como objetivo principal explorar y probar distintas técnicas y arquitecturas de redes neuronales convolucionales (CNNs), en un contexto de clasificación de imágenes médicas con un conjunto de datos reducido. Al no tener experiencia previa en el ámbito del aprendizaje profundo, el propósito inicial fue también formativo, motivado por un interés personal en este campo y el deseo de adquirir competencias prácticas en inteligencia artificial aplicada a la visión por computador.

A lo largo del desarrollo del proyecto se han implementado y comparado seis modelos CNN preentrenados: Inception V3, MobileNet V2, DenseNet 201, EfficientNet V2B0, ResNet 50V2 y VGG16. A pesar de disponer únicamente de 374 imágenes, se ha conseguido mejorar de forma considerable el rendimiento de los modelos mediante el uso estratégico de técnicas de data augmentation, tanto implícitas como explícitas. Asimismo, el entrenamiento se ha llevado a cabo en dos fases: primero con feature extraction, y posteriormente mediante fine tuning, permitiendo refinar los modelos partiendo de pesos previamente entrenados en ImageNet.

De todos los modelos evaluados, VGG16 fue el que alcanzó el mejor rendimiento, logrando un 87% de precisión en su mejor experimento. Le siguieron EfficientNet V2B0 con un 84% y DenseNet 201 con un 82%. Estas arquitecturas demostraron ser especialmente eficaces, incluso en contextos con pocos datos, cuando se aplicaban técnicas adecuadas de aumento de datos. En cambio, MobileNet V2 obtuvo los resultados más discretos; en este caso, se observó que su rendimiento empeoraba tras realizar fine tuning, lo que indica que su estructura ligera podría no ser la más adecuada para este tipo de tarea con datos limitados.

Un aspecto fundamental ha sido el diseño sistemático de más de 50 experimentos, cuyos resultados se encuentran documentados en el Anexo. Esta planificación ha permitido realizar una evaluación comparativa rigurosa entre los modelos y las distintas configuraciones de entrenamiento, aportando claridad sobre qué estrategias funcionaban mejor en cada arquitectura.

Desde el punto de vista personal, el proyecto ha sido altamente enriquecedor. Ha supuesto una curva de aprendizaje intensa, tanto en el plano teórico como en el práctico, en áreas como las CNNs, el entorno TensorFlow/Keras, el preprocesamiento de imágenes, y las técnicas modernas de entrenamiento con recursos computacionales limitados.

En conclusión, este trabajo ha demostrado que es posible alcanzar resultados competitivos incluso con conjuntos de datos reducidos, siempre que se apliquen estrategias adecuadas. Además, se ha cumplido el objetivo formativo y exploratorio del proyecto, consolidando un conocimiento sólido que servirá como base para futuros proyectos o estudios en el campo del aprendizaje profundo.

## 15 Trabajos Futuros

Este proyecto sienta una base sólida para múltiples líneas de mejora y profundización. Una de las direcciones más relevantes sería incrementar la cantidad de datos disponibles, ya sea mediante la recopilación de nuevas imágenes o utilizando técnicas de data augmentation más avanzadas como GANs (Generative Adversarial Networks). También sería de gran interés incorporar técnicas de interpretación de modelos, como Grad-CAM o saliency maps, con el fin de identificar visualmente qué regiones de las imágenes están influyendo en la decisión del modelo, especialmente en un contexto médico donde la interpretabilidad es crucial. Otra línea prometedora consistiría en ajustar y comparar más arquitecturas modernas o incluso probar modelos vision transformers (ViT), que están demostrando buenos resultados en visión por computador. Finalmente, una mejora en la infraestructura computacional permitiría entrenar modelos más pesados o realizar fine tuning en mayor profundidad, algo que en este proyecto estuvo limitado por las restricciones de hardware. Estas extensiones no solo mejorarían el rendimiento del sistema, sino que aportarían mayor robustez, comprensión y aplicabilidad al problema tratado.

## 16 Agradecimientos

Quisiera expresar mi más sincero agradecimiento a mi tutor de Trabajo de Fin de Grado, Jordi Delgado Pin, por haber confiado en mí desde el inicio, aceptando la propuesta de este proyecto a pesar de que partía sin experiencia previa en el área de redes neuronales convolucionales. Su orientación fue clave desde las primeras etapas, proporcionándome bibliografía fundamental para iniciarme en el uso de Keras, TensorFlow y en la comprensión teórica de las CNNs.

A lo largo de todo el desarrollo del proyecto, su labor de supervisión ha sido constante y cercana, ofreciéndome apoyo en los momentos en los que me encontraba estancado, ya fuera sugiriéndome soluciones técnicas, proponiendo caminos alternativos o ayudándome a reformular ciertas ideas. Su disponibilidad, seguimiento regular y comentarios constructivos han contribuido decisivamente a que este trabajo haya podido completarse con éxito.

# 17 Glosario

**Bazo:** Órgano que forma parte de los sistemas linfático y sanguíneo.

**Biopsia:** Extracción de células o tejidos para que un patólogo los examine al microscopio o realice otras pruebas.

**Enfermedad oncológica:** Esta es una enfermedad que se produce cuando un grupo de células normales del organismo evolucionan de forma anómala generándose un bulto o tumor.

**Glóbulos blancos:** Son parte del sistema inmunitario del cuerpo y ayudan a combatir infecciones y otras enfermedades.

**Hematólogo:** Especialista que analiza las principales características y comportamientos de los componentes de la sangre.

**Hematoxilina/Eosina (H&E):** La hematoxilina tiñe de violeta azulado intenso los ribosomas, la cromatina (material genético) dentro del núcleo y otras estructuras. La eosina tiñe de rosa anaranjado o rosado el citoplasma, el colágeno, el tejido conjuntivo y otras estructuras que rodean y sostienen la célula.

**Heterogeneidad:** Combinación, mezcla o composición de elementos de diversa naturaleza en un todo o conjunto.

**Imágenes histológicas:** Imágenes obtenidas a partir de muestras de tejido humano.

**Linfocitos:** Son células del sistema inmunitario.

**Médula ósea:** Tejido suave y esponjoso, con muchos vasos sanguíneos, que se encuentra en el centro de la mayoría de los huesos.

**Oncólogo:** Profesional de la medicina que se especializa en el diagnóstico y tratamiento del cáncer.

**Patólogo:** Profesional de la medicina que se especializa en el diagnóstico de enfermedades mediante el estudio de células o tejidos con un microscopio.

**Pooling:** Es una operación que permite analizar el contenido de una imagen por regiones (o bloques) para extraer la información más representativa de las mismas.

**Proceso de tinción:** Inmersión de la muestra (antes o después de la fijación y el montaje) en la solución colorante, seguido del aclarado que es un lavado para eliminar el exceso de colorante y la observación.

**Sistema inmunitario:** Red compleja de células, tejidos, órganos y las sustancias que estos producen, y que ayudan al cuerpo a combatir infecciones y otras enfermedades.

## 18 Bibliografía

- [1] «Linfoma | ¿Qué es el Linfoma? | PortalCLÍNICA», Clínic Barcelona. Accedido: 22 de febrero de 2025. [En línea]. Disponible en: <https://www.clinicbarcelona.org/asistencia/enfermedades/linfoma>
- [2] «Qué es el linfoma, el cáncer de la sangre más común en adultos (y cómo detectarlo)», BBC News Mundo. Accedido: 22 de febrero de 2025. [En línea]. Disponible en: <https://www.bbc.com/mundo/noticias-58523986>
- [3] «Tipos de Linfomas», gpp. Accedido: 23 de febrero de 2025. [En línea]. Disponible en: <https://rochepacientes.es/cancer/linfoma/tipos.html>
- [4] «NCI Search Results - NCI». Accedido: 23 de febrero de 2025. [En línea]. Disponible en: <https://www.cancer.gov/search/results>
- [5] «Tratamiento del linfoma no Hodgkin». Accedido: 24 de febrero de 2025. [En línea]. Disponible en: <https://www.cancer.gov/espanol/tipos/linfoma/paciente/tratamiento-lnh-adultos-pdq>
- [6] «Linfoma Australia - Linfoma de células del manto (MCL)», Lymphoma Australia. Accedido: 24 de febrero de 2025. [En línea]. Disponible en: <https://es.lymphoma.org.au/tipos-de-linfoma/no-linfoma-de-Hodgkin/nhl-agresivo-de-c%C3%A9lulas-b-de-crecimiento-r%C3%A1pido/linfoma-de-c%C3%A9lulas-del-manto-mcl/>
- [7] B. Zhang, H. Shi, y H. Wang, «Machine Learning and AI in Cancer Prognosis, Prediction, and Treatment Selection: A Critical Approach», J. Multidiscip. Healthc., vol. 16, pp. 1779-1791, jun. 2023, doi: 10.2147/JMDH.S410301.
- [8] «Red neuronal artificial», Wikipedia, la enciclopedia libre. 24 de febrero de 2025. Accedido: 24 de febrero de 2025. [En línea]. Disponible en: [https://es.wikipedia.org/w/index.php?title=Red\\_neuronal\\_artificial&oldid=165657141](https://es.wikipedia.org/w/index.php?title=Red_neuronal_artificial&oldid=165657141)
- [9] «¿Qué son las redes neuronales convolucionales? | IBM». Accedido: 24 de febrero de 2025. [En línea]. Disponible en: <https://www.ibm.com/mx-es/think/topics/convolutional-neural-networks>
- [10] J. Zhang, W. Cui, X. Guo, B. Wang, y Z. Wang, «Classification of digital pathological images of non-Hodgkin's lymphoma subtypes based on the fusion of transfer learning and principal component analysis», Med. Phys., vol. 47, n.º 9, pp. 4241-4253, 2020, doi: 10.1002/mp.14357.
- [11] N. Brancati, G. De Pietro, M. Frucci, y D. Riccio, «A Deep Learning Approach for Breast Invasive Ductal Carcinoma Detection and Lymphoma Multi-Classification in Histological Images», IEEE Access, vol. 7, pp. 44709-44720, 2019, doi: 10.1109/ACCESS.2019.2908724.
- [12] «Búsqueda de empleo en Glassdoor», Glassdoor. Accedido: 11 de marzo de 2025. [En línea]. Disponible en: <https://www.glassdoor.es/index.htm>
- [13] Linfoma no hodgkiniano: Vídeo, Anatomía & Definición | Osmosis. Accedido: 14 de marzo de 2025. [En línea Video]. Disponible en: [https://www.osmosis.org/learn/es/Non-Hodgkin\\_lymphoma](https://www.osmosis.org/learn/es/Non-Hodgkin_lymphoma)

- [14] «Leucemia linfoide crónica». Accedido: 14 de marzo de 2025. [En línea]. Disponible en: <https://www.fesemi.org/informacion-pacientes/conozca-mejor-su-enfermedad/leucemia-linfoide-cronica>
- [15] Ingoh, «Linfoma Folicular (LF): um tipo indolente de linfoma não-Hodgkin de células B.», INGOH. Accedido: 14 de marzo de 2025. [En línea]. Disponible en: <https://ingoh.com.br/linfoma-folicular-lf-um-tipo-indolente-de-linfoma-nao-hodgkin-de-celulas-b/>
- [16] M. Dreyling *et al.*, «Newly diagnosed and relapsed mantle cell lymphoma: ESMO Clinical Practice Guidelines for diagnosis, treatment and follow-up †», *Ann. Oncol.*, vol. 25, pp. iii83-iii92, sep. 2014, doi: 10.1093/annonc/mdu264.
- [17] «Deep Learning». Accedido: 14 de marzo de 2025. [En línea]. Disponible en: <https://apmonitor.com/do/index.php/Main/DeepLearning>
- [18] «Neural network types», Questions and Answers in MRI. Accedido: 14 de marzo de 2025. [En línea]. Disponible en: <http://mriquestions.com/deep-network-types.html>
- [19] «Aprendizaje profundo», *Wikipedia, la enciclopedia libre*. 24 de febrero de 2025. Accedido: 14 de marzo de 2025. [En línea]. Disponible en: [https://es.wikipedia.org/w/index.php?title=Aprendizaje\\_profundo&oldid=165647625](https://es.wikipedia.org/w/index.php?title=Aprendizaje_profundo&oldid=165647625)
- [20] J. Carreras, R. Hamoudi, y N. Nakamura, «Artificial intelligence and classification of mature lymphoid neoplasms», *Explor. Target. Anti-Tumor Ther.*, vol. 5, n.º 2, pp. 332-348, 2024, doi: 10.37349/etat.2024.00221.
- [21] «Definition of mantle cell lymphoma - NCI Dictionary of Cancer Terms - NCI». Accedido: 24 de febrero de 2025. [En línea]. Disponible en: <https://www.cancer.gov/publications/dictionaries/cancer-terms/def/mantle-cell-lymphoma>
- [22] S. Brondfield, N. Bochatay, y C. Perlis, «Developing a Community for Patients With Cancer Through Longer-Term Art Therapy», *JCO Oncol. Pract.*, vol. 17, n.º 4, pp. e506-e516, abr. 2021, doi: 10.1200/OP.20.00419.
- [23] C. Laurent *et al.*, «Impact of Expert Pathologic Review of Lymphoma Diagnosis: Study of Patients From the French Lymphopath Network», *J. Clin. Oncol.*, vol. 35, n.º 18, pp. 2008-2017, jun. 2017, doi: 10.1200/JCO.2016.71.2083.
- [24] R. S. Robetorye *et al.*, «Incorporation of digital gene expression profiling for cell-of-origin determination (Lymph2Cx testing) into the routine work-up of diffuse large B cell lymphoma», *J. Hematop.*, vol. 12, n.º 1, pp. 3-10, mar. 2019, doi: 10.1007/s12308-019-00344-0.
- [25] «Linfoma», Cancer Research Institute. Accedido: 22 de febrero de 2025. [En línea]. Disponible en: <https://www.cancerresearch.org/es/cancer-types/lymphoma>
- [26] R. S. of N. A. (RSNA) and A. C. of Radiology (ACR), «lymphoma», Radiologyinfo.org. Accedido: 22 de febrero de 2025. [En línea]. Disponible en: <https://www.radiologyinfo.org/es/info/lymphoma>
- [27] «¿Qué es el deep learning? | IBM». Accedido: 14 de marzo de 2025. [En línea]. Disponible en: <https://www.ibm.com/es-es/topics/deep-learning>



- [28] BBVA, «Te contamos qué es el “machine learning” y cómo funciona», BBVA NOTICIAS. Accedido: 24 de febrero de 2025. [En línea]. Disponible en: <https://www.bbva.com/es/innovacion/machine-learning-que-es-y-como-funciona/>
- [29] SensIO, *Redes Neuronales Convolucionales - Arquitecturas*, (2020). Accedido: 3 de marzo de 2025. [En línea Video]. Disponible en: <https://www.youtube.com/watch?v=IKB8rzGU8Wo>
- [30] Nicholas Renotte, *Tensorflow Tutorial for Python in 10 Minutes*, (2020). Accedido: 1 de marzo de 2025. [En línea Video]. Disponible en: [https://www.youtube.com/watch?v=6\\_2hzRopPbQ](https://www.youtube.com/watch?v=6_2hzRopPbQ)
- [31] When Maths Meet Coding, *Train Neural Network by loading your images |TensorFlow, CNN, Keras tutorial*, (2020). Accedido: 1 de marzo de 2025. [En línea Video]. Disponible en: [https://www.youtube.com/watch?v=uqomO\\_BZ44g](https://www.youtube.com/watch?v=uqomO_BZ44g)
- [32] «densenet201 - (No recomendado) Red neuronal convolucional DenseNet-201 - MATLAB». Accedido: 11 de junio de 2025. [En línea]. Disponible en: <https://es.mathworks.com/help/deeplearning/ref/densenet201.html>
- [33] «Inceptionv3», Viquipèdia, l'enciclopèdia lliure. 23 de marzo de 2024. Accedido: 29 de mayo de 2025. [En línea]. Disponible en: <https://ca.wikipedia.org/w/index.php?title=Inceptionv3&oldid=33288707>
- [34] «¿Qué es el transfer learning o aprendizaje por transferencia?», UNIR. Accedido: 27 de mayo de 2025. [En línea]. Disponible en: <https://www.unir.net/revista/ingenieria/transfer-learning/>
- [35] «What is Feature Extraction? Feature Extraction Techniques Explained». Accedido: 1 de junio de 2025. [En línea]. Disponible en: <https://domino.ai/data-science-dictionary/feature-extraction>
- [36] zalando research/fashion-mnist. (11 de junio de 2025). Python. Zalando Research. Accedido: 11 de junio de 2025. [En línea]. Disponible en: <https://github.com/zalando research/fashion-mnist>
- [37] Greg Hogg, How to Code the AlexNet Convolutional Neural Network Architecture from Scratch in TensorFlow / Keras, (2022). Accedido: 22 de abril de 2025. [En línea Video]. Disponible en: <https://www.youtube.com/watch?v=c2kKFSkAF10>
- [38] KGP Talkie, How to Install TensorFlow GPU on Windows 11 | GPU Setup on Windows | TensorFlow GPU Setup, (2023). Accedido: 16 de abril de 2025. [En línea Video]. Disponible en: <https://www.youtube.com/watch?v=QUjtDlalh0k>
- [39] R. [Admin Jarroba, «¿Qué debemos saber para instalar correctamente TensorFlow con GPU?», Jarroba. Accedido: 16 de abril de 2025. [En línea]. Disponible en: <https://jarroba.com/que-debemos-saber-para-instalar-correctamente-tensorflow-con-gpu/>
- [40] Welch Labs, The moment we stopped understanding AI [AlexNet], (1 de julio de 2024). Accedido: 16 de abril de 2025. [En línea Video]. Disponible en: <https://www.youtube.com/watch?v=UZDiGooFs54>
- [41] «Architecture of EfficientNetV2B0 model.», ResearchGate. Accedido: 11 de junio de 2025. [En línea]. Disponible en: [https://www.researchgate.net/figure/Architecture-of-EfficientNetV2B0-model\\_fig5\\_389466676](https://www.researchgate.net/figure/Architecture-of-EfficientNetV2B0-model_fig5_389466676)

- [42] «Figure 3. Detailed architecture of an Efficient DenseNet-201.», ResearchGate. Accedido: 11 de junio de 2025. [En línea]. Disponible en: [https://www.researchgate.net/figure/Detailed-architecture-of-an-Efficient-DenseNet-201\\_fig3\\_360070131](https://www.researchgate.net/figure/Detailed-architecture-of-an-Efficient-DenseNet-201_fig3_360070131)
- [43] «Illustration of ResNet-50V2 network architecture.», ResearchGate. Accedido: 11 de junio de 2025. [En línea]. Disponible en: [https://www.researchgate.net/figure/Illustration-of-ResNet-50V2-network-architecture\\_fig5\\_354550914](https://www.researchgate.net/figure/Illustration-of-ResNet-50V2-network-architecture_fig5_354550914)
- [44] «inceptionv3onc--oview\_vjAbOfw.png (1000×389)». Accedido: 29 de mayo de 2025. [En línea]. Disponible en: [https://production-media.paperswithcode.com/methods/inceptionv3onc--oview\\_vjAbOfw.png](https://production-media.paperswithcode.com/methods/inceptionv3onc--oview_vjAbOfw.png)
- [45] «Understanding Transfer Learning and Fine Tuning for Face Recognition». Accedido: 1 de junio de 2025. [En línea]. Disponible en: <https://www.linkedin.com/pulse/understanding-transfer-learning-fine-tuning-face-rahul-bairwa>
- [46] M. ul Hassan, «VGG16 - Convolutional Network for Classification and Detection». Accedido: 11 de junio de 2025. [En línea]. Disponible en: <https://neurohive.io/en/popular-networks/vgg16/>
- [47] Futurology — An Optimistic Future, Convolutional Neural Networks Explained (CNN Visualized), (2020). Accedido: 28 de abril de 2025. [En línea Video]. Disponible en: <https://www.youtube.com/watch?v=pj9-rr1wDhM>
- [48] codebasics, Data augmentation to address overfitting | Deep Learning Tutorial 26 (Tensorflow, Keras & Python), (1 de noviembre de 2020). Accedido: 21 de mayo de 2025. [En línea Video]. Disponible en: <https://www.youtube.com/watch?v=mTVf7BN7S8w>
- [49] Eran Feit, Inception V3 Practical Implementation | Classify more than 500 Birds Species (!!!), (5 de marzo de 2024). Accedido: 28 de abril de 2025. [En línea Video]. Disponible en: [https://www.youtube.com/watch?v=d\\_JB9GA2U\\_c](https://www.youtube.com/watch?v=d_JB9GA2U_c)
- [50] K. Team, «Keras documentation: Transfer learning & fine-tuning». Accedido: 5 de mayo de 2025. [En línea]. Disponible en: [https://keras.io/guides/transfer\\_learning/](https://keras.io/guides/transfer_learning/)
- [51] SensIO, Redes Neuronales Convolucionales - Transfer Learning, (2020). Accedido: 28 de abril de 2025. [En línea Video]. Disponible en: <https://www.youtube.com/watch?v=4NgPVGt67Es>
- [52] «Transfer learning and fine-tuning | TensorFlow Core». Accedido: 10 de mayo de 2025. [En línea]. Disponible en: [https://www.tensorflow.org/tutorials/images/transfer\\_learning](https://www.tensorflow.org/tutorials/images/transfer_learning)
- [53] Neuralearn, Understanding Transfer Learning - Feature Extraction and Finetuning - Human Emotions Detection, (12 de noviembre de 2022). Accedido: 5 de mayo de 2025. [En línea Video]. Disponible en: <https://www.youtube.com/watch?v=1ENyc24yOYs>
- [54] Tech With Tim, Python Virtual Environments - Full Tutorial for Beginners, (16 de marzo de 2024). Accedido: 20 de marzo de 2025. [En línea Video]. Disponible en: <https://www.youtube.com/watch?v=Y21OR1OPC9A>
- [55] Niam Yaraghi, Train Neural Networks to Classify Objects, (30 de marzo de 2022). Accedido: 23 de

marzo de 2025. [En línea Video]. Disponible en: <https://www.youtube.com/watch?v=GEq3VerUm7o>

- [56] Aftab5550, Aftab5550/TFG. (15 de junio de 2025). Jupyter Notebook. Accedido: 20 de junio de 2025. [En línea]. Disponible en: <https://github.com/Aftab5550/TFG>
- [57] E. Daehnhardt, «TensorFlow: Transfer Learning (Fine-Tuning) in Image Classification», Elena's AI and Python Coding Blog, Living with AI daehnhardt.com. Accedido: 1 de junio de 2025. [En línea]. Disponible en: <https://daehnhardt.com/blog/2022/04/06/tensorflow-transfer-learning-image-classification-fine-tuning-data-augmentation-predictive-modeling-image-classification/>
- [58] N. Sharma, «What is MobileNetV2? Features, Architecture, Application and More», Analytics Vidhya. Accedido: 20 de junio de 2025. [En línea]. Disponible en: <https://www.analyticsvidhya.com/blog/2023/12/what-is-mobilenetv2/>

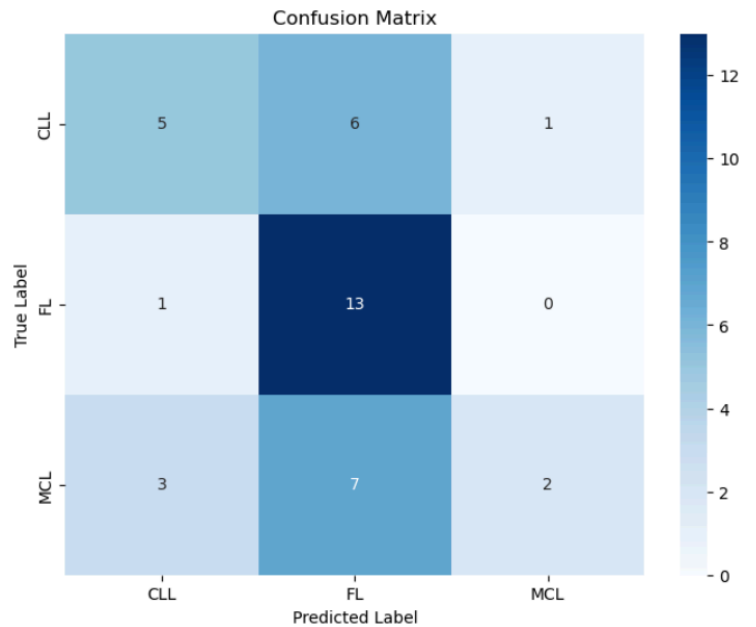
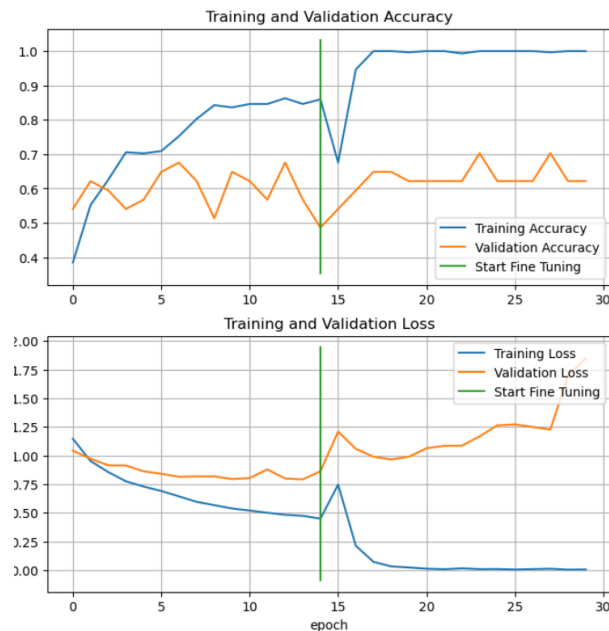
## 19 Anexos

A continuació se presenten los resultados obtenidos en los distintos experimentos realizados durante el desarrollo del Trabajo de Fin de Grado. Para cada uno de los experimentos se incluyen cuatro representaciones gráficas o salidas relevantes que permiten evaluar el rendimiento del modelo:

- **Curvas de precisión y pérdida:** La primera figura muestra la evolución del accuracy y la loss tanto en entrenamiento como en validación a lo largo de todas las épocas (combinando la fase de Feature Extraction y la de Fine Tuning). Se incluye una línea vertical que marca el inicio del Fine Tuning, permitiendo visualizar el posible impacto de esta fase en el aprendizaje del modelo.
- **Matriz de confusión:** La segunda figura corresponde a la matriz de confusión del modelo, la cual permite analizar visualmente los aciertos y errores de clasificación para cada una de las clases. Se presenta como un mapa de calor etiquetado, con las clases verdaderas en el eje vertical y las clases predichas en el eje horizontal.
- **Informe de clasificación:** La tercera salida es el classification report, que ofrece métricas detalladas para cada clase, incluyendo precisión (precision), exhaustividad (recall), puntuación F1 (F1-score) y soporte (support). Esta información es útil para entender el comportamiento del modelo en cada categoría específica.
- **Curvas ROC multi-clase:** La cuarta figura muestra las curvas ROC para las distintas clases, permitiendo evaluar gráficamente la capacidad de discriminación del modelo. Para cada clase, se presenta su correspondiente área bajo la curva (AUC), siendo un indicador de su rendimiento en clasificación binaria frente a las demás.

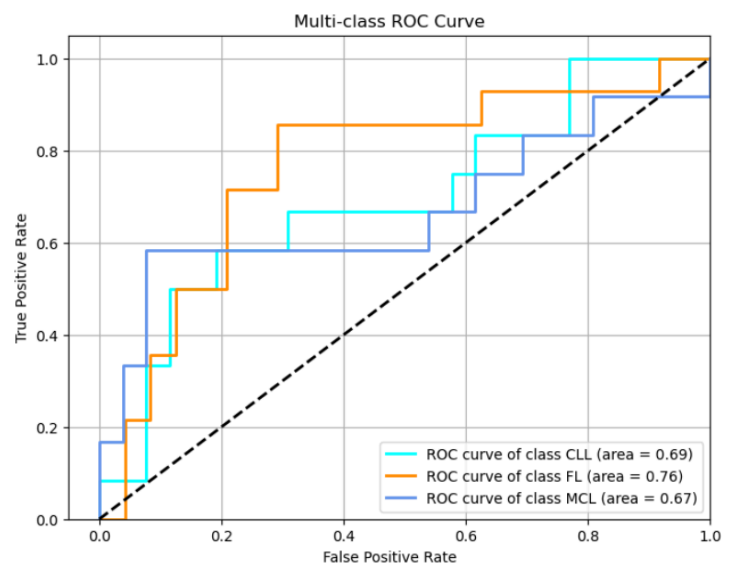
Estas representaciones permiten hacer una evaluación exhaustiva y comparativa del rendimiento de cada configuración de modelo y técnica de entrenamiento utilizada.

# Experimento 1 – InceptionV3 – Sin data augmentation – 15 epochs

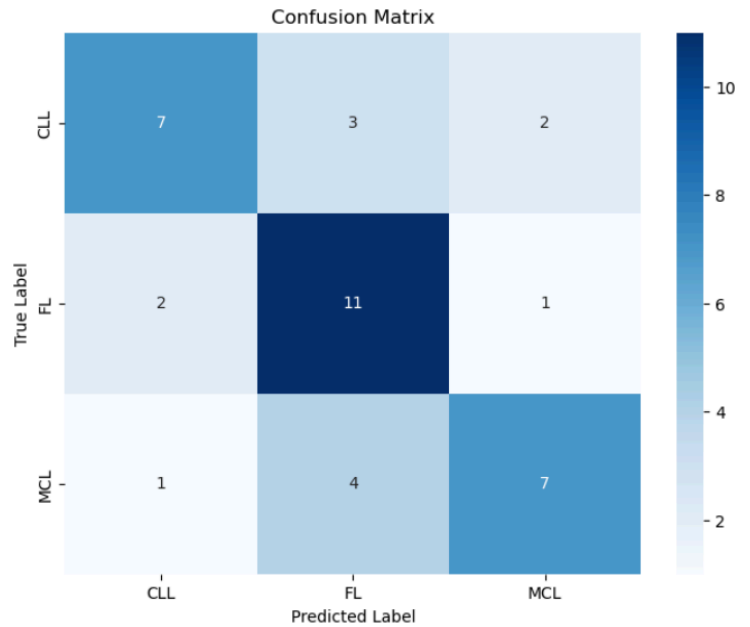
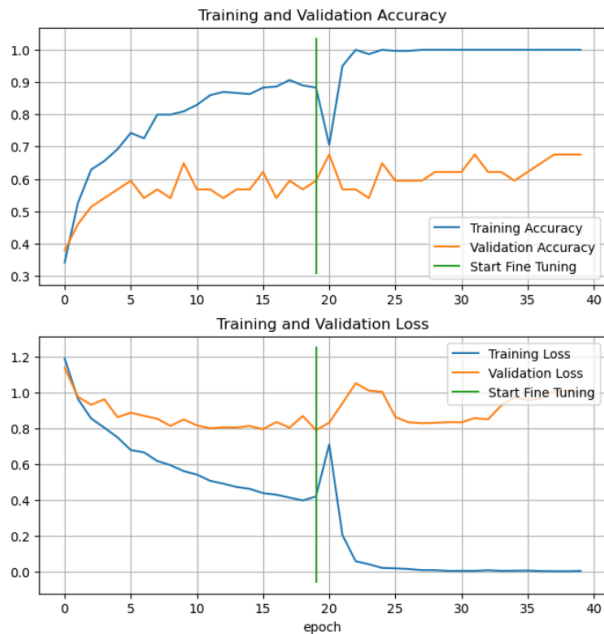


## Classification Report:

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| CLL          | 0.56      | 0.42   | 0.48     | 12      |
| FL           | 0.50      | 0.93   | 0.65     | 14      |
| MCL          | 0.67      | 0.17   | 0.27     | 12      |
| accuracy     |           |        | 0.53     | 38      |
| macro avg    | 0.57      | 0.50   | 0.46     | 38      |
| weighted avg | 0.57      | 0.53   | 0.47     | 38      |

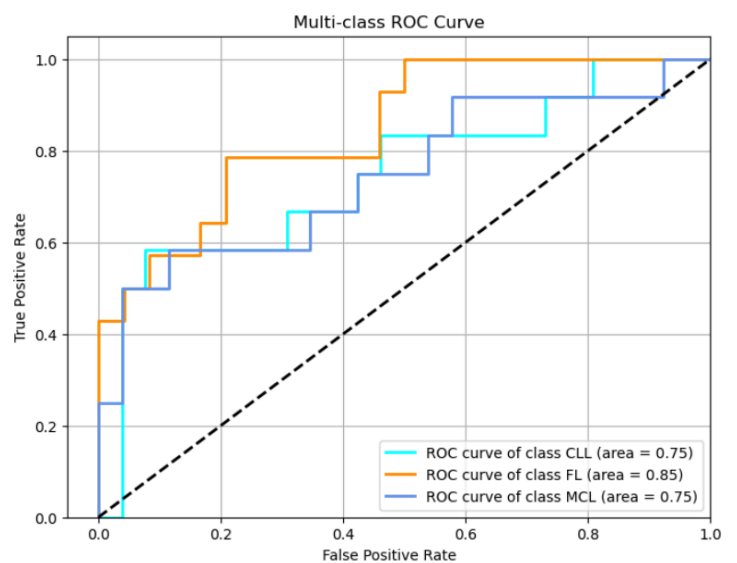


## Experimento 2 – InceptionV3 – Sin data augmentation – 20 epochs

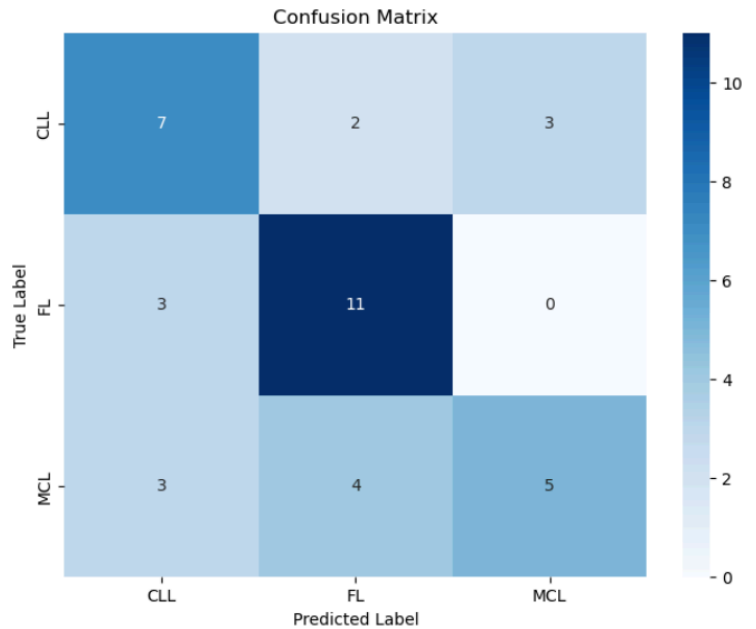
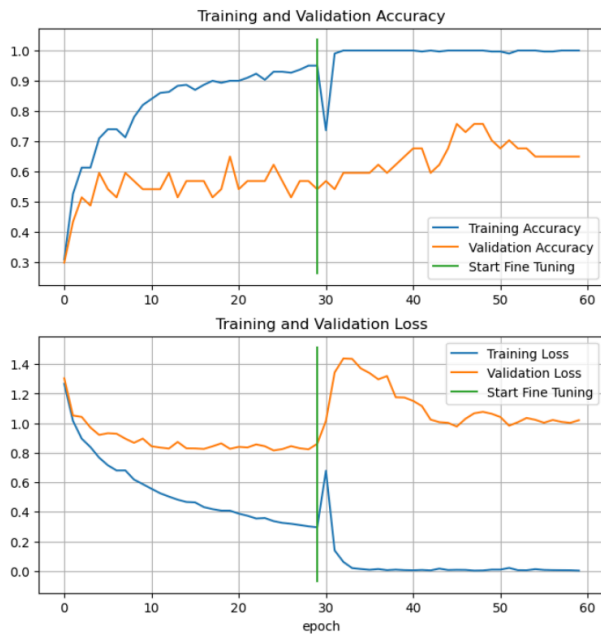


### Classification Report:

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| CLL          | 0.70      | 0.58   | 0.64     | 12      |
| FL           | 0.61      | 0.79   | 0.69     | 14      |
| MCL          | 0.70      | 0.58   | 0.64     | 12      |
| accuracy     |           |        | 0.66     | 38      |
| macro avg    | 0.67      | 0.65   | 0.65     | 38      |
| weighted avg | 0.67      | 0.66   | 0.66     | 38      |

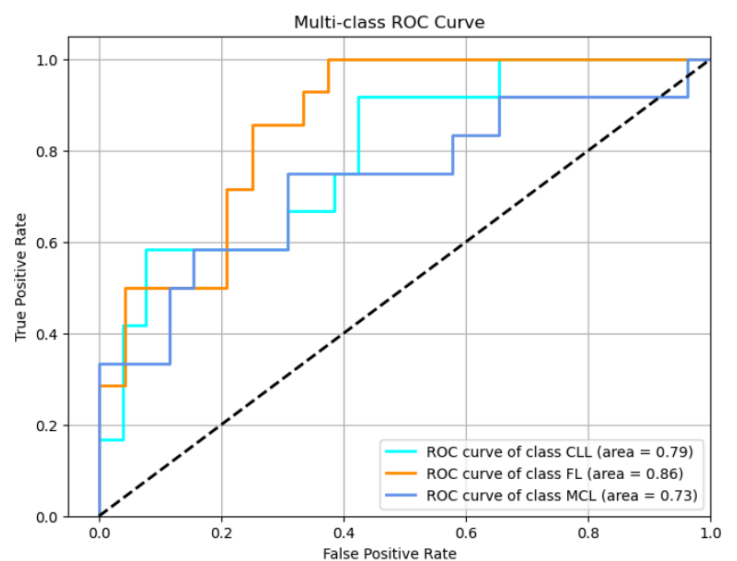


## Experimento 3 – InceptionV3 – Sin data augmentation – 30 epochs



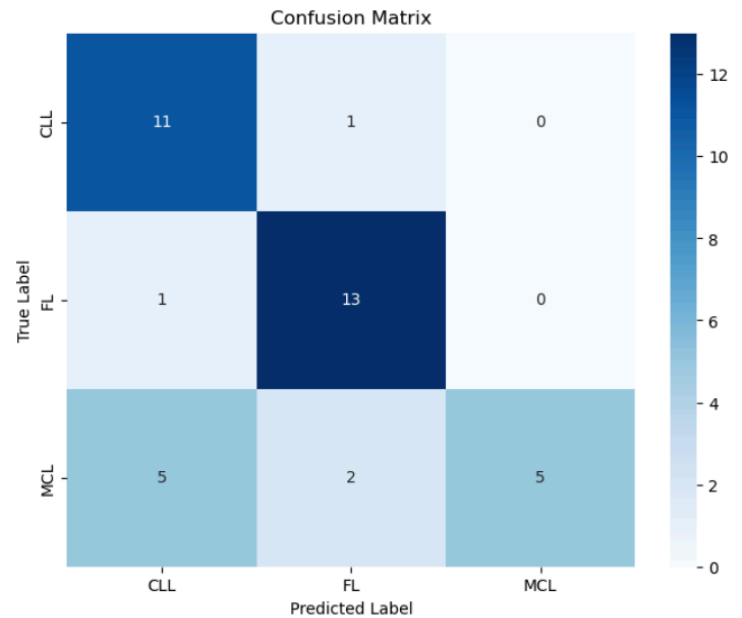
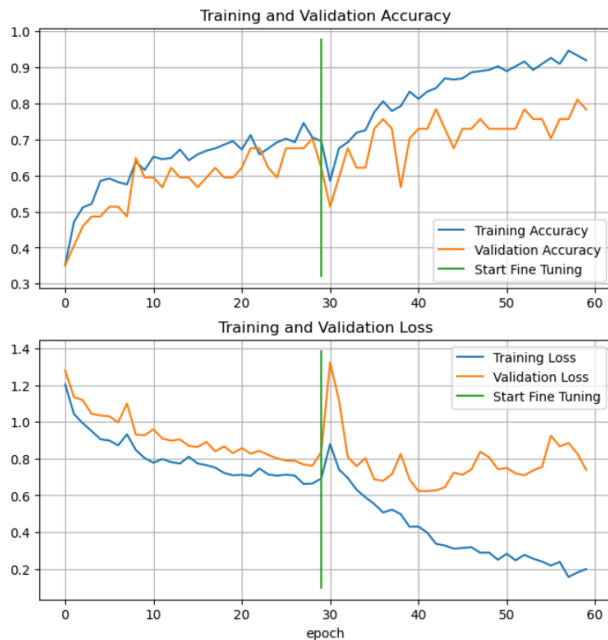
### Classification Report:

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| CLL          | 0.54      | 0.58   | 0.56     | 12      |
| FL           | 0.65      | 0.79   | 0.71     | 14      |
| MCL          | 0.62      | 0.42   | 0.50     | 12      |
| accuracy     |           |        | 0.61     | 38      |
| macro avg    | 0.60      | 0.60   | 0.59     | 38      |
| weighted avg | 0.61      | 0.61   | 0.60     | 38      |



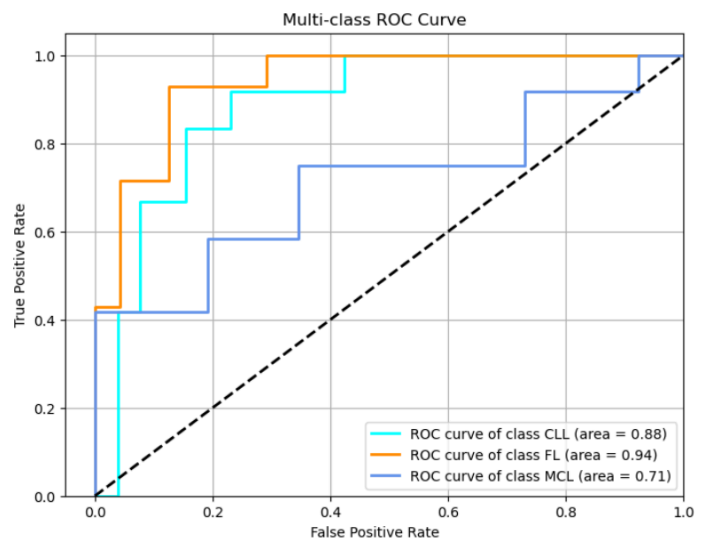


## Experimento 4 – InceptionV3 – Augmentación implícita – 30 epochs



### Classification Report:

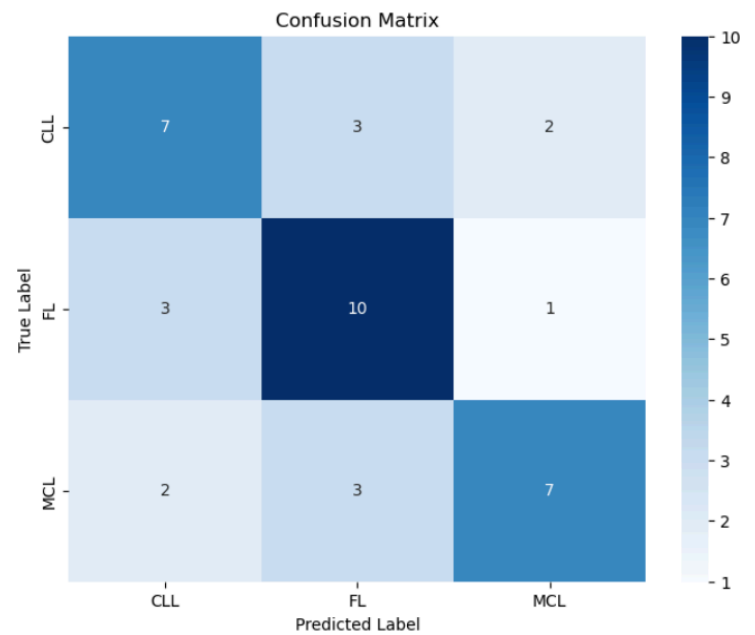
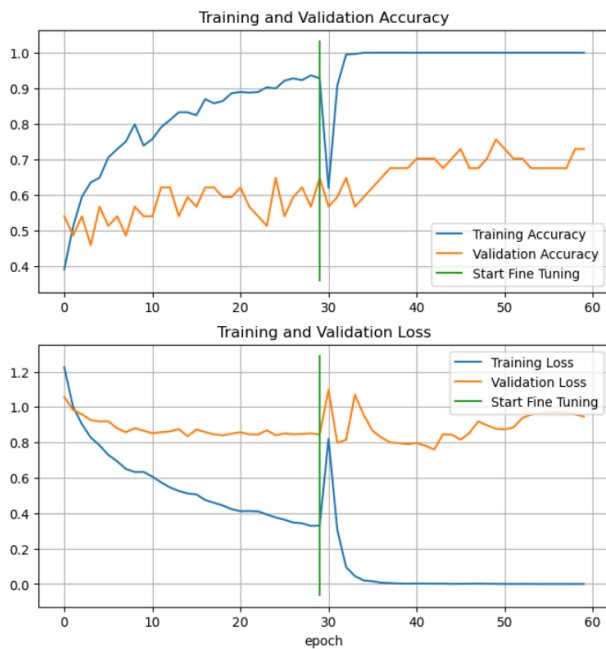
|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| CLL          | 0.65      | 0.92   | 0.76     | 12      |
| FL           | 0.81      | 0.93   | 0.87     | 14      |
| MCL          | 1.00      | 0.42   | 0.59     | 12      |
| accuracy     |           |        | 0.76     | 38      |
| macro avg    | 0.82      | 0.75   | 0.74     | 38      |
| weighted avg | 0.82      | 0.76   | 0.74     | 38      |



## Experimento 5 – InceptionV3 – Augmentación explícita (x2) – 30 epochs

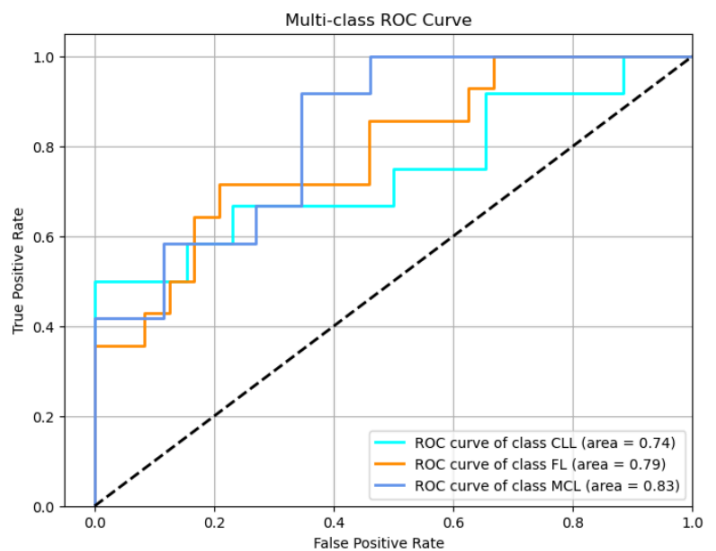
Original training set size: (299, 224, 224, 3)

Augmented training set size: (598, 224, 224, 3)



### Classification Report:

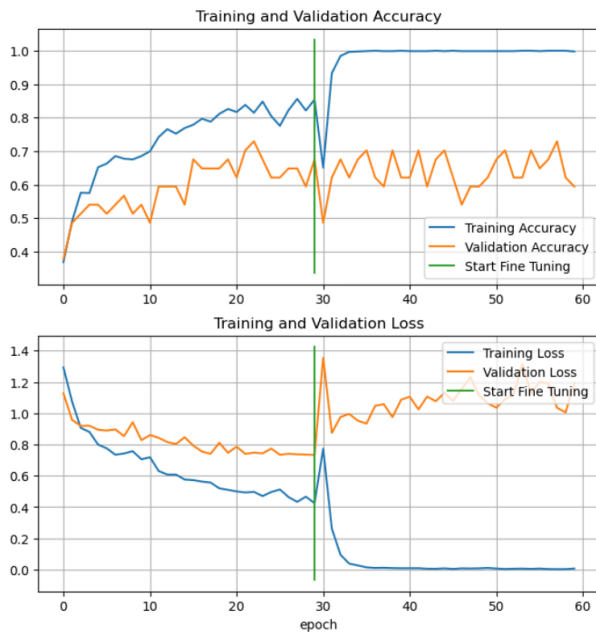
|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| CLL          | 0.58      | 0.58   | 0.58     | 12      |
| FL           | 0.62      | 0.71   | 0.67     | 14      |
| MCL          | 0.70      | 0.58   | 0.64     | 12      |
| accuracy     |           |        | 0.63     | 38      |
| macro avg    | 0.64      | 0.63   | 0.63     | 38      |
| weighted avg | 0.64      | 0.63   | 0.63     | 38      |



## Experimento 6 – InceptionV3 – Augmentación explícita (x3) – 30 epochs

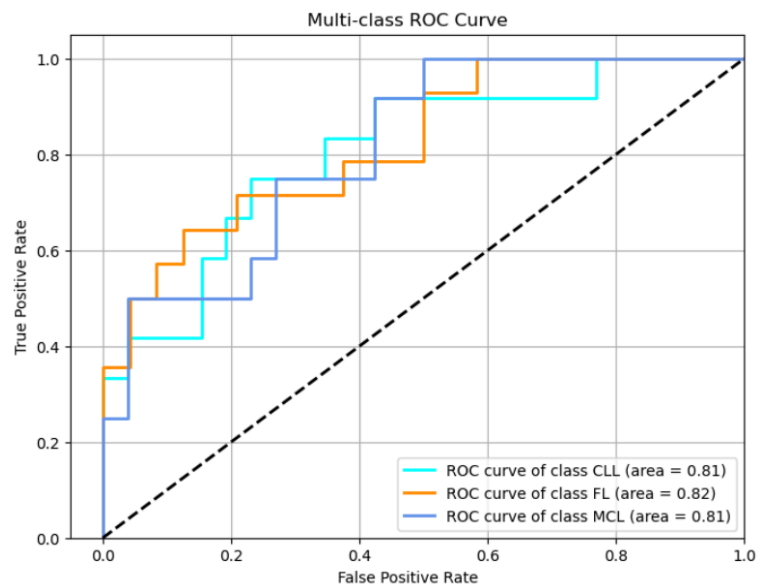
Original training set size: (299, 224, 224, 3)

Augmented training set size: (897, 224, 224, 3)



Classification Report:

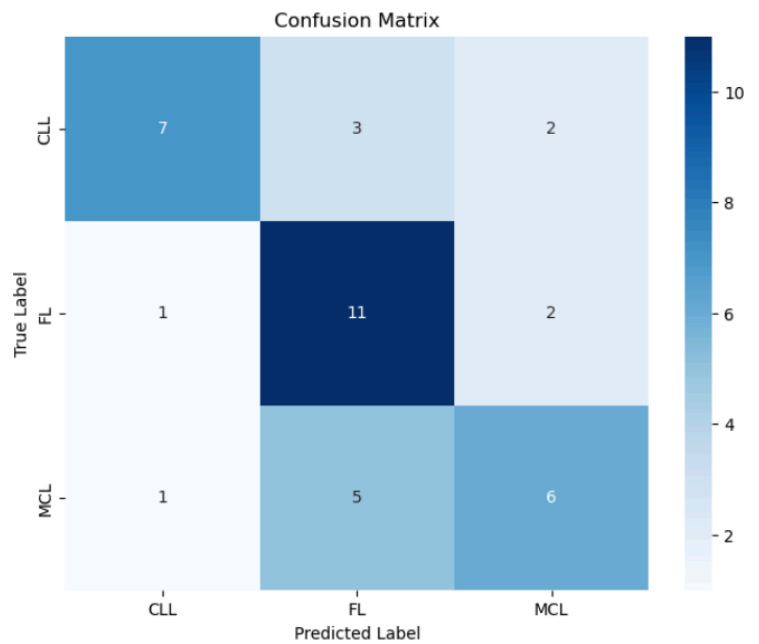
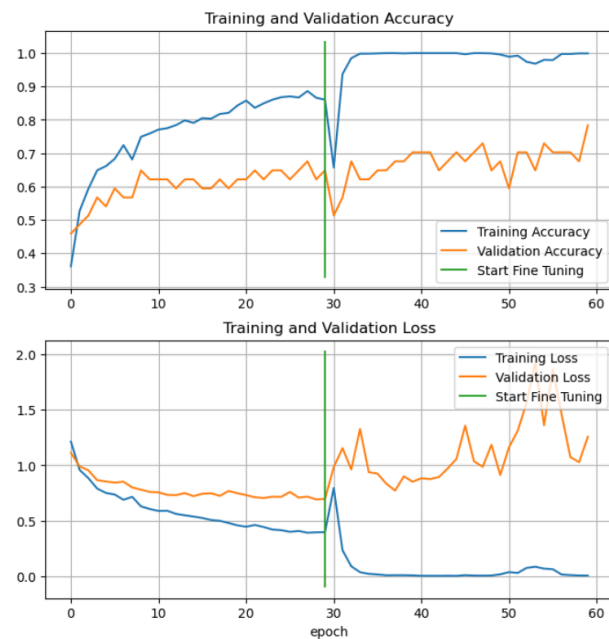
|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| CLL          | 0.56      | 0.75   | 0.64     | 12      |
| FL           | 0.62      | 0.71   | 0.67     | 14      |
| MCL          | 0.83      | 0.42   | 0.56     | 12      |
| accuracy     |           |        | 0.63     | 38      |
| macro avg    | 0.67      | 0.63   | 0.62     | 38      |
| weighted avg | 0.67      | 0.63   | 0.62     | 38      |



## Experimento 7 – InceptionV3 – Augmentación explícita (x4) – 30 epochs

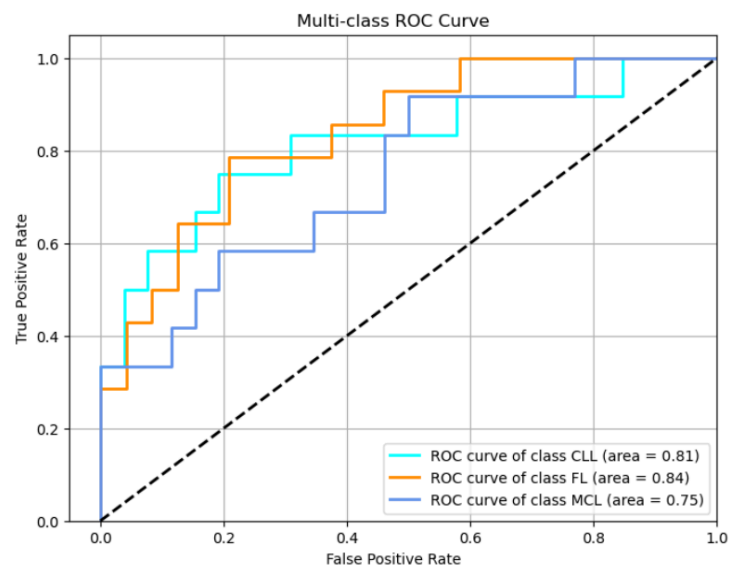
Original training set size: (299, 224, 224, 3)

Augmented training set size: (1196, 224, 224, 3)



Classification Report:

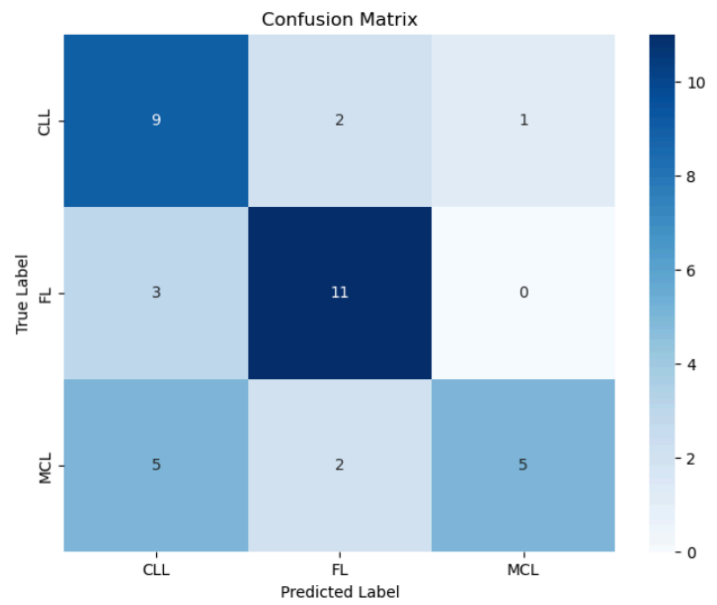
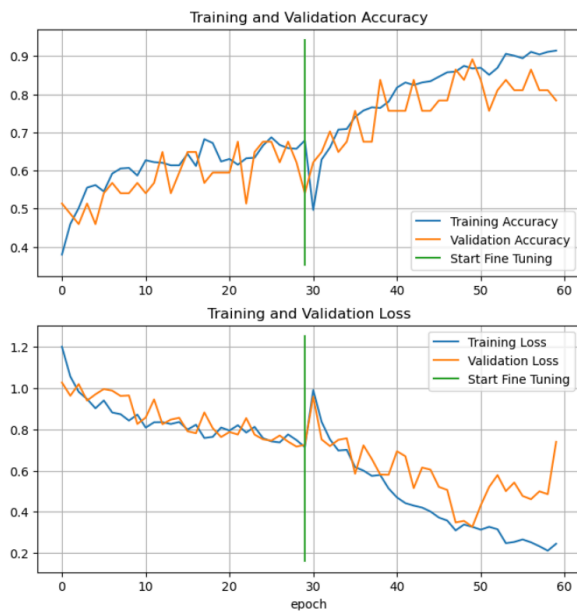
|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| CLL          | 0.78      | 0.58   | 0.67     | 12      |
| FL           | 0.58      | 0.79   | 0.67     | 14      |
| MCL          | 0.60      | 0.50   | 0.55     | 12      |
| accuracy     |           |        | 0.63     | 38      |
| macro avg    | 0.65      | 0.62   | 0.63     | 38      |
| weighted avg | 0.65      | 0.63   | 0.63     | 38      |



## Experimento 8 – InceptionV3 – Augmentación combinada (x2) – 30 epochs

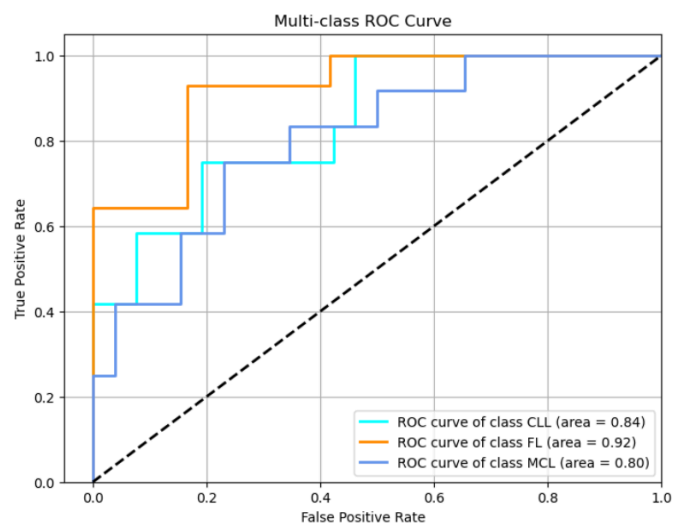
Original training set size: (299, 224, 224, 3)

Augmented training set size: (598, 224, 224, 3)



### Classification Report:

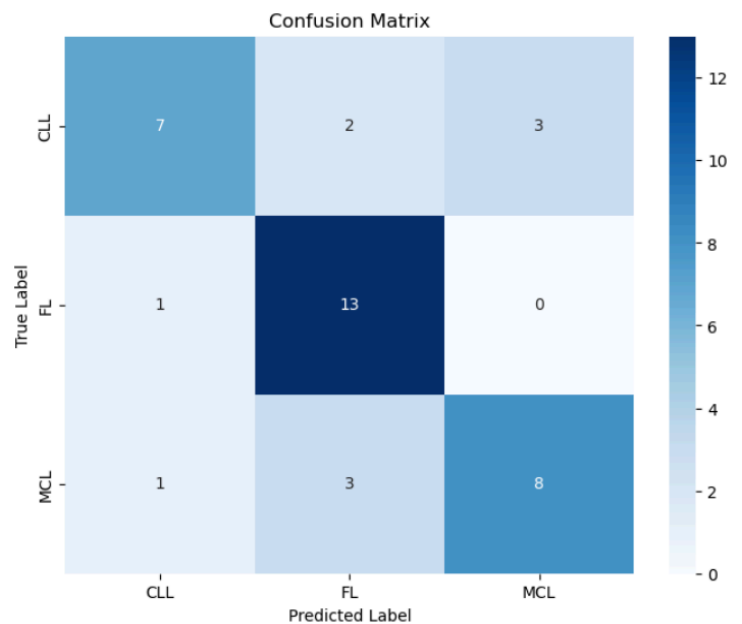
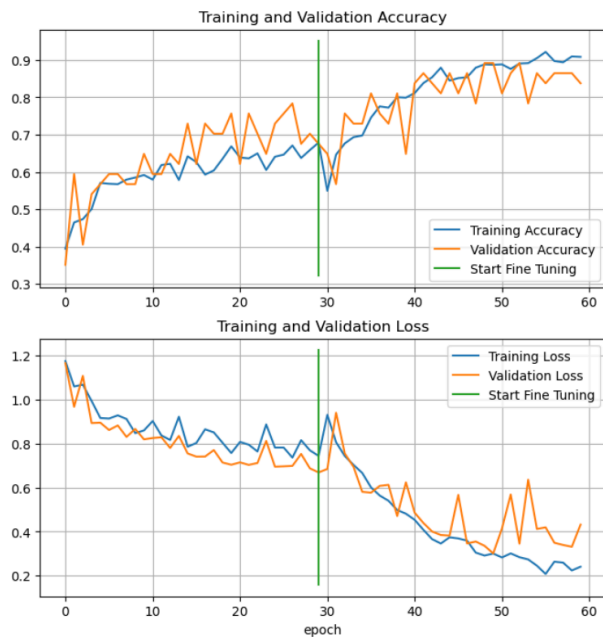
|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| CLL          | 0.53      | 0.75   | 0.62     | 12      |
| FL           | 0.73      | 0.79   | 0.76     | 14      |
| MCL          | 0.83      | 0.42   | 0.56     | 12      |
| accuracy     |           |        | 0.66     | 38      |
| macro avg    | 0.70      | 0.65   | 0.64     | 38      |
| weighted avg | 0.70      | 0.66   | 0.65     | 38      |



# Experimento 9 – InceptionV3 – Augmentación combinada (x3) – 30 epochs

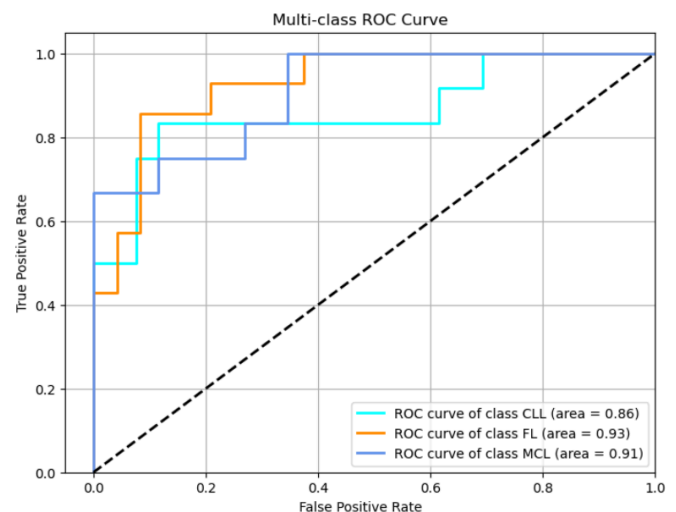
Original training set size: (299, 224, 224, 3)

Augmented training set size: (897, 224, 224, 3)



## Classification Report:

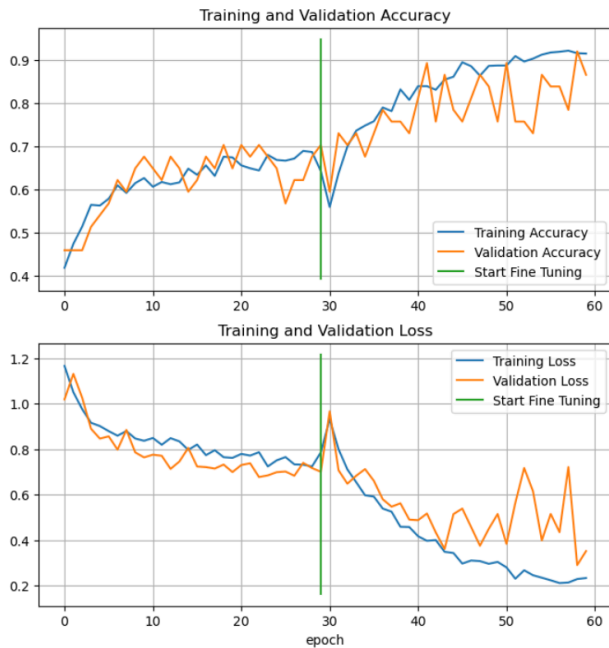
|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| CLL          | 0.78      | 0.58   | 0.67     | 12      |
| FL           | 0.72      | 0.93   | 0.81     | 14      |
| MCL          | 0.73      | 0.67   | 0.70     | 12      |
| accuracy     |           |        | 0.74     | 38      |
| macro avg    | 0.74      | 0.73   | 0.72     | 38      |
| weighted avg | 0.74      | 0.74   | 0.73     | 38      |



# Experimento 10 – InceptionV3 – Augmentación combinada (x4) – 30 epochs

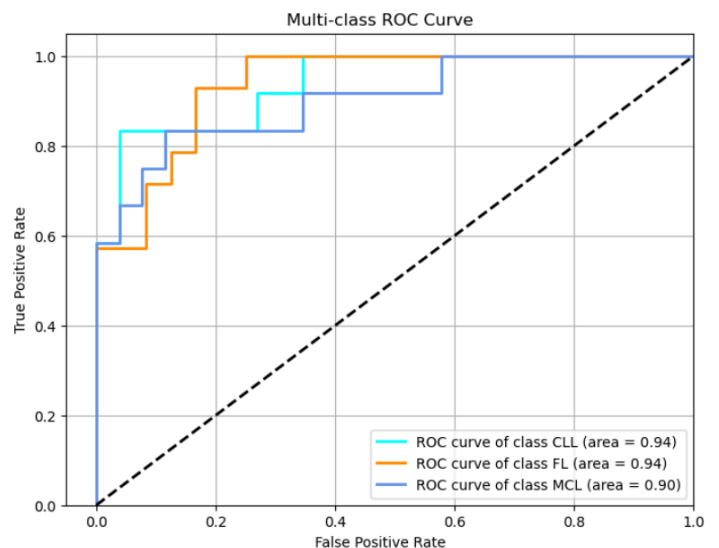
Original training set size: (299, 224, 224, 3)

Augmented training set size: (1196, 224, 224, 3)



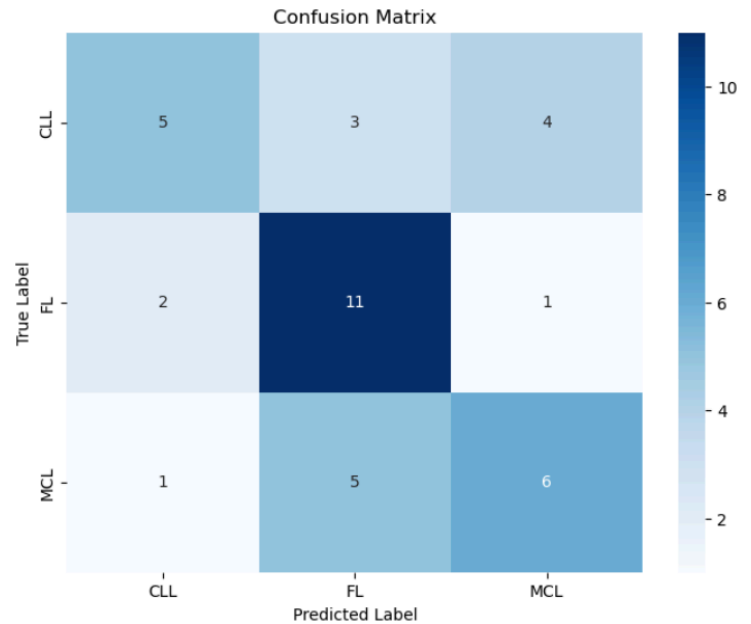
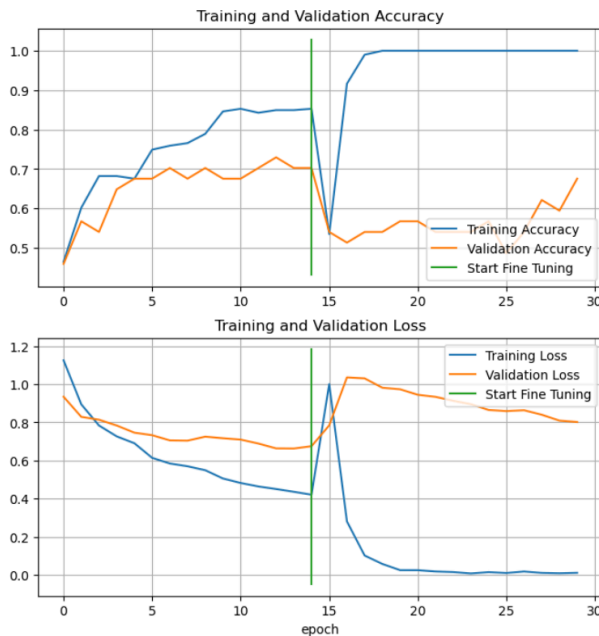
## Classification Report:

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| CLL          | 0.89      | 0.67   | 0.76     | 12      |
| FL           | 0.76      | 0.93   | 0.84     | 14      |
| MCL          | 0.75      | 0.75   | 0.75     | 12      |
| accuracy     |           |        | 0.79     | 38      |
| macro avg    | 0.80      | 0.78   | 0.78     | 38      |
| weighted avg | 0.80      | 0.79   | 0.79     | 38      |



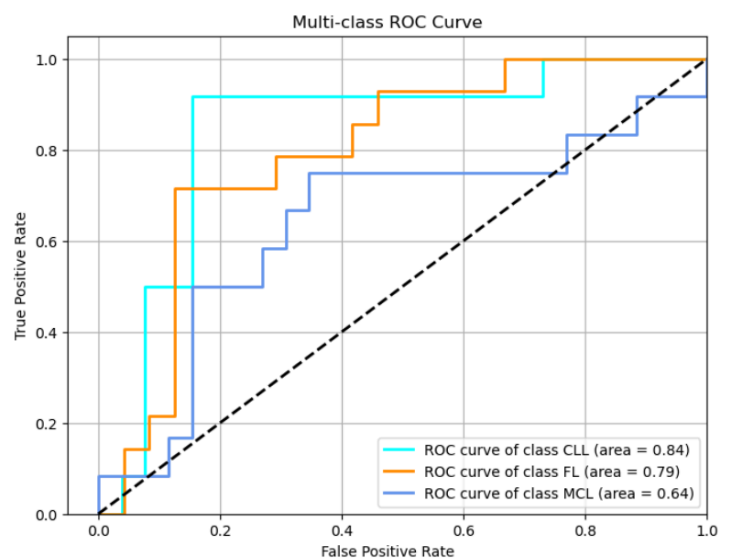


# Experimento 11 – MobileNetV2 – Sin data augmentation – 15 epochs

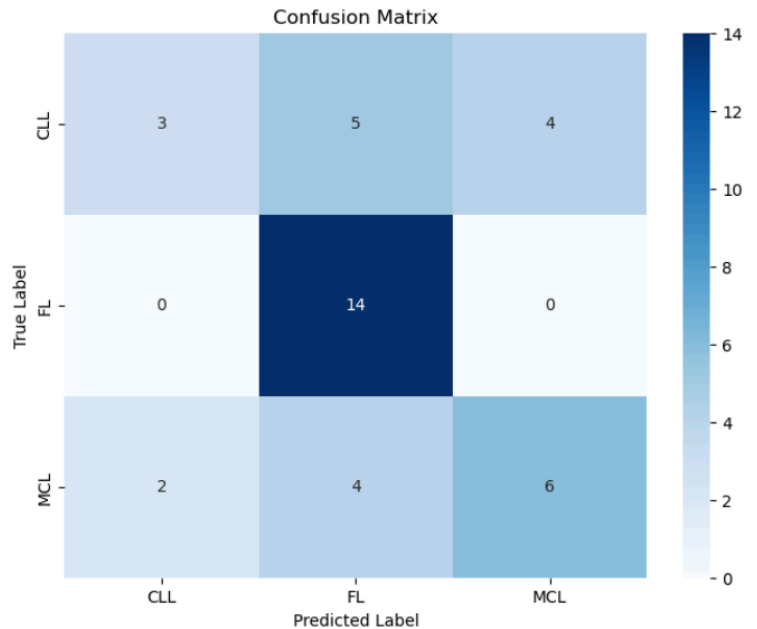
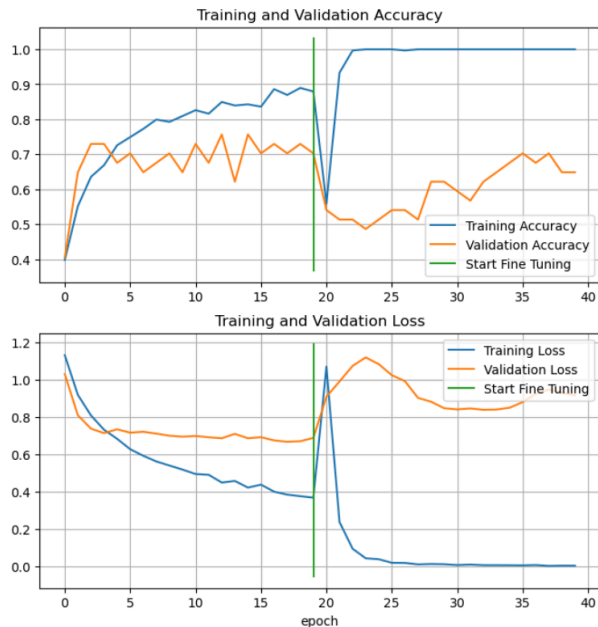


## Classification Report:

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| CLL          | 0.62      | 0.42   | 0.50     | 12      |
| FL           | 0.58      | 0.79   | 0.67     | 14      |
| MCL          | 0.55      | 0.50   | 0.52     | 12      |
| accuracy     |           |        | 0.58     | 38      |
| macro avg    | 0.58      | 0.57   | 0.56     | 38      |
| weighted avg | 0.58      | 0.58   | 0.57     | 38      |

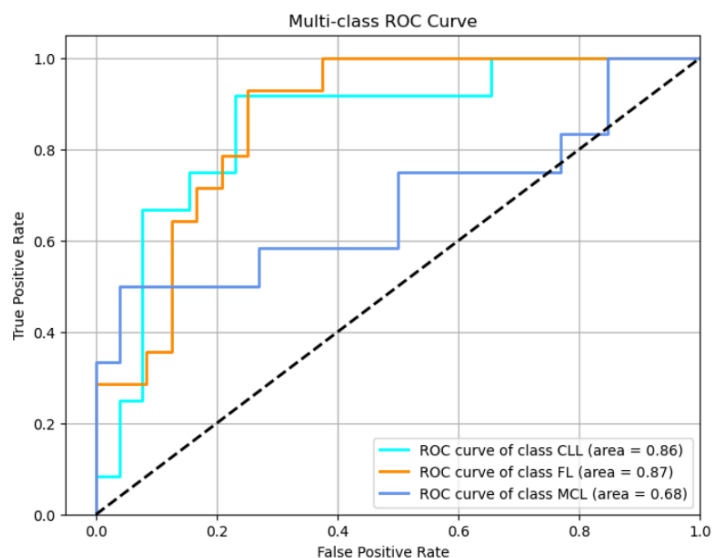


## Experimento 12 – MobileNetV2 – Sin data augmentation – 20 epochs

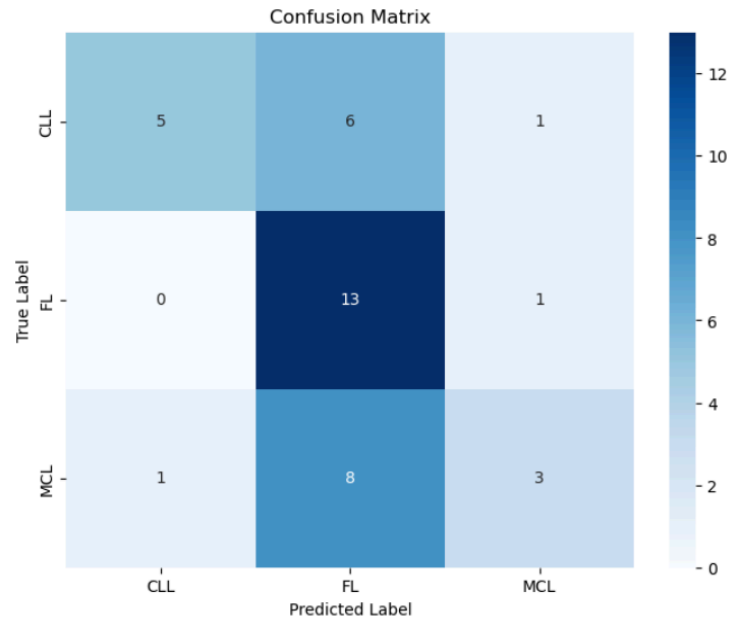
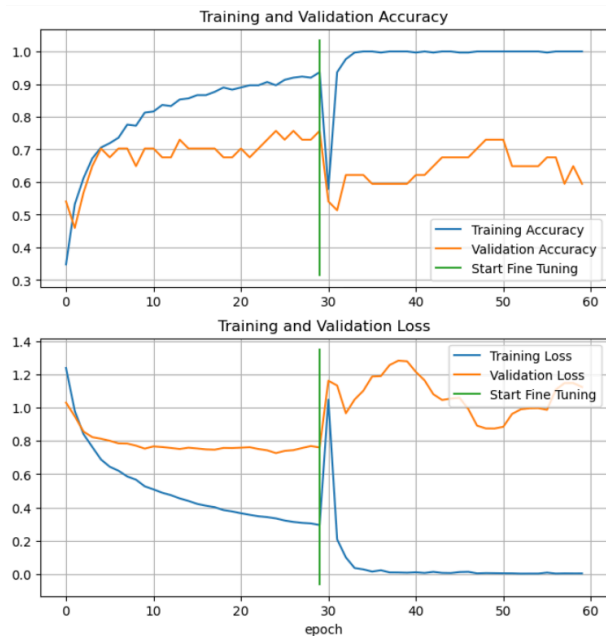


### Classification Report:

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| CLL          | 0.60      | 0.25   | 0.35     | 12      |
| FL           | 0.61      | 1.00   | 0.76     | 14      |
| MCL          | 0.60      | 0.50   | 0.55     | 12      |
| accuracy     |           |        | 0.61     | 38      |
| macro avg    | 0.60      | 0.58   | 0.55     | 38      |
| weighted avg | 0.60      | 0.61   | 0.56     | 38      |

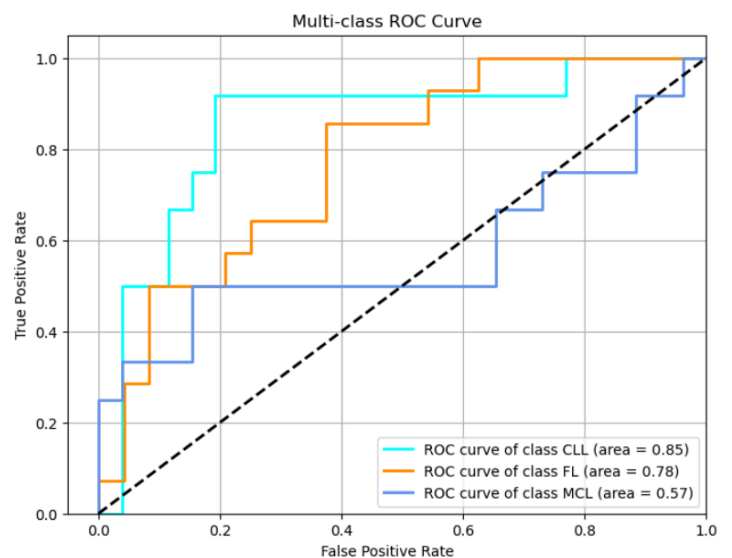


# Experimento 13 – MobileNetV2 – Sin data augmentation – 30 epochs

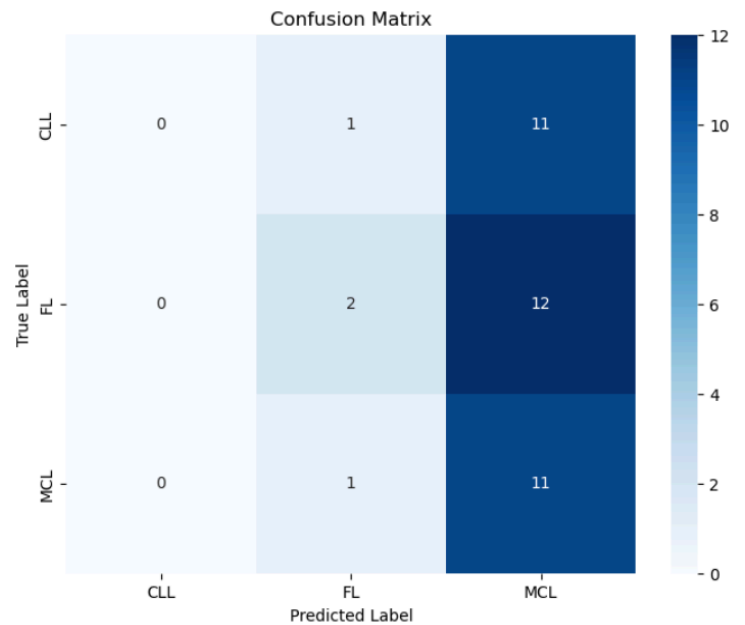
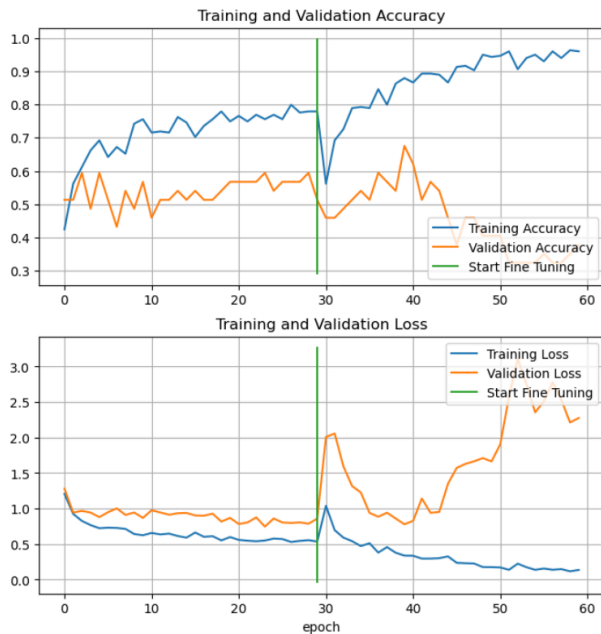


## Classification Report:

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| CLL          | 0.83      | 0.42   | 0.56     | 12      |
| FL           | 0.48      | 0.93   | 0.63     | 14      |
| MCL          | 0.60      | 0.25   | 0.35     | 12      |
| accuracy     |           |        | 0.55     | 38      |
| macro avg    | 0.64      | 0.53   | 0.51     | 38      |
| weighted avg | 0.63      | 0.55   | 0.52     | 38      |

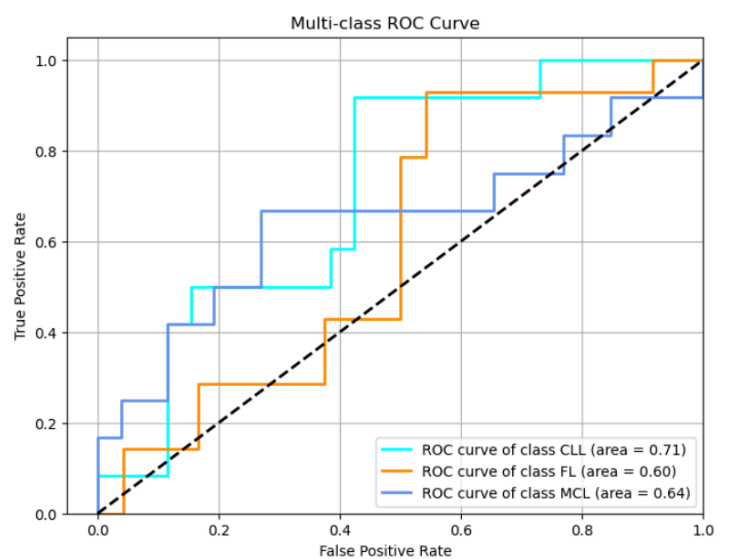


# Experimento 14 – MobileNetV2 – Augmentación implícita – 30 epochs



## Classification Report:

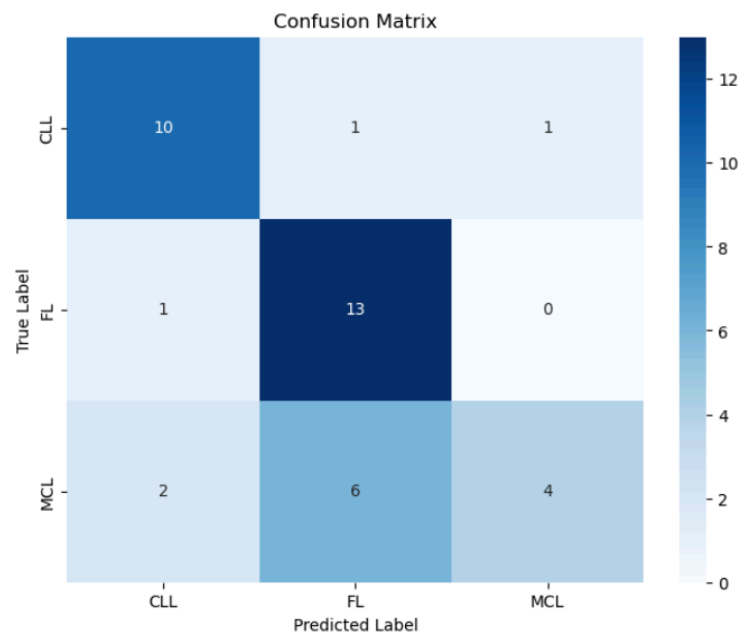
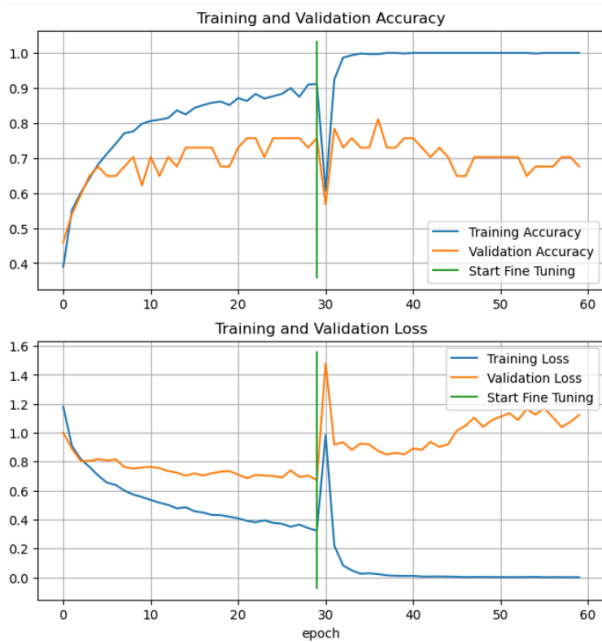
|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| CLL          | 0.00      | 0.00   | 0.00     | 12      |
| FL           | 0.50      | 0.14   | 0.22     | 14      |
| MCL          | 0.32      | 0.92   | 0.48     | 12      |
| accuracy     |           |        | 0.34     | 38      |
| macro avg    | 0.27      | 0.35   | 0.23     | 38      |
| weighted avg | 0.29      | 0.34   | 0.23     | 38      |



# Experimento 15 – MobileNetV2 – Augmentación explícita (x2) – 30 epochs

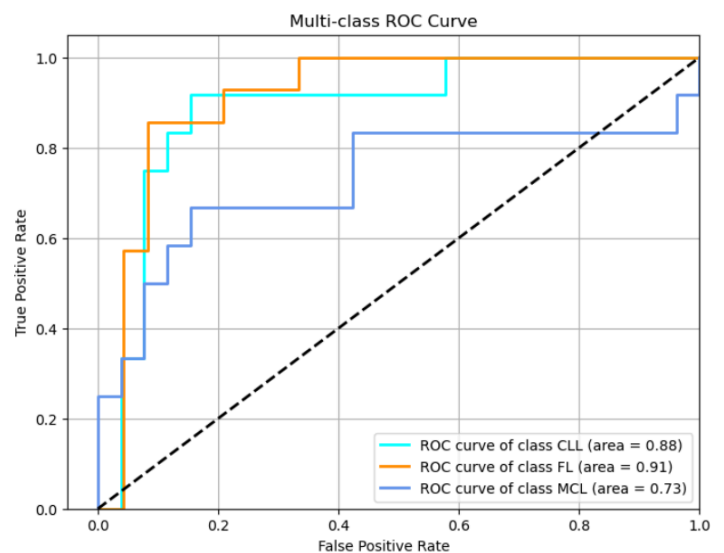
Original training set size: (299, 224, 224, 3)

Augmented training set size: (598, 224, 224, 3)



Classification Report:

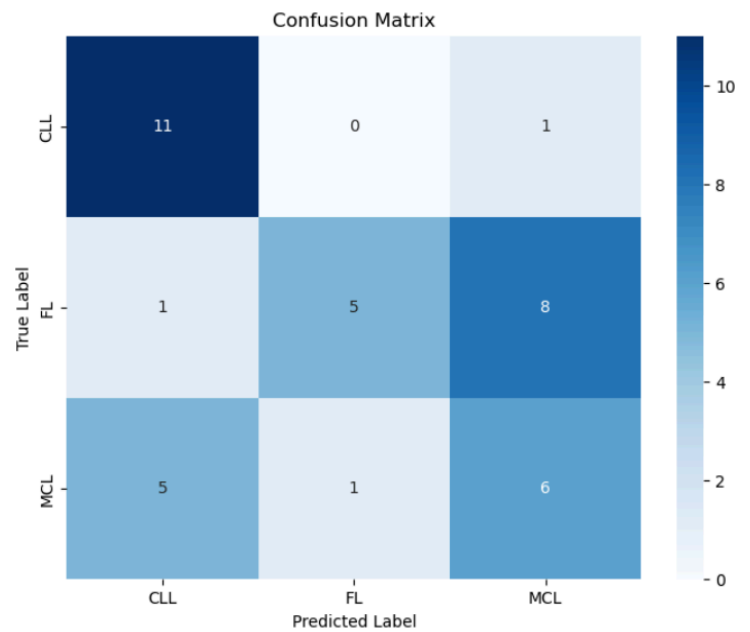
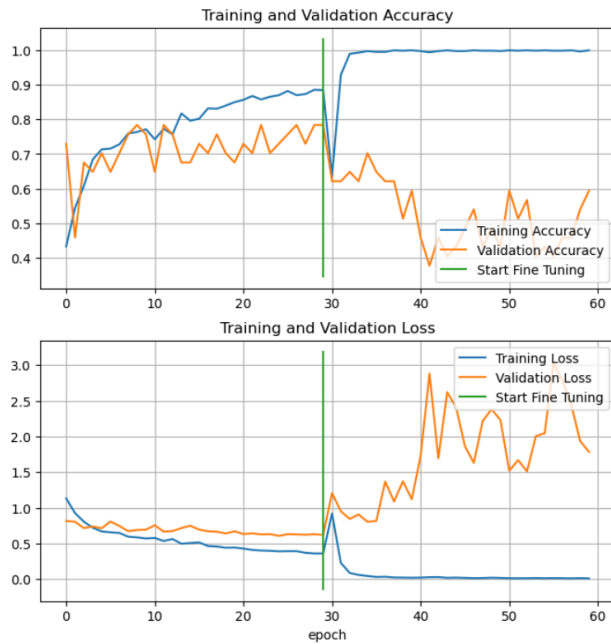
|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| CLL          | 0.77      | 0.83   | 0.80     | 12      |
| FL           | 0.65      | 0.93   | 0.76     | 14      |
| MCL          | 0.80      | 0.33   | 0.47     | 12      |
| accuracy     |           |        | 0.71     | 38      |
| macro avg    | 0.74      | 0.70   | 0.68     | 38      |
| weighted avg | 0.74      | 0.71   | 0.68     | 38      |



## Experimento 16 – MobileNetV2 – Augmentación explícita (x3) – 30 epochs

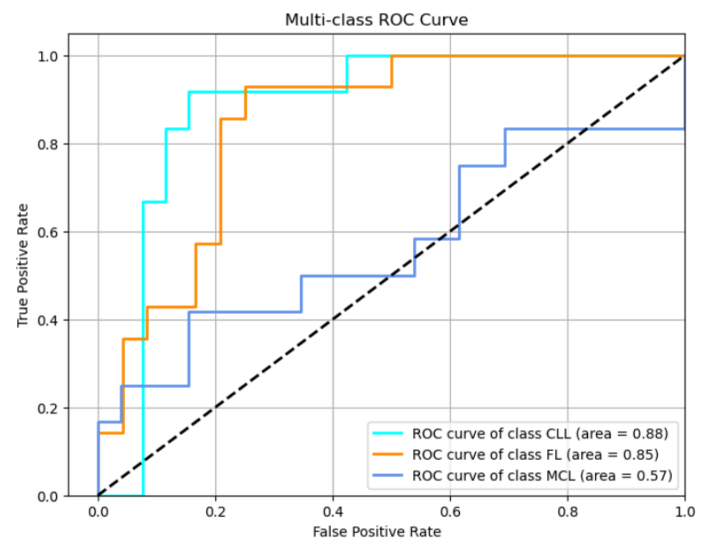
Original training set size: (299, 224, 224, 3)

Augmented training set size: (897, 224, 224, 3)



Classification Report:

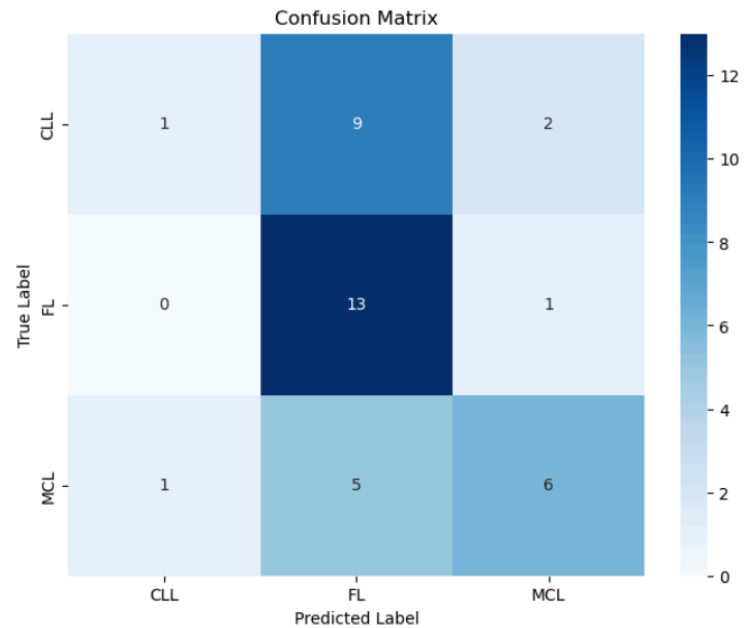
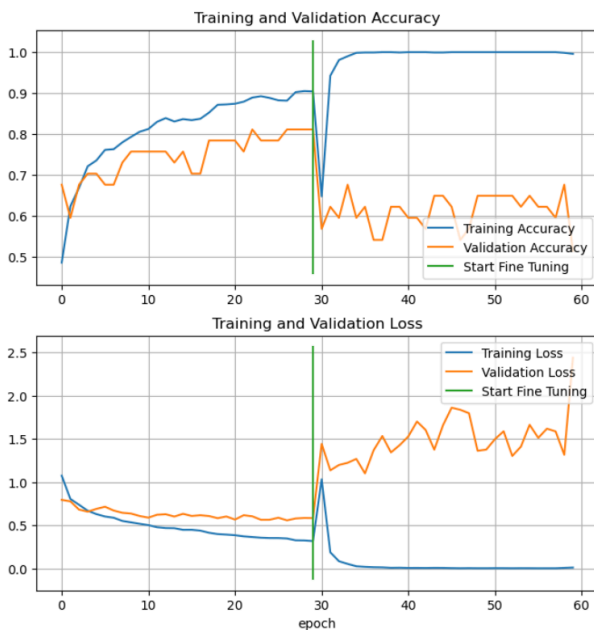
|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| CLL          | 0.65      | 0.92   | 0.76     | 12      |
| FL           | 0.83      | 0.36   | 0.50     | 14      |
| MCL          | 0.40      | 0.50   | 0.44     | 12      |
| accuracy     |           |        | 0.58     | 38      |
| macro avg    | 0.63      | 0.59   | 0.57     | 38      |
| weighted avg | 0.64      | 0.58   | 0.56     | 38      |



# Experimento 17 – MobileNetV2 – Augmentación explícita (x4) – 30 epochs

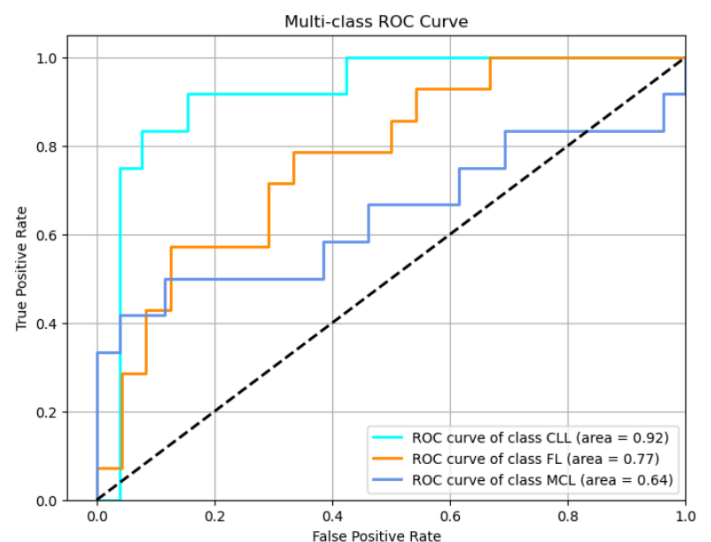
Original training set size: (299, 224, 224, 3)

Augmented training set size: (1196, 224, 224, 3)



## Classification Report:

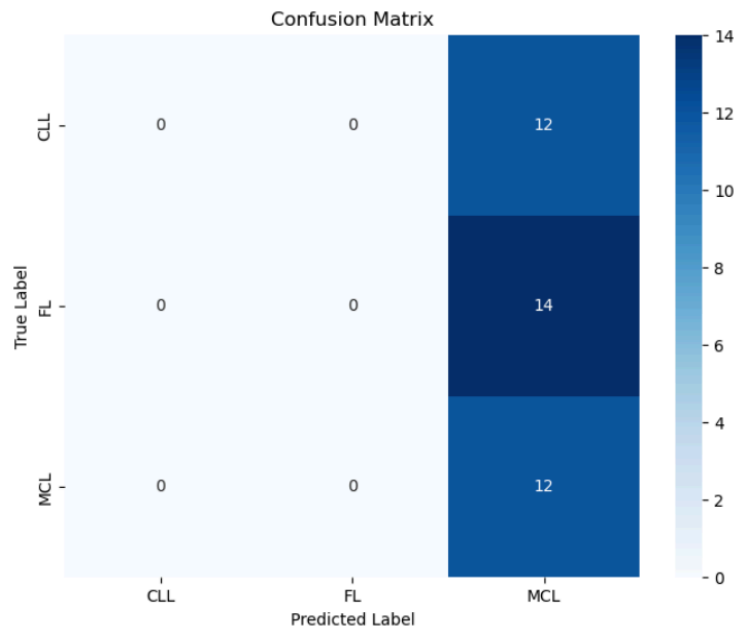
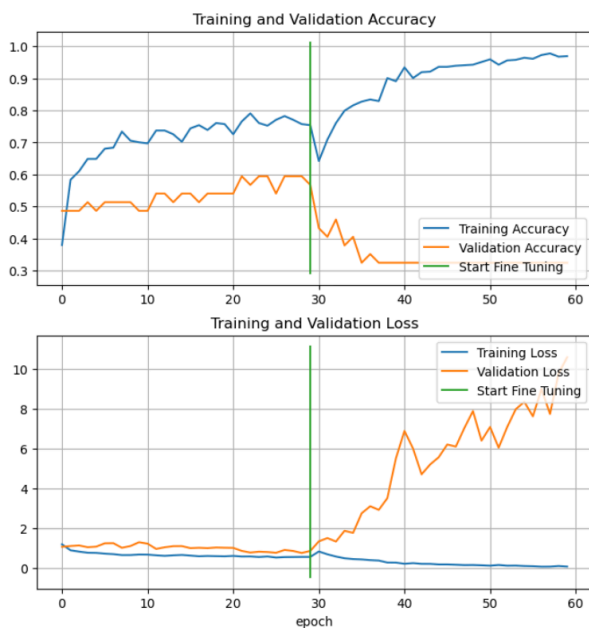
|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| CLL          | 0.50      | 0.08   | 0.14     | 12      |
| FL           | 0.48      | 0.93   | 0.63     | 14      |
| MCL          | 0.67      | 0.50   | 0.57     | 12      |
| accuracy     |           |        | 0.53     | 38      |
| macro avg    | 0.55      | 0.50   | 0.45     | 38      |
| weighted avg | 0.55      | 0.53   | 0.46     | 38      |



## Experimento 18 – MobileNetV2 – Augmentación combinada (x2) – 30 epochs

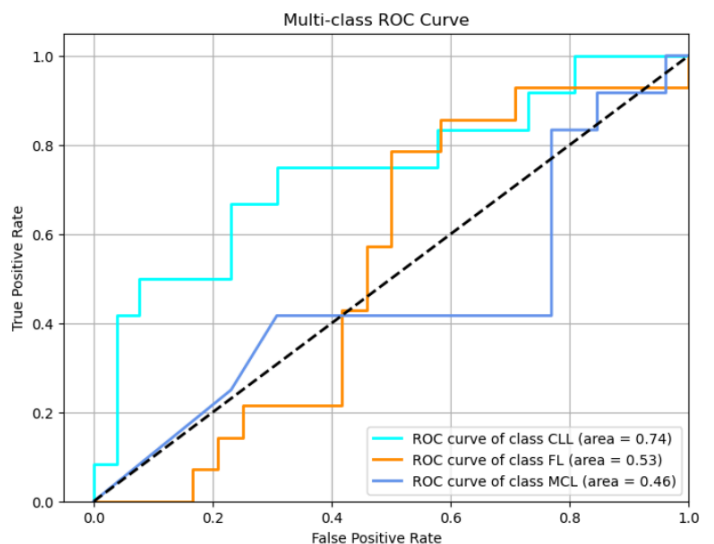
Original training set size: (299, 224, 224, 3)

Augmented training set size: (598, 224, 224, 3)



### Classification Report:

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| CLL          | 0.00      | 0.00   | 0.00     | 12      |
| FL           | 0.00      | 0.00   | 0.00     | 14      |
| MCL          | 0.32      | 1.00   | 0.48     | 12      |
| accuracy     |           |        | 0.32     | 38      |
| macro avg    | 0.11      | 0.33   | 0.16     | 38      |
| weighted avg | 0.10      | 0.32   | 0.15     | 38      |

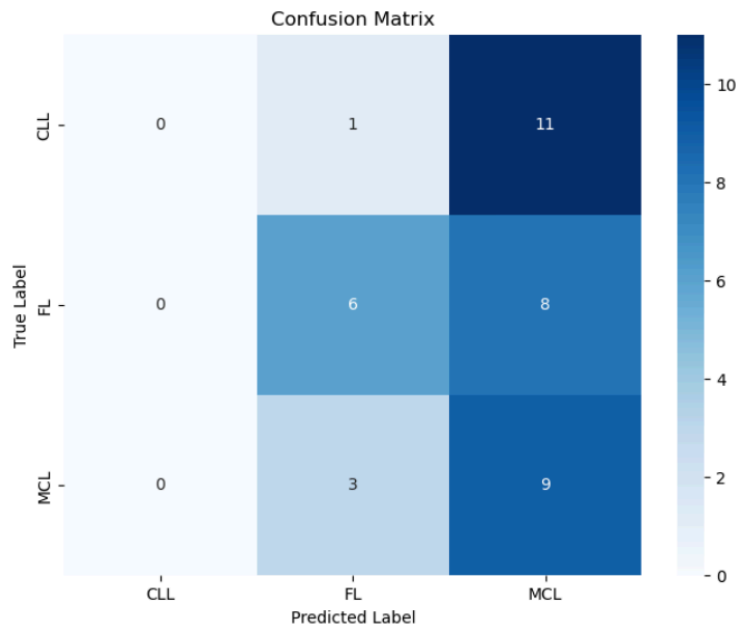
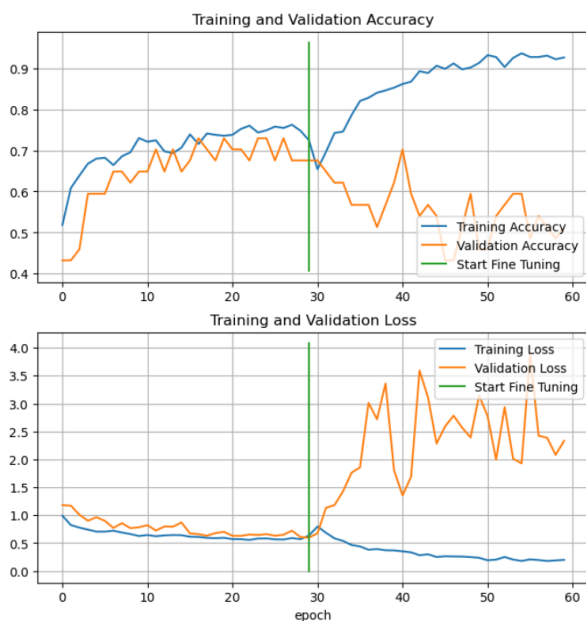




# Experimento 19 – MobileNetV2 – Augmentación combinada (x3) – 30 epochs – Fine-tuning con 20 capas

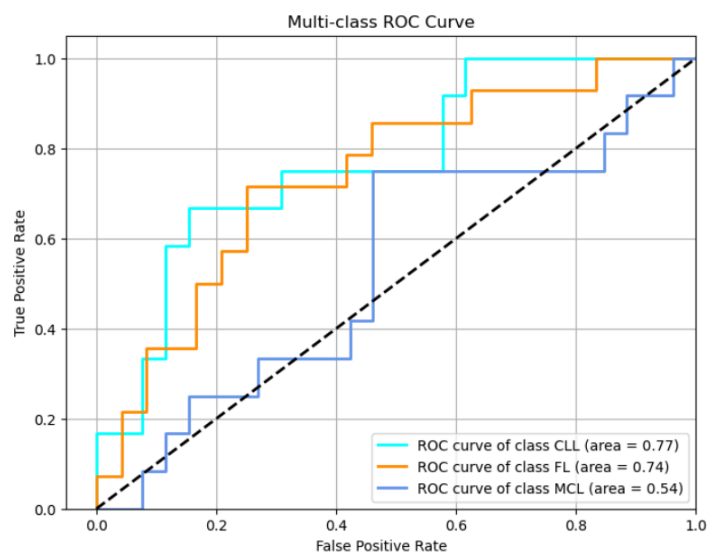
Original training set size: (299, 224, 224, 3)

Augmented training set size: (897, 224, 224, 3)



## Classification Report:

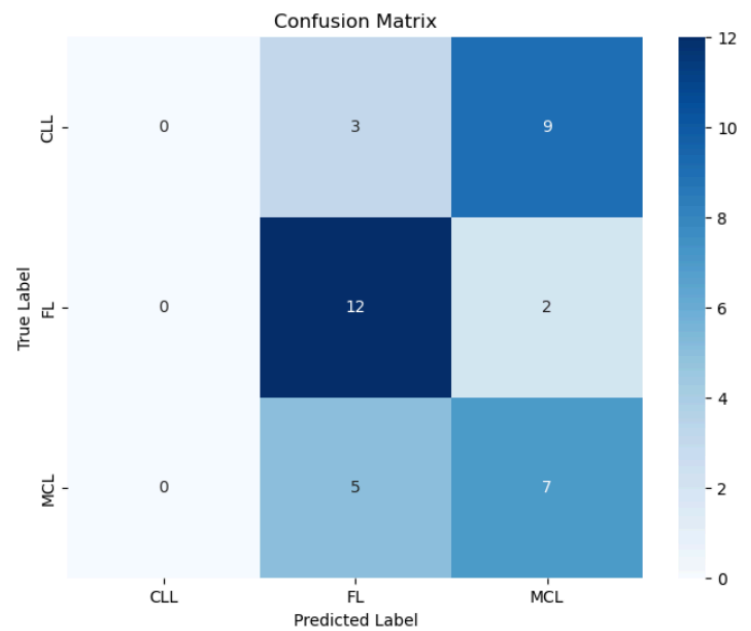
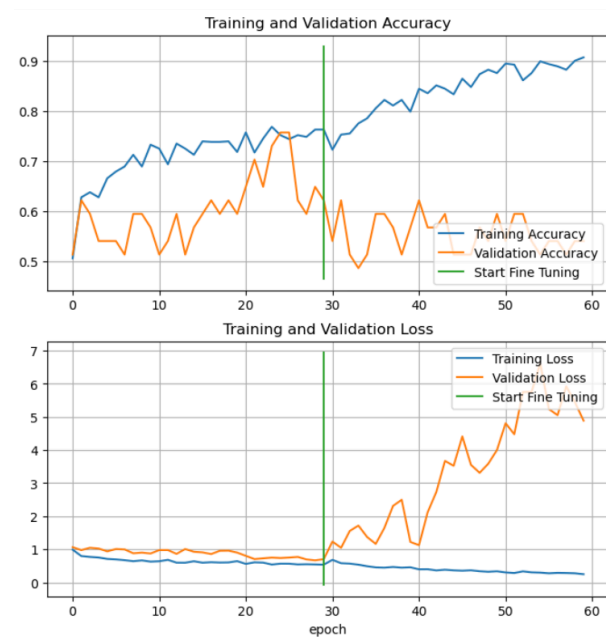
|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| CLL          | 0.00      | 0.00   | 0.00     | 12      |
| FL           | 0.60      | 0.43   | 0.50     | 14      |
| MCL          | 0.32      | 0.75   | 0.45     | 12      |
| accuracy     |           |        | 0.39     | 38      |
| macro avg    | 0.31      | 0.39   | 0.32     | 38      |
| weighted avg | 0.32      | 0.39   | 0.33     | 38      |



# Experimento 20 – MobileNetV2 – Augmentación combinada (x4) – 30 epochs – Fine-tuning con 5 capas

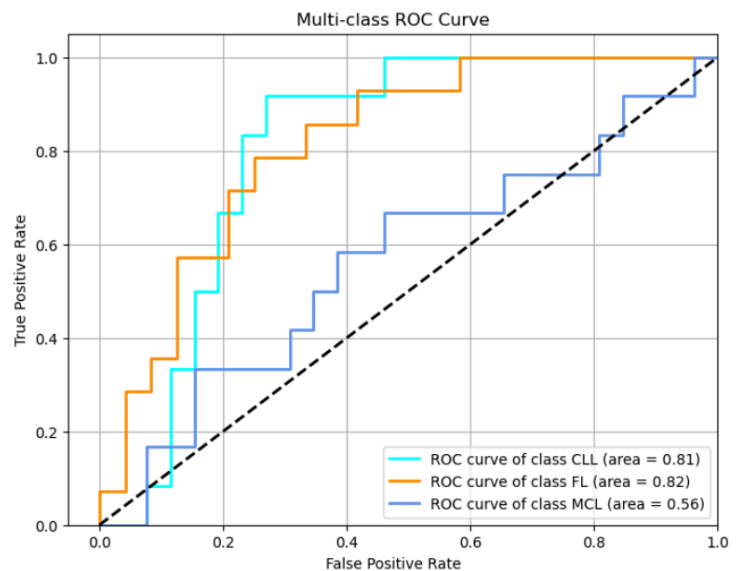
Original training set size: (299, 224, 224, 3)

Augmented training set size: (1196, 224, 224, 3)

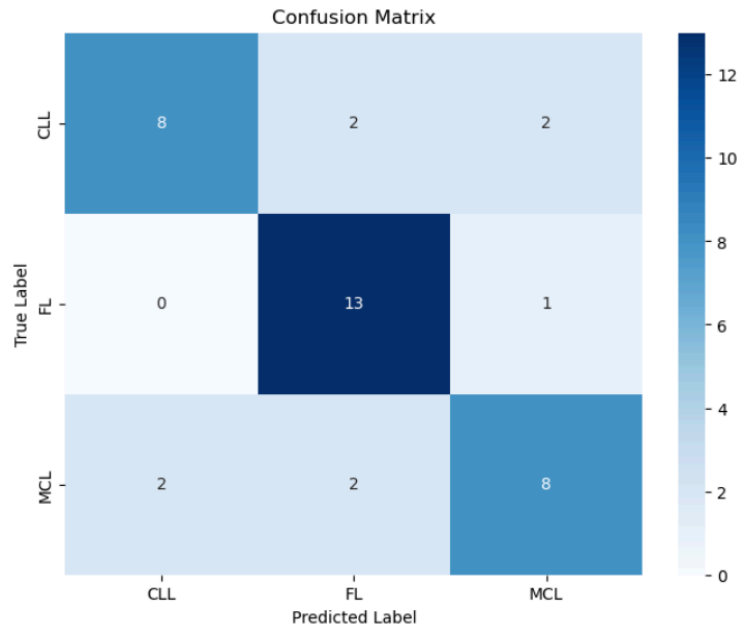
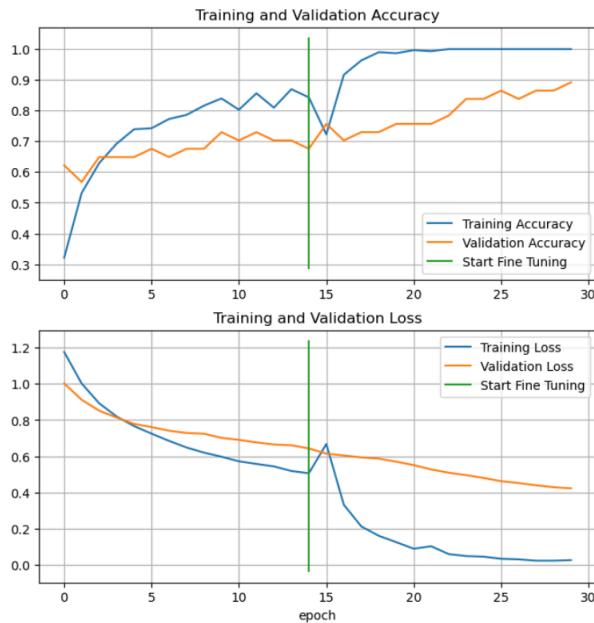


## Classification Report:

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| CLL          | 0.00      | 0.00   | 0.00     | 12      |
| FL           | 0.60      | 0.86   | 0.71     | 14      |
| MCL          | 0.39      | 0.58   | 0.47     | 12      |
| accuracy     |           |        | 0.50     | 38      |
| macro avg    | 0.33      | 0.48   | 0.39     | 38      |
| weighted avg | 0.34      | 0.50   | 0.41     | 38      |

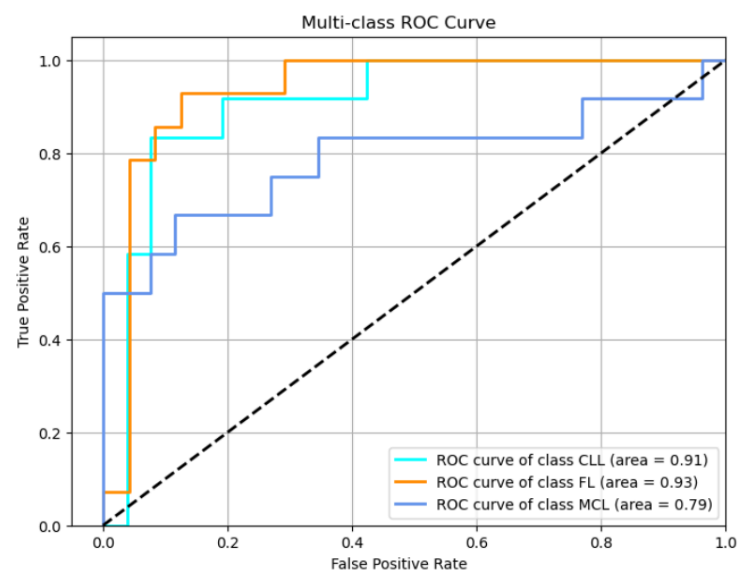


## Experimento 21 – DenseNet201 – Sin data augmentation – 15 epochs

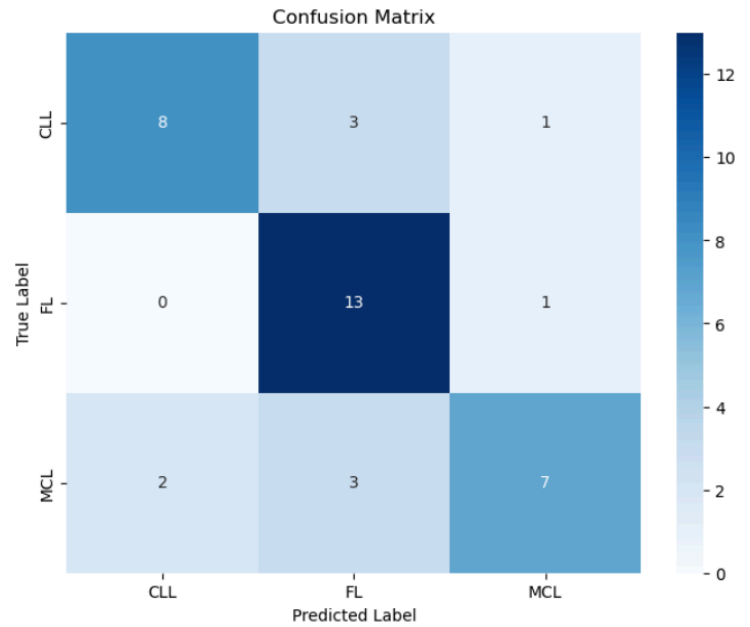
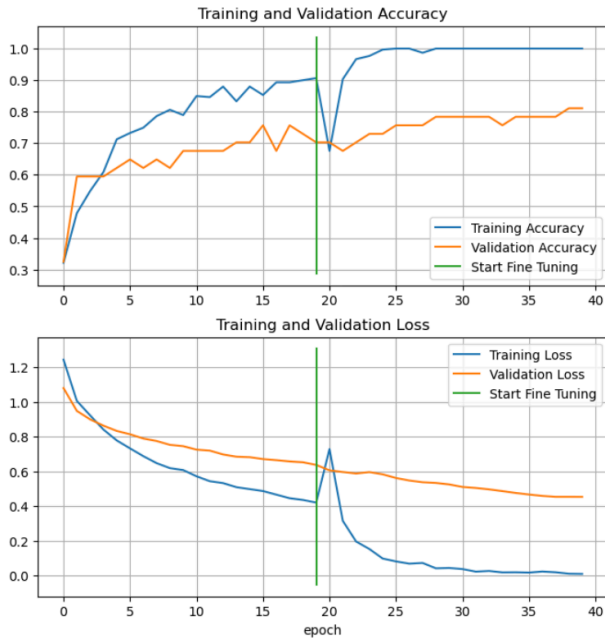


### Classification Report:

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| CLL          | 0.80      | 0.67   | 0.73     | 12      |
| FL           | 0.76      | 0.93   | 0.84     | 14      |
| MCL          | 0.73      | 0.67   | 0.70     | 12      |
| accuracy     |           |        | 0.76     | 38      |
| macro avg    | 0.76      | 0.75   | 0.75     | 38      |
| weighted avg | 0.76      | 0.76   | 0.76     | 38      |

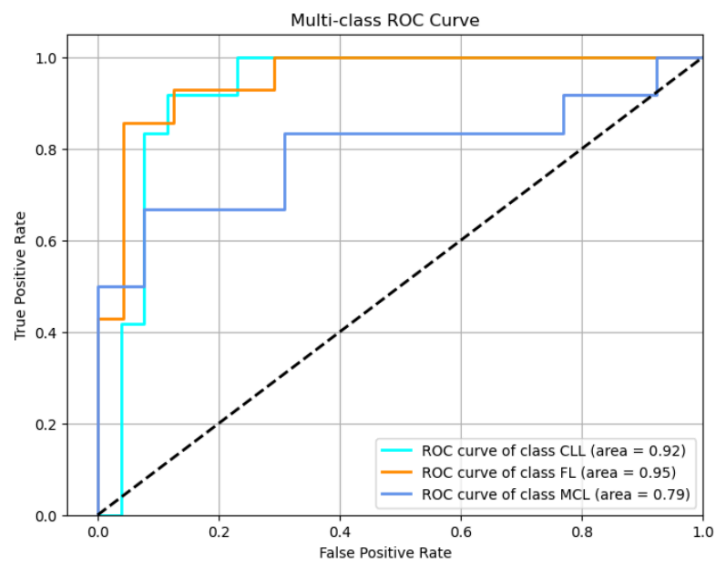


## Experimento 22 – DenseNet201 – Sin data augmentation – 20 epochs

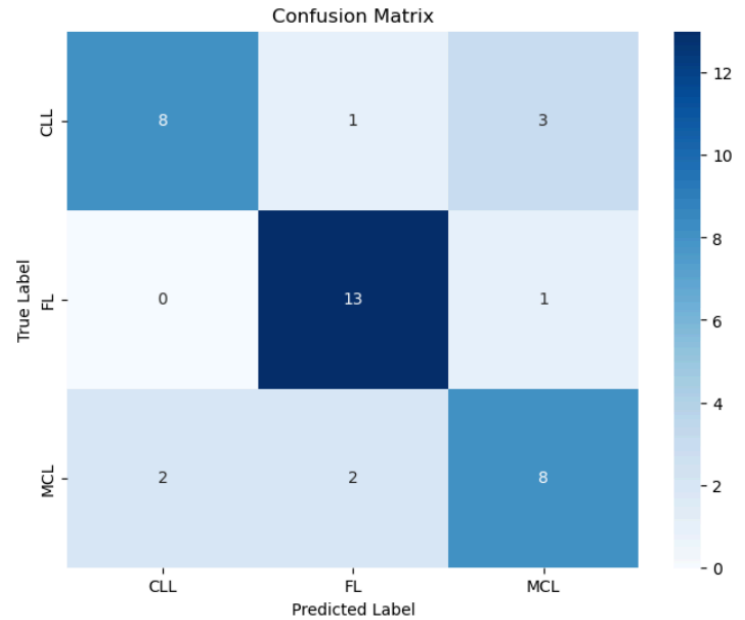
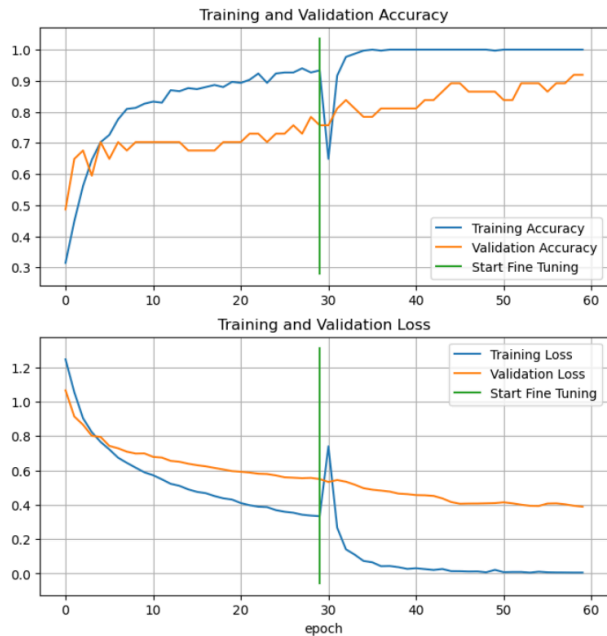


### Classification Report:

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| CLL          | 0.80      | 0.67   | 0.73     | 12      |
| FL           | 0.68      | 0.93   | 0.79     | 14      |
| MCL          | 0.78      | 0.58   | 0.67     | 12      |
| accuracy     |           |        | 0.74     | 38      |
| macro avg    | 0.75      | 0.73   | 0.73     | 38      |
| weighted avg | 0.75      | 0.74   | 0.73     | 38      |

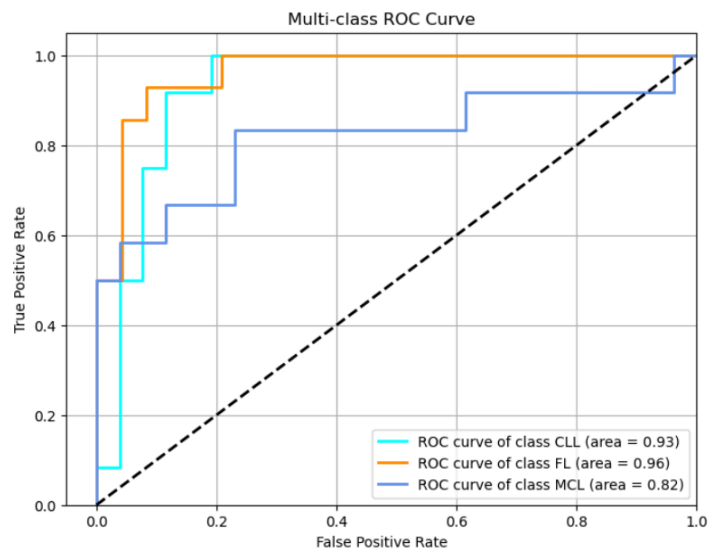


## Experimento 23 – DenseNet201 – Sin data augmentation – 30 epochs

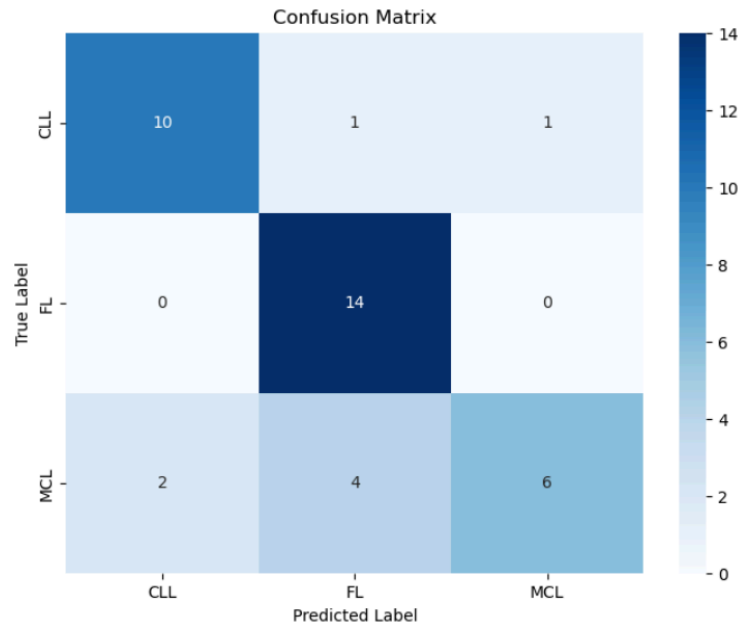
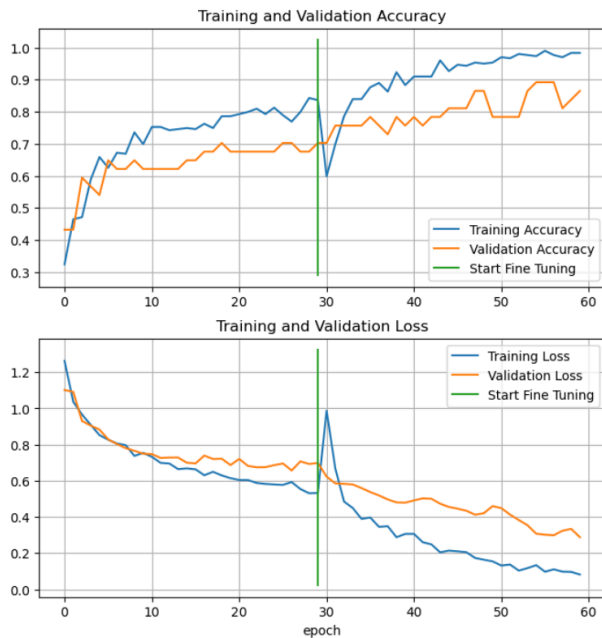


### Classification Report:

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| CLL          | 0.80      | 0.67   | 0.73     | 12      |
| FL           | 0.81      | 0.93   | 0.87     | 14      |
| MCL          | 0.67      | 0.67   | 0.67     | 12      |
| accuracy     |           |        | 0.76     | 38      |
| macro avg    | 0.76      | 0.75   | 0.75     | 38      |
| weighted avg | 0.76      | 0.76   | 0.76     | 38      |

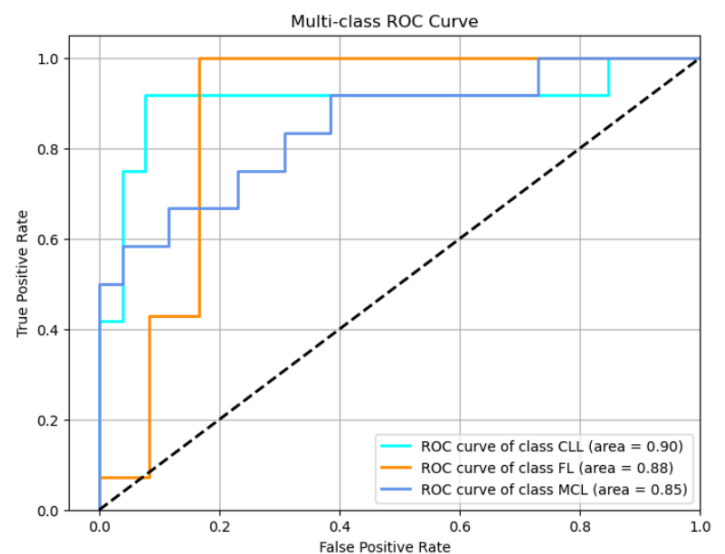


## Experimento 24 – DenseNet201 – Augmentación implícita – 30 epochs



### Classification Report:

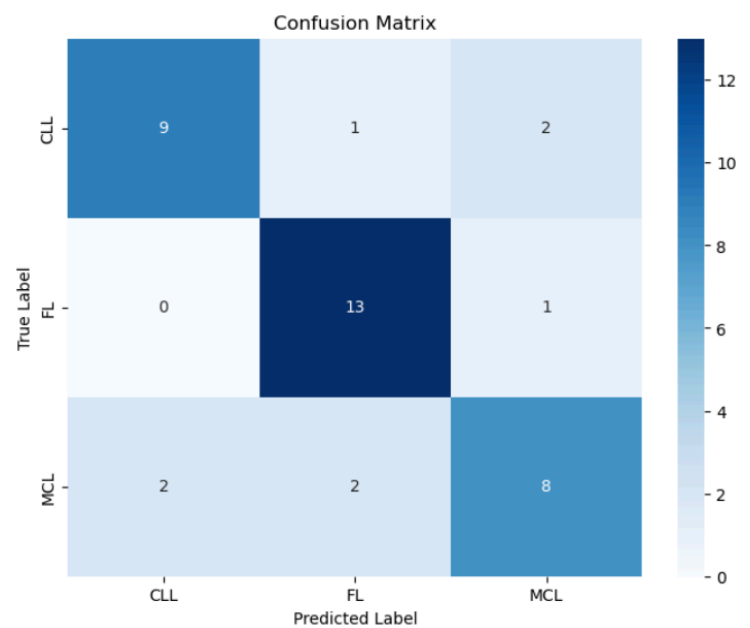
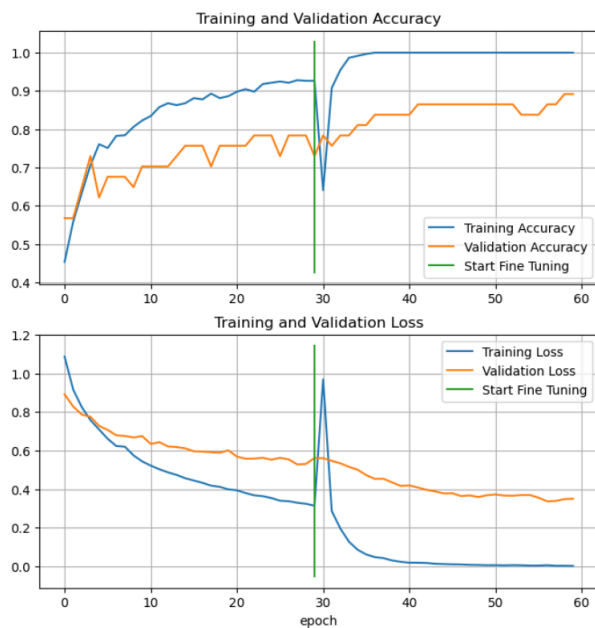
|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| CLL          | 0.83      | 0.83   | 0.83     | 12      |
| FL           | 0.74      | 1.00   | 0.85     | 14      |
| MCL          | 0.86      | 0.50   | 0.63     | 12      |
| accuracy     |           |        | 0.79     | 38      |
| macro avg    | 0.81      | 0.78   | 0.77     | 38      |
| weighted avg | 0.81      | 0.79   | 0.78     | 38      |



## Experimento 25 – DenseNet201 – Augmentación explícita (x2) – 30 epochs

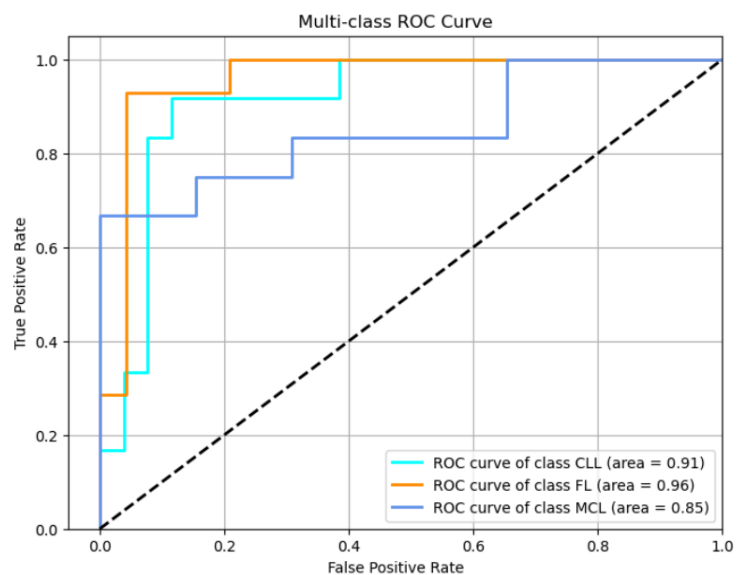
Original training set size: (299, 224, 224, 3)

Augmented training set size: (598, 224, 224, 3)



### Classification Report:

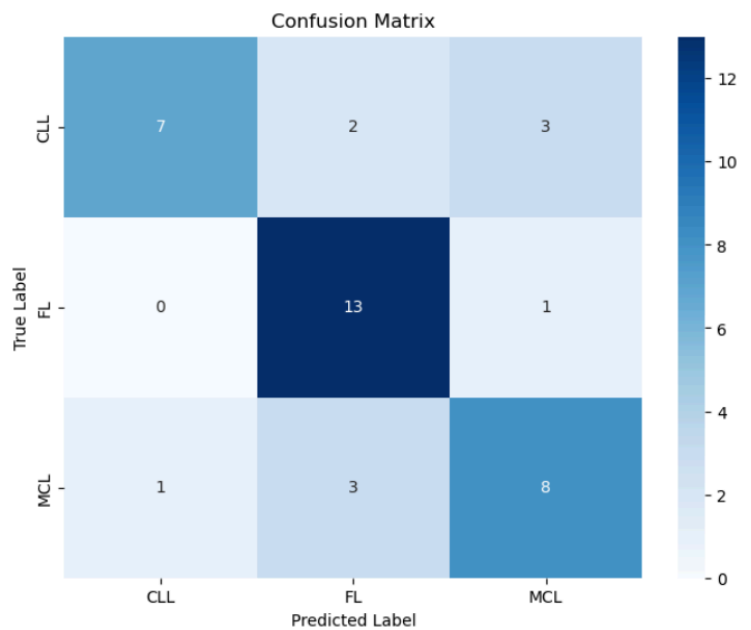
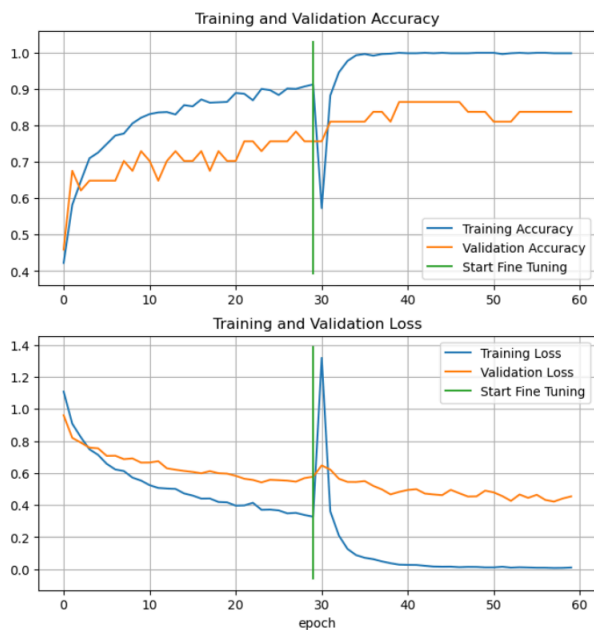
|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| CLL          | 0.82      | 0.75   | 0.78     | 12      |
| FL           | 0.81      | 0.93   | 0.87     | 14      |
| MCL          | 0.73      | 0.67   | 0.70     | 12      |
| accuracy     |           |        | 0.79     | 38      |
| macro avg    | 0.79      | 0.78   | 0.78     | 38      |
| weighted avg | 0.79      | 0.79   | 0.79     | 38      |



## Experimento 26 – DenseNet201 – Augmentación explícita (x3) – 30 epochs

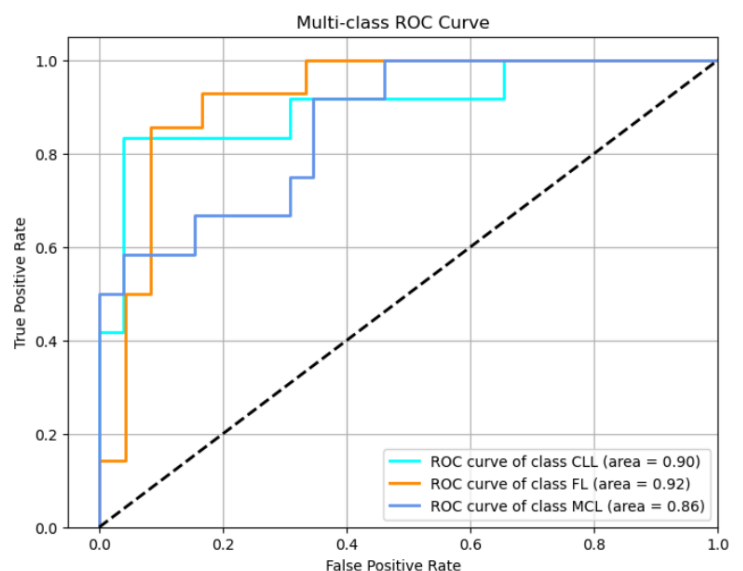
Original training set size: (299, 224, 224, 3)

Augmented training set size: (897, 224, 224, 3)



### Classification Report:

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| CLL          | 0.88      | 0.58   | 0.70     | 12      |
| FL           | 0.72      | 0.93   | 0.81     | 14      |
| MCL          | 0.67      | 0.67   | 0.67     | 12      |
| accuracy     |           |        | 0.74     | 38      |
| macro avg    | 0.75      | 0.73   | 0.73     | 38      |
| weighted avg | 0.75      | 0.74   | 0.73     | 38      |

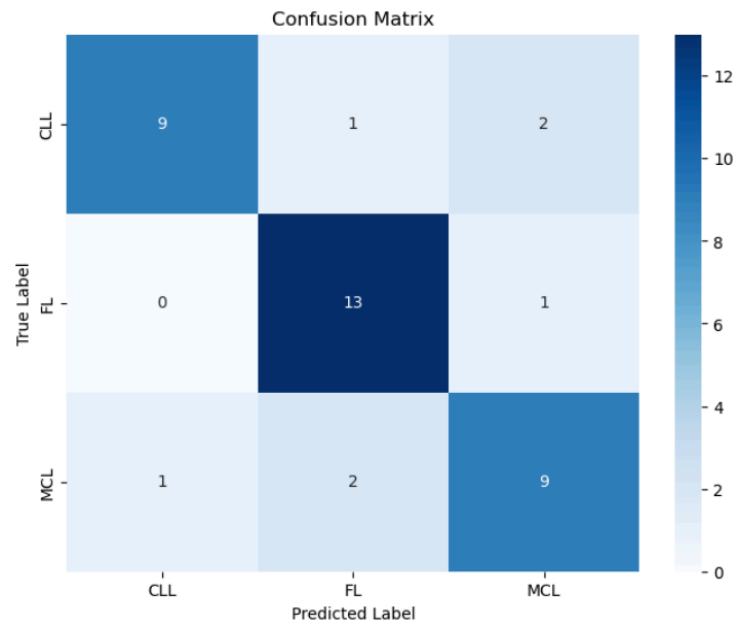
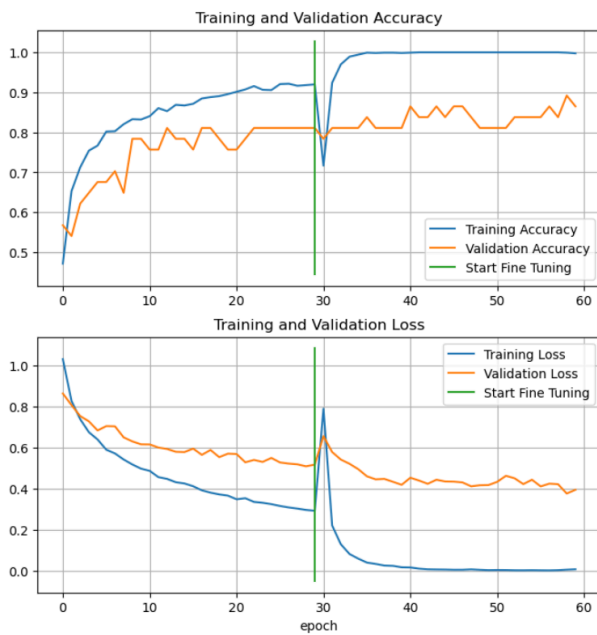




## Experimento 27 – DenseNet201 – Augmentación explícita (x4) – 30 epochs

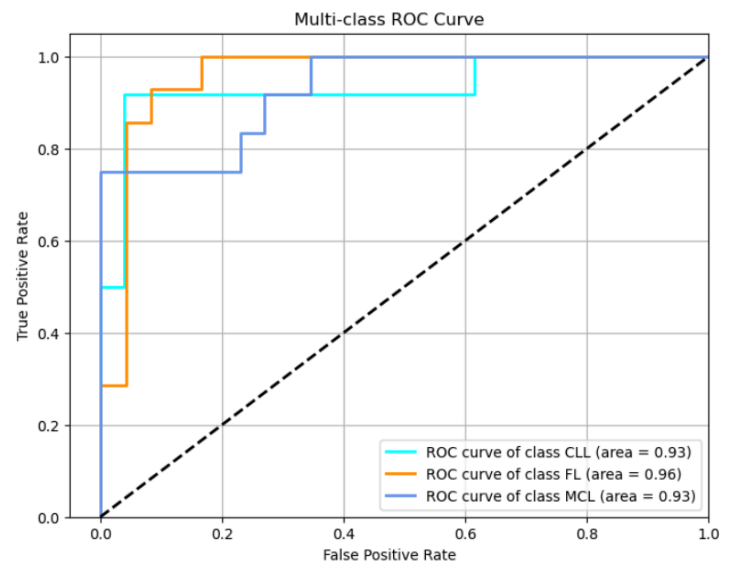
Original training set size: (299, 224, 224, 3)

Augmented training set size: (1196, 224, 224, 3)



### Classification Report:

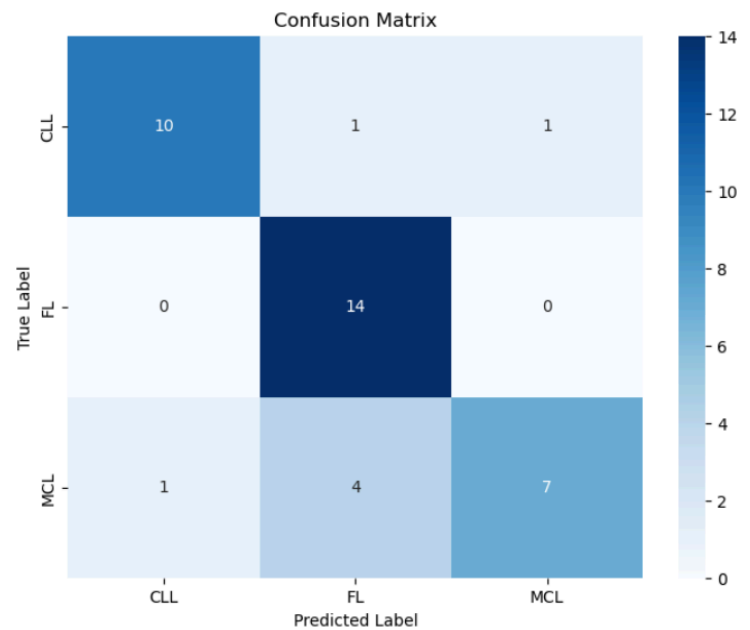
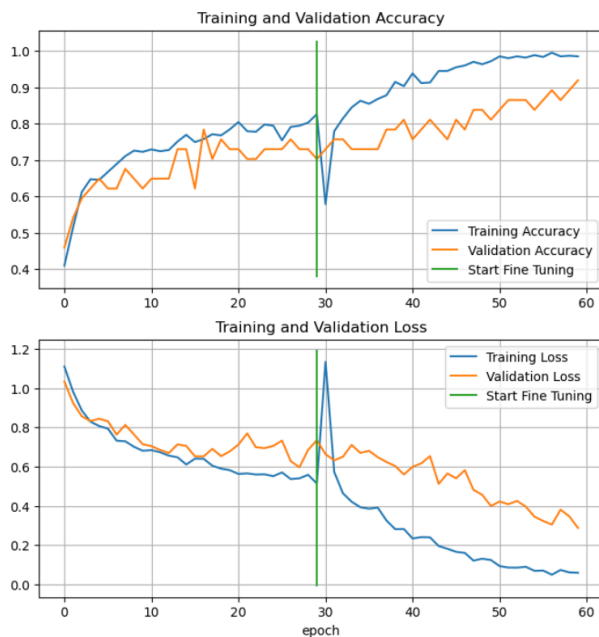
|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| CLL          | 0.90      | 0.75   | 0.82     | 12      |
| FL           | 0.81      | 0.93   | 0.87     | 14      |
| MCL          | 0.75      | 0.75   | 0.75     | 12      |
| accuracy     |           |        | 0.82     | 38      |
| macro avg    | 0.82      | 0.81   | 0.81     | 38      |
| weighted avg | 0.82      | 0.82   | 0.81     | 38      |



## Experimento 28 – DenseNet201 – Augmentación combinada (x2) – 30 epochs

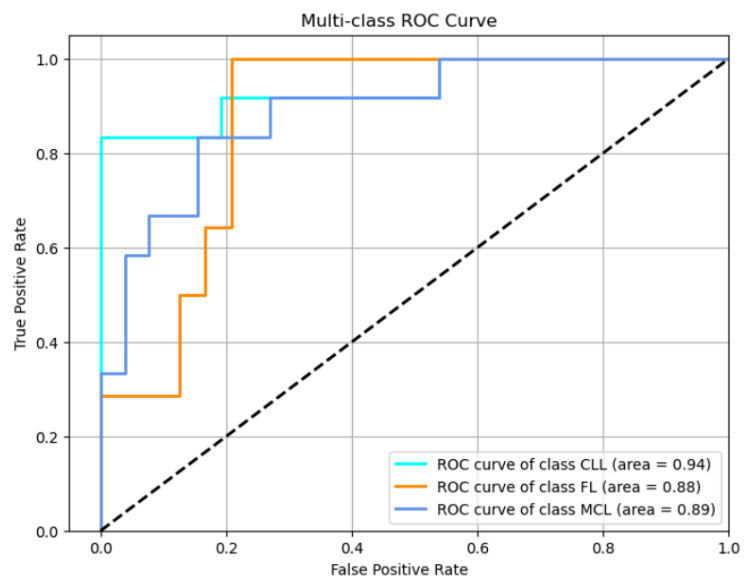
Original training set size: (299, 224, 224, 3)

Augmented training set size: (598, 224, 224, 3)



### Classification Report:

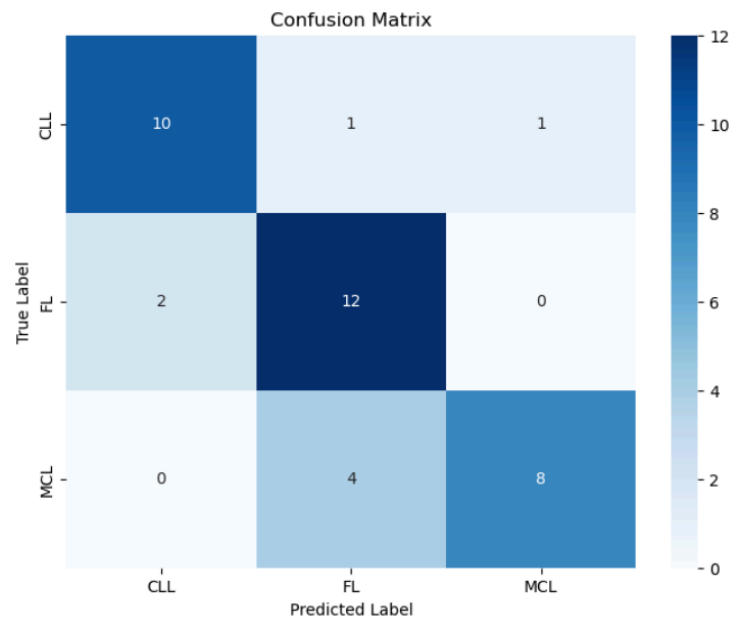
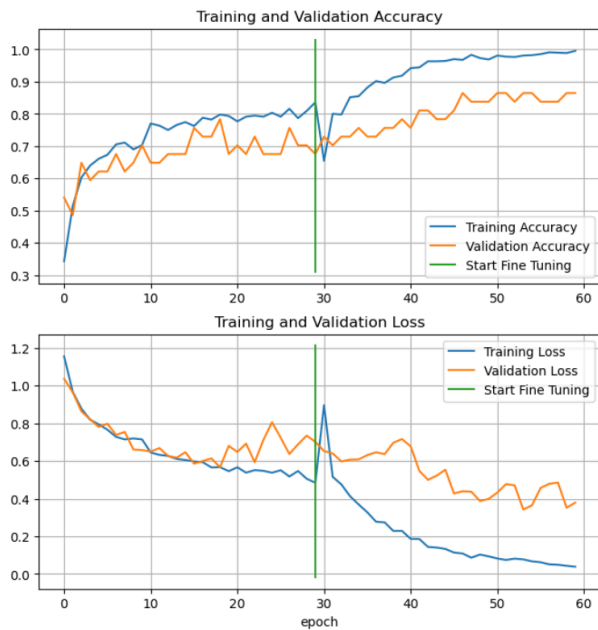
|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| CLL          | 0.91      | 0.83   | 0.87     | 12      |
| FL           | 0.74      | 1.00   | 0.85     | 14      |
| MCL          | 0.88      | 0.58   | 0.70     | 12      |
| accuracy     |           |        | 0.82     | 38      |
| macro avg    | 0.84      | 0.81   | 0.81     | 38      |
| weighted avg | 0.83      | 0.82   | 0.81     | 38      |



## Experimento 29 – DenseNet201 – Augmentación combinada (x3) – 30 epochs

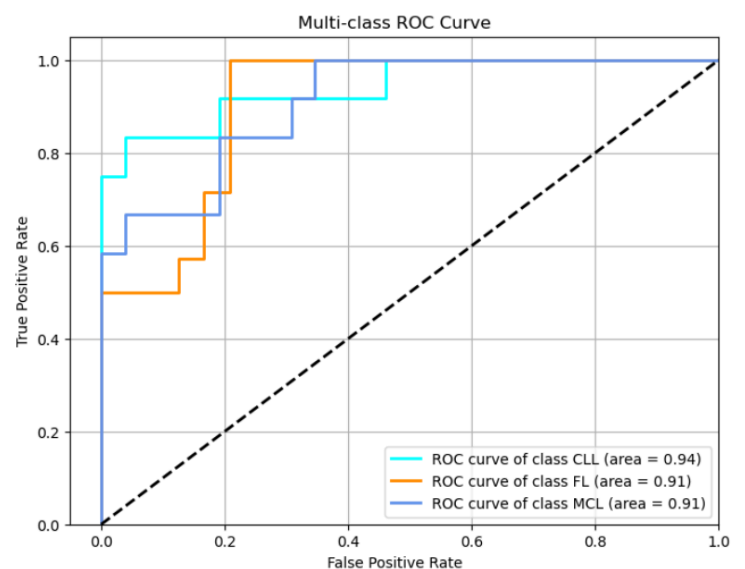
Original training set size: (299, 224, 224, 3)

Augmented training set size: (897, 224, 224, 3)



Classification Report:

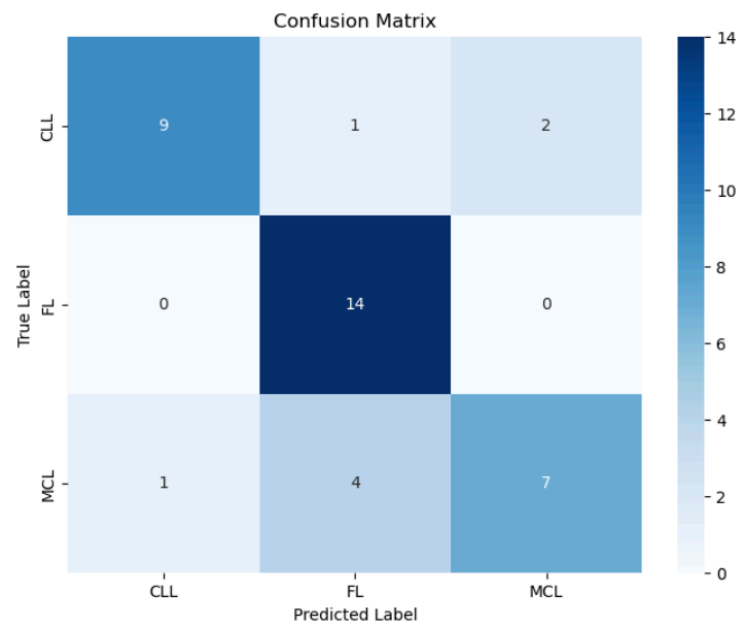
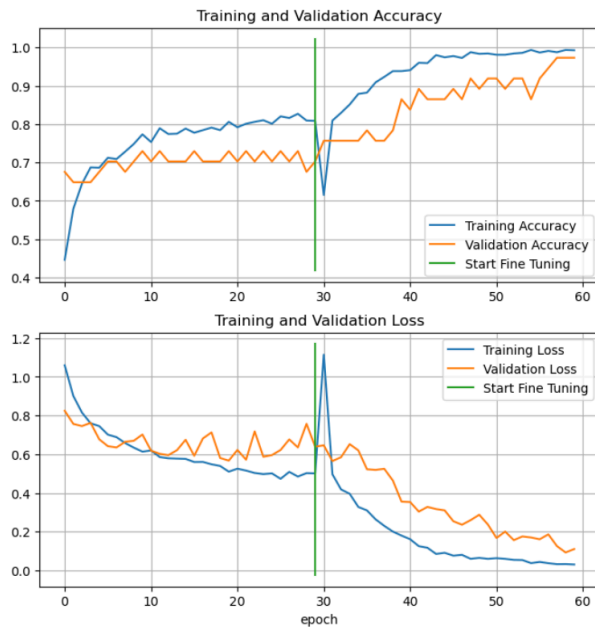
|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| CLL          | 0.83      | 0.83   | 0.83     | 12      |
| FL           | 0.71      | 0.86   | 0.77     | 14      |
| MCL          | 0.89      | 0.67   | 0.76     | 12      |
| accuracy     |           |        | 0.79     | 38      |
| macro avg    | 0.81      | 0.79   | 0.79     | 38      |
| weighted avg | 0.80      | 0.79   | 0.79     | 38      |



# Experimento 30 – DenseNet201 – Augmentación combinada (x4) – 30 epochs

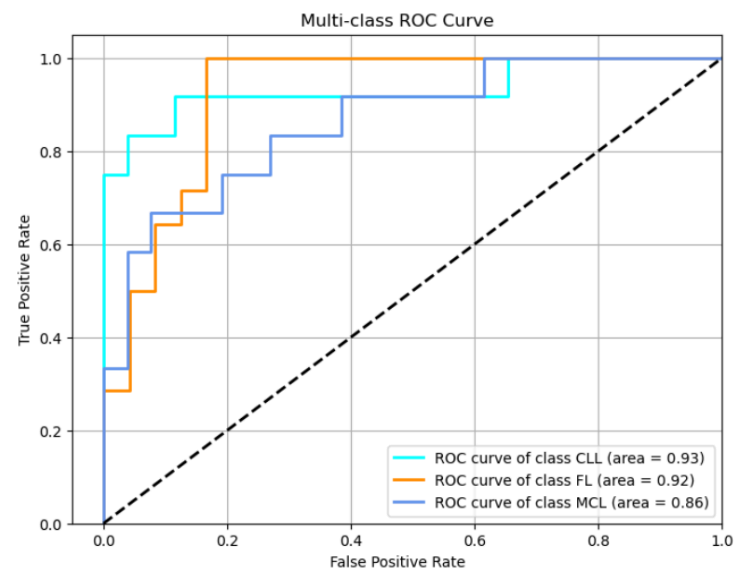
Original training set size: (299, 224, 224, 3)

Augmented training set size: (1196, 224, 224, 3)



## Classification Report:

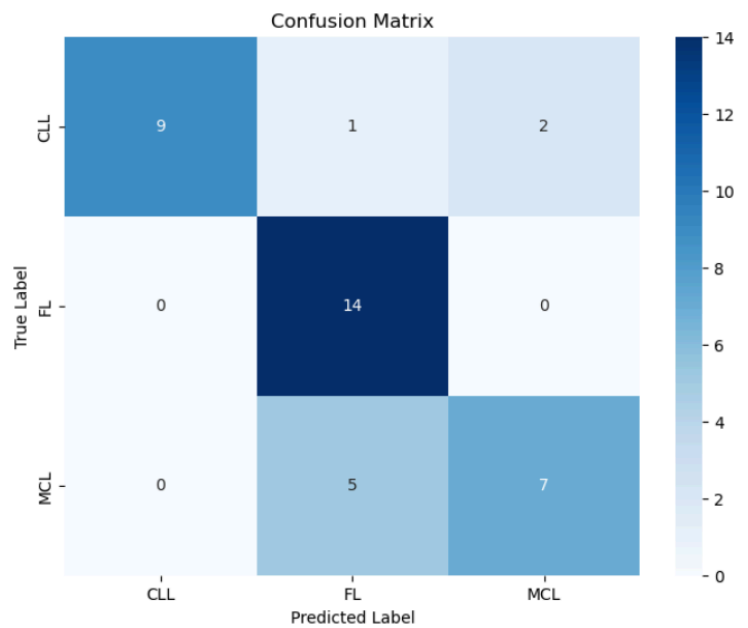
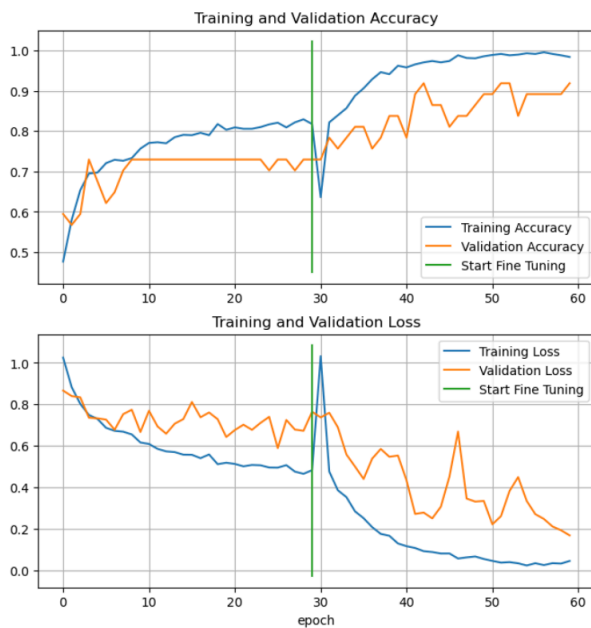
|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| CLL          | 0.90      | 0.75   | 0.82     | 12      |
| FL           | 0.74      | 1.00   | 0.85     | 14      |
| MCL          | 0.78      | 0.58   | 0.67     | 12      |
| accuracy     |           |        | 0.79     | 38      |
| macro avg    | 0.80      | 0.78   | 0.78     | 38      |
| weighted avg | 0.80      | 0.79   | 0.78     | 38      |



# Experimento 31 – DenseNet201 – Augmentación combinada (x4) – 30 epochs – Fine-tuning con 100 capas

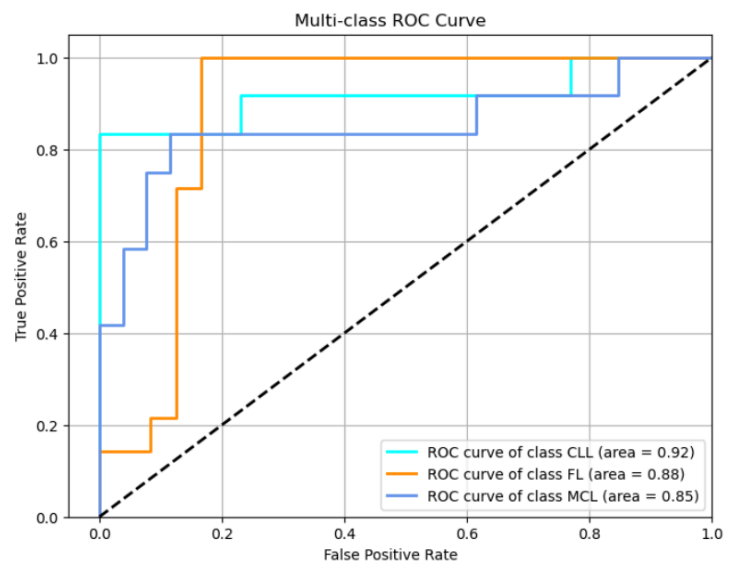
Original training set size: (299, 224, 224, 3)

Augmented training set size: (1196, 224, 224, 3)



## Classification Report:

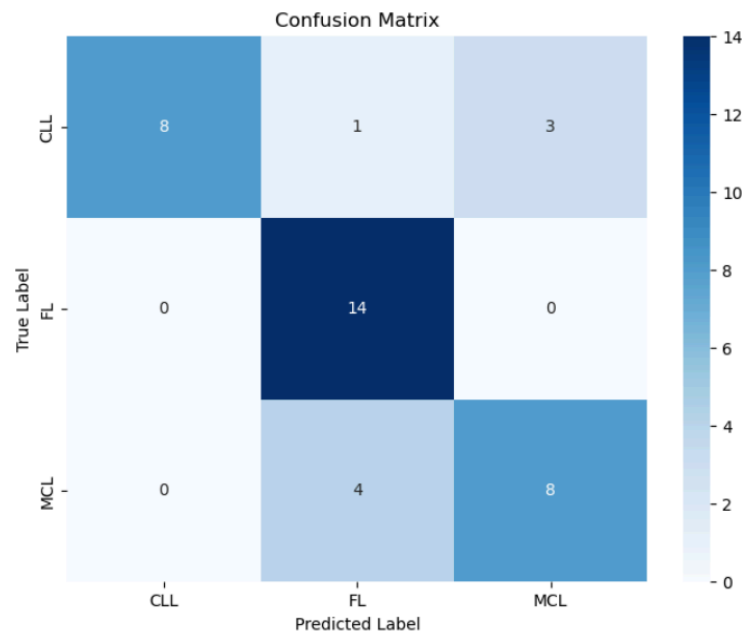
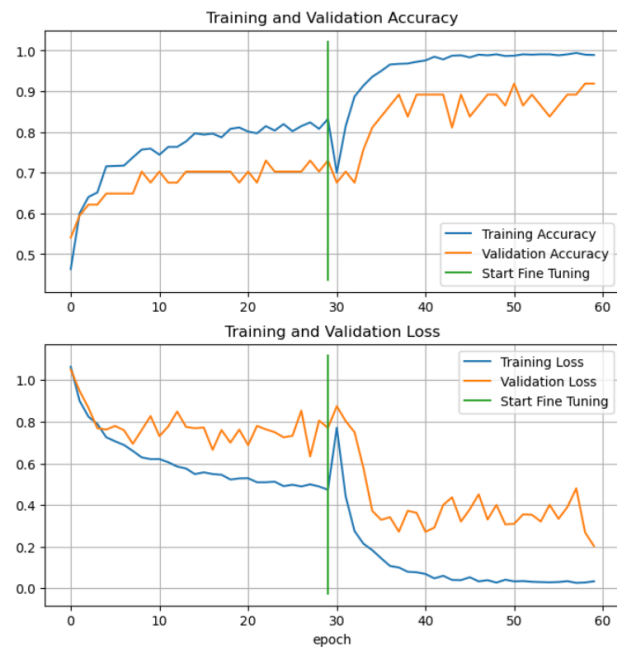
|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| CLL          | 1.00      | 0.75   | 0.86     | 12      |
| FL           | 0.70      | 1.00   | 0.82     | 14      |
| MCL          | 0.78      | 0.58   | 0.67     | 12      |
| accuracy     |           |        | 0.79     | 38      |
| macro avg    | 0.83      | 0.78   | 0.78     | 38      |
| weighted avg | 0.82      | 0.79   | 0.78     | 38      |



## Experimento 32 – DenseNet201 – Augmentación combinada (x4) – 30 epochs – Fine-tuning con 200 capas

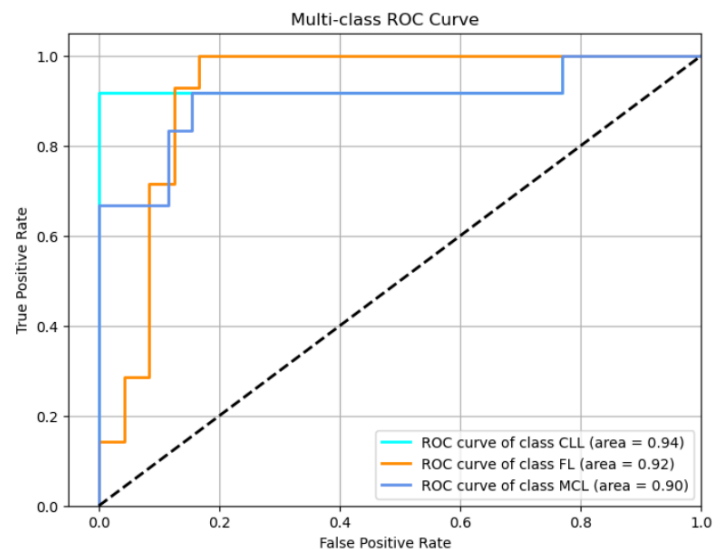
Original training set size: (299, 224, 224, 3)

Augmented training set size: (1196, 224, 224, 3)

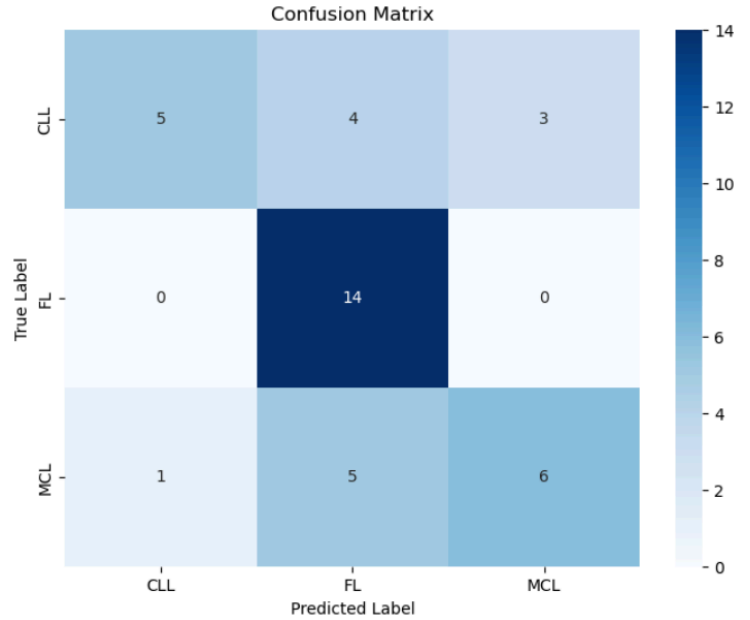
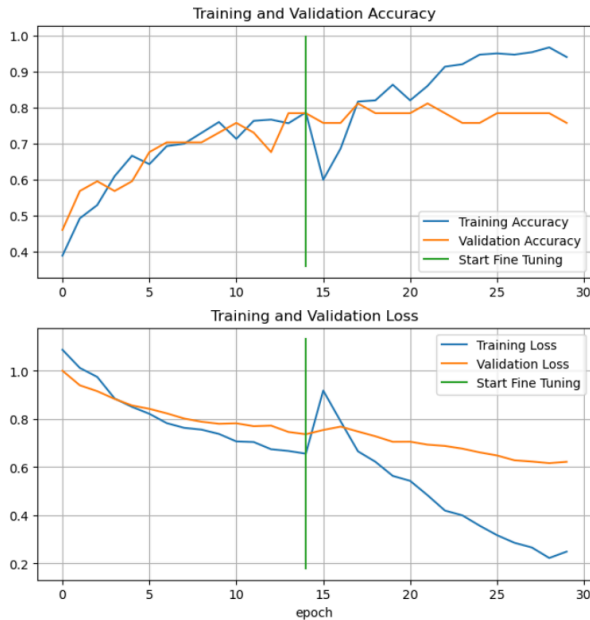


### Classification Report:

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| CLL          | 1.00      | 0.67   | 0.80     | 12      |
| FL           | 0.74      | 1.00   | 0.85     | 14      |
| MCL          | 0.73      | 0.67   | 0.70     | 12      |
| accuracy     |           |        | 0.79     | 38      |
| macro avg    | 0.82      | 0.78   | 0.78     | 38      |
| weighted avg | 0.82      | 0.79   | 0.78     | 38      |

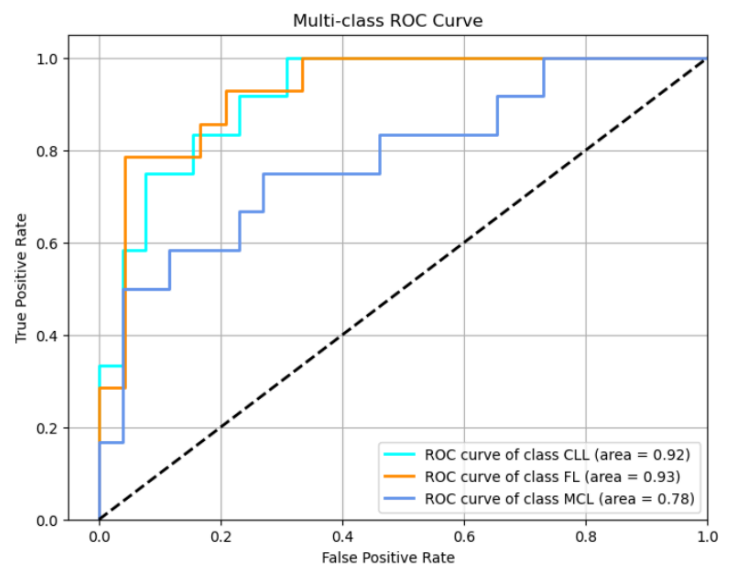


# Experimento 33 – EfficientNetV2B0 – Sin data augmentation – 15 epochs

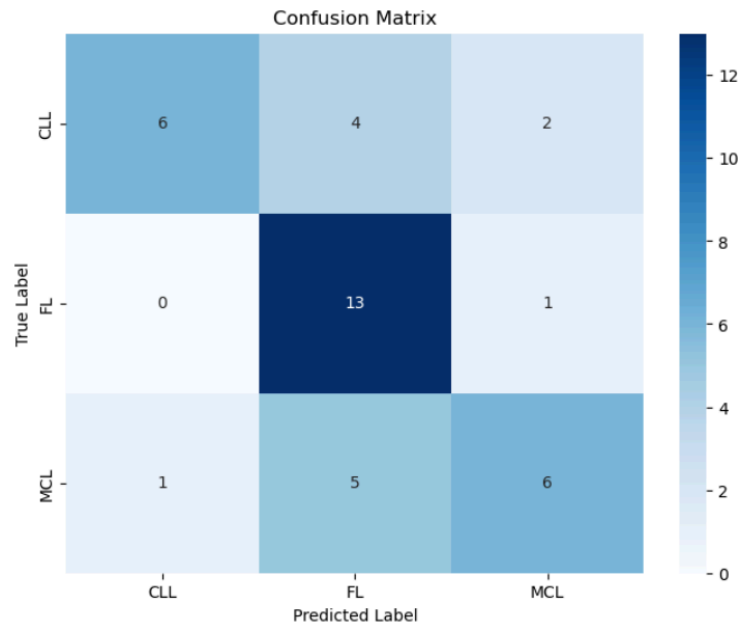
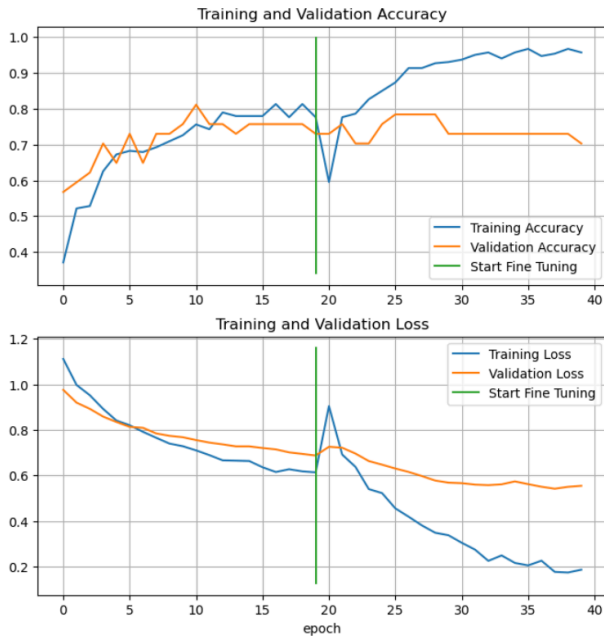


## Classification Report:

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| CLL          | 0.83      | 0.42   | 0.56     | 12      |
| FL           | 0.61      | 1.00   | 0.76     | 14      |
| MCL          | 0.67      | 0.50   | 0.57     | 12      |
| accuracy     |           |        | 0.66     | 38      |
| macro avg    | 0.70      | 0.64   | 0.63     | 38      |
| weighted avg | 0.70      | 0.66   | 0.63     | 38      |

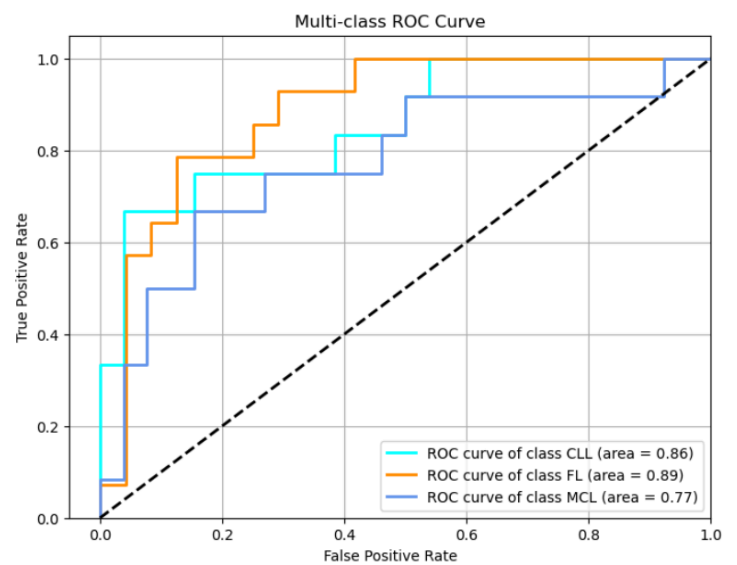


# Experimento 34 – EfficientNetV2B0 – Sin data augmentation – 20 epochs



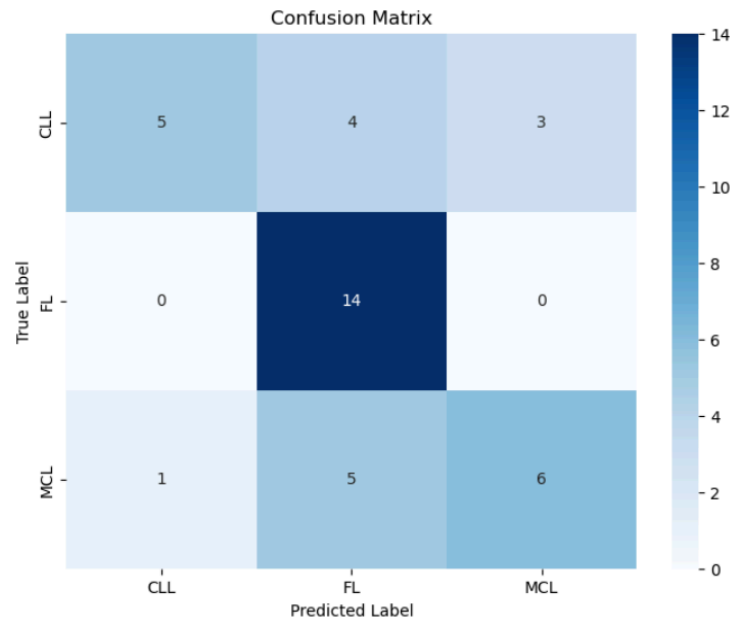
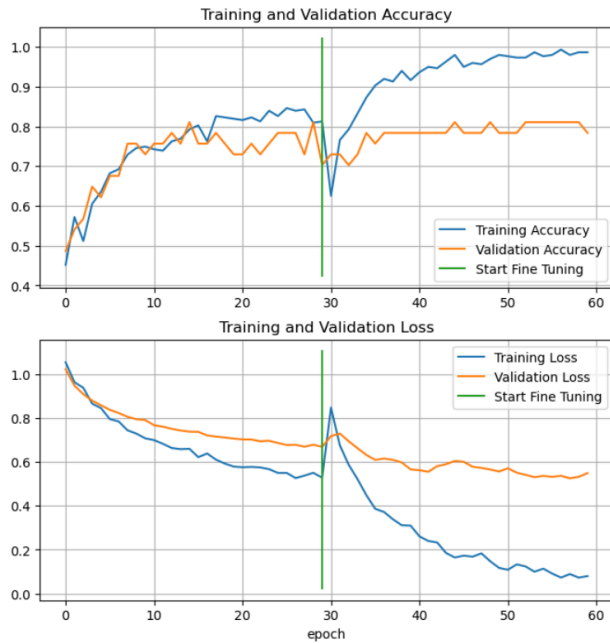
## Classification Report:

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| CLL          | 0.86      | 0.50   | 0.63     | 12      |
| FL           | 0.59      | 0.93   | 0.72     | 14      |
| MCL          | 0.67      | 0.50   | 0.57     | 12      |
| accuracy     |           |        | 0.66     | 38      |
| macro avg    | 0.70      | 0.64   | 0.64     | 38      |
| weighted avg | 0.70      | 0.66   | 0.65     | 38      |



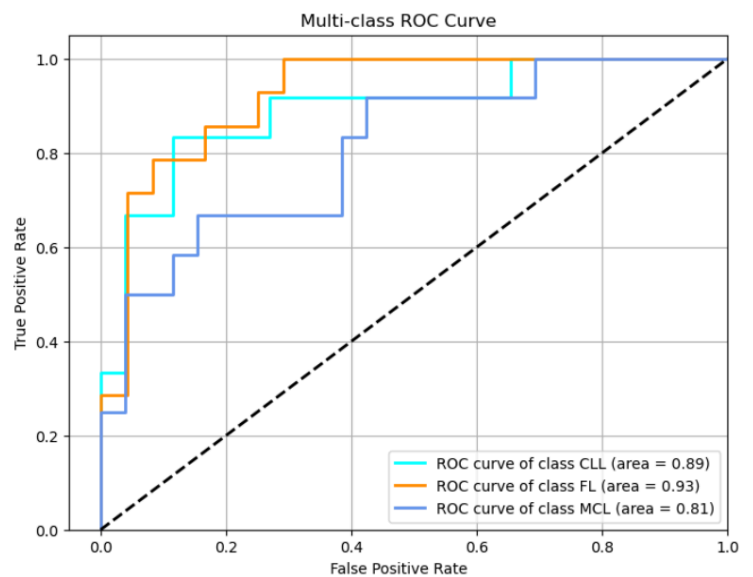


# Experimento 35 – EfficientNetV2B0 – Sin data augmentation – 30 epochs

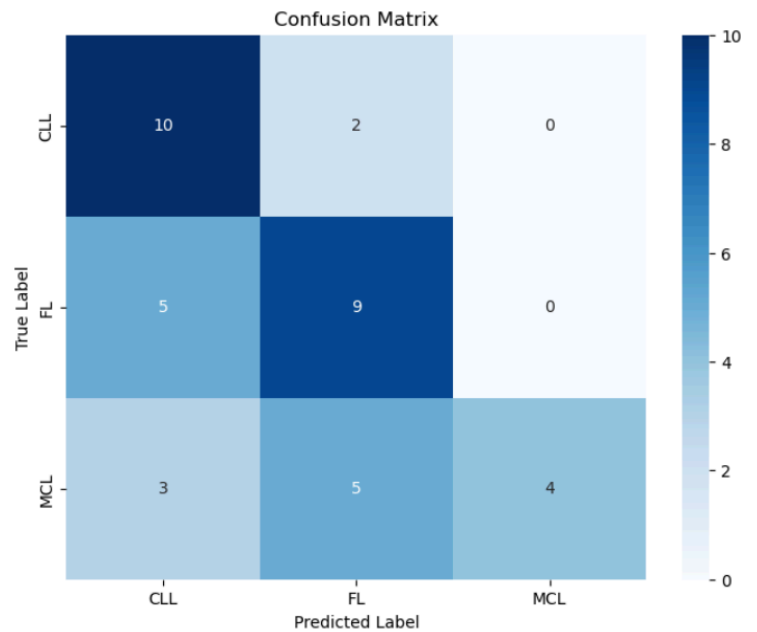
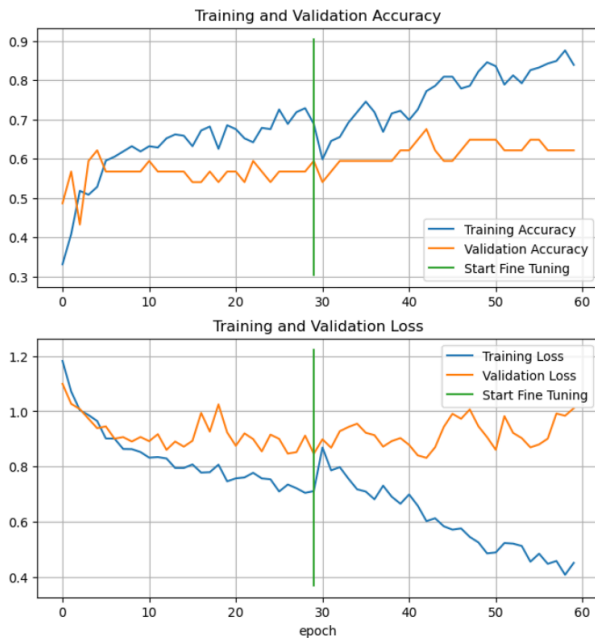


## Classification Report:

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| CLL          | 0.83      | 0.42   | 0.56     | 12      |
| FL           | 0.61      | 1.00   | 0.76     | 14      |
| MCL          | 0.67      | 0.50   | 0.57     | 12      |
| accuracy     |           |        | 0.66     | 38      |
| macro avg    | 0.70      | 0.64   | 0.63     | 38      |
| weighted avg | 0.70      | 0.66   | 0.63     | 38      |

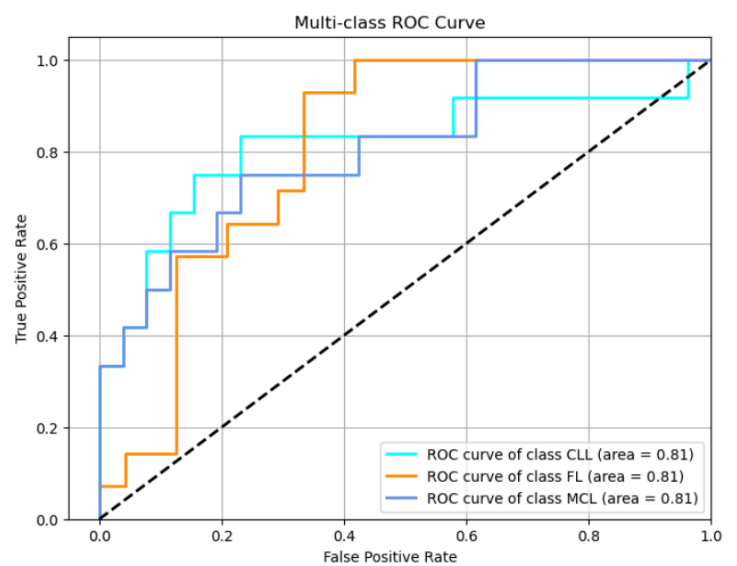


# Experimento 36 – EfficientNetV2B0 – Augmentación implícita – 30 epochs



## Classification Report:

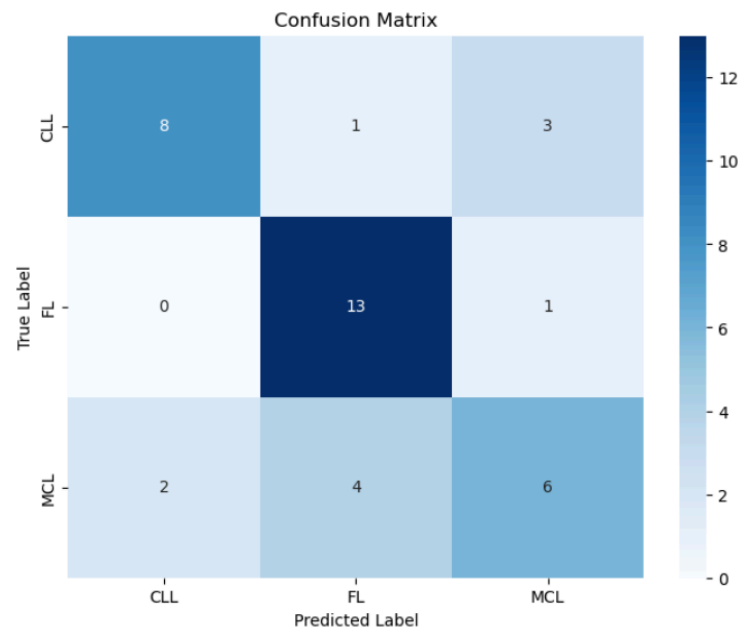
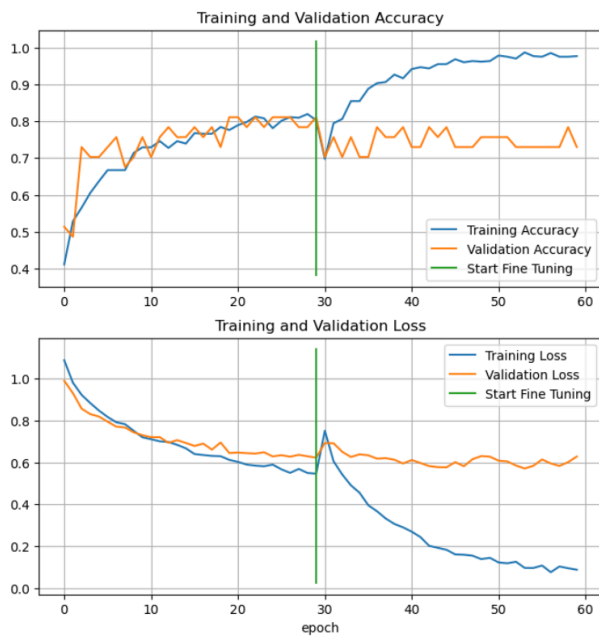
|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| CLL          | 0.56      | 0.83   | 0.67     | 12      |
| FL           | 0.56      | 0.64   | 0.60     | 14      |
| MCL          | 1.00      | 0.33   | 0.50     | 12      |
| accuracy     |           |        | 0.61     | 38      |
| macro avg    | 0.71      | 0.60   | 0.59     | 38      |
| weighted avg | 0.70      | 0.61   | 0.59     | 38      |



## Experimento 37 – EfficientNetV2B0 – Augmentación explícita (x2) – 30 epochs

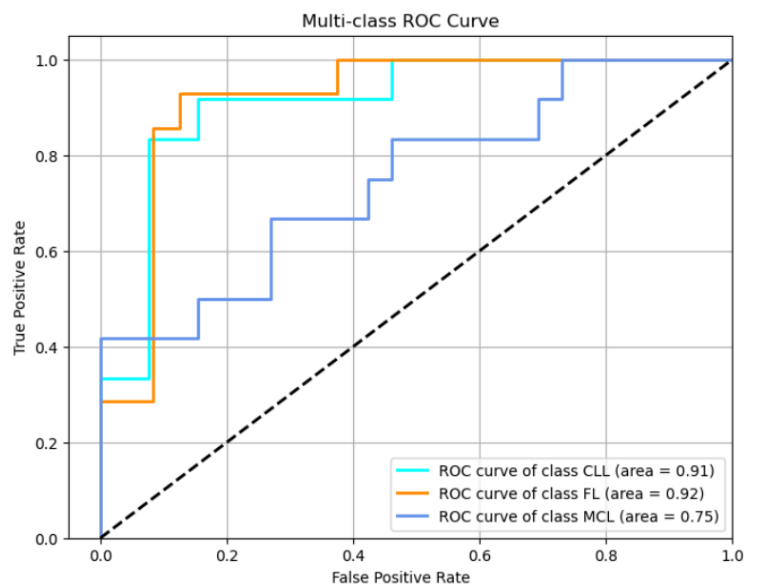
Original training set size: (299, 224, 224, 3)

Augmented training set size: (598, 224, 224, 3)



### Classification Report:

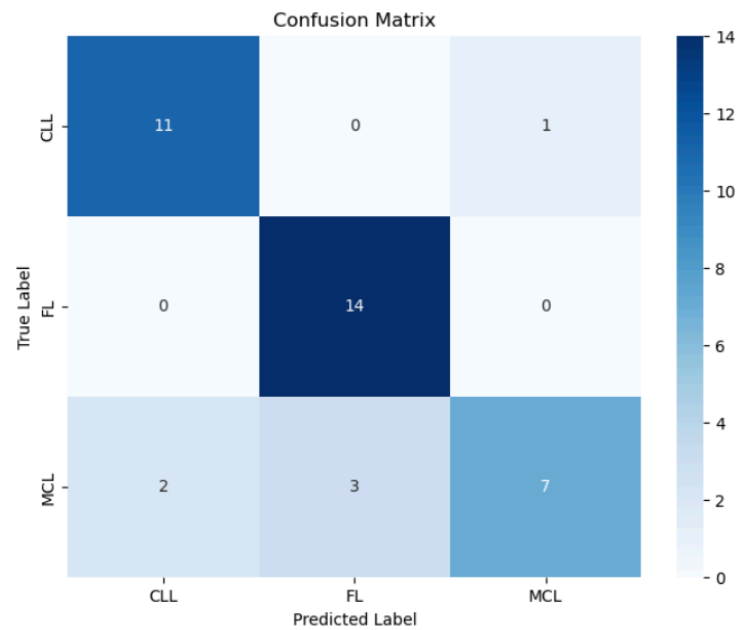
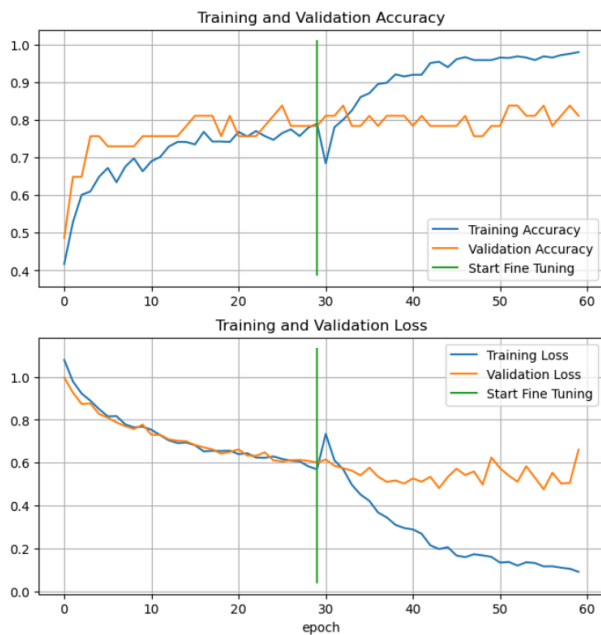
|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| CLL          | 0.80      | 0.67   | 0.73     | 12      |
| FL           | 0.72      | 0.93   | 0.81     | 14      |
| MCL          | 0.60      | 0.50   | 0.55     | 12      |
| accuracy     |           |        | 0.71     | 38      |
| macro avg    | 0.71      | 0.70   | 0.70     | 38      |
| weighted avg | 0.71      | 0.71   | 0.70     | 38      |



## Experimento 38 – EfficientNetV2B0 – Augmentación explícita (x3) – 30 epochs

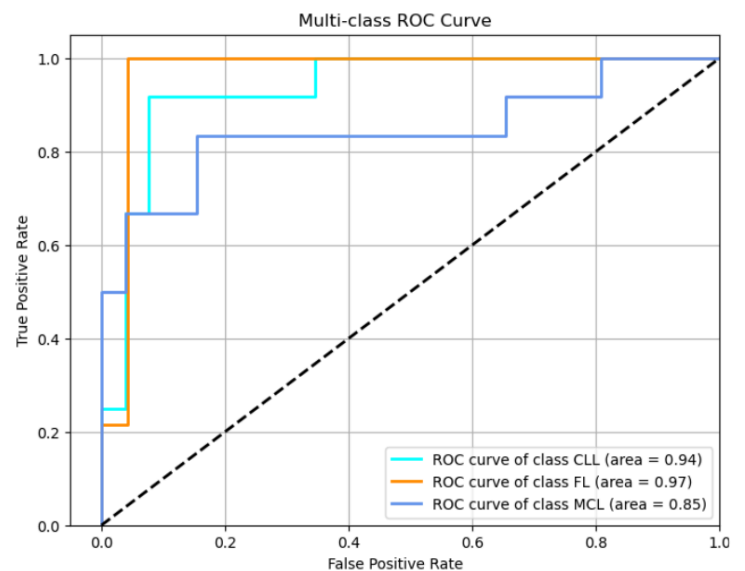
Original training set size: (299, 224, 224, 3)

Augmented training set size: (897, 224, 224, 3)



### Classification Report:

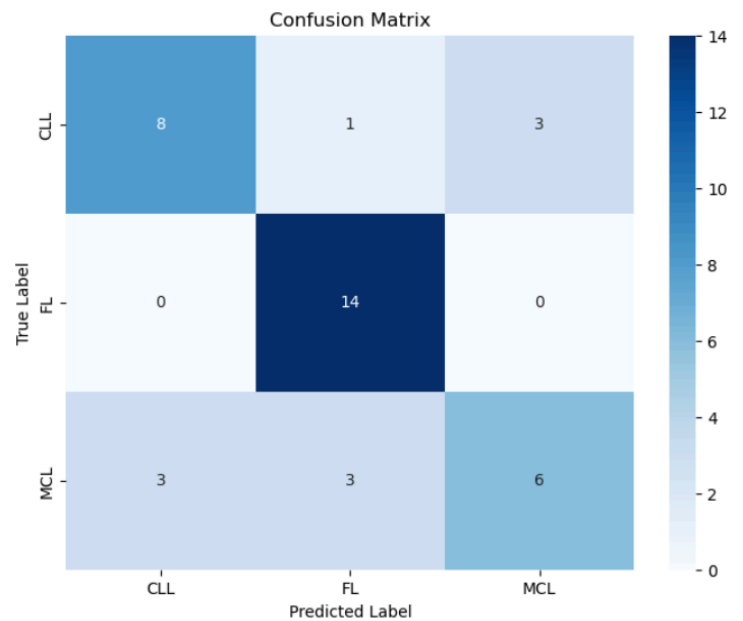
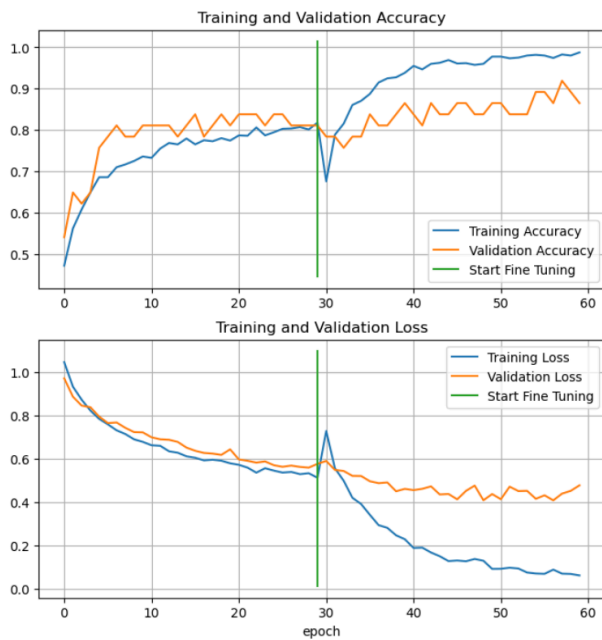
|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| CLL          | 0.85      | 0.92   | 0.88     | 12      |
| FL           | 0.82      | 1.00   | 0.90     | 14      |
| MCL          | 0.88      | 0.58   | 0.70     | 12      |
| accuracy     |           |        | 0.84     | 38      |
| macro avg    | 0.85      | 0.83   | 0.83     | 38      |
| weighted avg | 0.85      | 0.84   | 0.83     | 38      |



## Experimento 39 – EfficientNetV2B0 – Augmentación explícita (x4) – 30 epochs

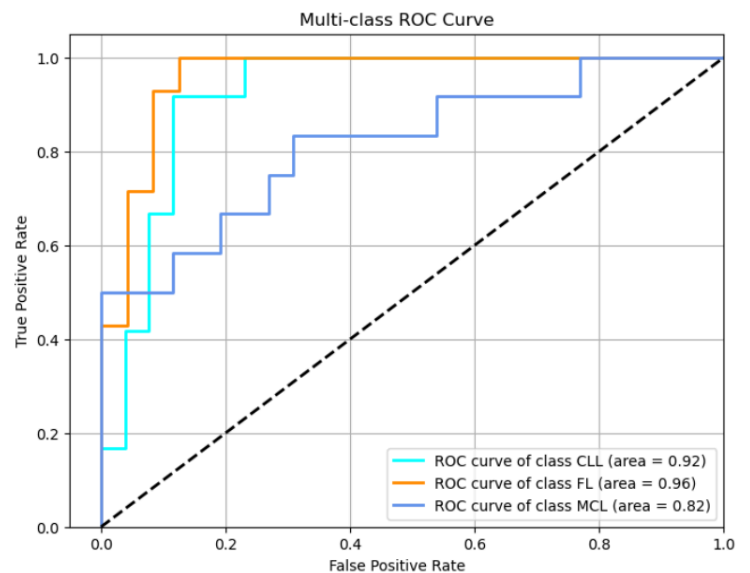
Original training set size: (299, 224, 224, 3)

Augmented training set size: (1196, 224, 224, 3)



### Classification Report:

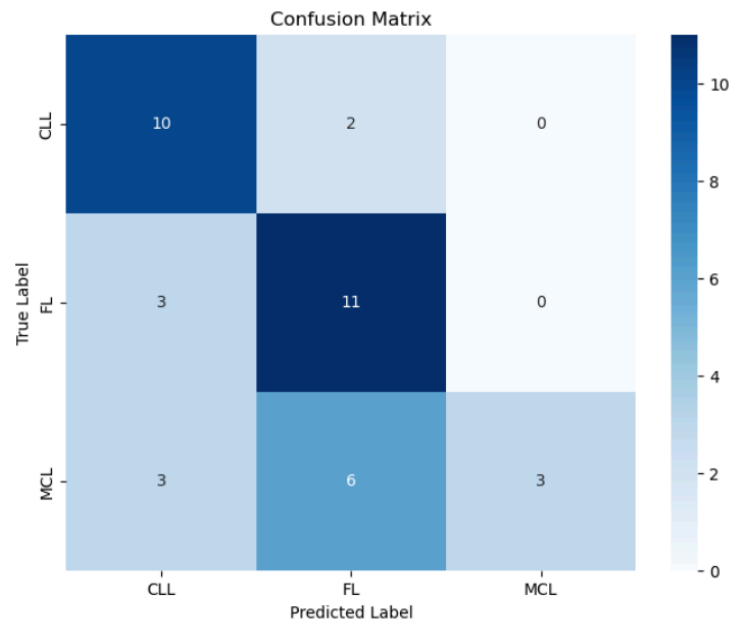
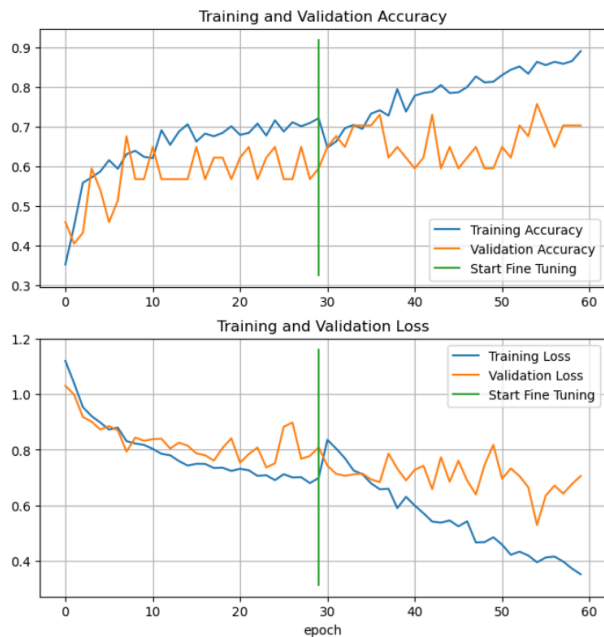
|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| CLL          | 0.73      | 0.67   | 0.70     | 12      |
| FL           | 0.78      | 1.00   | 0.88     | 14      |
| MCL          | 0.67      | 0.50   | 0.57     | 12      |
| accuracy     |           |        | 0.74     | 38      |
| macro avg    | 0.72      | 0.72   | 0.71     | 38      |
| weighted avg | 0.73      | 0.74   | 0.72     | 38      |



## Experimento 40 – EfficientNetV2B0 – Augmentación combinada (x2) – 30 epochs

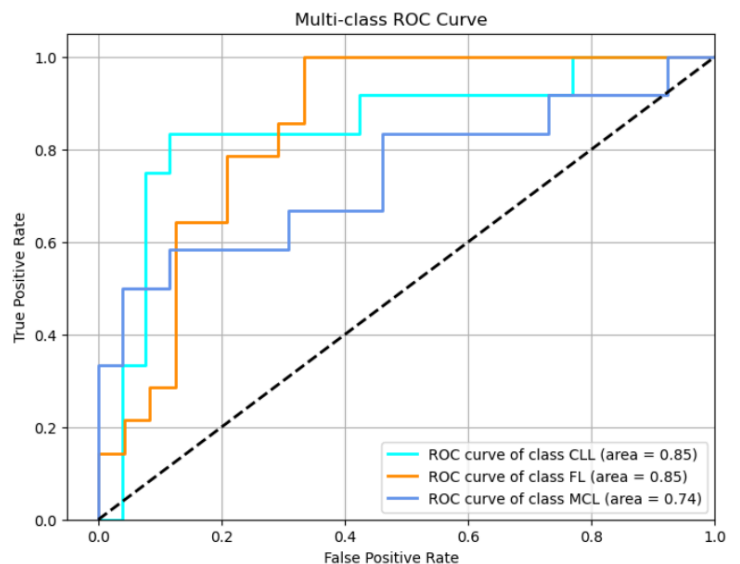
Original training set size: (299, 224, 224, 3)

Augmented training set size: (598, 224, 224, 3)



### Classification Report:

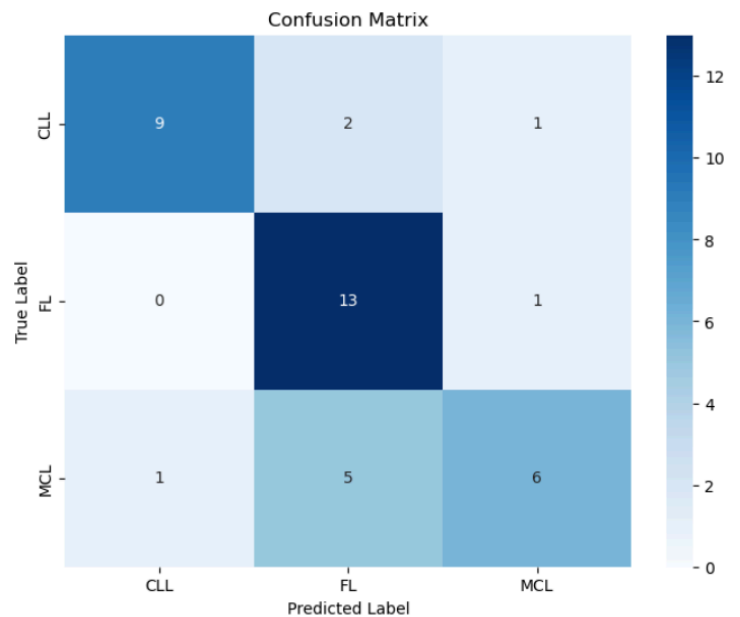
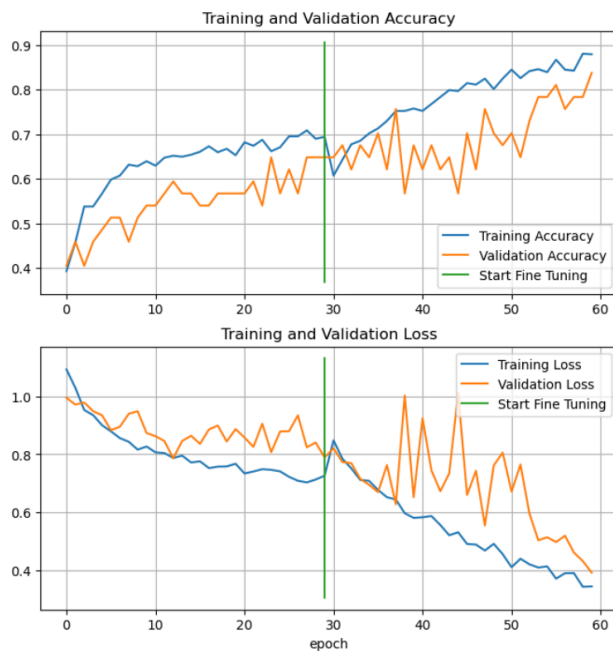
|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| CLL          | 0.62      | 0.83   | 0.71     | 12      |
| FL           | 0.58      | 0.79   | 0.67     | 14      |
| MCL          | 1.00      | 0.25   | 0.40     | 12      |
| accuracy     |           |        | 0.63     | 38      |
| macro avg    | 0.73      | 0.62   | 0.59     | 38      |
| weighted avg | 0.73      | 0.63   | 0.60     | 38      |



# Experimento 41 – EfficientNetV2B0 – Augmentación combinada (x3) – 30 epochs

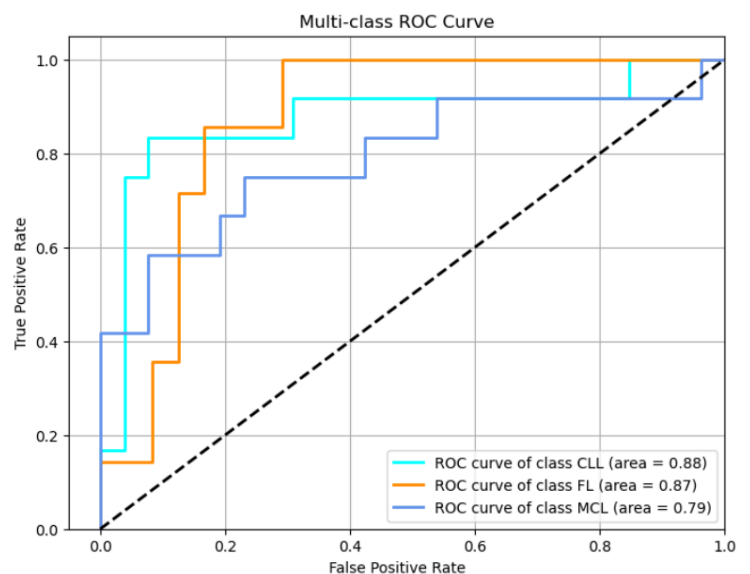
Original training set size: (299, 224, 224, 3)

Augmented training set size: (897, 224, 224, 3)



## Classification Report:

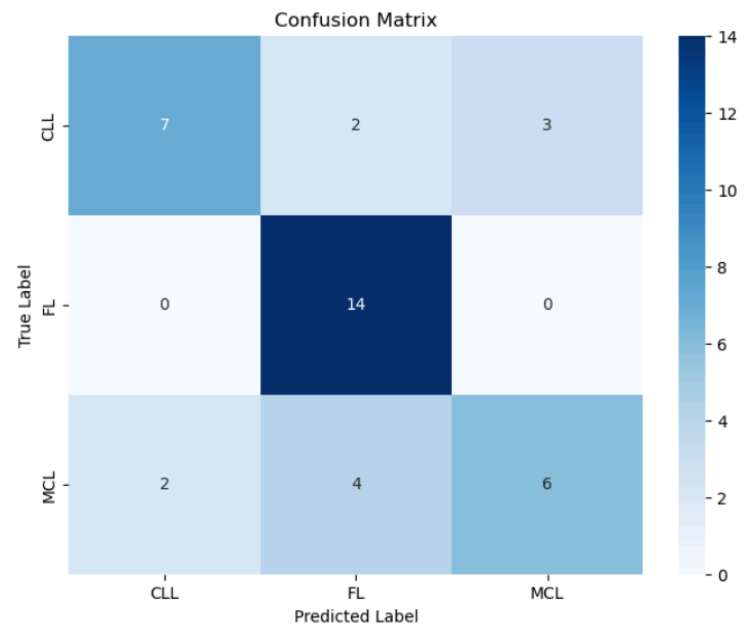
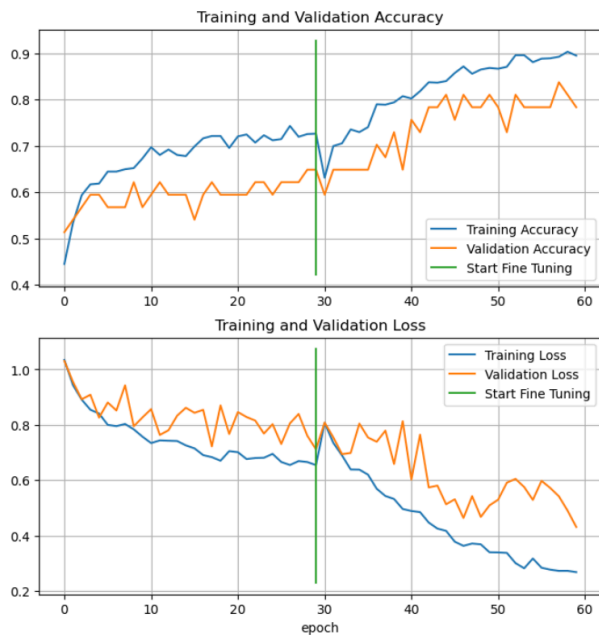
|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| CLL          | 0.90      | 0.75   | 0.82     | 12      |
| FL           | 0.65      | 0.93   | 0.76     | 14      |
| MCL          | 0.75      | 0.50   | 0.60     | 12      |
| accuracy     |           |        | 0.74     | 38      |
| macro avg    | 0.77      | 0.73   | 0.73     | 38      |
| weighted avg | 0.76      | 0.74   | 0.73     | 38      |



## Experimento 42 – EfficientNetV2B0 – Augmentación combinada (x4) – 30 epochs

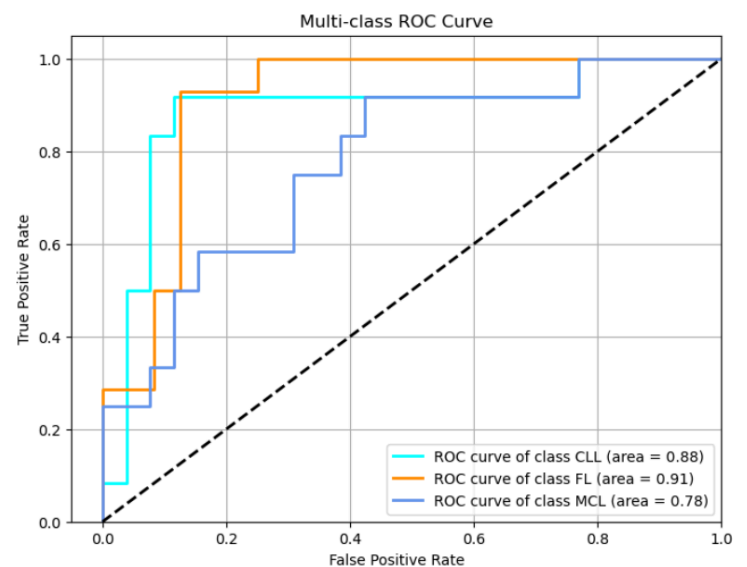
Original training set size: (299, 224, 224, 3)

Augmented training set size: (1196, 224, 224, 3)



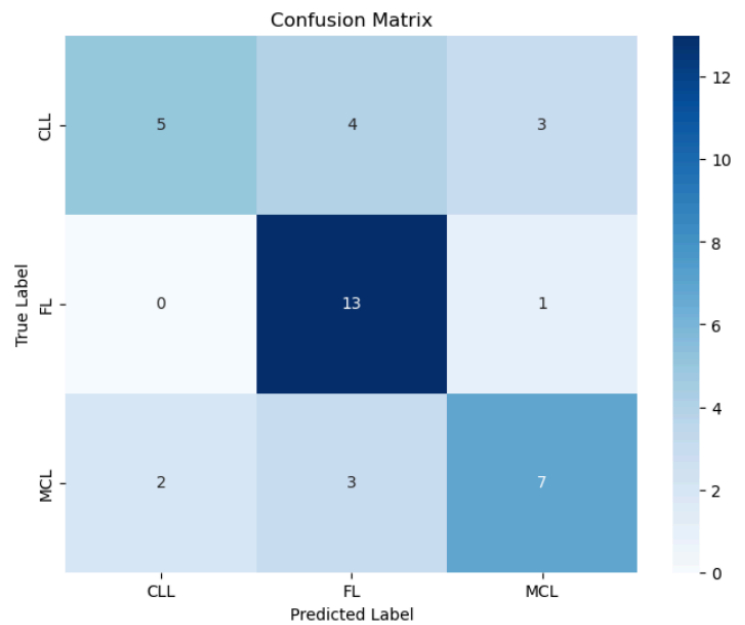
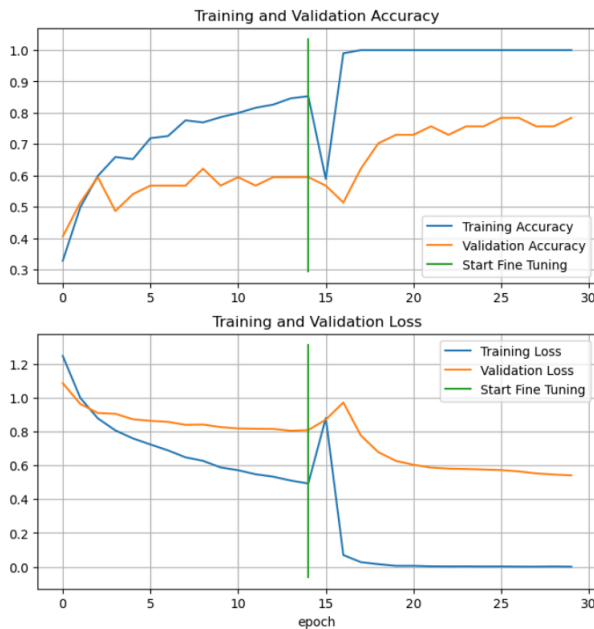
### Classification Report:

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| CLL          | 0.78      | 0.58   | 0.67     | 12      |
| FL           | 0.70      | 1.00   | 0.82     | 14      |
| MCL          | 0.67      | 0.50   | 0.57     | 12      |
| accuracy     |           |        | 0.71     | 38      |
| macro avg    | 0.71      | 0.69   | 0.69     | 38      |
| weighted avg | 0.71      | 0.71   | 0.69     | 38      |



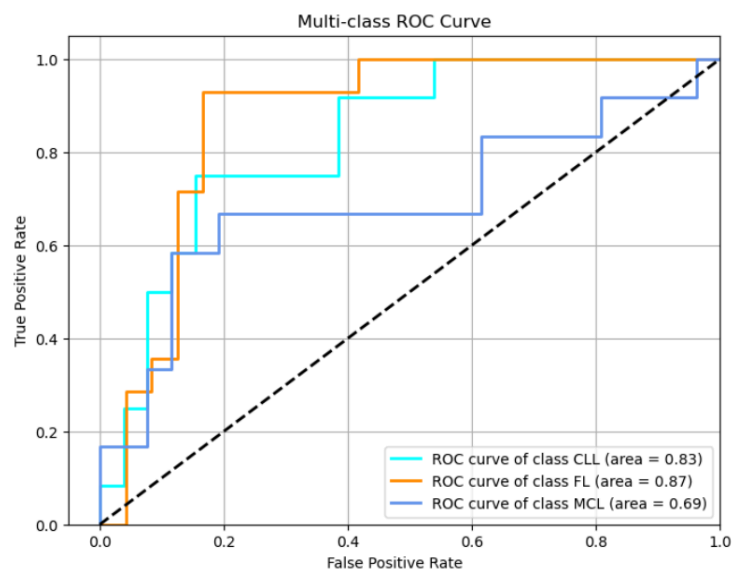


## Experimento 43 – ResNet50V2 – Sin data augmentation – 15 epochs

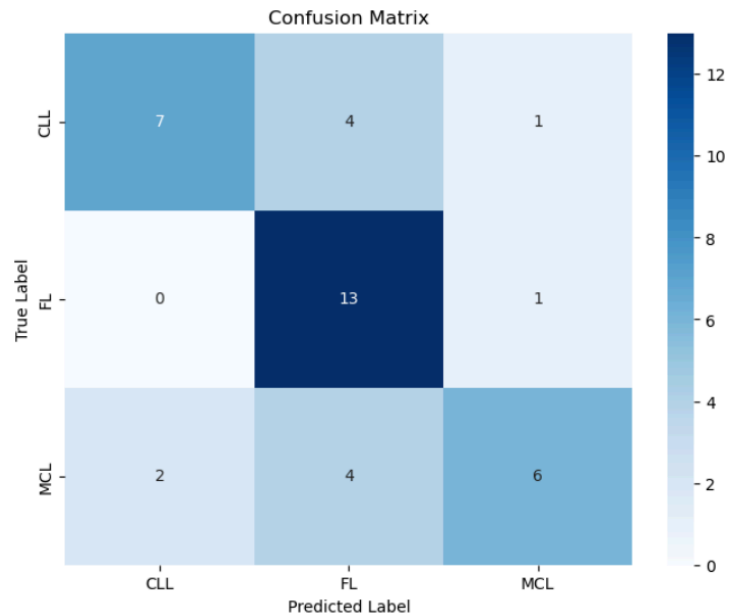
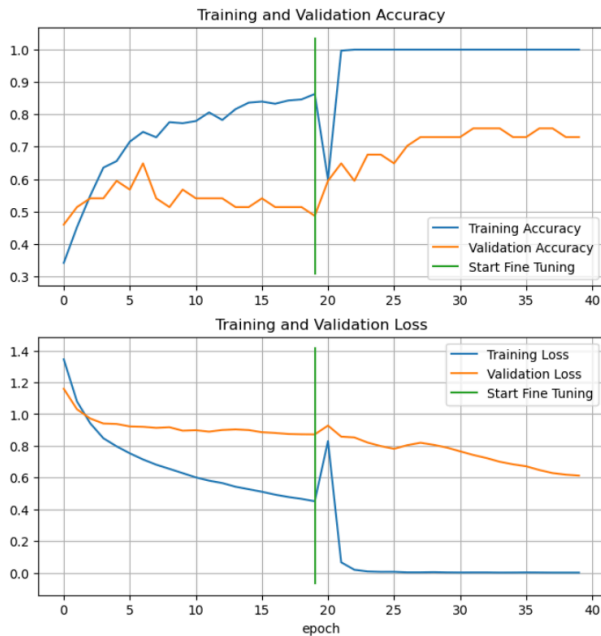


### Classification Report:

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| CLL          | 0.71      | 0.42   | 0.53     | 12      |
| FL           | 0.65      | 0.93   | 0.76     | 14      |
| MCL          | 0.64      | 0.58   | 0.61     | 12      |
| accuracy     |           |        | 0.66     | 38      |
| macro avg    | 0.67      | 0.64   | 0.63     | 38      |
| weighted avg | 0.67      | 0.66   | 0.64     | 38      |

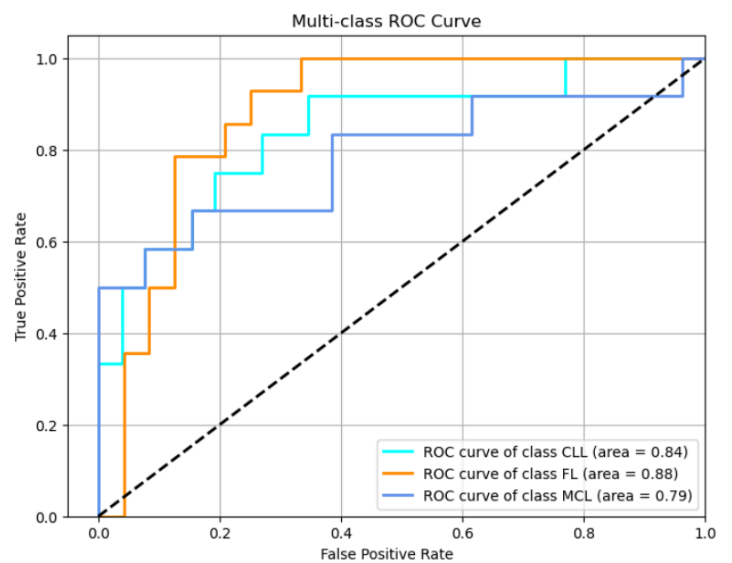


# Experimento 44 – ResNet50V2 – Sin data augmentation – 20 epochs

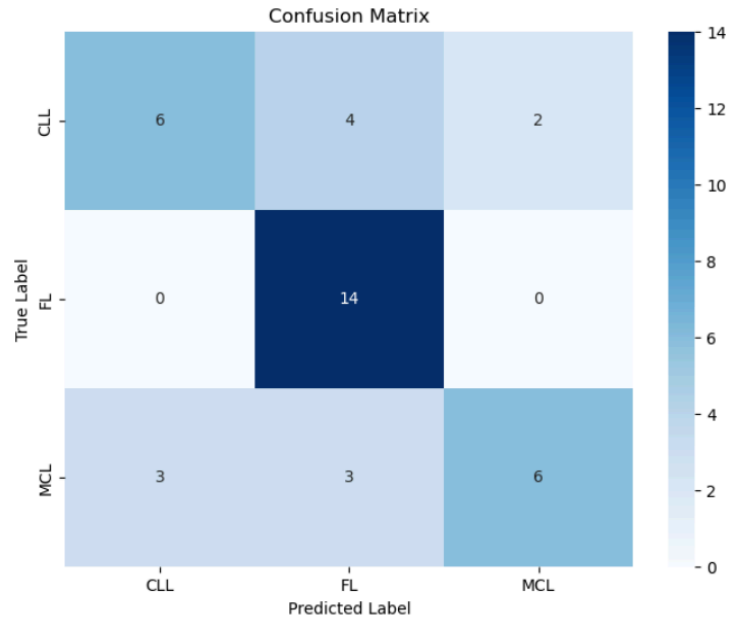
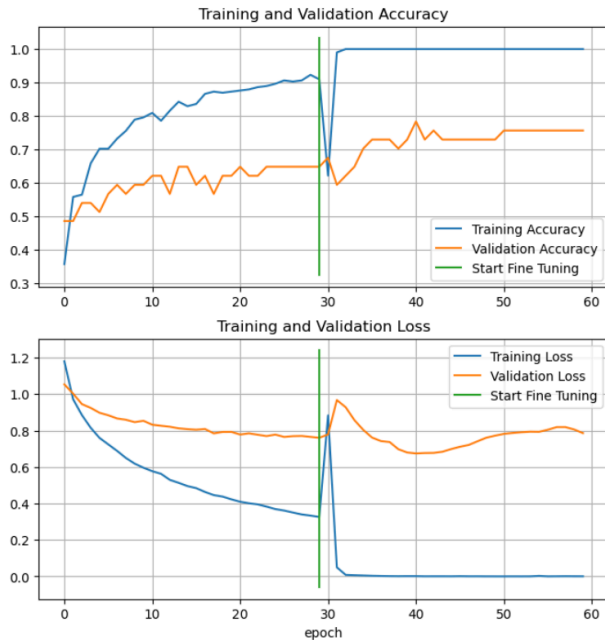


## Classification Report:

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| CLL          | 0.78      | 0.58   | 0.67     | 12      |
| FL           | 0.62      | 0.93   | 0.74     | 14      |
| MCL          | 0.75      | 0.50   | 0.60     | 12      |
| accuracy     |           |        | 0.68     | 38      |
| macro avg    | 0.72      | 0.67   | 0.67     | 38      |
| weighted avg | 0.71      | 0.68   | 0.67     | 38      |

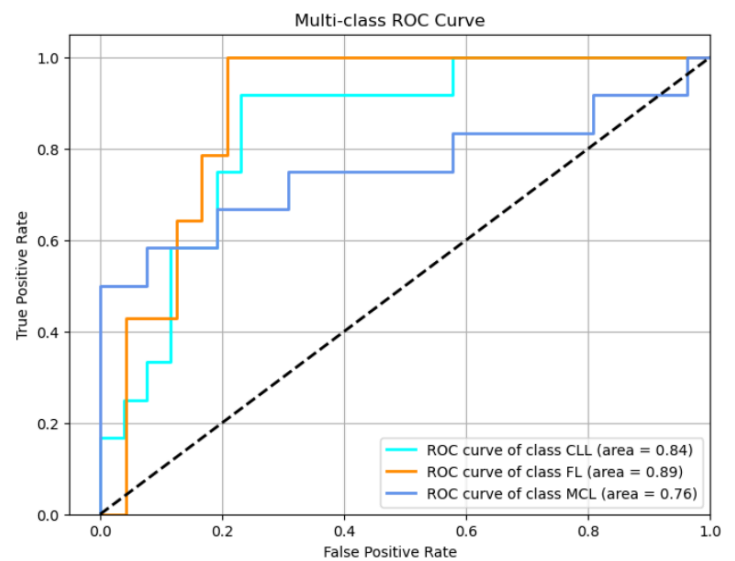


## Experimento 45 – ResNet50V2 – Sin data augmentation – 30 epochs

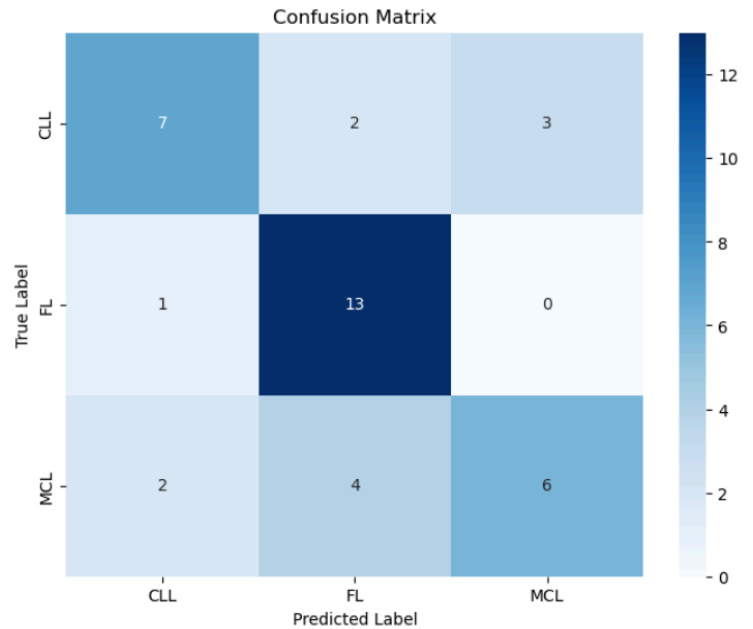
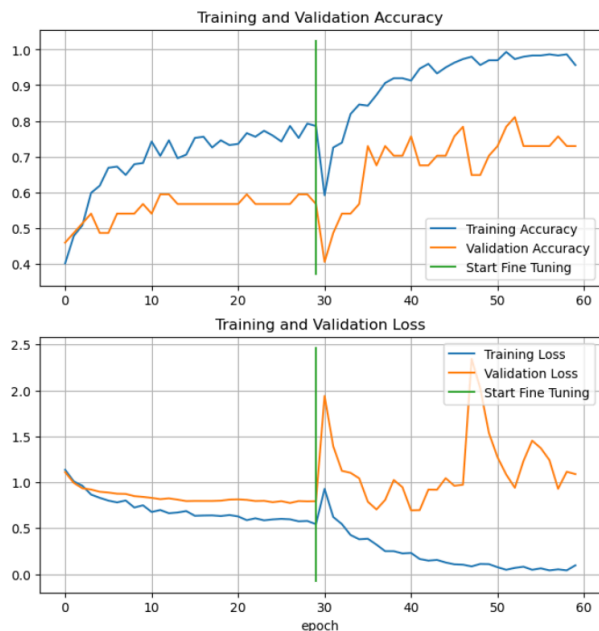


### Classification Report:

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| CLL          | 0.67      | 0.50   | 0.57     | 12      |
| FL           | 0.67      | 1.00   | 0.80     | 14      |
| MCL          | 0.75      | 0.50   | 0.60     | 12      |
| accuracy     |           |        | 0.68     | 38      |
| macro avg    | 0.69      | 0.67   | 0.66     | 38      |
| weighted avg | 0.69      | 0.68   | 0.66     | 38      |

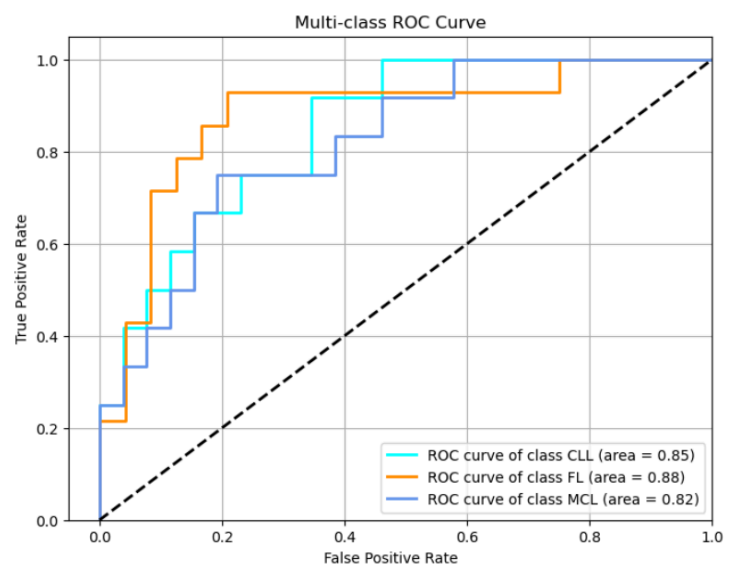


# Experimento 46 – ResNet50V2 – Augmentación implícita – 30 epochs



## Classification Report:

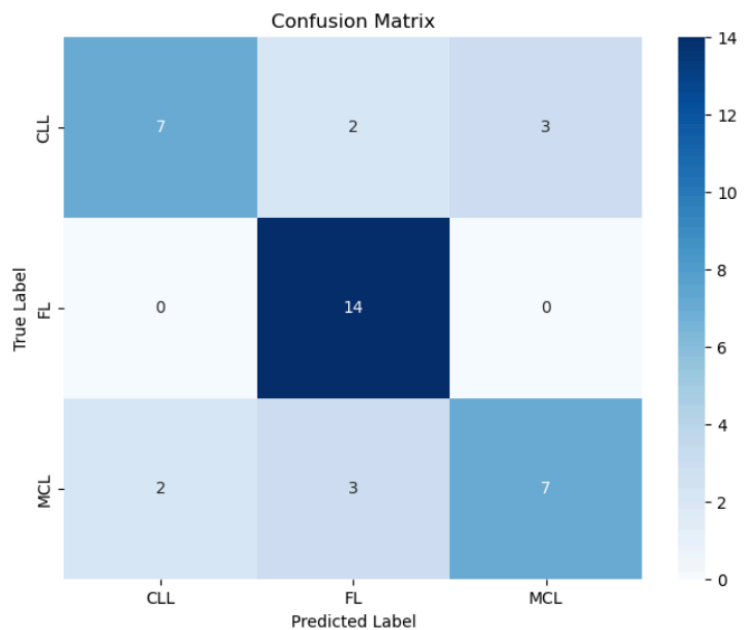
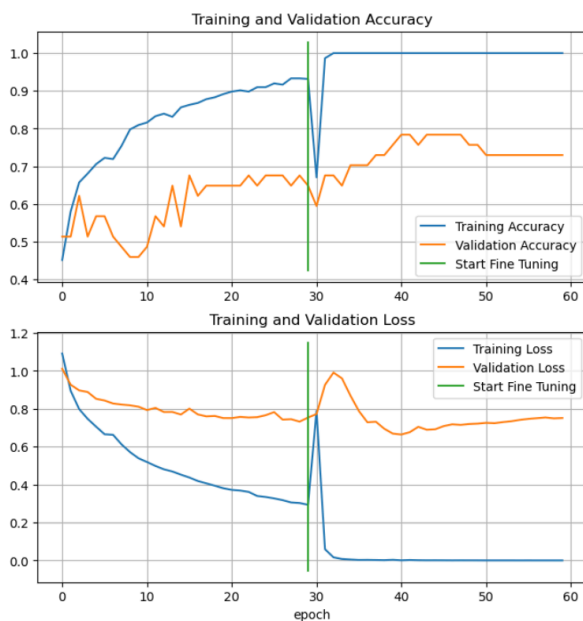
|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| CLL          | 0.70      | 0.58   | 0.64     | 12      |
| FL           | 0.68      | 0.93   | 0.79     | 14      |
| MCL          | 0.67      | 0.50   | 0.57     | 12      |
| accuracy     |           |        | 0.68     | 38      |
| macro avg    | 0.68      | 0.67   | 0.67     | 38      |
| weighted avg | 0.68      | 0.68   | 0.67     | 38      |



# Experimento 47 – ResNet50V2 – Augmentación explícita (x2) – 30 epochs

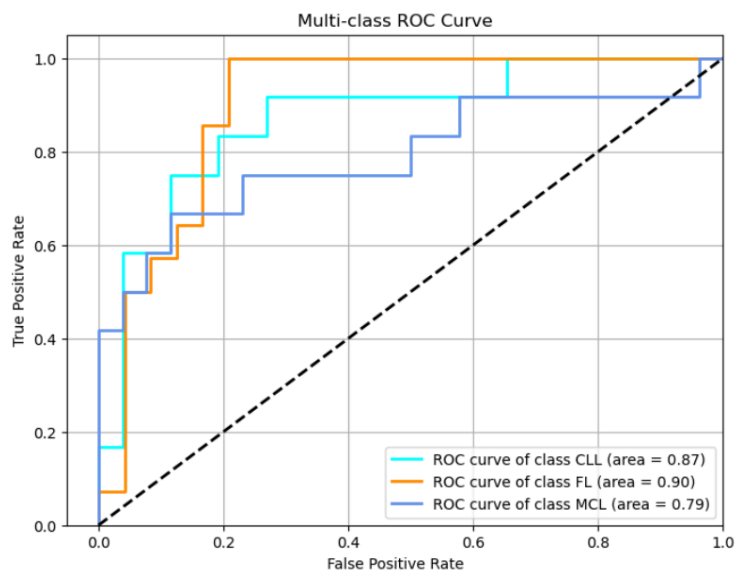
Original training set size: (299, 224, 224, 3)

Augmented training set size: (598, 224, 224, 3)



## Classification Report:

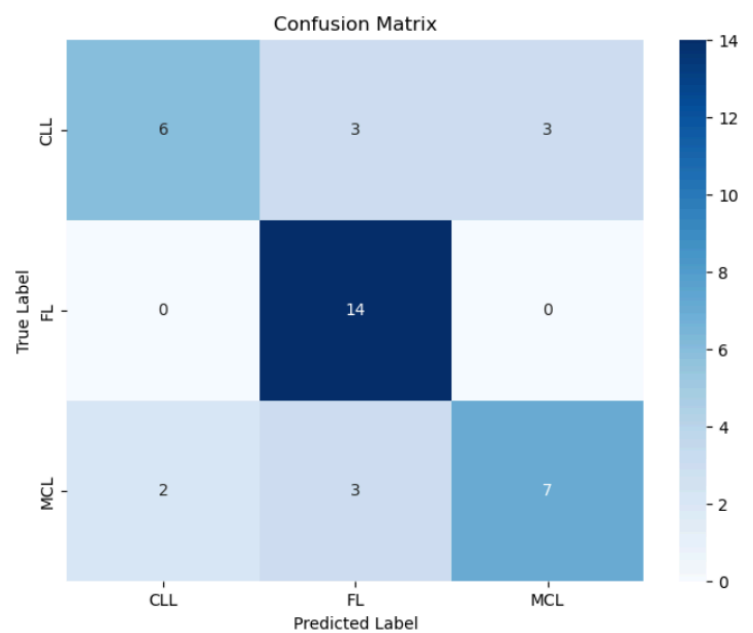
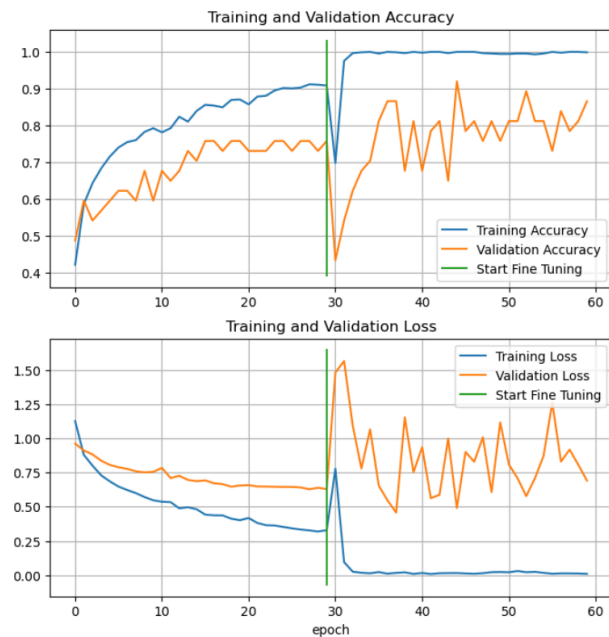
|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| CLL          | 0.78      | 0.58   | 0.67     | 12      |
| FL           | 0.74      | 1.00   | 0.85     | 14      |
| MCL          | 0.70      | 0.58   | 0.64     | 12      |
| accuracy     |           |        | 0.74     | 38      |
| macro avg    | 0.74      | 0.72   | 0.72     | 38      |
| weighted avg | 0.74      | 0.74   | 0.72     | 38      |



# Experimento 48 – ResNet50V2 – Augmentación explícita (x3) – 30 epochs

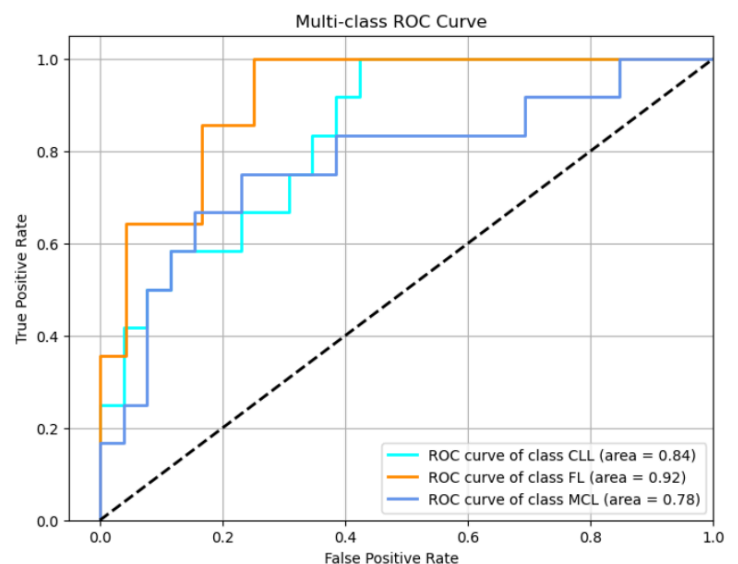
Original training set size: (299, 224, 224, 3)

Augmented training set size: (897, 224, 224, 3)



## Classification Report:

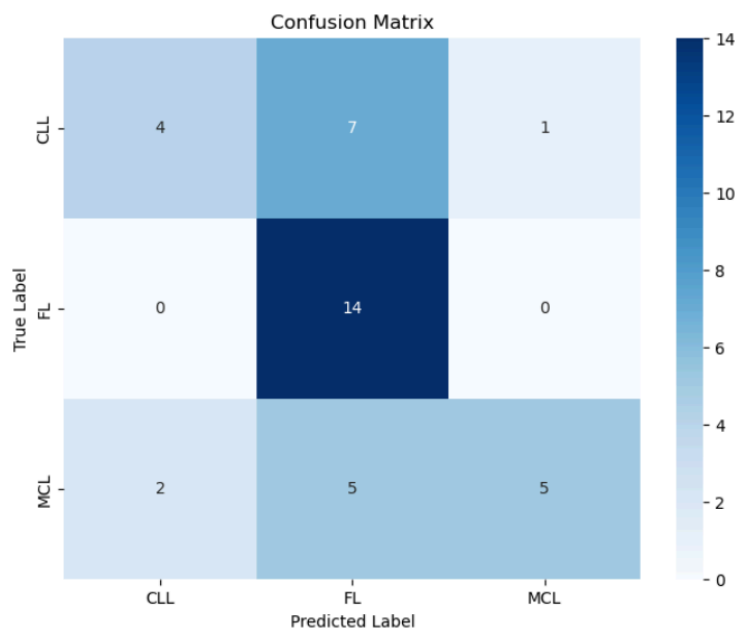
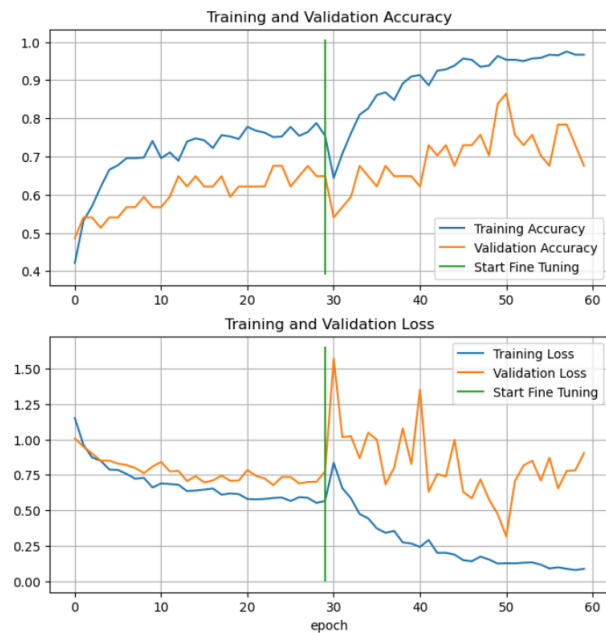
|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| CLL          | 0.75      | 0.50   | 0.60     | 12      |
| FL           | 0.70      | 1.00   | 0.82     | 14      |
| MCL          | 0.70      | 0.58   | 0.64     | 12      |
| accuracy     |           |        | 0.71     | 38      |
| macro avg    | 0.72      | 0.69   | 0.69     | 38      |
| weighted avg | 0.72      | 0.71   | 0.69     | 38      |



## Experimento 49 – ResNet50V2 – Augmentación combinada (x2) – 30 epochs

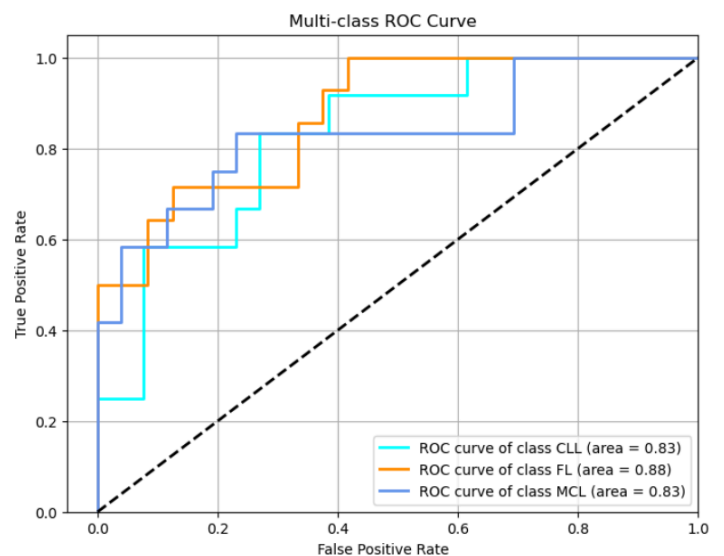
Original training set size: (299, 224, 224, 3)

Augmented training set size: (598, 224, 224, 3)



Classification Report:

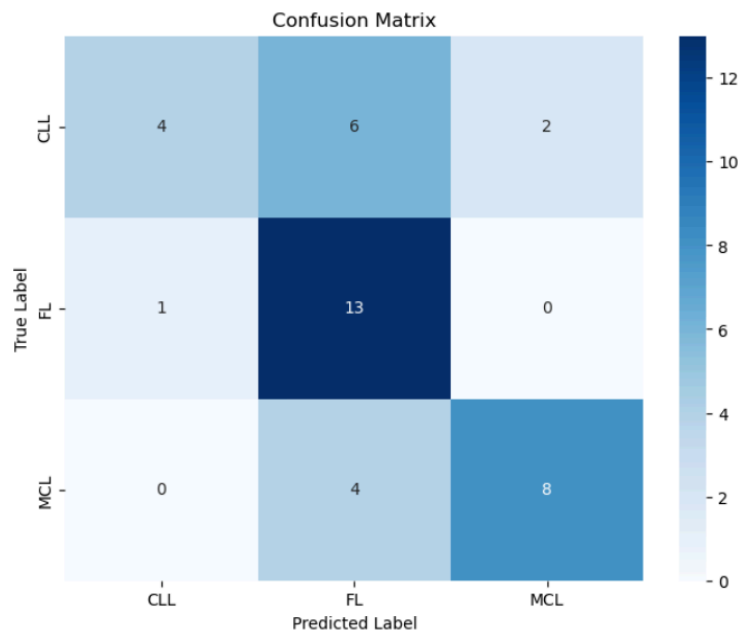
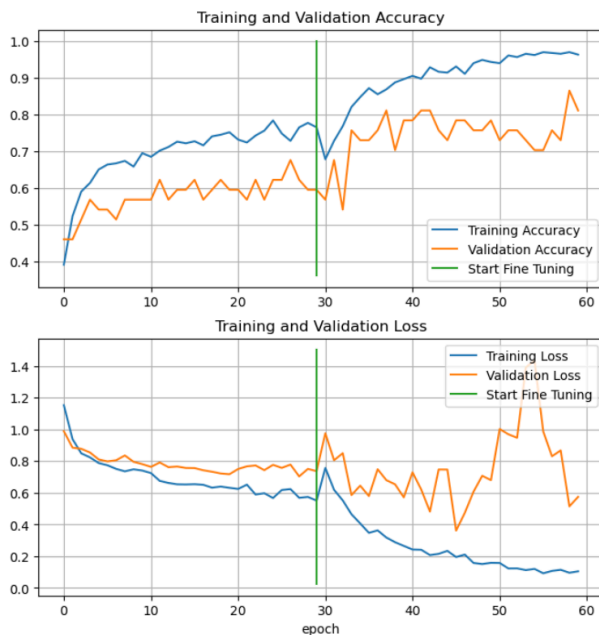
|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| CLL          | 0.67      | 0.33   | 0.44     | 12      |
| FL           | 0.54      | 1.00   | 0.70     | 14      |
| MCL          | 0.83      | 0.42   | 0.56     | 12      |
| accuracy     |           |        | 0.61     | 38      |
| macro avg    | 0.68      | 0.58   | 0.57     | 38      |
| weighted avg | 0.67      | 0.61   | 0.57     | 38      |



# Experimento 50 – ResNet50V2 – Augmentación combinada (x3) – 30 epochs

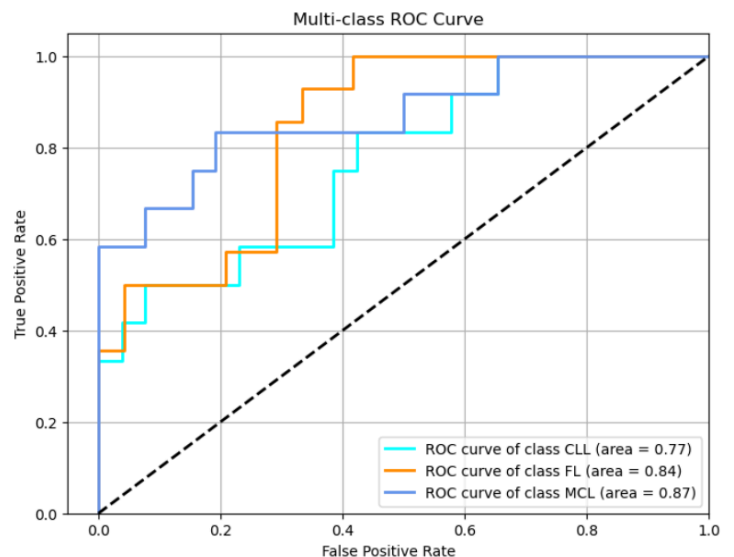
Original training set size: (299, 224, 224, 3)

Augmented training set size: (897, 224, 224, 3)



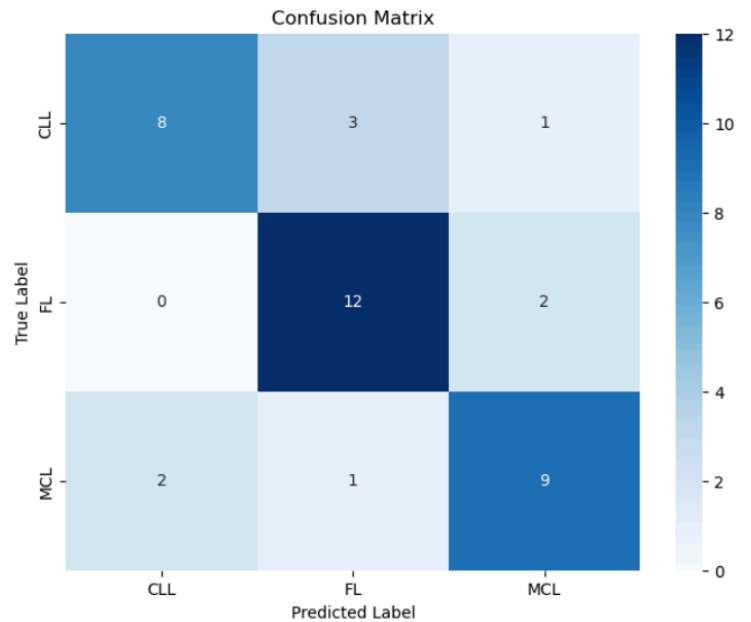
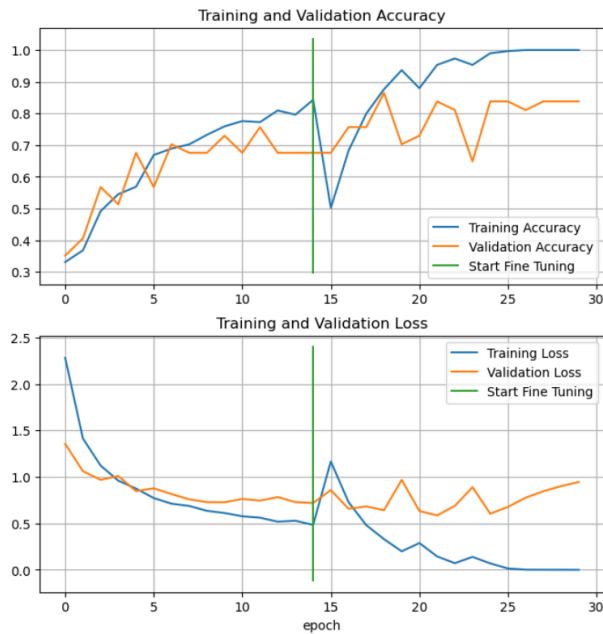
## Classification Report:

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| CLL          | 0.80      | 0.33   | 0.47     | 12      |
| FL           | 0.57      | 0.93   | 0.70     | 14      |
| MCL          | 0.80      | 0.67   | 0.73     | 12      |
| accuracy     |           |        | 0.66     | 38      |
| macro avg    | 0.72      | 0.64   | 0.63     | 38      |
| weighted avg | 0.71      | 0.66   | 0.64     | 38      |



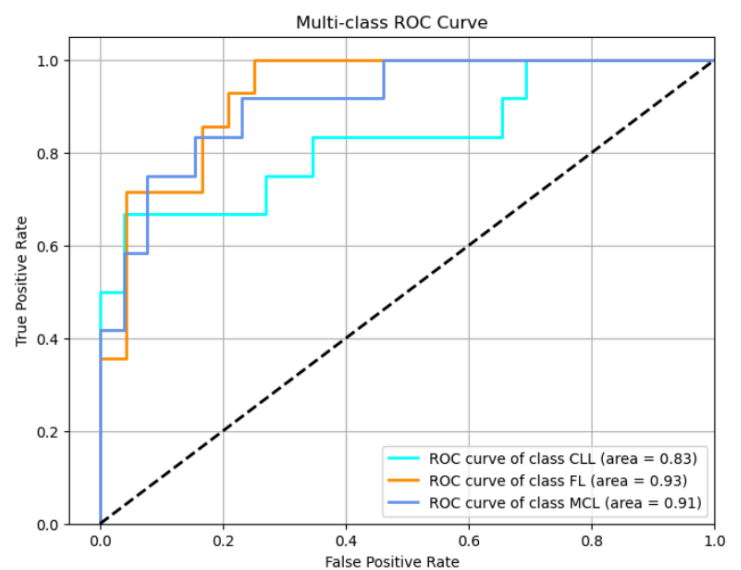


## Experimento 51 – VGG16 – Sin data augmentation – 15 epochs – Batch size 8

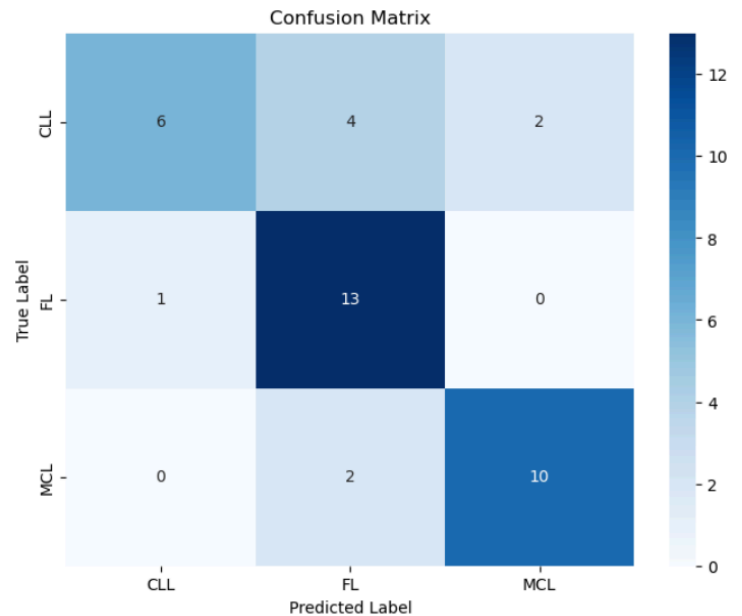
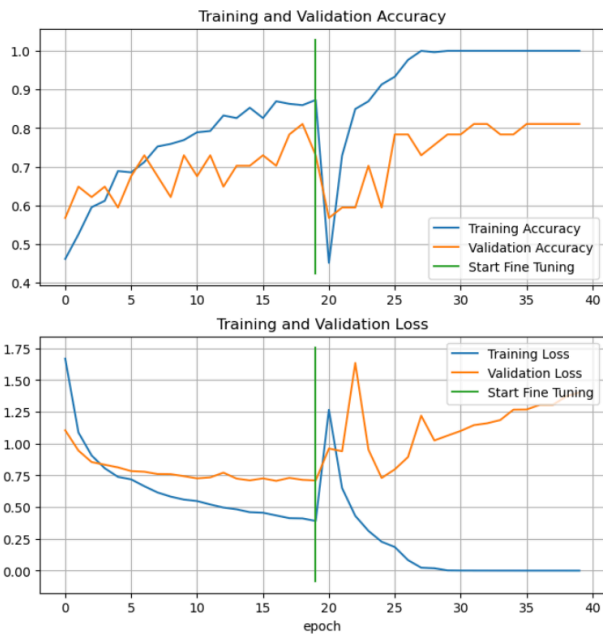


### Classification Report:

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| CLL          | 0.80      | 0.67   | 0.73     | 12      |
| FL           | 0.75      | 0.86   | 0.80     | 14      |
| MCL          | 0.75      | 0.75   | 0.75     | 12      |
| accuracy     |           |        | 0.76     | 38      |
| macro avg    | 0.77      | 0.76   | 0.76     | 38      |
| weighted avg | 0.77      | 0.76   | 0.76     | 38      |

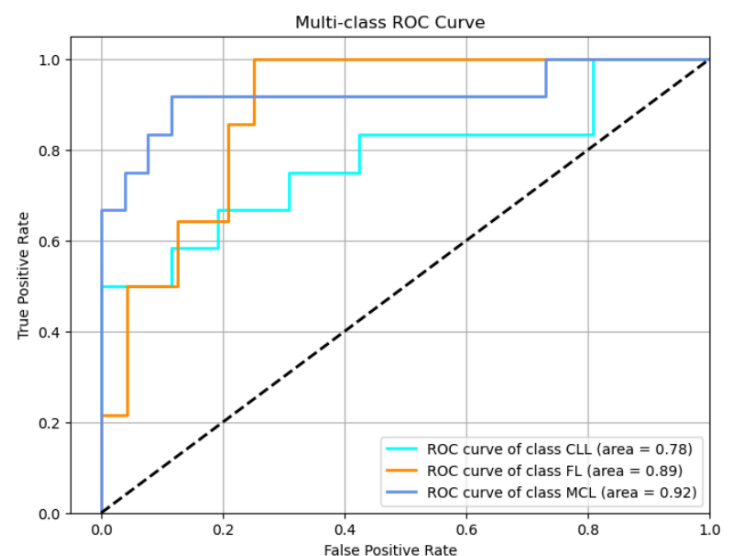


# Experimento 52 – VGG16 – Sin data augmentation – 20 epochs – Batch size 8

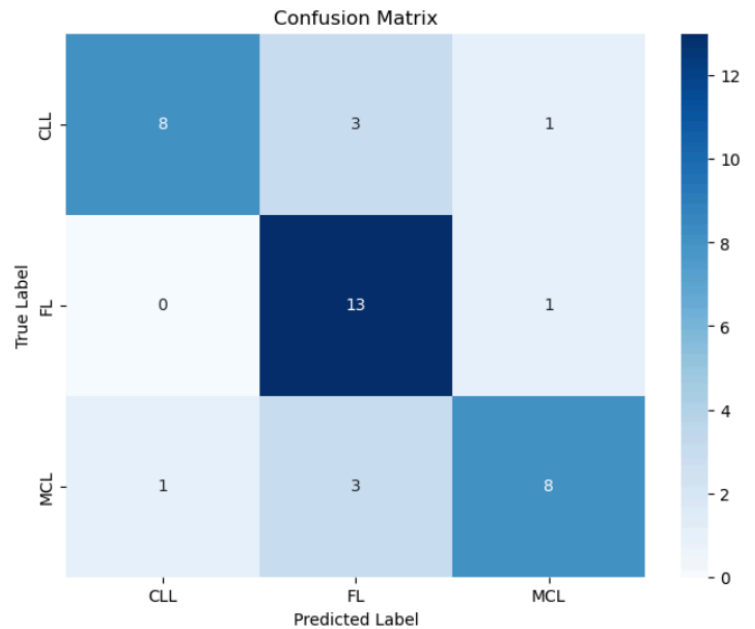
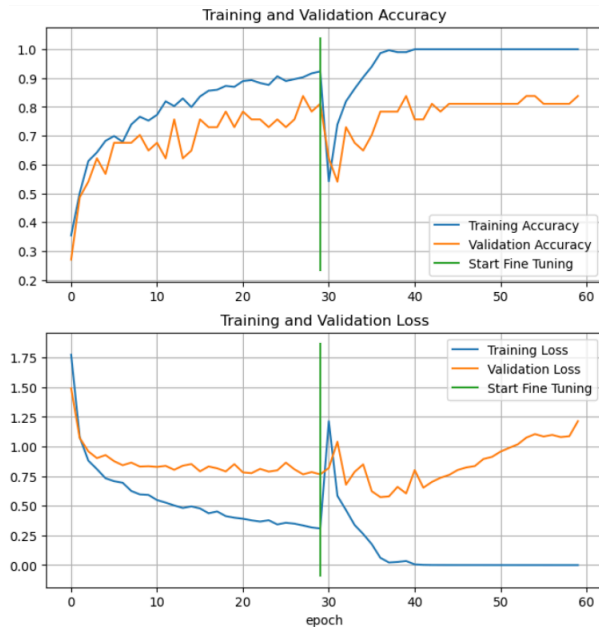


## Classification Report:

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| CLL          | 0.86      | 0.50   | 0.63     | 12      |
| FL           | 0.68      | 0.93   | 0.79     | 14      |
| MCL          | 0.83      | 0.83   | 0.83     | 12      |
| accuracy     |           |        | 0.76     | 38      |
| macro avg    | 0.79      | 0.75   | 0.75     | 38      |
| weighted avg | 0.79      | 0.76   | 0.75     | 38      |

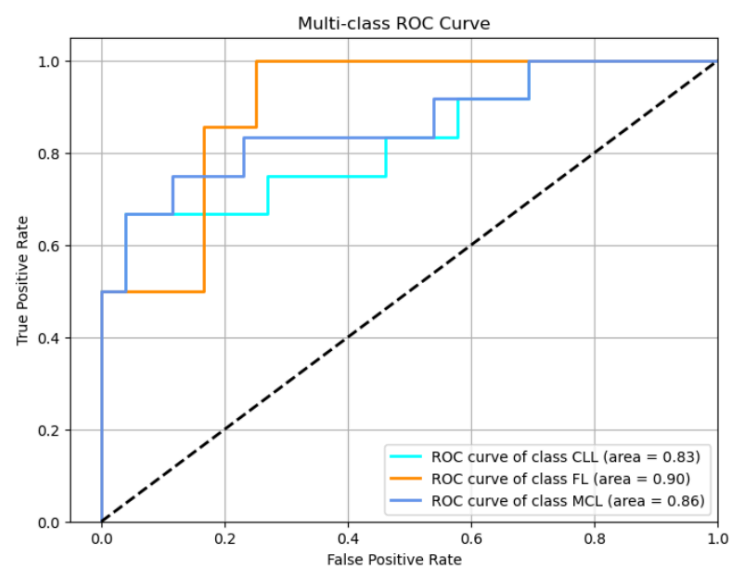


## Experimento 53 – VGG16 – Sin data augmentation – 30 epochs – Batch size 8

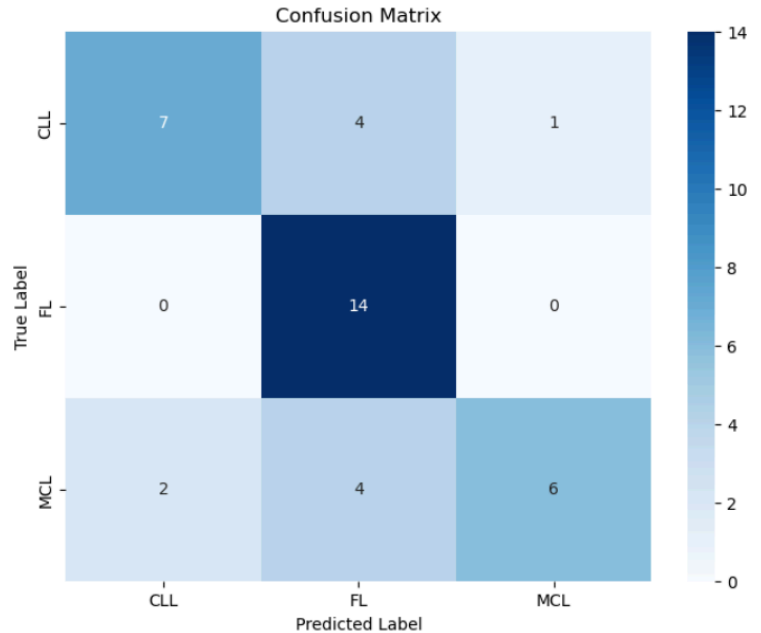
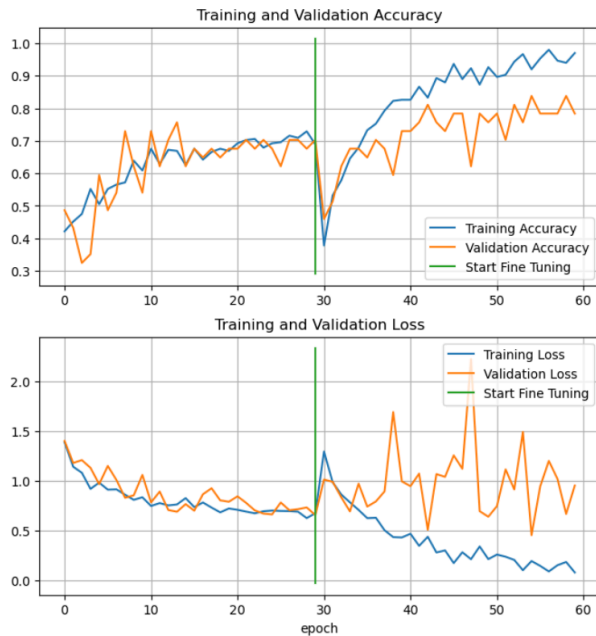


### Classification Report:

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| CLL          | 0.89      | 0.67   | 0.76     | 12      |
| FL           | 0.68      | 0.93   | 0.79     | 14      |
| MCL          | 0.80      | 0.67   | 0.73     | 12      |
| accuracy     |           |        | 0.76     | 38      |
| macro avg    | 0.79      | 0.75   | 0.76     | 38      |
| weighted avg | 0.79      | 0.76   | 0.76     | 38      |

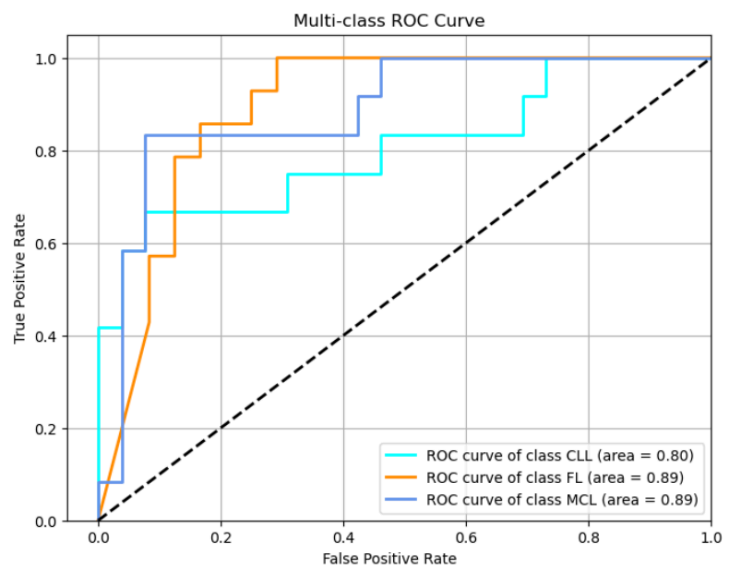


## Experimento 54 – VGG16 – Augmentación implícita – 30 epochs – Batch size 8



### Classification Report:

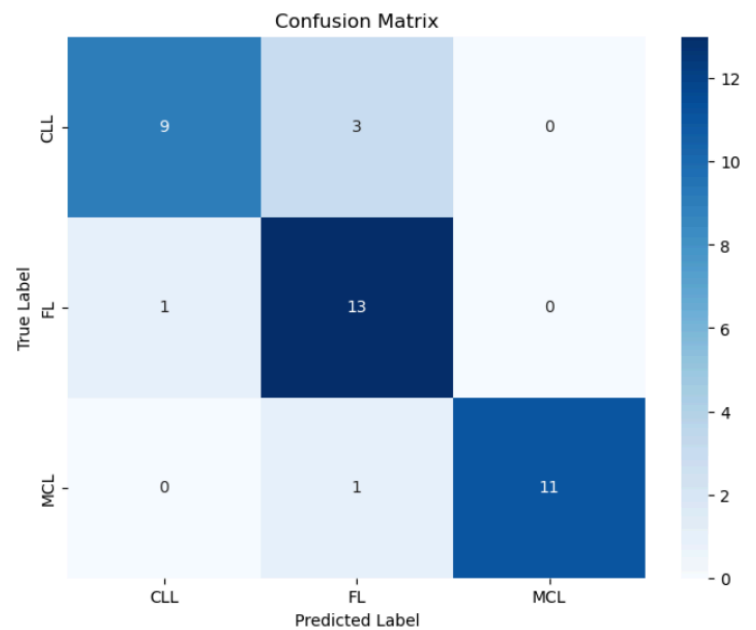
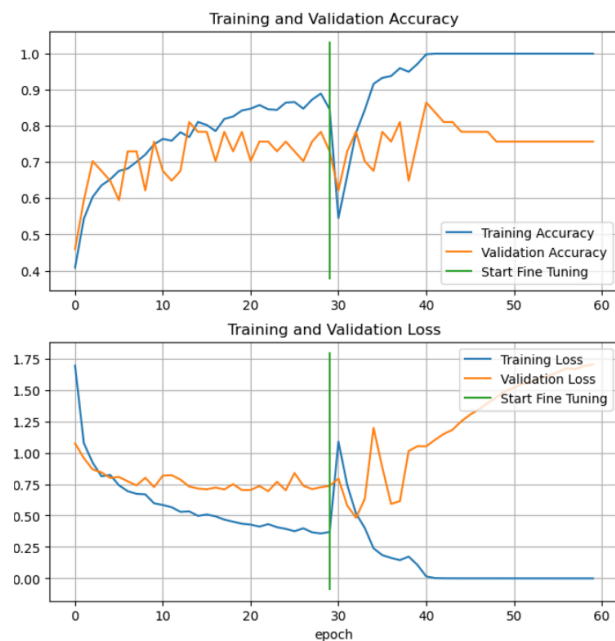
|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| CLL          | 0.78      | 0.58   | 0.67     | 12      |
| FL           | 0.64      | 1.00   | 0.78     | 14      |
| MCL          | 0.86      | 0.50   | 0.63     | 12      |
| accuracy     |           |        | 0.71     | 38      |
| macro avg    | 0.76      | 0.69   | 0.69     | 38      |
| weighted avg | 0.75      | 0.71   | 0.70     | 38      |



## Experimento 55 – VGG16 – Augmentación explícita (x2) – 30 epochs – Batch size 8

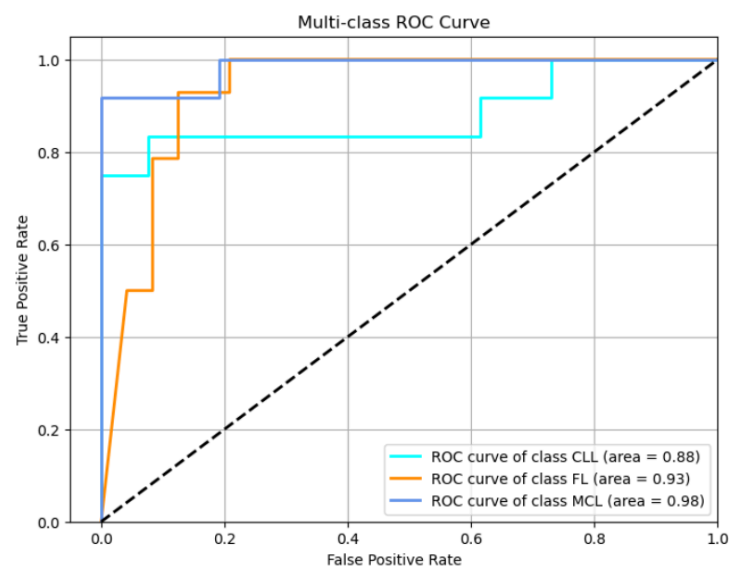
Original training set size: (299, 224, 224, 3)

Augmented training set size: (598, 224, 224, 3)



Classification Report:

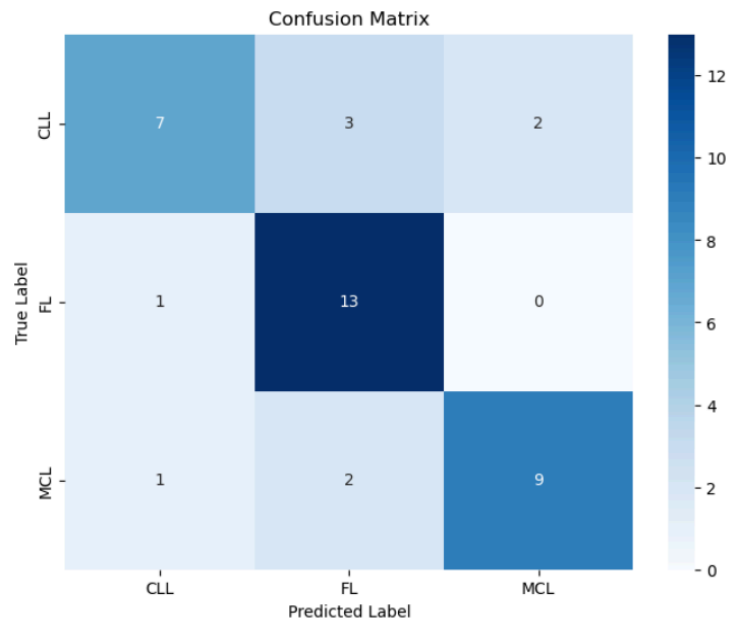
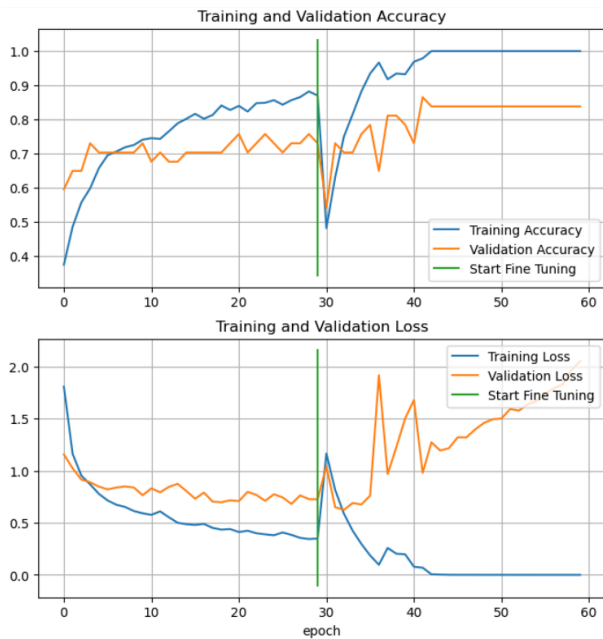
|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| CLL          | 0.90      | 0.75   | 0.82     | 12      |
| FL           | 0.76      | 0.93   | 0.84     | 14      |
| MCL          | 1.00      | 0.92   | 0.96     | 12      |
| accuracy     |           |        | 0.87     | 38      |
| macro avg    | 0.89      | 0.87   | 0.87     | 38      |
| weighted avg | 0.88      | 0.87   | 0.87     | 38      |



## Experimento 56 – VGG16 – Augmentación explícita (x3) – 30 epochs – Batch size 8

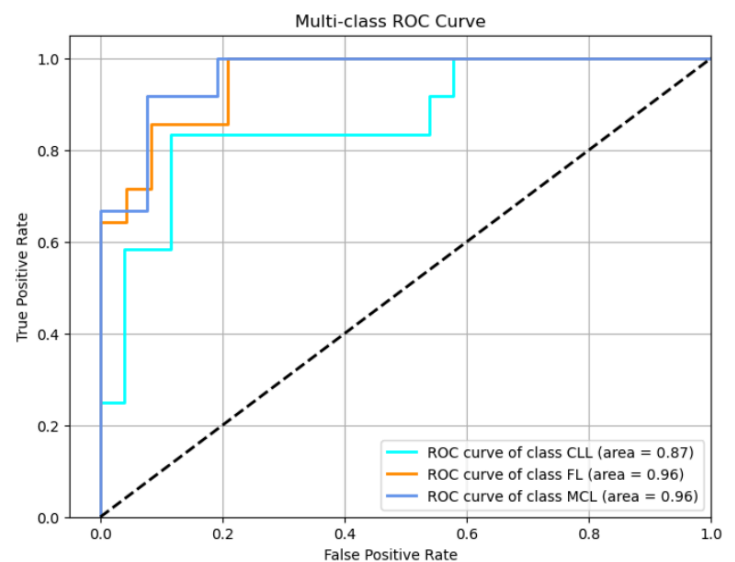
Original training set size: (299, 224, 224, 3)

Augmented training set size: (897, 224, 224, 3)



### Classification Report:

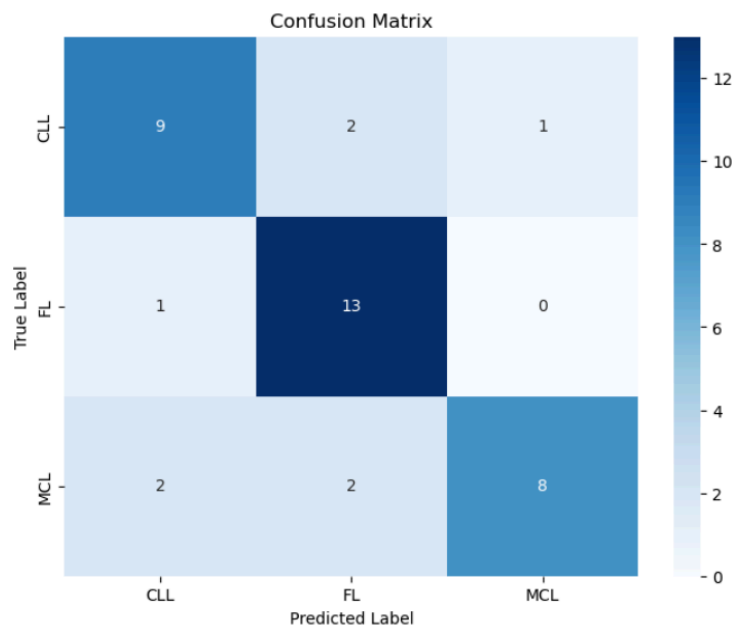
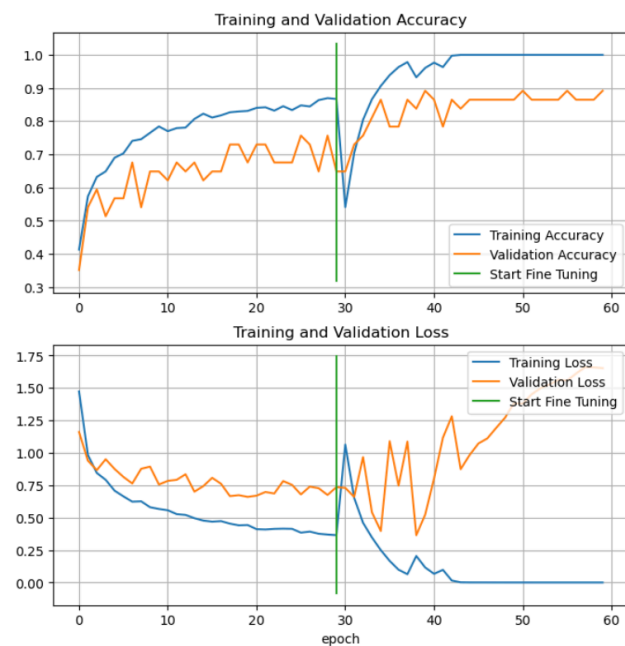
|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| CLL          | 0.78      | 0.58   | 0.67     | 12      |
| FL           | 0.72      | 0.93   | 0.81     | 14      |
| MCL          | 0.82      | 0.75   | 0.78     | 12      |
| accuracy     |           |        | 0.76     | 38      |
| macro avg    | 0.77      | 0.75   | 0.75     | 38      |
| weighted avg | 0.77      | 0.76   | 0.76     | 38      |



## Experimento 57 – VGG16 – Augmentación explícita (x4) – 30 epochs – Batch size 8

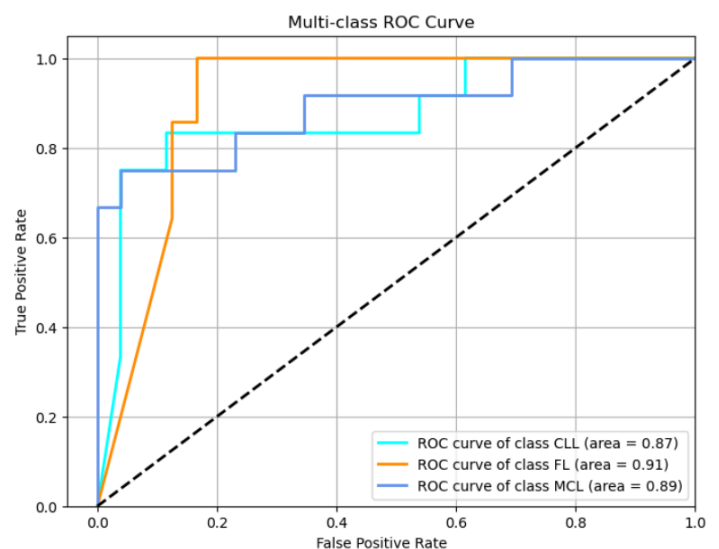
Original training set size: (299, 224, 224, 3)

Augmented training set size: (1196, 224, 224, 3)



### Classification Report:

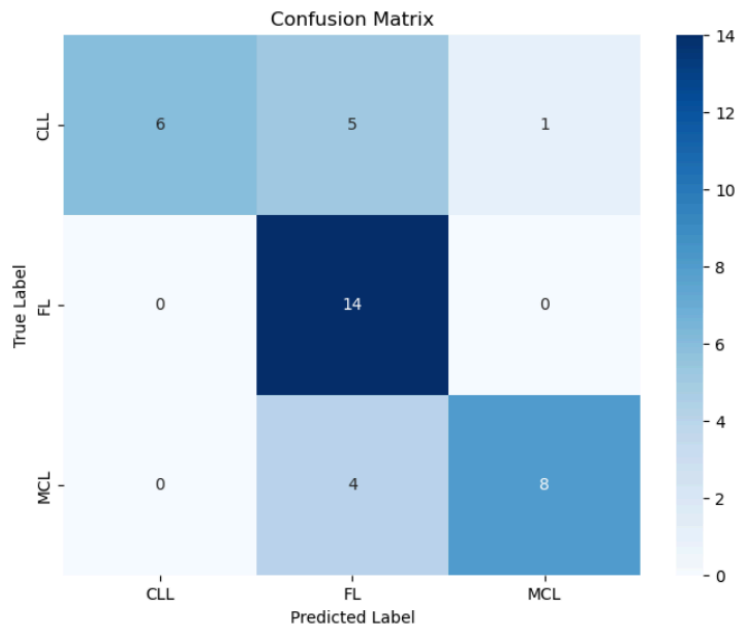
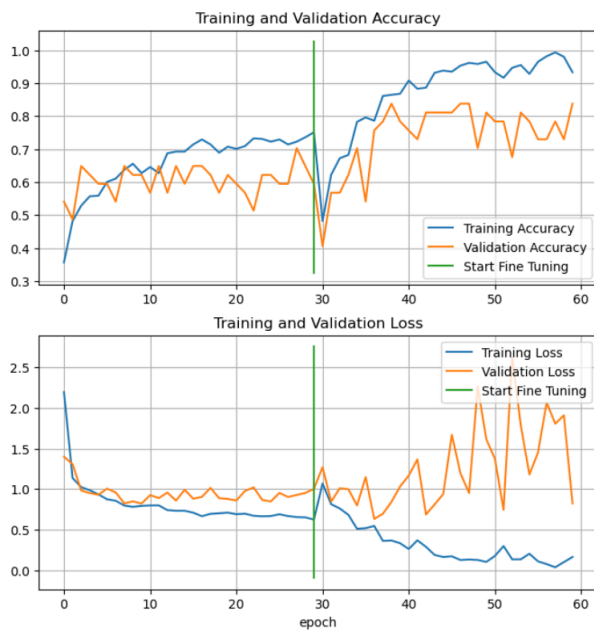
|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| CLL          | 0.75      | 0.75   | 0.75     | 12      |
| FL           | 0.76      | 0.93   | 0.84     | 14      |
| MCL          | 0.89      | 0.67   | 0.76     | 12      |
| accuracy     |           |        | 0.79     | 38      |
| macro avg    | 0.80      | 0.78   | 0.78     | 38      |
| weighted avg | 0.80      | 0.79   | 0.79     | 38      |



## Experimento 58 – VGG16 – Augmentación combinada (x2) – 30 epochs – Batch size 8

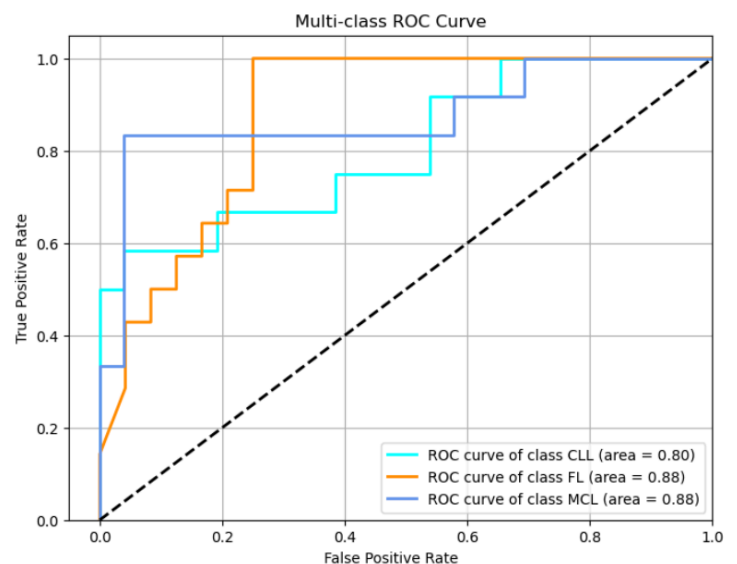
Original training set size: (299, 224, 224, 3)

Augmented training set size: (598, 224, 224, 3)



### Classification Report:

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| CLL          | 1.00      | 0.50   | 0.67     | 12      |
| FL           | 0.61      | 1.00   | 0.76     | 14      |
| MCL          | 0.89      | 0.67   | 0.76     | 12      |
| accuracy     |           |        | 0.74     | 38      |
| macro avg    | 0.83      | 0.72   | 0.73     | 38      |
| weighted avg | 0.82      | 0.74   | 0.73     | 38      |

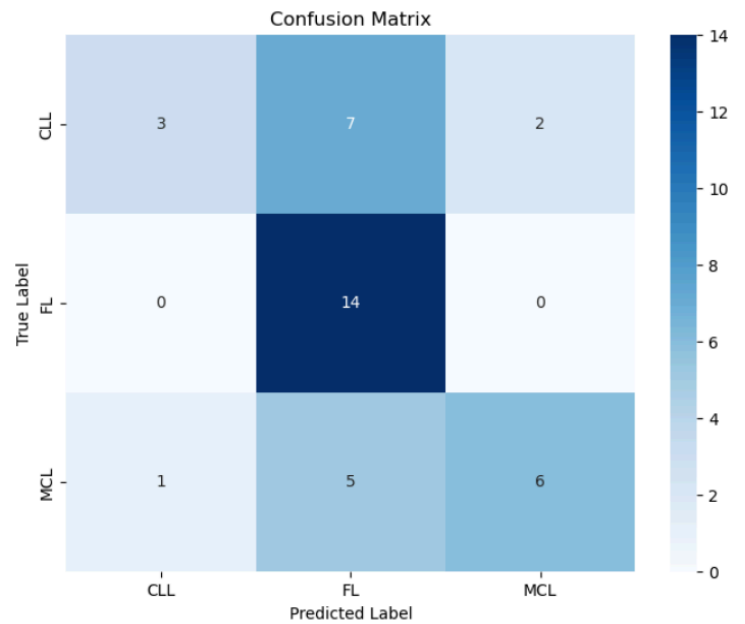
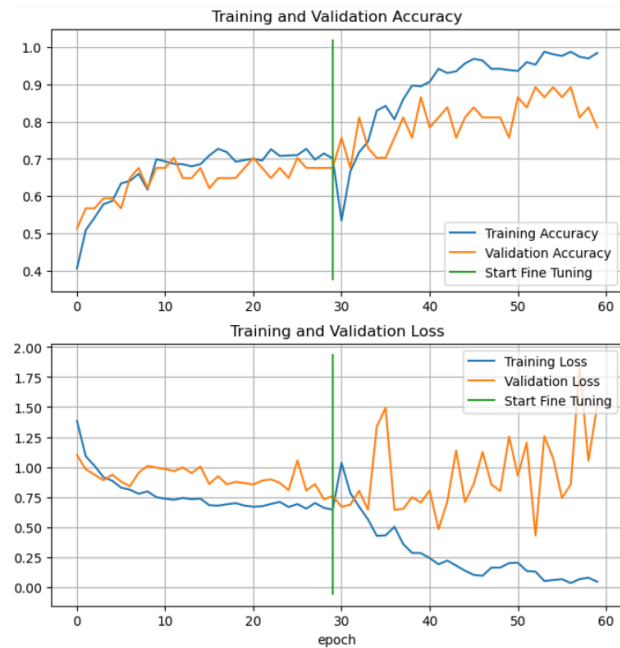




## Experimento 59 – VGG16 – Augmentación combinada (x3) – 30 epochs – Batch size 8

Original training set size: (299, 224, 224, 3)

Augmented training set size: (897, 224, 224, 3)



### Classification Report:

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| CLL          | 0.75      | 0.25   | 0.38     | 12      |
| FL           | 0.54      | 1.00   | 0.70     | 14      |
| MCL          | 0.75      | 0.50   | 0.60     | 12      |
| accuracy     |           |        | 0.61     | 38      |
| macro avg    | 0.68      | 0.58   | 0.56     | 38      |
| weighted avg | 0.67      | 0.61   | 0.57     | 38      |

